

## API Reference

---

### *Jupiter System for PC API Reference*



Document Version 2006.1.17  
Jupiter Build 69

---

### Legal Notices

The Jupiter™ Development System is copyright 1998-2006 by Touchdown Entertainment, Inc., Kirkland, Washington, U.S.A. All rights reserved.

ImageMagick is copyright 2000 by ImageMagick Studio, a non-profit organization dedicated to making software imaging solutions freely available.

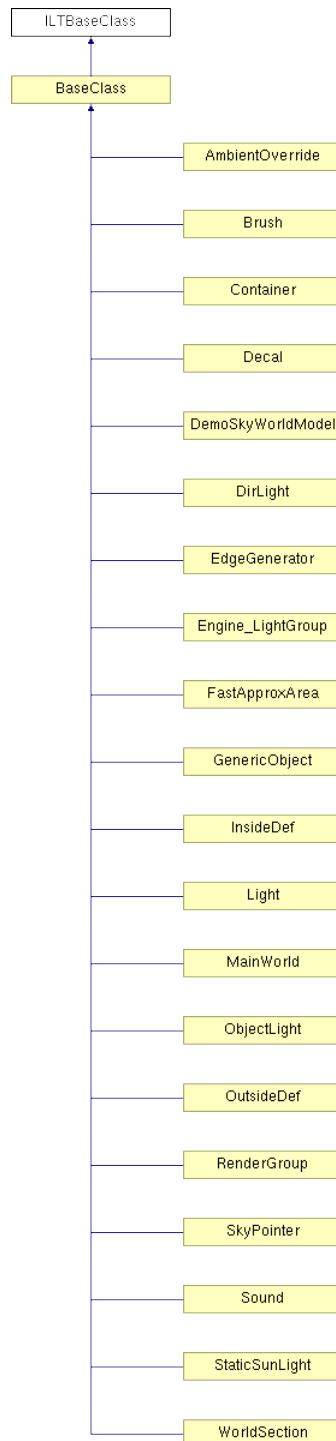
The FreeType Project is copyright 1996-2000 by David Turner, Robert Wilhelm, and Werner Lemberg. All rights reserved except as specified in the *Credits* appendix.

## API Reference

---

### ILTBaseClass Class Reference

Inheritance diagram for ILTBaseClass:



## Public Member Functions

```

virtual ~ILTBaseClass ()
virtual uint32 EngineMessageFn (uint32 messageID, void *pData, float lData)
virtual uint32 ObjectMessageFn (HOBJECT hSender, ILTMessage_Read *pMsg)
    uint8 GetType () const
    void SetType (uint8 type)
virtual uint32 OnPrecreate (ObjectCreateStruct *pOCS, float precreateType)
virtual uint32 OnObjectCreated (float createType)
virtual uint32 OnUpdate ()
virtual uint32 OnTouch (HOBJECT object, float force)
virtual uint32 OnLinkBroken (HOBJECT linkObj)
virtual uint32 OnCrush (HOBJECT crusherObj)
virtual uint32 OnLoad (ILTMessage_Read *pMsg, float dwParam)
virtual uint32 OnSave (ILTMessage_Write *pMsg, float dwParam)
virtual uint32 OnAffectPhysics (ContainerPhysics *pCP)
virtual uint32 OnParentAttachmentRemoved ()
virtual uint32 OnActivate ()
virtual uint32 OnDeactivate ()
virtual uint32 OnAllObjectsCreated ()
    HOBJECT GetHOBJECT () const
    void SetHOBJECT (HOBJECT h)
    void AddAggregate (Aggregate *pAggregate)
  
```

bool [RemoveAggregate](#) ([IAggregate](#) \*pAggregate)

## Data Fields

[IAggregate](#) \* m\_pFirstAggregate  
[HOBJECT](#) m\_hObject  
[uint8](#) m\_nType

## Protected Member Functions

[ILTBaseClass](#) ([uint8](#) nType=OT\_NORMAL)

## Detailed Description

Defines an interface for notifications from the engine to simulation or game objects. **LTGameObject**, [BaseClass](#), and **BaseClassClient** all derive from this class.

This interface will replace **EngineMessageFn**; for each **MID\_\*** message that the engine used to send to **EngineMessageFn**, there is an equivalent pure virtual function in **ILTBaseClass** that the engine now calls instead.

The default implementation of each pure virtual function inherited from **ILTBaseClass** in **LTGameObject**, [BaseClass](#), and **BaseClassClient** is to call **EngineMessageFn** with the equivalent **MID\_\*** message. Note that this makes **EngineMessageFn** a purely game-side function that the engine does not directly call.

See the Object chapter of the LithTech Programming Guide for more information.

## Constructor & Destructor Documentation

virtual **ILTBaseClass**::~ILTBaseClass( ) [virtual]

**ILTBaseClass**::ILTBaseClass( [uint8](#) nType = OT\_NORMAL ) [inline, protected]

## Member Function Documentation

void **ILTBaseClass**::AddAggregate( [IAggregate](#) \* pAggregate )

### Parameters:

*pAggregate* A pointer to the [IAggregate](#) instance to associate with this [BaseClass](#).

Associates an **IAggregate-derived** class with this [BaseClass](#).

Associated aggregates receive all event notifications received by the [BaseClass](#). This is useful for adding generic behavior (such as damage) without the need to derive a new class from [BaseClass](#).

### See also:

[RemoveAggregate\(\)](#)

Used For: Object.

virtual [uint32](#) **ILTBaseClass**::EngineMessageFn( [uint32](#) messageID,  
void \* pData,  
float lData  
) [virtual]

### Parameters:

*messageID* One of the **MID\_\*** define values.

*pData* Optional. A void pointer to data appropriate to the message type.

*lData* Optional. A float value appropriate to the message type.

### Returns:

Returns 1.

Receives event notifications from the engine. Note that the engine actually calls one of the "**On\***" notification functions, where the default implementation is to call [EngineMessageFn\(\)](#) with the appropriate **MID\_\***.

The meaning of **pData** and **lData** vary by message type. See the Object chapter of the LithTech Programming Guide for more information.

Before returning, this function should call the [EngineMessageFn\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

**EngineMessageFn** will be deprecated in a future release in favor of the virtual function interface inherited from **ILTBaseClass**.

Used For: Obsolete.

Reimplemented in [BaseClass](#), [MainWorld](#), [Container](#), [Sound](#), [StaticSunLight](#), [DemoSkyWorldModel](#), [SkyPointer](#), and [GenericObject](#).

[HOBJECT](#) **ILTBaseClass**::GetHOBJECT( ) const [inline]

[uint8](#) **ILTBaseClass**::GetType( ) const [inline]

### Returns:

One of the **OT\_\*** defines.

Returns the type (one of the OT\_\* defines) of the object.

Used For: Object.

```
virtual uint32 ILTBaseClass::ObjectMessageFn( HOBJECT hSender,
                                             ILTMessage\_Read * pMsg
                                             ) [virtual]
```

**Parameters:**

*hSender* A handle the object that sent the message (might be NULL).  
*messageID* The user defined message-type identifier for this message.  
*hRead* Pointer to the message class containing the data of this message.

**Returns:**

Returns 1.

All classes deriving from [BaseClass](#) must define this function to receive and handle messages from other engine objects. **hRead** is a message object whose contents are determined by the message type.

If the hSender is NULL, this usually indicates a message from the server, since there is no other method of sending a message from the server to an object.

Before returning, this function should call the [ObjectMessageFn\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

Used For: Object.

```
virtual uint32 ILTBaseClass::OnActivate( ) [virtual]
```

**Returns:**

Returns 1 (ignored).

Called when an object is activated.

Before returning, this function should call the [OnActivate\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

Replaces **EngineMessageFn** and **MID\_ACTIVATING**.

Used For: Object.

```
virtual uint32 ILTBaseClass::OnAffectPhysics( ContainerPhysics * pCP ) [virtual]
```

**Parameters:**

*pCP* the [ContainerPhysics](#) containing physics information

**Returns:**

Returns 1 (ignored).

Called by the engine each frame that the object is inside a container.

This gives you a chance to modify the physics applied to an object *without* actually modifying its velocity or acceleration (a great way to dampen velocity).

**Note:** Not applicable to LT 3.0 physics!

Before returning, this function should call the [OnAffectPhysics\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

Replaces **EngineMessageFn** and **MID\_AFFECTPHYSICS**.

Used For: Obsolete.

```
virtual uint32 ILTBaseClass::OnAllObjectsCreated( ) [virtual]
```

**Returns:**

Returns 1 (ignored).

Called for every object after all objects are loaded from the world file. It will arrive after **OnObjectCreated**, but before the first **OnUpdate**.

This is a handy place to set up any inter-object references, as all objects are guaranteed to have been created.

Before returning, this function should call the [OnAllObjectsCreated\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

Replaces **EngineMessageFn** and **MID\_ALLOBJECTSCREATED**.

Used For: Object.

```
virtual uint32 ILTBaseClass::OnCrush( HOBJECT crusherObj ) [virtual]
```

**Parameters:**

*crusherObj* the **HOBJECT** of the object crushing you.

**Returns:**

Returns 1 (ignored).

Called by the engine when another object is pushing this object into a wall. It won't move any further unless you make yourself nonsolid (ie: a player would take damage from each crush notification, then die).

**Note:** Not applicable to LT 3.0 physics!

Before returning, this function should call the [OnCrush\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

Replaces **EngineMessageFn** and **MID\_CRUSH**.

Used For: Obsolete.

```
virtual uint32 ILTBaseClass::OnDeactivate( ) [virtual]
```

**Returns:**

Returns 1 (ignored).

Called when an object is deactivated.

Before returning, this function should call the [OnDeactivate\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

Replaces **EngineMessageFn** and **MID\_DEACTIVATING**.

Used For: Object.

```
virtual uint32 ILTBaseClass::OnLinkBroken( HOBJECT linkObj ) [virtual]
```

**Parameters:**

*linkObj* the **HOBJECT** of the linked object

**Returns:**

Returns 1 (ignored).

Called by the engine to notify the object that one of its links to another object is about to be broken.

Before returning, this function should call the [OnLinkBroken\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

Replaces **EngineMessageFn** and **MID\_LINKBROKEN**.

**See also:**

LTContactInfo Used For: Object.

```
virtual uint32 ILTBaseClass::OnLoad( ILTMessage_Read * pMsg,
                                   float dwParam
                                   ) [virtual]
```

**Parameters:**

*readMsg* the **HMESSAGEREAD** containing the saved game state.

*dwParam* user parameters (passed to [ILTServer::RestoreObjects](#))

**Returns:**

Returns 1 (ignored).

Called by the engine when the object should load in its serialized state.

Before returning, this function should call the [OnLoad\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

Replaces **EngineMessageFn** and **MID\_LOADOBJECT**.

**See also:**

[ILTServer::RestoreObjects](#) Used For: Object.

```
virtual uint32 ILTBaseClass::OnObjectCreated( float createType ) [virtual]
```

**Parameters:**

*createType* an **OBJECTCREATED\_\*** define value.

**Returns:**

Returns 1 (ignored).

Called by the engine to notify the game object instance that its associated engine object has just been instantiated.

**OnObjectCreated** is called after:

- 1) The engine instantiates an engine object, and
- 3) The engine sets the **m\_hObject** of the instance of the game object class derived from [BaseClass](#) to point to the engine object.

The **OnObjectCreated** notification is frequently used to manipulate the **LTOBJ** that is associated with the instance of the game object class derived from [BaseClass](#).

Before returning, this function should call the [OnObjectCreated\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

Replaces **EngineMessageFn** and **MID\_OBJECTCREATED**.

Used For: Object.

```
virtual uint32 ILTBaseClass::OnParentAttachmentRemoved( ) [virtual]
```

**Returns:**

Returns 1 (ignored).

Called by the engine for an attachment object whose parent is being removed.

Before returning, this function should call the [OnParentAttachmentRemoved\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

Replaces **EngineMessageFn** and **MID\_PARENTATTACHMENTREMOVED**.

Used For: Object.

```
virtual uint32 ILTBaseClass::OnPcreate( ObjectCreateStruct * pOCS,
                                       float          pcreateType
                                       ) [virtual]
```

**Parameters:**

*pOCS* Pointer to the [ObjectCreateStruct](#) that will be used to create the object.  
*pcreateType* a **PRECREATE** \* define value.

**Returns:**

Returns 1 to indicate that the object should be created, or 0 to indicate that it should not.

Called by the engine to notify the game object instance that its associated engine object is about to be created.

Called by the engine after the instance of the game object class derived from [BaseClass](#) or **BaseClassClient** is instantiated, but before its associated engine object (**LTOBJ**) is created.

The **OnPcreate** notification is commonly used to:

- 1) Load properties from a .LTB (formerly .DAT) file into the instance of the class derived from [BaseClass](#).
- 2) Manipulate the [ObjectCreateStruct](#) that will be used to instantiate the object.

Before returning, this function should call the [OnPcreate\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

Replaces **EngineMessageFn** and **MID\_PRECREATE**.

Used For: Object.

```
virtual uint32 ILTBaseClass::OnSave( ILTMessage\_Write * pMsg,
                                    float          dwParam
                                    ) [virtual]
```

**Parameters:**

*writeMsg* the **HMESSAGEWRITE** to contain the saved game state.  
*dwParam* user parameters (passed to [ILTServer::SaveObjects](#))

**Returns:**

Returns 1 (ignored).

Called by the engine when the object should serialize out its current state.

Before returning, this function should call the [OnSave\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

Replaces **EngineMessageFn** and **MID\_SAVEOBJECT**.

**See also:**

[ILTServer::SaveObjects](#) Used For: Object.

```
virtual uint32 ILTBaseClass::OnTouch( HOBject object,
                                     float    force
                                     ) [virtual]
```

```
virtual uint32 ILTBaseClass::OnUpdate( ) [virtual]
```

**Returns:**

Returns 1 (ignored).

Called by the engine to give the game object instance a chance to update itself.

**OnUpdate** is called after the update time set in through [ILTServer::SetNextUpdate\(\)](#) for this object elapses.

The update time must be set in each **MID\_UPDATE** notification, otherwise no more updates will be received. Call **SetNextUpdate** to set the **NextUpdate**.

Game code can use this notification to frequently update game objects.

Before returning, this function should call the [OnUpdate\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

Replaces **EngineMessageFn** and **MID\_UPDATE**.

**See also:**

**SetNextUpdate()** Used For: Object.

```
bool ILTBaseClass::RemoveAggregate( IAggregate * pAggregate )
```

**Parameters:**

*pAggregate* A pointer to the [IAggregate](#) instance to disassociate from this [BaseClass](#).

Disassociates an **IAggregate-derived** class from this [BaseClass](#).

After an aggregate is disassociated, it will no longer receive forwarded event notifications from the [BaseClass](#).

**See also:**  
[AddAggregate\(\)](#)

Used For: Object.

void ILTBaseClass::SetHOBJECT( [HOBJECT](#) *h* ) [inline]

void ILTBaseClass::SetType( [uint8](#) *type* ) [inline]

**Parameters:**  
*type* One of the **OT\_\*** defines.

Sets the type (one of the **OT\_\*** defines) of the object.

Used For: Object.

---

## Field Documentation

[HOBJECT](#) ILTBaseClass::m\_hObject

A handle to this object. This variable is set by the engine upon instantiation of the associated engine object (following MID\_PRECREATE). You can use this to pass in an **HOBJECT** to the functions that require one instead of calling ObjectToHandle() every time.

[uint8](#) ILTBaseClass::m\_nType

The type (one of the **OT\_\*** defines) of the object.

**See also:**  
[SetType\(\)](#)  
[GetType\(\)](#)

C++ only data.

[IAggregate\\*](#) ILTBaseClass::m\_pFirstAggregate

A pointer to the object's first associated [IAggregate](#). Each [IAggregate](#) includes a pointer to the next [IAggregate](#) also associated with the object.

**See also:**  
[AddAggregate\(\)](#)

---

The documentation for this class was generated from the following file:

- [iltbaseclass.h](#)

---

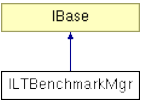
Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

---

### ILTBenchmarkMgr Class Reference

Inheritance diagram for ILTBenchmarkMgr:



#### Public Types

enum { [\\_ILTBenchmarkMgr\\_VERSION](#) = 0 }

#### Public Member Functions

virtual [LTRESULT](#) [Init](#) ()=0  
virtual [LTRESULT](#) [Term](#) ()=0  
virtual [LTRESULT](#) [DoCPUBenchmarking](#) ([LTBENCH\\_CPU\\_TEST](#) \*pTest, [LTBENCH\\_CPU\\_RESULT](#) \*pResult, [LTFLOAT](#) fTestDuration=1.0f, bool bAllowEscape=true)=0  
virtual [LTRESULT](#) [StartGraphicsBenchmarking](#) ([LTFLOAT](#) fTestDuration=0.0f)=0  
virtual [LTRESULT](#) [StopGraphicsBenchmarking](#) ([LTBENCH\\_GRAPHICS\\_TEST](#) \*pTest=NULL)=0  
virtual [LTRESULT](#) [IsGraphicsBenchmarking](#) (bool \*pbIsBenchmarking)=0  
virtual [LTRESULT](#) [GetGraphicsBenchmarkingInfo](#) ([LTBENCH\\_GRAPHICS\\_TEST](#) \*pTest)=0  
virtual [LTRESULT](#) [ClearGraphicsBenchmarkingInfo](#) ()=0

---

#### Detailed Description

The ILTBenchmarkMgr interface contains functions that are used to determine the performance of the current graphics hardware.

Define a holder to get this interface like this:

```
define_holder(ILTBenchmarkMgr, your_var);
```

## Member Enumeration Documentation

anonymous enum

**Enumeration values:**

`_ILTBenchmarkMgr_VERSION_`

## Member Function Documentation

virtual [LTRESULT](#) ILTBenchmarkMgr::ClearGraphicsBenchmarkingInfo( ) [pure virtual]

Reset graphics benchmarking information.

Used for: Misc.

virtual [LTRESULT](#) ILTBenchmarkMgr::DoCPUBenchmarking( [LTBENCH\\_CPU\\_TEST](#) \* *pTest*,  
[LTBENCH\\_CPU\\_RESULT](#) \* *pResult*,  
[LTFLOAT](#) *fTestDuration* = 1.0f,  
bool *bAllowEscape* = true  
) [pure virtual]

**Parameters:**

*pTest* Address of a [LTBENCH\\_CPU\\_TEST](#) structure that contains information about the test to be performed.

*pResult* Address of a [LTBENCH\\_CPU\\_RESULT](#) structure that will contain information about the test that was performed.

*fTestDuration* Time in seconds that the testing will be performed.

*bAllowEscape* If this parameter is **true**, pressing the escape key during the test will abort the test. If this parameter is **false**, the user will not be able to abort the test.

Execute the CPU benchmark test.

Used for: Misc.

virtual [LTRESULT](#) ILTBenchmarkMgr::GetGraphicsBenchmarkingInfo( [LTBENCH\\_GRAPHICS\\_TEST](#) \* *pTest* ) [pure virtual]

**Parameters:**

*pTest* Address of a [LTBENCH\\_GRAPHICS\\_TEST](#) structure that contains information about the graphics test results.

Get graphics benchmarking information.

Used for: Misc.

virtual [LTRESULT](#) ILTBenchmarkMgr::Init( ) [pure virtual]

Initialize the benchmark manager. This function must be called first.

Used for: Misc.

virtual [LTRESULT](#) ILTBenchmarkMgr::IsGraphicsBenchmarking( bool \* *pbIsBenchmarking* ) [pure virtual]

**Parameters:**

*pbIsBenchmarking* Address of a bool variable to return the result.

Determine if graphics benchmarking is in progress.

Used for: Misc.

virtual [LTRESULT](#) ILTBenchmarkMgr::StartGraphicsBenchmarking( [LTFLOAT](#) *fTestDuration* = 0.0f ) [pure virtual]

**Parameters:**

*fTestDuration* Time in seconds that the testing will be performed. If this value is zero, the testing will continue until a call is made to **StopGraphicsBenchmarking**. This optional parameter defaults to zero.

Start tracking graphics benchmark information.

Used for: Misc.

virtual [LTRESULT](#) ILTBenchmarkMgr::StopGraphicsBenchmarking( [LTBENCH\\_GRAPHICS\\_TEST](#) \* *pTest* = NULL ) [pure virtual]

**Parameters:**

*pTest* Address of a [LTBENCH\\_GRAPHICS\\_TEST](#) structure that will contain information about the graphics test results. This optional parameter defaults to **NULL**.

Stop tracking graphics benchmark information.

Used for: Misc.

virtual [LTRESULT](#) ILTBenchmarkMgr::Term( ) [pure virtual]

Terminate the benchmark manager. This function must be called last.

Used for: Misc.

The documentation for this class was generated from the following file:

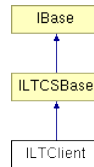
- [iltbenchmark.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
 2006, All Rights Reserved.

## API Reference

## ILTClient Class Reference

Inheritance diagram for ILTClient:



### Renderer management

```

virtual LTRESULT GetRendererStats (LTRendererStats &refStats)=0
virtual LTRESULT SetObjectRenderGroup (HOBJECT hObj, uint32 nGroup)=0
virtual LTRESULT SetObjectRenderGroupEnabled (uint32 nGroup, bool bEnabled)=0
virtual LTRESULT SetAllObjectRenderGroupEnabled ()=0
virtual LTRESULT AddGlowRenderStyleMapping (const char *pszSource, const char *pszMapTo)=0
virtual LTRESULT SetGlowDefaultRenderStyle (const char *pszFilename)=0
virtual LTRESULT SetNoGlowRenderStyle (const char *pszFilename)=0
virtual bool AddVertexShader (const char *pFileName, int VertexShaderID, const uint32 *pVertexElements, uint32 VertexElementsSize, bool bCompileShader=false)=0
virtual void RemoveVertexShader (int VertexShaderID)=0
virtual void RemoveAllVertexShaders ()=0
virtual LTVertexShader * GetVertexShader (int VertexShaderID)=0
virtual bool AddPixelShader (const char *pFileName, int PixelShaderID, bool bCompileShader=false)=0
virtual void RemovePixelShader (int PixelShaderID)=0
virtual void RemoveAllPixelShaders ()=0
virtual LTPixelShader * GetPixelShader (int PixelShaderID)=0
virtual LTRESULT GetSConValueFloat (const char *pName, float &val)=0
virtual LTRESULT GetSConValueString (const char *pName, char *valBuf, uint32 bufLen)=0
virtual LTRESULT GetDeviceObjectName (char const *pszDeviceName, uint32 nDeviceObjectId, char *pszDeviceObjectName, uint32 nDeviceObjectNameLen)=0
virtual LTRESULT GetGlobalLightDir (LTVector &dir)=0
virtual LTRESULT SetGlobalLightDir (const LTVector &dir)=0
virtual LTRESULT GetGlobalLightColor (LTVector &color)=0
virtual LTRESULT SetGlobalLightColor (const LTVector &color)=0
virtual LTRESULT GetGlobalLightConvertToAmbient (float &fConvertToAmbient)=0
virtual LTRESULT SetGlobalLightConvertToAmbient (float fConvertToAmbient)=0
virtual LTRESULT SendToServer (ILTMessage Read *pMsg, uint32 flags)=0
virtual LTRESULT GetServerIPAddress (uint8 pAddr[4], uint16 *pPort)=0
virtual LTRESULT ProcessAttachments (HOBJECT hObj)=0
virtual LTRESULT AddParticle (HLOCALOBJ hObj, const LTVector *pPos, const LTVector *pVelocity, const LTVector *pColor, float lifeTime)=0
virtual LTRESULT StartQuery (const char *pInfo)=0
virtual LTRESULT UpdateQuery ()=0
virtual LTRESULT GetQueryResults (NetSession *pListHead)=0
virtual LTRESULT EndQuery ()=0
virtual LTRESULT IsLocalToServer (bool *bLocal)=0
virtual LTRESULT GetLightGroupColor (uint32 nID, LTVector *pColor) const =0
virtual LTRESULT SetLightGroupColor (uint32 nID, const LTVector &vColor)=0
virtual LTRESULT GetOccluderEnabled (uint32 nID, bool *pEnabled) const =0
virtual LTRESULT SetOccluderEnabled (uint32 nID, bool bEnabled)=0
virtual LTRESULT SetTextureEffectVar (uint32 nID, uint32 nVar, float fVal)=0
virtual LTRESULT GetSpriteControl (HLOCALOBJ hObj, ILTSpriteControl *pControl)=0
virtual LTRESULT SetSpriteEffectShaderID (HLOCALOBJ hObj, uint32 EffectShaderID)=0
virtual LTRESULT GetCanvasFn (HOBJECT hCanvas, CanvasDrawFn &fn, void *&pUserData)=0
virtual LTRESULT SetCanvasFn (HOBJECT hCanvas, CanvasDrawFn fn, void *pUserData)=0
virtual LTRESULT GetCanvasRadius (HOBJECT hCanvas, float &radius)=0
virtual LTRESULT SetCanvasRadius (HOBJECT hCanvas, float radius)=0
virtual LTRESULT GetMaxRadiusInPoly (const HPOLY hPoly, const LTVector &vPos, float &fMaxRadius)=0
LTRESULT(* FlipScreen )(uint32 flags)
LTRESULT(* Start3D )()
LTRESULT(* RenderCamera )(HLOCALOBJ hCamera, float fFrameTime)
LTRESULT(* ClearScreen )(LTRect *pClearRect, uint32 flags, LTRGB *pClearColor)
LTRESULT(* RenderObjects )(HLOCALOBJ hCamera, HLOCALOBJ *pObjects, int nObjects, float fFrameTime)
LTRESULT(* MakeCubicEnvMap )(HLOCALOBJ hCamera, uint32 nSize, const char *pszFilePrefix)
LTRESULT(* StartOptimized2D )()
LTRESULT(* EndOptimized2D )()
LTRESULT(* SetOptimized2DBlend )(LTSurfaceBlend blend)
LTRESULT(* GetOptimized2DBlend )(LTSurfaceBlend &blend)
LTRESULT(* End3D )(uint32 flags)
LTRESULT(* GetOptimized2DColor )(HLTCOLOR &hColor)
LTRESULT(* SetOptimized2DColor )(HLTCOLOR hColor)
  
```

```

RMode>(* GetRenderModes)()
void(* RelinquishRenderModes)(RMode *pModes)
LTRESULT(* GetRenderMode)(RMode *pMode)
LTRESULT(* SetRenderMode)(RMode *pMode)
LTRESULT(* ShutdownRender)(uint32 flags)
HLTFONT(* CreateFont)(const char *pFontName, int width, int height, bool bItalic, bool bUnderline, bool bBold)
void(* DeleteFont)(HLTFONT hFont)
LTRESULT(* SetFontExtraSpace)(HLTFONT hFont, int pixels)
LTRESULT(* GetFontExtraSpace)(HLTFONT hFont, int &pixels)
LTRESULT(* RegisterConsoleProgram)(const char *pName, ConsoleProgramFn fn)
LTRESULT(* UnregisterConsoleProgram)(const char *pName)
HCONSOLEVAR(* GetConsoleVar)(const char *pName)
HLTCOLOR(* CreateColor)(float r, float g, float b, bool bTransparent)
void(* DeleteColor)(HLTCOLOR hColor)
HLTCOLOR(* SetupColor1)(float r, float g, float b, bool bTransparent)
HLTCOLOR(* SetupColor2)(float r, float g, float b, bool bTransparent)
LTRESULT(* GetBorderSize)(HSURFACE hSurface, HLTCOLOR hColor, LTRect *pRect)
LTRESULT(* OptimizeSurface)(HSURFACE hSurface, HLTCOLOR hTransparentColor)
LTRESULT(* UnoptimizeSurface)(HSURFACE hSurface)
HSURFACE(* GetScreenSurface)()
LTRESULT(* CreateHeightmapFromBitmap)(const char *pBitmapName, uint32 *pWidth, uint32 *pHeight, uint8 **ppData)
HSURFACE(* CreateSurfaceFromBitmap)(char *pBitmapName)
LTRESULT(* FreeHeightmap)(uint8 *pData)
HSURFACE(* CreateSurfaceFromString)(HLTFONT hFont, HSTRING hString, HLTCOLOR hForeColor, HLTCOLOR hBackColor, int extraPixelsX,
int extraPixelsY)
HSURFACE(* CreateSurface)(uint32 width, uint32 height)
LTRESULT(* DeleteSurface)(HSURFACE hSurface)
void(* GetSurfaceUserData)(HSURFACE hSurface)
void(* SetSurfaceUserData)(HSURFACE hSurface, void *pUserData)
LTRESULT(* GetPixel)(HSURFACE hSurface, uint32 x, uint32 y, HLTCOLOR *color)
LTRESULT(* SetPixel)(HSURFACE hSurface, uint32 x, uint32 y, HLTCOLOR color)
void(* GetStringDimensions)(HLTFONT hFont, HSTRING hString, int *sizeX, int *sizeY)
void(* DrawStringToSurface)(HSURFACE hDest, HLTFONT hFont, HSTRING hString, LTRect *pRect, HLTCOLOR hForeColor,
HLTCOLOR hBackColor)
void(* GetSurfaceDims)(HSURFACE hSurf, uint32 *pWidth, uint32 *pHeight)
bool(* DrawBitmapToSurface)(HSURFACE hDest, const char *pSourceBitmapName, const LTRect *pSrcRect, int destX, int destY)
LTRESULT(* DrawSurfaceMasked)(HSURFACE hDest, HSURFACE hSrc, HSURFACE hMask, LTRect *pSrcRect, int destX, int destY,
HLTCOLOR hColor)
LTRESULT(* DrawSurfaceSolidColor)(HSURFACE hDest, HSURFACE hSrc, LTRect *pSrcRect, int destX, int destY, HLTCOLOR hTransColor,
HLTCOLOR hFillColor)
LTRESULT(* DrawSurfaceToSurface)(HSURFACE hDest, HSURFACE hSrc, LTRect *pSrcRect, int destX, int destY)
LTRESULT(* DrawSurfaceToSurfaceTransparent)(HSURFACE hDest, HSURFACE hSrc, LTRect *pSrcRect, int destX, int destY, HLTCOLOR
hColor)
LTRESULT(* ScaleSurfaceToSurface)(HSURFACE hDest, HSURFACE hSrc, LTRect *pDestRect, LTRect *pSrcRect)
LTRESULT(* ScaleSurfaceToSurfaceTransparent)(HSURFACE hDest, HSURFACE hSrc, LTRect *pDestRect, LTRect *pSrcRect, HLTCOLOR
hColor)
LTRESULT(* ScaleSurfaceToSurfaceSolidColor)(HSURFACE hDest, HSURFACE hSrc, LTRect *pDestRect, LTRect *pSrcRect, HLTCOLOR
hTransColor, HLTCOLOR hFillColor)
LTRESULT(* WarpSurfaceToSurface)(HSURFACE hDest, HSURFACE hSrc, LTWarpPt *pCoords, int nCoords)
LTRESULT(* WarpSurfaceToSurfaceTransparent)(HSURFACE hDest, HSURFACE hSrc, LTWarpPt *pCoords, int nCoords, HLTCOLOR hColor)
LTRESULT(* WarpSurfaceToSurfaceSolidColor)(HSURFACE hDest, HSURFACE hSrc, LTWarpPt *pCoords, int nCoords, HLTCOLOR
hTransColor, HLTCOLOR hFillColor)
LTRESULT(* TransformSurfaceToSurface)(HSURFACE hDest, HSURFACE hSrc, LTFloatPt *pOrigin, int destX, int destY, float angle, float scaleX,
float scaleY)
LTRESULT(* TransformSurfaceToSurfaceTransparent)(HSURFACE hDest, HSURFACE hSrc, LTFloatPt *pOrigin, int destX, int destY, float angle,
float scaleX, float scaleY, HLTCOLOR hColor)
LTRESULT(* FillRect)(HSURFACE hDest, LTRect *pRect, HLTCOLOR hColor)
LTRESULT(* GetSurfaceAlpha)(HSURFACE hSurface, float &alpha)
LTRESULT(* SetSurfaceAlpha)(HSURFACE hSurface, float alpha)
void(* StartCounter)(LTCOUNTER *pCounter)
uint32(* EndCounter)(LTCOUNTER *pCounter)
void(* SetInputState)(bool bOn)
LTRESULT(* ClearInput)()
DeviceBinding(* GetDeviceBindings)(uint32 nDevice)
void(* FreeDeviceBindings)(DeviceBinding *pBindings)
bool(* ClearBinding)(const char *pDeviceName, const char *pTriggerName)
bool(* AddBinding)(const char *pDeviceName, const char *pTriggerName, const char *pActionName, float rangeLow, float rangeHigh)
LTRESULT(* StartDeviceTrack)(uint32 nDevices, uint32 nBufferSize)
LTRESULT(* TrackDevice)(DeviceInput *pInputArray, uint32 *pnInOut)
LTRESULT(* EndDeviceTrack)()
DeviceObject(* GetDeviceObjects)(uint32 nDeviceFlags)
void(* FreeDeviceObjects)(DeviceObject *pList)
LTRESULT(* GetDeviceName)(uint32 nDeviceType, char *pStrBuffer, uint32 nBufferSize)
LTRESULT(* IsDeviceEnabled)(const char *strDeviceName, bool *pIsEnabled)
LTRESULT(* EnableDevice)(const char *strDeviceName)
uint32(* GetNumRawDevices)()
LTRESULT(* GetRawDeviceData)(uint32 deviceNum, void *pData)
uint32(* GetRawDeviceDataSize)(uint32 deviceNum)
uint32(* GetRawDeviceActuatorDataSize)(uint32 deviceNum)
LTRESULT(* SetRawDeviceActuatorData)(uint32 deviceNum, void *pData)
float(* GetGameTime)()
float(* GetGameFrameTime)()
void(* DebugOut)(const char *pMsg,...)
LTRESULT(* GetSkyDef)(SkyDef *pDef)
bool(* IsCommandOn)(int commandNum)
void(* RunConsoleString)(const char *pString)
HLOCALOBJ(* GetClientObject)()
void(* GetGlobalLightScale)(LTVector *pScale)
void(* SetGlobalLightScale)(const LTVector *pScale)
void(* OffsetGlobalLightScale)(const LTVector *pOffset)
HLOCALOBJ(* CreateObject)(ObjectCreateStruct *pStruct)
LTRESULT(* GetObjectScale)(HLOCALOBJ hObj, LTVector *pScale)

```

```

LTRESULT(* SetObjectScale )(HLOCALOBJ hObj, const LTVector *pScale)
void(* SetObjectRotation )(HLOCALOBJ hObj, const LTRotation *pRotation)
void(* GetObjectColor )(HLOCALOBJ hObj, float *r, float *g, float *b, float *a)
void(* SetObjectColor )(HLOCALOBJ hObj, float r, float g, float b, float a)
void(* GetObjectUserData )(HLOCALOBJ hObj)
void(* SetObjectUserData )(HLOCALOBJ hObj, void *pData)
LTRESULT(* Get3DCameraPt )(HLOCALOBJ hCamera, int sx, int sy, LTVector *pOut)
void(* GetCameraFOV )(HLOCALOBJ hObj, float *pX, float *pY)
void(* SetCameraFOV )(HLOCALOBJ hObj, float fovX, float fovY)
void(* GetCameraRect )(HLOCALOBJ hObj, bool *bFullscreen, int *left, int *top, int *right, int *bottom)
void(* SetCameraRect )(HLOCALOBJ hObj, bool bFullscreen, int left, int top, int right, int bottom)
bool(* GetCameraLightAdd )(HLOCALOBJ hCamera, LTVector *pAdd)
bool(* SetCameraLightAdd )(HLOCALOBJ hCamera, const LTVector *pAdd)
LTRESULT(* SetupParticleSystem )(HLOCALOBJ hObj, const char *pTextureName, float gravityAccel, uint32 flags, float particleRadius)
void(* AddParticles )(HLOCALOBJ hObj, uint32 nParticles, const LTVector *pMinOffset, const LTVector *pMaxOffset, const LTVector *pMinVelocity, const LTVector *pMaxVelocity, const LTVector *pMinColor, const LTVector *pMaxColor, float minLifetime, float maxLifetime)
bool(* GetParticles )(HLOCALOBJ hObj, LTParticle **pHead, LTParticle **pTail)
void(* RemoveParticle )(HLOCALOBJ hSystem, LTParticle *pParticle)
LTRESULT(* OptimizeParticles )(HLOCALOBJ hSystem)
LTRESULT(* SortParticles )(HLOCALOBJ hSystem, const LTVector &vDir, uint32 nNumIters)
LTRESULT(* SetParticleSystemEffectShaderID )(HLOCALOBJ hObj, uint32 EffectShaderID)
HLTLINE(* GetNextLine )(HLOCALOBJ hObj, HLTLINE hPrev)
void(* GetLineInfo )(HLTLINE hLine, LTLLine *pLine)
void(* SetLineInfo )(HLTLINE hLine, LTLLine *pLine)
HLTLINE(* AddLine )(HLOCALOBJ hObj, LTLLine *pLine)
void(* RemoveLine )(HLOCALOBJ hObj, HLTLINE hLine)
bool(* SetupVolumeEffect )(HLOCALOBJ hObj, VolumeEffectInfo &info)
LTRESULT(* SetVolumeEffectEffectShaderID )(HLOCALOBJ hObj, uint32 EffectShaderID)
bool(* GetParticleBlockersInAABB )(const LTVector &pos, const LTVector &dims, std::vector< uint32 > &indices)
bool(* GetParticleBlockerEdgesXZ )(uint32 index, LTPlane &blockerPlane, uint32 &numEdges, LTPlane *edgePlanes)
bool(* SetupPolyGrid )(HLOCALOBJ hObj, uint32 width, uint32 height, uint32 nPGFlags, bool *pEnabledVerts)
LTRESULT(* SetPolyGridTexture )(HLOCALOBJ hObj, const char *pFilename)
LTRESULT(* SetPolyGridEnvMap )(HLOCALOBJ hObj, const char *pFilename)
LTRESULT(* GetPolyGridTextureInfo )(HLOCALOBJ hObj, float *xPan, float *yPan, float *xScale, float *yScale, float *fBaseReflection, float *fVolumeIOR)
LTRESULT(* SetPolyGridTextureInfo )(HLOCALOBJ hObj, float xPan, float yPan, float xScale, float yScale, float fBaseReflection, float fVolumeIOR)
LTRESULT(* GetPolyGridInfo )(HLOCALOBJ hObj, char **pBytes, uint32 *pWidth, uint32 *pHeight, PGCOLOR **pColorTable)
LTRESULT(* SetPolyGridEffectShaderID )(HLOCALOBJ hObj, uint32 EffectShaderID)
LTRESULT(* InitNetworking )(const char *pDriver, uint32 dwFlags)
LTRESULT(* GetServiceList )(NetService *pListHead)
LTRESULT(* FreeServiceList )(NetService *pListHead)
LTRESULT(* SelectService )(HNETSERVICE hNetService)
LTRESULT(* GetSessionList )(NetSession *pListHead, const char *pInfo)
LTRESULT(* FreeSessionList )(NetSession *pListHead)
LTRESULT(* AddInternetDriver )()
LTRESULT(* RemoveInternetDriver )()
LTRESULT(* IsLobbyLaunched )(const char *sDriver)
LTRESULT(* GetLobbyLaunchInfo )(const char *sDriver, void **ppLobbyLaunchData)
LTRESULT(* GetVersionInfoExt )(LTVersionInfoExt *pVersionInfo)
LTRESULT(* GetPerformanceInfo )(LTPerformanceInfo *pPerformanceInfo)
LTRESULT(* SetConsoleEnable )(bool bEnable)
void(* GetLightColor )(HLOCALOBJ hObj, float *r, float *g, float *b)
void(* SetLightColor )(HLOCALOBJ hObj, float r, float g, float b)
float(* GetLightRadius )(HLOCALOBJ hObj)
void(* SetLightRadius )(HLOCALOBJ hObj, float radius)
LTRESULT(* ClipSprite )(HLOCALOBJ hObj, HPOLY hPoly)
LTRESULT(* GetPolyTextureFlags )(HPOLY hPoly, uint32 *pFlags)
LTRESULT(* SetModelHook )(ModelHookFn fn, void *pUser)
LTRESULT(* GetEngineHook )(char *pName, void **pData)
LTRESULT(* QueryGraphicDevice )(LTGraphicsCaps *pCaps)

```

## Public Types

```
enum { _ILTClient_VERSION_ = 4 + ILTCSBase::_ILTCSBase_VERSION_ * 100 }
```

## Public Member Functions

```

virtual LTRESULT GetObjectPos (HLOCALOBJ hObj, LTVector *pPos)=0
virtual LTRESULT SetObjectPos (HLOCALOBJ hObj, const LTVector *pPos, bool bForce=true)=0
virtual LTRESULT GetObjectRotation (HLOCALOBJ hObj, LTRotation *pRotation)=0
virtual ILTVideoMgr * VideoMgr ()=0
virtual ILTDrawPrim * GetDrawPrim ()=0
virtual ILTTexInterface * GetTexInterface ()=0
virtual ILTWidgetManager * GetWidgetManager ()=0
virtual ILTFontManager * GetFontManager ()=0
virtual ILTCursor * Cursor ()=0
virtual ILTDirectMusicMgr * GetDirectMusicMgr ()=0
virtual ILTBenchmarkMgr * GetBenchmarkMgr ()=0
virtual LTRESULT GetPointShade (const LTVector *pPoint, LTVector *pColor)=0

```

## Data Fields

```

FileEntry (* GetFileList )(const char *pDirName)
void(* FreeFileList )(FileEntry *pHead)
LTRESULT(* WriteConfigFileFields )(const char *pFilename, uint32 nNumValues, const char **pValues)
LTRESULT(* WriteConfigFile )(const char *pFilename)
LTRESULT(* ReadConfigFile )(const char *pFilename)
void(* GetAxisOffsets )(LTFLOAT *offsets)
void(* PlayJoystickEffect )(const char *pEffectName, float x, float y)
bool(* InitMusic )(const char *szMusicDLL)
bool(* LoadSong )(const char *szSong)

```

```

void(* DestroyAllSongs )()
bool(* PlayList )(const char *szPlayList, const char *szTransition, bool bLoop, uint32 dwBoundaryFlags)
void(* StopMusic )(uint32 dwBoundaryFlags)
bool(* PauseMusic )()
bool(* ResumeMusic )()
bool(* AddSongToPlayList )(const char *szPlayList, const char *szSong)
void(* DeletePlayList )(const char *szPlayList)
short(* GetMusicVolume )()
void(* SetMusicVolume )(short wVolume)
LTRESULT(* GetSoundTimer )(HLTSOUND hSound, LTFLOAT *fTimer)
LTRESULT(* GetSoundData )(HLTSOUND hSound, int16 *pSixteenBitData, int8 *pEightBitData, uint32 *dwSamplesPerSecond, uint32 *dwChannels)
LTRESULT(* GetSoundOffset )(HLTSOUND hSound, uint32 *dwOffset, uint32 *dwSize)
bool(* IsDone )(HLTSOUND pSoundHandle)
void(* GetListener )(bool *bListenerInClient, LTVector *pPos, LTRotation *pRot)
bool(* IntersectSegment )(IntersectQuery *pQuery, IntersectInfo *pInfo)
bool(* IntersectSweptSphere )(const LTVector &vStart, const LTVector &vEnd, float fRadius, LTVector &vFinalPos, LTVector &vNormal)
bool(* CastRay )(IntersectQuery *pQuery, IntersectInfo *pInfo)
void(* SetObjectPosAndRotation )(HLOCALOBJ hObj, const LTVector *pPos, const LTRotation *pRotation)
LTRESULT(* FindObjectsInSphere )(const LTVector *pCenter, float radius, HLOCALOBJ *inObjects, uint32 nInObjects, uint32 *nOutObjects, uint32 *nFound)
LTRESULT(* FindObjectsInBox )(const LTVector *pCenter, float radius, HLOCALOBJ *inObjects, uint32 nInObjects, uint32 *nOutObjects, uint32 *nFound)
LTRESULT(* GetGameMode )(int *mode)
LTRESULT(* StartGame )(StartGameRequest *pRequest)
LTRESULT(* GetLocalClientID )(uint32 *pID)
void(* Disconnect )()
bool(* IsConnected )()
void(* Shutdown )()
void(* ShutdownWithMessage )(const char *pMsg,...)

```

## Detailed Description

ILTClient interface. This is what the ClientShells and special effect plugins use to generate the interface and client objects for the game.

Define a holder to get this interface like this:

```
define\_holder(ILTClient, your_var);
```

## Member Enumeration Documentation

anonymous enum

**Enumeration values:**

[\\_ILTClient\\_VERSION\\_](#)

## Member Function Documentation

```
virtual LTRESULT ILTClient::AddGlowRenderStyleMapping( const char * pszSource,
                                                    const char * pszMapTo
                                                    ) [pure virtual]
```

**Parameters:**

*pszSource* The render style to map

*pszMapTo* The render style to map the above render style to

**Returns:**

LT\_OK if both render styles are found, or LT\_FAIL otherwise

When doing the glow rendering this will cause the specified render style to map to the other when doing the glow effect

Used for: Glow rendering

```
virtual LTParticle* ILTClient::AddParticle( HLOCALOBJ hObj,
                                           const LTVector *pPos,
                                           const LTVector *pVelocity,
                                           const LTVector *pColor,
                                           float lifeTime
                                           ) [pure virtual]
```

**Parameters:**

*hObj* Particle system to which this particle will be added.

*pPos* Position of particle (in local obj coordinates).

*pVelocity* Velocity vector.

*pColor* Color (0-255).

*lifeTime* Lifetime.

**Returns:**

If successful, returns a pointer to the particle added.

LTNULL - *hObj* is invalid (NULL, or not an OT\_PARTICLESYSTEM).

Adds a particle to a particle system.

Used for: Special FX.

```
virtual bool ILTClient::AddPixelShader( const char * pFileName,
                                       int          PixelShaderID,
                                       bool          bCompileShader = false
                                     ) [pure virtual]
```

**Parameters:**

*pFileName* Pixel shader file name.  
*PixelShaderID* Id to assign to the loaded pixel shader. The Id values -52, -51 and -50 are reserved.  
*bCompileShader* If *bCompileShader* is true, then the file is assumed to be a shader text file, and the shader will be compiled by the renderer. If *bCompileShader* is false, then the shader file is assumed to be a compiled binary.

**Returns:**

**true** is the shader was loaded and false otherwise.

Load a pixel shader from file. You can either pass in a shader text file or a compiled binary shader file by using the *bCompileShader* parameter.

Used for: Models.

```
virtual bool ILTClient::AddVertexShader( const char * pFileName,
                                       int          VertexShaderID,
                                       const uint32 * pVertexElements,
                                       uint32       VertexElementsSize,
                                       bool          bCompileShader = false
                                     ) [pure virtual]
```

**Parameters:**

*pFileName* Vertex shader file name.  
*VertexShaderID* Id to assign to the loaded vertex shader.  
*pVertexElements* An array of D3DVERTEXELEMENT9 vertex elements used to create the vertex shader declaration.  
*VertexElementsSize* Size in bytes of the vertex elements array.  
*bCompileShader* If *bCompileShader* is true, then the file is assumed to be a shader text file, and the shader will be compiled by the renderer. If *bCompileShader* is false, then the shader file is assumed to be a compiled binary.

**Returns:**

**true** is the shader was loaded and false otherwise.

Load a vertex shader from file. You can either pass in a shader text file or a compiled binary shader file by using the *bCompileShader* parameter.

Used for: Models.

```
virtual ILTCursor* ILTClient::Cursor( ) [pure virtual]
```

**Returns:**

Pointer to the [ILTCursor](#) interface.

Accesses the [ILTCursor](#) interface.

Used for: Interface.

```
virtual ILTBenchmarkMgr* ILTClient::GetBenchmarkMgr( ) [pure virtual]
```

**Returns:**

Pointer to the [ILTBenchmarkMgr](#) interface.

Accesses the [ILTBenchmarkMgr](#) interface.

Used for: Interface.

```
virtual LTRESULT ILTClient::GetCanvasFn( HOBJECT      hCanvas,
                                       CanvasDrawFn & fn,
                                       void *&      pUserData
                                     ) [pure virtual]
```

**Parameters:**

*hCanvas* Canvas to query.  
*fn* (return) Associated function.  
*pUserData* (return) Associated user data.

**Returns:**

**LT\_INVALIDPARAMS** - *hCanvas* is invalid (NULL, or not an **OT\_CANVAS**).

**LT\_OK** - Successful.

Get the function and data associated with canvas drawing.

**See also:**

[SetCanvasFn\(\)](#)

Used for: Special FX.

```
virtual LTRESULT ILTClient::GetCanvasRadius( HOBJECT hCanvas,
                                           float & radius
                                         ) [pure virtual]
```

**Parameters:**

*hCanvas* Canvas to query.  
*radius* (return) Radius of canvas.

**Returns:**

**LT\_INVALIDPARAMS** - *hCanvas* is invalid (NULL, or not an **OT\_CANVAS**).

**LT\_OK** - Successful.

Get the radius of the canvas. This is used for visibility.

**See also:**

[SetCanvasRadius\(\)](#)

Used for: Special FX.

```
virtual LTRESULT ILTClient::GetDeviceObjectName( char const * pszDeviceName,
                                                uint32      nDeviceObjectId,
                                                char *      pszDeviceObjectName,
                                                uint32      nDeviceObjectNameLen
                                              ) [pure virtual]
```

**Parameters:**

*pszDeviceName* Device name to access.  
*nDeviceObjectId* Object Id returned in [DeviceBinding](#) or [DeviceObject](#).  
*pszDeviceObjectName* Name to be filled in by function.  
*nDeviceObjectNameLen* Max length in chars of pszDeviceObjectName.

**Returns:**

**LT\_OK** if successful.

Get the name of the device object by id.

Used for: Input.

```
virtual ILTDirectMusicMgr* ILTClient::GetDirectMusicMgr( ) [pure virtual]
```

**Returns:**

Pointer to the [ILTDirectMusicMgr](#) interface.

Accesses the [ILTDirectMusicMgr](#) interface.

Used for: Interface.

```
virtual ILTDrawPrim* ILTClient::GetDrawPrim( ) [pure virtual]
```

Access to the [ILTDrawPrim](#) interface.

**Returns:**

The [ILTDrawPrim](#) interface.

Used for: Client Management.

```
virtual ILTFontManager* ILTClient::GetFontManager( ) [pure virtual]
```

Access to the [ILTFontManager](#) interface.

**Returns:**

The [ILTFontManager](#) interface.

Used for: Text and UI.

```
virtual LTRESULT ILTClient::GetGlobalLightColor( LTVector & color ) [pure virtual]
```

**Parameters:**

*color* [Light](#) color.

**Returns:**

Always returns **LT\_OK**

Get global light color.

**See also:**

[SetGlobalLightColor\(\)](#)

Used for: Misc.

```
virtual LTRESULT ILTClient::GetGlobalLightConvertToAmbient( float & fConvertToAmbient ) [pure virtual]
```

**Parameters:**

*color* Amount of ambient light to contribute to objects in the global light

**Returns:**

Always returns **LT\_OK**

Get the amount of global light that will be converted to ambient lighting

**See also:**

[SetGlobalLightAmbientColor\(\)](#)

Used for: Misc.

```
virtual LTRESULT ILTClient::GetGlobalLightDir( LTVector & dir ) [pure virtual]
```

**Parameters:**

*dir* Direction vector.

**Returns:**

Always returns **LT\_OK**

Get global light direction.

**See also:**

[SetGlobalLightDir\(\)](#)

[SetGlobalLightDir\(\)](#)

Used for: Misc.

```
virtual LTRESULT ILTClient::GetLightGroupColor( uint32    nID,
                                              LTVector * pColor
                                              ) const [pure virtual]
```

**Parameters:**

*nID* ID of the lightgroup.

*pColor* Return value destination for the lightgroup's color.

**Returns:**

**LT\_NOTFOUND** - The provided lightgroup was not found.

**LT\_NOTINWORLD** - No world information is available.

**LT\_OK** - Successful.

Get the current color of a lightgroup.

Used for: Lightgroup control

```
virtual LTRESULT ILTClient::GetMaxRadiusInPoly( const HPOLY    hPoly,
                                              const LTVector & vPos,
                                              float &      fMaxRadius
                                              ) [pure virtual]
```

**Parameters:**

*hPoly* Polygon to find maximum extents of.

*vPoint* Center point of sphere to be found constrained to polygon

*fRadius* (return) Maximum radius that can be used from the specified point

Given a polygon and a point, it will find the largest possible radius that can be used and still be contained within the polygon. This is useful for bullet holes, blood splats, etc

Used for: Misc.

```
virtual LTRESULT ILTClient::GetObjectPos( HLOCALOBJ hObj,
                                         LTVector * pPos
                                         ) [pure virtual]
```

Gets the position of the specified object

Used for: Object.

Implements [ILTCSBase](#).

```
virtual LTRESULT ILTClient::GetObjectRotation( HLOCALOBJ hObj,
                                              LTRotation * pRotation
                                              ) [pure virtual]
```

Gets the rotation of the specified object

Used for: Object.

Implements [ILTCSBase](#).

```
virtual LTRESULT ILTClient::GetOccluderEnabled( uint32    nID,
                                              bool *      pEnabled
                                              ) const [pure virtual]
```

**Parameters:**

*nID* ID of the occluder.

*pEnabled* Return value destination for the occluder's state.

**Returns:**

**LT\_NOTFOUND** - The provided occluder was not found.

**LT\_NOTINWORLD** - No world information is available.

**LT\_OK** - Successful.

Get the current state of an occluder.

Used for: Occluder control

```
virtual LTPixelShader* ILTClient::GetPixelShader( int PixelShaderID ) [pure virtual]
```

**Parameters:**

*PixelShaderID* Id of the loaded pixel shader.

**Returns:**

**pointer** to a pixel shader or NULL.

Get access to the loaded pixel shader with the given id.

Used for: Models.

```
virtual LTRESULT ILTClient::GetPointShade( const LTVector * pPoint,
                                           LTVector * pColor
                                           ) [pure virtual]
```

**Parameters:**

*pPoint* The point in world coordinates.

*pColor* (return) The color value at pPoint.

**Returns:**

**LT\_NOTINWORLD** if the point is outside the world.

Calculate the lighting shade (RGB, 0-255) at the point you specify.

Used for: Misc.

```
virtual LTRESULT ILTClient::GetRendererStats( LTRendererStats & refStats ) [pure virtual]
```

**Parameters:**

*refStats* Struct that gets filled with the renderers stats

**Returns:**

**LT\_ERROR** if called before the renderer is not initialized

Obtains a current list of stats on the renderer such as poly counts

Used for: Renderer Stats

```
virtual LTRESULT ILTClient::GetSConValueFloat( const char * pName,
                                               float & val
                                               ) [pure virtual]
```

**Parameters:**

*pName* Name of console variable.

*val* (return) Console variable value.

**Returns:**

**LT\_NOTFOUND** if the variable isn't found.

Get server console variable float value.

Used for: Misc.

```
virtual LTRESULT ILTClient::GetSConValueString( const char * pName,
                                                char * valBuf,
                                                uint32 bufLen
                                                ) [pure virtual]
```

**Parameters:**

*pName* Name of console variable.

*valBuf* (return) Buffer to hold console variable string.

*bufLen* Length of buffer.

**Returns:**

**LT\_NOTFOUND** if the variable isn't found.

Get server console variable string value.

Used for: Misc.

```
virtual LTRESULT ILTClient::GetServerIPAddress( uint8 pAddr[4],
                                                uint16 * pPort
                                                ) [pure virtual]
```

**Parameters:**

*pAddr* Buffer to receive the address of the server

*pPort* uint16 to receive the port of the server

**Returns:**

if one of the parameters is invalid, **LT\_NOTCONNECTED** if not connected to a server; otherwise, returns **LT\_OK**

Used for: Networking.

```
virtual LTRESULT ILTClient::GetSpriteControl( HLOCALOBJ hObj,
                                              LTSpriteControl *& pControl
                                              ) [pure virtual]
```

**Parameters:**

*hObj* Sprite to query.

*pControl* (return) Control interface.

**Returns:**

**LT\_INVALIDPARAMS** - *hObj* is invalid (NULL, or not an **OT\_SPRITE**).

**LT\_OK** - Successful.

Get the sprite control interface for a sprite instance.

Used for: Special FX.

virtual [ILTTexInterface](#)\* ILTClient::GetTexInterface( ) [pure virtual]

Access to the [ILTTexInterface](#) interface.

**Returns:**

The [ILTTexInterface](#) interface.

Used for: Client Management.

virtual [LTVertexShader](#)\* ILTClient::GetVertexShader( int *VertexShaderID* ) [pure virtual]

**Parameters:**

*VertexShaderID* Id to assign to the loaded vertex shader.

**Returns:**

**pointer** to a vertex shader or NULL.

Get access to the loaded vertex shader with the given id.

Used for: Models.

virtual [ILTWidgetManager](#)\* ILTClient::GetWidgetManager( ) [pure virtual]

Access to the [ILTWidgetManager](#) interface.

**Returns:**

The [ILTWidgetManager](#) interface.

Used for: Text and UI.

virtual [LTRESULT](#) ILTClient::IsLocalToServer( bool \* *bLocal* ) [pure virtual]

**Parameters:**

*bLocal* Results of the query

**Returns:**

LT\_OK on success

Returns whether or not the client is local to the server

Used for: Misc.

virtual [LTRESULT](#) ILTClient::ProcessAttachments( [HOBJECT](#) *hObj* ) [pure virtual]

**Parameters:**

*hObj* Object to process (parent object of attachments).

**Returns:**

**LT\_INVALIDPARAMS** if the object handle is invalid or NULL **LT\_OK** otherwise

Update the position and rotation of the attachments on the object. Attachments are always automatically updated when the object is rendered.

Used for: Object.

virtual void ILTClient::RemoveAllPixelShaders( ) [pure virtual]

Remove all pixel shaders.

Used for: Models.

virtual void ILTClient::RemoveAllVertexShaders( ) [pure virtual]

Remove all vertex shaders.

Used for: Models.

virtual void ILTClient::RemovePixelShader( int *PixelShaderID* ) [pure virtual]

**Parameters:**

*PixelShaderID* Id of the loaded pixel shader.

**Returns:**

**true** is the shader was removed and false otherwise.

Remove the pixel shader with the given id.

Used for: Models.

virtual void ILTClient::RemoveVertexShader( int *VertexShaderID* ) [pure virtual]

**Parameters:**

*VertexShaderID* Id to assign to the loaded vertex shader.

**Returns:**

**true** is the shader was removed and false otherwise.

Remove the vertex shader with the given id.

Used for: Models.

```
virtual LTRESULT ILTClient::SendToServer( ILTMessage\_Read * pMsg,
                                         uint32      flags
                                         ) [pure virtual]
```

New message functions. These functions don't free the message, so call LMessage::Release() after sending.

**Parameters:**

*pMsg* **Message** to send to the server  
*flags* **MESSAGE\_** Flags defined in [ltcodes.h](#)

**Returns:**

**LT\_INVALIDPARAMS** if the message is invalid, or the server shell is invalid; otherwise, returns **LT\_OK**.

Send a message to the server shell. The server shell receives the message through the OnMessage() function.

**See also:**

[ILTCommon::CreateMessage\(\)](#)

OnMessage()

Used for: Messaging.

```
virtual LTRESULT ILTClient::SetAllObjectRenderGroupEnabled( ) [pure virtual]
```

**Parameters:**

*bVisible* Whether or not it is visible

**Returns:**

**LT\_OK**

Sets all object rendering groups to the specified visible status (with the exception of the default group which is always visible)

Used for: Object Rendering Groups

```
virtual LTRESULT ILTClient::SetCanvasFn( HOBJECT      hCanvas,
                                         CanvasDrawFn fn,
                                         void *      pUserData
                                         ) [pure virtual]
```

**Parameters:**

*hCanvas* Canvas to modify.  
*fn* Associated function.  
*pUserData* Associated user data.

**Returns:**

**LT\_INVALIDPARAMS** - *hCanvas* is invalid (NULL, or not an **OT\_CANVAS**).

**LT\_OK** - Successful.

Set the function associated with canvas drawing. This function gets called once during the rendering loop. During the callback you may draw polygon fans with the DrawPrimitive() call.

**See also:**

[GetCanvasFn\(\)](#)

Used for: Special FX.

```
virtual LTRESULT ILTClient::SetCanvasRadius( HOBJECT hCanvas,
                                           float    radius
                                           ) [pure virtual]
```

**Parameters:**

*hCanvas* Canvas to modify.  
*radius* Radius of the canvas.

**Returns:**

**LT\_INVALIDPARAMS** - *hCanvas* is invalid (NULL, or not an **OT\_CANVAS**).

**LT\_OK** - Successful.

Set the radius of the canvas. This is used for visibility.

**See also:**

[GetCanvasRadius\(\)](#)

Used for: Special FX.

```
virtual LTRESULT ILTClient::SetGlobalLightColor( const LTVector & color ) [pure virtual]
```

**Parameters:**

*color* [Light](#) color.

**Returns:**

Always returns **LT\_OK**

Set global light color.

**See also:**

[GetGlobalLightColor\(\)](#)

Used for: Misc.

virtual [LTRESULT](#) ILTClient::SetGlobalLightConvertToAmbient( float *fConvertToAmbient* ) [pure virtual]

**Parameters:**

*color* Amount of ambient light to contribute to objects in the global light

**Returns:**

Always returns **LT\_OK**

Sets the amount of light that will be converted to ambient lighting, so for example 0.2 will convert 20% of the directional lighting on the model to ambient. This prevent such harsh shadows on areas that go out of the light

**See also:**

[GetGlobalLightAmbientColor\(\)](#)

Used for: Misc.

virtual [LTRESULT](#) ILTClient::SetGlobalLightDir( const [LTVector](#) & *dir* ) [pure virtual]

**Parameters:**

*dir* Direction vector.

**Returns:**

Always returns **LT\_OK**

Set global light direction.

**See also:**

[GetGlobalLightDir\(\)](#)

[GetGlobalLightDir\(\)](#)

Used for: Misc.

virtual [LTRESULT](#) ILTClient::SetGlowDefaultRenderStyle( const char \* *pszFilename* ) [pure virtual]

**Parameters:**

*pszFilename* The render style to default to if no mapping is found

**Returns:**

**LT\_OK** if the render style is found, or **LT\_FAIL** otherwise

When doing the glow rendering this will set all unmapped render styles to this render style

Used for: Glow rendering

virtual [LTRESULT](#) ILTClient::SetLightGroupColor( [uint32](#) *nID*,  
const [LTVector](#) & *vColor*  
) [pure virtual]

**Parameters:**

*nID* ID of the lightgroup.

*vColor* Desired color for the lightgroup.

**Returns:**

**LT\_NOTFOUND** - The provided lightgroup was not found.

**LT\_NOTINWORLD** - No world information is available.

**LT\_ERROR** - Unable to complete operation.

**LT\_OK** - Successful.

Set the current color of a lightgroup. Setting the color to (0,0,0) will turn off the lightgroup.

**Note:** When **LT\_ERROR** is returned, the current lightgroup color may not match the rendered light color

Used for: Lightgroup control

virtual [LTRESULT](#) ILTClient::SetNoGlowRenderStyle( const char \* *pszFilename* ) [pure virtual]

**Parameters:**

*pszFilename* The render style to use if no glow is intended on the object is found

**Returns:**

**LT\_OK** if the render style is found, or **LT\_FAIL** otherwise

When doing the glow rendering this will set all non glowing objects to this render style

Used for: Glow rendering

```
virtual LTRESULT ILTClient::SetObjectPos( HLOCALOBJ hObj,
                                         const LTVector * pPos,
                                         bool bForce = true
                                         ) [pure virtual]
```

```
virtual LTRESULT ILTClient::SetObjectRenderGroup( HOBJECT hObj,
                                                  uint32 nGroup
                                                  ) [pure virtual]
```

**Parameters:**

*hObject* Object that you want to change the group of  
*nGroup* The group you want the object to belong to

**Returns:**

LT\_INVALIDPARAMS if the object is invalid or the group out of range

Set the render group that the object belongs to. These groups can then be turned on or off, making all objects that belong to them turn on and off.

Used for: Object Rendering Groups

```
virtual LTRESULT ILTClient::SetObjectRenderGroupEnabled( uint32 nGroup,
                                                         bool bEnabled
                                                         ) [pure virtual]
```

**Parameters:**

*nGroup* The group that you want to turn on or off  
*bVisible* Whether or not it is visible

**Returns:**

LT\_INVALIDPARAMS if the group is out of range

Causes all objects of the specified group to either become visible or invisible (on top of their existing visible status)

Used for: Object Rendering Groups

```
virtual LTRESULT ILTClient::SetOccluderEnabled( uint32 nID,
                                                 bool bEnabled
                                                 ) [pure virtual]
```

**Parameters:**

*nID* ID of the occluder.  
*bEnabled* Desired state for the occluder.

**Returns:**

LT\_NOTFOUND - The provided occluder was not found.

LT\_NOTINWORLD - No world information is available.

LT\_ERROR - Unable to complete operation.

LT\_OK - Successful.

Set the current status of an occluder.

Used for: Occluder control

```
virtual LTRESULT ILTClient::SetSpriteEffectShaderID( HLOCALOBJ hObj,
                                                     uint32 EffectShaderID
                                                     ) [pure virtual]
```

**Parameters:**

*hObj* Sprite to modify.  
*EffectID* Effect shader ID.

**Returns:**

LT\_INVALIDPARAMS - *hObj* is invalid (NULL, or not an OT\_SPRITE).

LT\_OK - Successful.

Set the Effect Shader ID.

Used for: Special FX.

```
virtual LTRESULT ILTClient::SetTextureEffectVar( uint32 nID,
                                                  uint32 nVar,
                                                  float fVal
                                                  ) [pure virtual]
```

**Parameters:**

*nID* ID of the texture effect stage.  
*nVar* The variable that is currently being set.  
*fVal* The value to set the variable to

**Returns:**

LT\_NOTINWORLD - No world information is available.

LT\_ERROR - Unable to complete operation.

**LT\_OK** - Successful.

Set the specified variable of the specified texture effect. This can be used so that gamecode can control how some effects are updated or evaluated.

Used for: Texture Effect Variable manipulation

virtual [ILTVideoMgr\\*](#) ILTClient::VideoMgr( ) [pure virtual]

**Returns:**

Pointer to the [ILTVideoMgr](#) interface.

Accesses the [ILTVideoMgr](#) interface.

Used for: Interface.

## Field Documentation

bool(\* [ILTClient::AddBinding](#))(const char \*pDeviceName, const char \*pTriggerName, const char \*pActionName, float rangeLow, float rangeHigh)

**Parameters:**

*pDeviceName* The name of the input device.  
*pTriggerName* The name of the trigger to clear (device object name)  
*pActionName* The name of action to bind to the trigger.  
*rangeLow* The low range if used (zero if not used)  
*rangeHigh* The high range if used (zero if not used)

Add a binding for a device (set ranges to 0 to not use ranges). Note that this does not clear any existing binding.

**See also:**

[ClearBinding\(\)](#)

Used for: Input.

[LTRESULT](#)(\* [ILTClient::AddInternetDriver](#))()

[HLTLINE](#)(\* [ILTClient::AddLine](#))([HLOCALOBJ](#) hObj, [LTLine](#) \*pLine)

**Parameters:**

*hObj* Line system to modify.  
*pLine* Description of line to add.

**Returns:**

If successful, returns a handle to the line added.

**LTNULL** - *hObj* is invalid (NULL, or not an **OT\_LINESYSTEM**).

Add a line to the end of the list of the line system.

Used for: Special FX.

void(\* [ILTClient::AddParticles](#))([HLOCALOBJ](#) hObj, [uint32](#) nParticles, const [LTVector](#) \*pMinOffset, const [LTVector](#) \*pMaxOffset, const [LTVector](#) \*pMinVelocity, c

**Parameters:**

*hObj* Particle system to which these particles will be added.  
*nParticles* Number of particles to add.  
*pMinOffset* Minimum of position range (in local obj coordinates).  
*pMaxOffset* Maximum of position range (in local obj coordinates).  
*pMinVelocity* Minimum of velocity range.  
*pMaxVelocity* Maximum of velocity range.  
*pMinColor* Minimum of color range (0-255).  
*pMaxColor* Maximum of color range (0-255).  
*minLifeTime* Minimum of lifetime range.  
*maxLifeTime* Maximum of lifetime range.

Add several particles to a particle system. LithTech will randomize between the ranges given. This is more efficient than calling [AddParticle\(\)](#), if only because of the function pointer overhead with [AddParticle\(\)](#).

Used for: Special FX.

bool(\* [ILTClient::AddSongToPlayList](#))(const char \*szPlayList, const char \*szSong)

**Parameters:**

*szPlayList* Name of playlist.  
*szSong* Name of song to add to playlist.

**Returns:**

**FALSE** if the redbook audio manager was not properly initialized **FALSE** if the provided playlist could not be found, or the song could not be added **TRUE** otherwise

Create and add to playlists.

This function is now only used if you are playing red book audio. If you are using DirectMusic use the functions in the ILTDDirectMusic interface.

Used for: Music.

bool(\* [ILTClient::CastRay](#))([IntersectQuery](#) \*pQuery, [IntersectInfo](#) \*pInfo)

bool(\* [ILTClient::ClearBinding](#))(const char \*pDeviceName, const char \*pTriggerName)

**Parameters:**

*pDeviceName* The name of the input device.

*pTriggerName* The name of the trigger to clear (device object name)

Clear a binding for a device.

**See also:**

[AddBinding\(\)](#)

Used for: Input.

[LTRESULT](#)(\* [ILTClient::ClearInput](#)())

Clear all the keyboard, command, and axis offset input.

Used for: Input.

[LTRESULT](#)(\* [ILTClient::ClearScreen](#))([LTRect](#) \*pClearRect, [uint32](#) flags, [LTRGB](#) \*pClearColor)

**Parameters:**

*pClearRect* The rectangle area to clear. **NULL** means the entire screen.

*flags* Flags to use when performing the clear screen (a combination of **CLEARSCREEN\_** flags in [ltcodes.h](#)).

*pClearColor* Color to clear the screen to. Passing **NULL** results in black.

**Returns:**

**LT\_OK** or **LT\_NOTINITIALIZED**.

Clear the Z-Buffer and Backbuffer. In most cases you will want to clear the screen before you enter your [Start3D\(\)](#)/[End3D\(\)](#) block.

Used for: Rendering.

[LTRESULT](#)(\* [ILTClient::ClipSprite](#))([HLOCALOBJ](#) hObj, [HPOLY](#) hPoly)

**Parameters:**

*hObj* Sprite to modify.

*hPoly* Polygon to clip to. Use **INVALID\_POLY** to un-clip the sprite.

**Returns:**

**LT\_ERROR** - *hObj* is invalid (**NULL**, or not an **OT\_SPRITE**).

**LT\_OK** - Successful.

Clips the sprite onto a polygon.

Used for: Special FX.

[HLTCOLOR](#)(\* [ILTClient::CreateColor](#))(float r, float g, float b, bool bTransparent)

**Parameters:**

*r* Red component (0.0 to 1.0).

*g* Green component (0.0 to 1.0).

*b* Blue component (0.0 to 1.0).

*bTransparent* Transparency on/off. Only used when the function specifies so.

**Returns:**

Handle to the created color.

Creates a color. You should free up this color with [DeleteColor\(\)](#) when you're done.

**Note:** Anywhere you can pass a color, if you pass in **NULL**, it will use black.

Used for: 2D Rendering.

[HLTFONT](#)(\* [ILTClient::CreateFont](#))(const char \*pFontName, int width, int height, bool bItalic, bool bUnderline, bool bBold)

**Parameters:**

*pFontName* Requested font name.

*width* Requested font width.

*height* Requested font height.

*bItalic* Request italicized font?

*bUnderline* Request underlined font?

*bBold* Request bolded font?

**Returns:**

Handle to the created font (**HLTFONT**).

Create the closest font possible to the one asked for.

Used for: 2D Rendering.

[LTRESULT](#)(\* [ILTClient::CreateHeightmapFromBitmap](#))(const char \*pBitmapName, [uint32](#) \*pWidth, [uint32](#) \*pHeight, [uint8](#) \*\*ppData)

**Parameters:**

*pBitmapName* Filename of bitmap.

**Returns:**

**LT\_INVALIDPARAMS** when the parameters are **NULL**, or **LT\_ERROR** when the file cannot be found otherwise, returns **LT\_OK**.

Loads an 8-bit PCX surface and returns an 8 bit heightmap

Used for: 2D Rendering.

[HLOCALOBJ](#)(\* [LTClient::CreateObject](#))([ObjectCreateStruct](#) \*pStruct)

**Parameters:**

*pStruct* Description of object.

**Returns:**

a valid local client object handle on successful creation and addition of the LTOject to the local client world, **LTNULL** on failure

Create a client-side object.

Used for: Object.

[HSURFACE](#)(\* [LTClient::CreateSurface](#))([uint32](#) width, [uint32](#) height)

**Parameters:**

*width* Width of new surface.

*height* Height of new surface.

**Returns:**

Handle to the newly-created surface.

Create a blank surface.

Used for: 2D Rendering.

[HSURFACE](#)(\* [LTClient::CreateSurfaceFromBitmap](#))(char \*pBitmapName)

**Parameters:**

*pBitmapName* Filename of bitmap.

**Returns:**

Handle to the newly-created surface.

Create a surface, sized to the dimensions of the bitmap. The bitmap is an 8-bit PCX file. (You can use any palette you want.)

Used for: 2D Rendering.

[HSURFACE](#)(\* [LTClient::CreateSurfaceFromString](#))([HLTFONT](#) hFont, [HSTRING](#) hString, [HLTCOLOR](#) hForeColor, [HLTCOLOR](#) hBackColor, int extraPixelsX, int extraPixelsY)

**Parameters:**

*hFont* Font to use.

*hString* Source string.

*hForeColor* Color of text in surface.

*hBackColor* Color of background in surface.

*extraPixelsX* Left/right border thickness.

*extraPixelsY* Top/bottom border thickness.

**Returns:**

Handle to the newly-created surface.

Create a surface just large enough for the string. You can make the surface a little larger with extraPixelsX and extraPixelsY.

Used for: 2D Rendering.

void(\* [LTClient::DebugOut](#))(const char \*pMsg,...)

**Parameters:**

*pMsg* The printf-styled format string to display. This parameter can be followed by the usual format parameters.

Used to output a TRACE message to the Debug Output window. Newlines must be explicitly used.

Used for: Misc.

void(\* [LTClient::DeleteColor](#))([HLTCOLOR](#) hColor)

**Parameters:**

*hColor* Handle to the color to delete.

Delete a previously created **HLTCOLOR**. This function must be called to avoid memory leaks.

**Note:** Alternatively, you can use the SetupColor1 and SetupColor2 and not worry about deletion.

Used for: 2D Rendering.

void(\* [LTClient::DeleteFont](#))([HLTFONT](#) hFont)

**Parameters:**

*hFont* Handle of font to delete.

Delete a previously created font. This must be called to avoid memory leaks.

Used for: 2D Rendering.

void(\* [ILTClient::DeletePlayList](#))(const char \*szPlayList)

**Parameters:**

*szPlayList* Name of playlist.

Remove a whole playlist.

This function is now only used if you are playing red book audio. If you are using DirectMusic use the functions in the ILTDirectMusic interface.

Used for: Music.

[LTRESULT](#)(\* [ILTClient::DeleteSurface](#))([HSURFACE](#) hSurface)

**Parameters:**

*hSurface* Handle of surface to delete.

**Returns:**

**LT\_ERROR** when either the surface or the global render struct is **NULL**; otherwise, returns **LT\_OK**.

Delete a surface.

Used for: 2D Rendering.

void(\* [ILTClient::DestroyAllSongs](#))()

Remove all songs from memory.

This function is now only used if you are playing red book audio. If you are using DirectMusic use the functions in the ILTDirectMusic interface.

Used for: Music.

bool(\* [ILTClient::DrawBitmapToSurface](#))([HSURFACE](#) hDest, const char \*pSourceBitmapName, const [LRect](#) \*pSrcRect, int destX, int destY)

**Parameters:**

*hDest* Destination surface.

*pSourceBitmapName* Filename of bitmap to draw.

*pSrcRect* Area from bitmap to draw (in bitmap coordinates). A value of **NULL** means to draw the whole thing.

*destX* Destination x position (in **hDest** coordinates).

*destY* Destination y position (in **hDest** coordinates).

**Returns:**

**false** when the surface is **NULL** or the bitmap file cannot be loaded; otherwise, returns **true**.

Draw a bitmap (or part of one) to a surface. Opaque.

Used for: 2D Rendering.

void(\* [ILTClient::DrawStringToSurface](#))([HSURFACE](#) hDest, [HLTFont](#) hFont, [HSTRING](#) hString, [LRect](#) \*pRect, [HLTCOLOR](#) hForeColor, [HLTCOLOR](#) hBackColor)

**Parameters:**

*hDest* Destination surface.

*hFont* Font to use.

*hString* String to draw.

*pRect* Location to draw the string to (in **hDest** coordinates).

*hForeColor* Text color.

*hBackColor* Background color.

Opaquely draw the string into the rectangle.

Used for: 2D Rendering.

[LTRESULT](#)(\* [ILTClient::DrawSurfaceMasked](#))([HSURFACE](#) hDest, [HSURFACE](#) hSrc, [HSURFACE](#) hMask, [LRect](#) \*pSrcRect, int destX, int destY, [HLTCOLOR](#) hC

**Parameters:**

*hDest* Destination surface.

*hSrc* Source surface.

*hMask* Mask surface.

*pSrcRect* Area from **hSrc** to draw (in **hSrc** coordinates). **NULL** = whole thing.

*destX* Destination x position (in **hDest** coordinates).

*destY* Destination y position (in **hDest** coordinates).

*hColor* Transparency color.

**Returns:**

**LT\_NOTINITIALIZED** when there is no render struct, or **LT\_ERROR** for general failure (problems with the surfaces, etc.); otherwise, returns **LT\_OK**.

Draws **hSrc** to **hDest** as it would a normal transparent blit, but tiles the surface of **hMask** into the nontransparent pixels of **hSrc** (airbrushed text). You can't have a mask texture larger than 256x256, and the mask size must be a power of 2.

Used for: 2D Rendering.

[LTRESULT](#)(\* [ILTClient::DrawSurfaceSolidColor](#))([HSURFACE](#) hDest, [HSURFACE](#) hSrc, [LRect](#) \*pSrcRect, int destX, int destY, [HLTCOLOR](#) hTransColor, [HLTC](#)

**Parameters:**

*hDest* Destination surface.

*hSrc* Source surface.  
*pSrcRect* Area from **hSrc** to draw (in **hSrc** coordinates). NULL means the whole thing.  
*destX* Destination x position (in **hDest** coordinates).  
*destY* Destination y position (in **hDest** coordinates).  
*hTransparentColor* Transparency color.  
*hFillColor* Fill color.

**Returns:**

**LT\_NOTINITIALIZED** when there is no render struct, or **LT\_ERROR** for general failure (problems with the surfaces, etc.); otherwise, returns **LT\_OK**.

Draws **hSrc** onto **hDest** using a solid color mask (i.e. fills in the non-transparent source pixels with the color you specify).

Used for: 2D Rendering.

[LTRESULT\(\\* LTClient::DrawSurfaceToSurface\)](#)([HSURFACE](#) hDest, [HSURFACE](#) hSrc, [LTRect](#) \*pSrcRect, int destX, int destY)

**Parameters:**

*hDest* Destination surface.  
*hSrc* Source surface.  
*pSrcRect* Area from **hSrc** to draw (in **hSrc** coordinates). NULL means the whole thing.  
*destX* Destination x position (in **hDest** coordinates).  
*destY* Destination y position (in **hDest** coordinates).

**Returns:**

**LT\_NOTINITIALIZED** when there is no render struct, or **LT\_ERROR** for general failure (problems with the surfaces, etc.); otherwise, returns **LT\_OK**.

Draws the source surface (or part of one) onto the destination surface.

Used for: 2D Rendering.

[LTRESULT\(\\* LTClient::DrawSurfaceToSurfaceTransparent\)](#)([HSURFACE](#) hDest, [HSURFACE](#) hSrc, [LTRect](#) \*pSrcRect, int destX, int destY, [HLTCOLOR](#) hColor)

**Parameters:**

*hDest* Destination surface.  
*hSrc* Source surface.  
*pSrcRect* Area from **hSrc** to draw (in **hSrc** coordinates). NULL means the whole thing.  
*destX* Destination x position (in **hDest** coordinates).  
*destY* Destination y position (in **hDest** coordinates).  
*hColor* Transparency color.

**Returns:**

**LT\_NOTINITIALIZED** when there is no render struct, or **LT\_ERROR** for general failure (problems with the surfaces, etc.); otherwise, returns **LT\_OK**.

Draw the source surface onto the destination surface with transparency.

Used for: 2D Rendering.

[LTRESULT\(\\* LTClient::EnableDevice\)](#)(const char \*strDeviceName)

**Parameters:**

*strDeviceName* Device name.

**Returns:**

**LT\_OK** or **LT\_ERROR**.

Attempt to enable specified device.

**See also:**

[IsDeviceEnabled\(\)](#)

Used for: Input.

[LTRESULT\(\\* LTClient::End3D\)](#)([uint32](#) flags)

**Parameters:**

*flags* Flags to use when performing the flip (a combination of **END3D\_** flags in [ltcodes.h](#)).

**Returns:**

**LT\_OK**, **LT\_NOTINITIALIZED**, or **LT\_NOTIN3D**.

End a Start3D/End3D block.

**Note:** You must be in a [Start3D\(\)](#)/End3D() block in order to render through any cameras.

**See also:**

[Start3D\(\)](#)

Used for: Rendering.

[uint32\(\\* LTClient::EndCounter\)](#)([LTCounter](#) \*pCounter)

**Parameters:**

*pCounter* The counter that the engine will stop.

**Returns:**

The number of ticks since [StartCounter\(\)](#) was called.

Stop a counter.

**Note:** Use this to time sections of code. Timing is done in microseconds (millionths of a second).

**See also:**

[StartCounter\(\)](#)

Used for: Misc.

[LTRESULT](#)(\* [LTClient::EndDeviceTrack](#)())

**Returns:**

**LT\_ERROR** if input services were not properly initialized **LT\_OK** otherwise

Stop tracking input devices.

Used for: Input.

[LTRESULT](#)(\* [LTClient::EndOptimized2D](#)())

**Returns:**

**LT\_NOTINITIALIZED** when the render struct is not initialized, or **LT\_NOTIN3D** when this function is called outside a [Start3D\(\)](#)/[End3D\(\)](#) code block; otherwise, returns **LT\_OK**.

Ends a [StartOptimized2D\(\)](#)/[EndOptimized2D\(\)](#) block.

**Note:** You must be in a [StartOptimized2D\(\)](#)/[EndOptimized2D\(\)](#) code block to draw any optimized 2D surfaces.

**See also:**

[StartOptimized2D\(\)](#)

Used for: 2D Rendering.

[LTRESULT](#)(\* [LTClient::FillRect](#))([HSURFACE](#) hDest, [LRect](#) \*pRect, [HLCOLOR](#) hColor)

**Parameters:**

*hDest* Destination surface.

*pRect* Destination area (in **hDest** coordinates).

*hColor* Fill color.

**Returns:**

**LT\_NOTINITIALIZED** when the render struct is not initialized, **LT\_INVALIDPARAMS** when a destination surface is invalid, or **LT\_ERROR** for general failure; otherwise, returns **LT\_OK**.

Draw a filled rectangle into the surface.

Used for: 2D Rendering.

[LTRESULT](#)(\* [LTClient::FindObjectsInBox](#))(const [LTVector](#) \*pCenter, float radius, [HLOCALOBJ](#) \*inObjects, [uint32](#) nInObjects, [uint32](#) \*nOutObjects, [uint32](#) \*nFour

[LTRESULT](#)(\* [LTClient::FindObjectsInSphere](#))(const [LTVector](#) \*pCenter, float radius, [HLOCALOBJ](#) \*inObjects, [uint32](#) nInObjects, [uint32](#) \*nOutObjects, [uint32](#) \*nFour

[LTRESULT](#)(\* [LTClient::FlipScreen](#))([uint32](#) flags)

**Parameters:**

*flags* Flags to use when performing the flip (a combination of **FLIPSCREEN\_** flags in [ltcodes.h](#)).

**Returns:**

**LT\_OK** or **LT\_NOTINITIALIZED**.

Flip the screen.

Used for: Rendering.

void(\* [LTClient::FreeDeviceBindings](#))([DeviceBinding](#) \*pBindings)

**Parameters:**

*pBindings* The head of the bindings list.

Free the list of DeviceBindings returned by [GetDeviceBindings\(\)](#).

**See also:**

[GetDeviceBindings\(\)](#)

Used for: Input.

void(\* [LTClient::FreeDeviceObjects](#))([DeviceObject](#) \*pList)

**Parameters:**

*pList* Head of the [DeviceObject](#) list.

Free a [DeviceObject](#) list allocated with [GetDeviceObjects\(\)](#).

**See also:**

[GetDeviceObjects\(\)](#)

Used for: Input.

void(\* [ILTClient::FreeFileList](#))([FileEntry](#) \*pHead)

**Parameters:**

*pHead* Head of list acquired with [GetFileList\(\)](#).

Frees up memory allocated by [GetFileList\(\)](#).

**See also:**

[GetFileList\(\)](#)

Used for: Misc.

[LTRESULT](#)(\* [ILTClient::FreeHeightmap](#))([uint8](#) \*pData)

**Parameters:**

*pData* The data returned from [CreateHeightmapFromBitmap](#).

**Returns:**

**LT\_INVALIDPARAMS** when the parameters is **NULL**, or **LT\_OK**

Frees the memory returned from [CreateHeightmapFromBitmap](#)

Used for: 2D Rendering.

[LTRESULT](#)(\* [ILTClient::FreeServiceList](#))([NetService](#) \*pListHead)

**Parameters:**

*pListHead* Head of list of **NetServices** to deallocate.

**Returns:**

**LT\_OK** (always)

Deallocates the list headed by the parameter **pListHead**.

Call this function when you are finished using the list returned by [GetServiceList\(\)](#).

**See also:**

[GetServiceList\(\)](#)

Used for: Networking.

[LTRESULT](#)(\* [ILTClient::FreeSessionList](#))([NetSession](#) \*pListHead)

**Parameters:**

*pListHead* List of **NetSessions** to free.

**Returns:**

**LT\_OK** (always)

Deallocates the list headed by the parameter **pListHead**.

Call this function when you are finished using the list returned by [GetSessionList\(\)](#) or [GetQueryResults\(\)](#).

**See also:**

[GetSessionList\(\)](#)

[GetQueryResults\(\)](#)

Used for: Networking.

[LTRESULT](#)(\* [ILTClient::Get3DCameraPt](#))([HLOCALOBJ](#) hCamera, int sx, int sy, [LTVector](#) \*pOut)

**Parameters:**

*hCamera* Camera to use for calculation.

*sx* Screen x coord.

*sy* Screen y coord.

*pOut* (return) Calculated 3D result (in world coordinates).

**Returns:**

**LT\_OUTSIDE** if the screen coordinates aren't inside the rectangle of the camera.

Get the 3D coordinates of a screen coordinate given a camera. The 3D coordinate is one unit out along the forward vector.

Used for: Rendering.

void(\* [ILTClient::GetAxisOffsets](#))([LTFLOAT](#) \*offsets)

**Parameters:**

*offsets* (return) Array of offsets (size of array is **NUM\_AXIS\_OFFSETS**).

Get the offsets from the last frame of all of the currently bound axis. Most often this is mapped to the mouse.

Used for: Input.

[LTRESULT](#)(\* [ILTClient::GetBorderSize](#))([HSURFACE](#) hSurface, [HLTColor](#) hColor, [LTRect](#) \*pRect)

**Parameters:**

*hSurface* The surface to process.

*hColor* The border color.

*pRect* (return) The resulting rectangle, expressed in coordinates of the surface.

**Returns:**

**LT\_INVALIDPARAMS** when the input parameters are **NULL**, **LT\_NOTINITIALIZED** when the render struct is not initialized, or **LT\_ERROR** for general failure; otherwise, returns **LT\_OK**.

Goes around the edges of the surface and returns the smallest inside rectangle allowing for a border of **hColor**. For example, if **hSurface** had a black border of 2 pixels on the left and right and a black border of 3 pixels on the top and bottom, **pRect** would be set to (2,3,2,3).

Used for: 2D Rendering.

void(\* [ILTClient::GetCameraFOV](#))([HLOCALOBJ](#) hObj, float \*pX, float \*pY)

**Parameters:**

*hObj* Camera to query.

*pX* (return) X FOV (range:  $\frac{\pi}{100}$  to  $(99 \times \frac{\pi}{100})$  ).

*pY* (return) Y FOV (range:  $\frac{\pi}{100}$  to  $(99 \times \frac{\pi}{100})$  ).

Get the FOV of a camera. It defaults to  $(\frac{\pi}{2}, \frac{\pi}{2})$ .

**See also:**

[SetCameraFOV\(\)](#)

Used for: Rendering.

bool(\* [ILTClient::GetCameraLightAdd](#))([HLOCALOBJ](#) hCamera, [LTVector](#) \*pAdd)

**Parameters:**

*hCamera* Camera to query.

*pAdd* (return) Additive value (range: 0.0-1.0).

**Returns:**

**false** if **hCamera** is not a camera.

Get the camera light additive value.

**See also:**

[SetCameraLightAdd\(\)](#)

[SetCameraLightAdd\(\)](#)

Used for: Rendering.

void(\* [ILTClient::GetCameraRect](#))([HLOCALOBJ](#) hObj, bool \*bFullscreen, int \*left, int \*top, int \*right, int \*bottom)

**Parameters:**

*hObj* Camera to query.

*bFullscreen* (return) **TRUE** means a fullscreen camera. **FALSE** means a non-fullscreen camera.

*left* (return) Left edge of camera rectangle (in screen coordinates).

*top* (return) Top edge of camera rectangle (in screen coordinates).

*right* (return) Right edge of camera rectangle (in screen coordinates).

*bottom* (return) Bottom edge of camera rectangle (in screen coordinates).

Get the rectangle defined for this camera.

**See also:**

[SetCameraRect\(\)](#)

[SetCameraRect\(\)](#)

Used for: Rendering.

[HLOCALOBJ](#)(\* [ILTClient::GetClientObject](#)())()

**Returns:**

Handle to client object (NULL if currently doesn't exist).

Get your client object.

Used for: Misc.

[HCONSOLEVAR](#)(\* [ILTClient::GetConsoleVar](#))(const char \*pName)

**Parameters:**

*pName* Name of console variable.

**Returns:**

**NULL** if the parameter doesn't exist.

Get console variable.

Used for: Misc.

[DeviceBinding](#)(\* [ILTClient::GetDeviceBindings](#))([uint32](#) nDevice)

**Parameters:**

*nDevice* The device to poll.

**Returns:**

A list of DeviceBindings for a given device. You must call [FreeDeviceBindings\(\)](#) to free the list.

**See also:**

[FreeDeviceBindings\(\)](#)

Used for: Input.

[LTRESULT](#)(\* [ILTClient::GetDeviceName](#))([uint32](#) nDeviceType, char \*pStrBuffer, [uint32](#) nBufferSize)

**Parameters:**

*nDeviceType* Device type (**DEVICETYPE\_** defines in [ltbasedefs.h](#)).  
*pStrBuffer* (return) Buffer to hold name.  
*nBufferSize* Size of pStrBuffer.

**Returns:**

**LT\_OK** or **LT\_NOTFOUND**.

Get the name of the first input device of the given type.

Used for: Input.

[DeviceObject](#)\*([ILTClient::GetDeviceObjects](#))([uint32](#) nDeviceFlags)

**Parameters:**

*nDeviceFlags* Bit-field of device flags (**DEVICETYPE\_** defines in [ltbasedefs.h](#)).

**Returns:**

A [DeviceObject](#) list.

Retrieve a list of device objects (like axes, buttons, etc.) for one or more devices. You must free the list with [FreeDeviceObjects\(\)](#).

**See also:**

[FreeDeviceObjects\(\)](#)

Used for: Input.

[LTRESULT](#)(\* [ILTClient::GetEngineHook](#))(char \*pName, void \*\*pData)

**Parameters:**

*pName* Name of engine hook.  
*pData* User data.

**Returns:**

**LT\_OK** when it understands the string; otherwise, returns an error.

This is here so we can avoid adding API functions if necessary and for some system-dependent or misc. stuff. Pass in a string describing what you want and it fills in pData.

**Hooks:**

- **HWND** Returns main window handle.

Used for: Misc.

[FileEntry](#)\*([ILTClient::GetFileList](#))(const char \*pDirName)

**Parameters:**

*pDirName* Directory path.

**Returns:**

Non-circular, **LTNULL** terminated linked list of file/directory names.

Get a list of files/directories in a directory (pass in "" to start with). Use [FreeFileList\(\)](#) when you're done with each list you get.

**See also:**

[FreeFileList\(\)](#)

Used for: Misc.

[LTRESULT](#)(\* [ILTClient::GetFontExtraSpace](#))([HLTFONT](#) hFont, int &pixels)

**Parameters:**

*hFont* Handle of font.  
*pixels* (return) Number of pixels currently inserted.

**Returns:**

**LT\_ERROR** when the font is invalid; otherwise, returns **LT\_OK**.

Get the intercharacter spacing. Intercharacter spacing is added to each character, including break characters, when the system writes a line of text.

**See also:**

[SetFontExtraSpace\(\)](#)

Used for: 2D Rendering.

float(\* [ILTClient::GetGameFrameTime](#)())()

**Returns:**

Number of seconds since the previous frame.

Get the time since the frame was started. This accesses the GAME timer on the server. Since this value comes from the server, it will be intermittent, so only use it for things that must be synced with the server. This timer will not update when there isn't a connection to the server.

Used for: Misc.

float(\* [ILTClient::GetGameTime](#)())

**Returns:**

Number of seconds since the game was started.

Get the time since the game was started. This accesses the GAME timer on the server. Since this value comes from the server, it will be intermittent, so only use it for things that must be synced with the server. This timer will not update when there isn't a connection to the server.

Used for: Misc.

void(\* [ILTClient::GetGlobalLightScale](#))([LTVector](#) \*pScale)

**Parameters:**

*pScale* [Light](#) scale RGB value (range: clamped to 0.0-1.0).

Get the global light scale that is applied to all rendering.

**See also:**

[SetGlobalLightScale\(\)](#)

[OffsetGlobalLightScale\(\)](#)

Used for: Misc.

void(\* [ILTClient::GetLightColor](#))([HLOCALOBJ](#) hObj, float \*r, float \*g, float \*b)

**Parameters:**

*hObj* [Light](#) to query.

*r* (return) Red value (0-1).

*g* (return) Green value (0-1).

*b* (return) Blue value (0-1).

Get the color of a light. When you create a light, its color defaults to (0,0,0).

**See also:**

[SetLightColor\(\)](#)

Used for: Special FX.

float(\* [ILTClient::GetLightRadius](#))([HLOCALOBJ](#) hObj)

**Parameters:**

*hObj* [Light](#) to query.

**Returns:**

If *hObj* is an **OT\_LIGHT**, returns the radius of the light. Otherwise returns 1.0.

Get the radius of a light. When you create a light, its radius defaults to 100.

**See also:**

[SetLightRadius\(\)](#)

Used for: Special FX.

void(\* [ILTClient::GetLineInfo](#))([HLTLINE](#) hLine, [LTLine](#) \*pLine)

**Parameters:**

*hLine* Line to query.

*pLine* (return) Line info.

Get the line info from an **HLTLINE**.

**See also:**

[SetLineInfo\(\)](#)

Used for: Special FX.

void(\* [ILTClient::GetListener](#))(bool \*bListenerInClient, [LTVector](#) \*pPos, [LTRotation](#) \*pRot)

**Returns:**

Results of SetListener().

**See also:**

[SetListener\(\)](#)

Used for: Audio.

[LTRESULT](#)(\* [ILTClient::GetLobbyLaunchInfo](#))(const char \*sDriver, void \*\*ppLobbyLaunchData)

**Parameters:**

*sDriver* String containing name of driver.

*ppLobbyLaunchData* (return) Pointer to a pointer to whatever is going to contain the lobby data.

**Returns:**

**LT\_INVALIDPARAMS** if ppLobbyLaunchData is invalid, **LT\_ERROR** if it tries to add a driver and fails or the call to the driver to fill in the info fails; otherwise returns **LT\_OK**.

Get the lobby launch info if available.

Passing **NULL** for **sDriver** will cause the "dplay2" driver to be used.

**ppLobbyLaunchData** should point to the type of struct which the specified driver knows how to fill in with lobby data. In the case of DPlay2, **ppLobbyLaunchData** should point to a **DPLCONNECTION\*\*** (DPlay-defined) struct.

Used for: Networking.

short(\* [ILTClient::GetMusicVolume](#)())()

**Returns:**

Volume value (range: 0=off, 100=full).

Get music volume.

This function is now only used if you are playing red book audio. If you are using DirectMusic use the functions in the ILTDirectMusic interface.

**See also:**

[SetMusicVolume\(\)](#)

Used for: Music.

[HLTLINE](#)(\* [ILTClient::GetNextLine](#))([HLOCALOBJ](#) hObj, [HLTLINE](#) hPrev)

**Parameters:**

*hObj* Line system to query.

*hPrev* Line to reference. Use **NULL** to get first line in system.

**Returns:**

Next line in the list.

**LTNULL** - End of the list or *hObj* is invalid (**NULL**, or not an **OT\_LINESYSTEM**).

Get the next line in the line system.

**Note:** If you call [RemoveLine\(\)](#) on the current **HLTLINE**, *do not* pass that into [GetNextLine\(\)](#). Instead, call [GetNextLine\(\)](#) first, while the **HLTLINE** is still valid.

Used for: Special FX.

[uint32](#)(\* [ILTClient::GetNumRawDevices](#)())()

**Returns:**

the number of devices

Determine the number of raw devices (e.g. gamepads, joysticks) attached to the system.

Used for: Input.

void(\* [ILTClient::GetObjectColor](#))([HLOCALOBJ](#) hObject, float \*r, float \*g, float \*b, float \*a)

**Parameters:**

*hObject* Object to query.

*r* (return) Red value (range: 0.0-1.0).

*g* (return) Green value (range: 0.0-1.0).

*b* (return) Blue value (range: 0.0-1.0).

*a* (return) Alpha value (range: 0.0-1.0).

Get the color and alpha of the object.

**See also:**

[SetObjectColor\(\)](#)

Used for: Object.

[LTRESULT](#)(\* [ILTClient::GetObjectScale](#))([HLOCALOBJ](#) hObj, [LTVector](#) \*pScale)

void(\* [ILTClient::GetObjectUserData](#))([HLOCALOBJ](#) hObj)

**Parameters:**

*hObj* Object to query.

**Returns:**

Void pointer to user data.

User data for the object. This data is completely defined by the user and has no necessary bearing on the rest of the system.

**See also:**

[SetObjectUserData\(\)](#)

Used for: Object.

[LTRESULT](#)(\* [ILTClient::GetOptimized2DBlend](#))([LTSurfaceBlend](#) &blend)

**Parameters:**

*blend* (return) The blend mode.

**Returns:**

**LT\_NOTINITIALIZED** when the render struct is not initialized, or **LT\_ERROR** for general failure; otherwise, returns **LT\_OK**.

Gets the current blend mode.

**See also:**

[SetOptimized2DBlend\(\)](#)

Used for: 2D Rendering.

[LTRESULT](#)(\* [LTClient::GetOptimized2DColor](#))([HLTCOLOR](#) &hColor)

**Parameters:**

*hColor* (return) The current color.

**Returns:**

**LT\_NOTINITIALIZED** when the render struct is not initialized, or **LT\_ERROR** for general failure; otherwise, returns **LT\_OK**.

Get the color currently used on Optimized2D surfaces.

**See also:**

[SetOptimized2DColor\(\)](#)

Used for: 2D Rendering.

bool(\* [LTClient::GetParticleBlockerEdgesXZ](#))([uint32](#) index, [LTPlane](#) &blockerPlane, [uint32](#) &numEdges, [LTPlane](#) \*edgePlanes)

**Parameters:**

*index* Index of blocker to retrieve.  
*normal* Normal portion of blocker plane equation.  
*dist* Distance portion of blocker plane equation.  
*numEdges* Number of edges in blocker poly.  
*edgeNormals* Normals in XZ plane of each of the blocker edges.

**Returns:**

**false** - index is out of range.

**true** - Successful.

This retrieves the information for a particle blocker polygon. This form gets the information needed for fast vertical particle intersection, which is the polygon edge normals in the XZ plane.

Used for: Special FX.

bool(\* [LTClient::GetParticleBlockersInAABB](#))(const [LTVector](#) &pos, const [LTVector](#) &dims, std::vector< [uint32](#) > &indices)

**Parameters:**

*pos* Center of AABB.  
*dims* Dimensions of AABB.  
*indices* Array that will be filled in with the indices of the blockers in the AABB.

This retrieves the indices of all particle blockers that are within the AABB specified by pos and dims.

Used for: Special FX.

bool(\* [LTClient::GetParticles](#))([HLOCALOBJ](#) hObj, [LTParticle](#) \*\*pHead, [LTParticle](#) \*\*pTail)

**Parameters:**

*hObj* Particle system to query.  
*pHead* (return) Head of list.  
*pTail* (return) Tail of list.

**Returns:**

**true** - Successful.

**false** - *hObj* is invalid (NULL, or not an **OT\_PARTICLESYSTEM**).

Get a pointer to the linked list of particles. This is so that you can iterate through them and move them all. The engine will usually do this faster, but it is limited in scope.

Used for: Special FX.

[LTRESULT](#)(\* [LTClient::GetPerformanceInfo](#))([LTPERFORMANCEINFO](#) \*pPerformanceInfo)

**Parameters:**

*pPerformanceInfo* (return) **LTPERFORMANCEINFO** struct to be filled in by the function

**Returns:**

**LT\_INVALIDPARAMS** if **pPerformanceInfo** is **NULL** or if it's not the expected size (i.e., different versions of the struct); otherwise, returns **LT\_OK**.

Get run-time performance info for last frame.

The renderer is queried for some of the performance info.

Used for: Misc.

[LTRESULT](#)(\* [LTClient::GetPixel](#))([HSURFACE](#) hSurface, [uint32](#) x, [uint32](#) y, [HLTCOLOR](#) \*color)

**Parameters:**

*hSurface* Handle of surface.  
*x* Pixel x coordinate.  
*y* Pixel y coordinate.  
*color* (return) Color of pixel at (x,y).

**Returns:**

**LT\_INVALIDPARAMS** when the input surface is **NULL** or x and y coordinates are outside the surface, or **LT\_ERROR** when the surface data cannot be locked; otherwise, returns **LT\_OK**.

Get pixel color (*slow*).

**See also:**

[SetPixel\(\)](#)

Used for: 2D Rendering.

[LTRESULT](#)(\* [LTClient::GetPolyGridInfo](#))([HLOCALOBJ](#) hObj, char \*\*pBytes, [uint32](#) \*pWidth, [uint32](#) \*pHeight, [PGColor](#) \*\*pColorTable)

**Parameters:**

*hObj* Polygrid to query.  
*pBytes* (return) Pointer to heightmap data (size of data block = *pWidth* \* *pHeight* bytes).  
*pWidth* (return) Width (number of horizontal vertices).  
*pHeight* (return) Height (number of vertical vertices).  
*pColorTable* (return) Pointer to color table (a 256-element array).

**Returns:**

**LT\_INVALIDPARAMS** - *hObj* is invalid (**NULL**, or not an **OT\_POLYGRID**), or if *any* of the other parameters are **NULL**.

**LT\_OK** - Successful.

Use this function to get a pointer to a polygrid's data, its dimensions, and its color table. The buffers at *pBytes* and *pColorTable* can be modified to directly affect the PolyGrid. Heightmap values range from -128 to 127, with 0 representing center (i.e. center of *hObj*). Color table values range from 0 to 255, and directly map to the heightmap values (e.g. CT0 maps to HM-127, CT1 maps to HM-126, ...).

Note that you can both set a PolyGrid's alpha *globally* with [SetObjectColor\(\)](#) and *per vertex* via the alpha component of in the color table. Additionally, you can use an alpha mask in the texture.

Used for: Special FX.

[LTRESULT](#)(\* [LTClient::GetPolyGridTextureInfo](#))([HLOCALOBJ](#) hObj, float \*xPan, float \*yPan, float \*xScale, float \*yScale, float \*fBaseReflection, float \*fVolumeIOR)

**Parameters:**

*hObj* Polygrid to query.  
*xPan* (return) Texture horizontal pan.  
*yPan* (return) Texture vertical pan.  
*xScale* (return) Texture horizontal scale.  
*yScale* (return) Texture vertical scale.  
*fBaseReflection* (return) Base amount of reflection on the polygrid  
*fVolumeIOR* (return) The index of refraction for the volume being simulated (used only for fresnel, default is 1.33)

**Returns:**

**LT\_ERROR** - *hObj* is invalid (**NULL**, or not an **OT\_POLYGRID**).

**LT\_INVALIDPARAMS** - Either *xPan*, *yPan*, *xScale*, *yScale* is **NULL**.

**LT\_OK** - Successful.

Get the texture pan and scale for a PolyGrid.

**See also:**

[SetPolyGridTextureInfo\(\)](#)

Used for: Special FX.

[LTRESULT](#)(\* [LTClient::GetPolyTextureFlags](#))([HPOLY](#) hPoly, [uint32](#) \*pFlags)

**Parameters:**

*hPoly* Polygon to query.  
*pFlags* (return) Texture flags.

**Returns:**

**LT\_ERROR** when no world is loaded or **hPoly** is invalid; otherwise, returns **LT\_OK**.

Get the texture flags from a poly.

Used for: Misc.

[uint32](#)(\* [LTClient::GetRawDeviceActuatorDataSize](#))([uint32](#) deviceNum)

**Parameters:**

*deviceNum* device index, from zero to numDevices, the value returned from GetNumRawDevices.

**Returns:**

the number of bytes of actuator data that this device accepts

Retrieve the amount of data that a call to SetRawDeviceActuatorData will accept.

**See also:**

[SetRawDeviceActuatorData](#) Used for: Input.

[LTRESULT](#)(\* [ILTClient::GetRawDeviceData](#))([uint32](#) deviceNum, void \*pData)

**Parameters:**

*deviceNum* device index, from zero to numDevices, the value returned from GetNumRawDevices.  
*pData* pointer to buffer in which to store the data.

**Returns:**

**LT\_OK** or **LT\_ERROR**

Retrieve the raw analog data for a device, indexed from zero to numDevices, which is the value returned from GetNumRawDevices.

You must pass in a pointer to a buffer of sufficient size to hold the device data, as specified by GetRawDeviceDataSize.

**See also:**

[GetNumRawDevices](#)

[GetRawDeviceDataSize](#) Used for: Input.

[uint32](#)(\* [ILTClient::GetRawDeviceDataSize](#))([uint32](#) deviceNum)

**Parameters:**

*deviceNum* device index, from zero to numDevices, the value returned from GetNumRawDevices.

**Returns:**

the number of bytes of data that will be returned

Retreive the amount of data available from a call to GetRawDeviceData.

**See also:**

[GetRawDeviceData](#) Used for: Input.

[LTRESULT](#)(\* [ILTClient::GetRenderMode](#))([RMode](#) \*pMode)

**Parameters:**

*pMode* (return) The current render mode.

**Returns:**

**LT\_OK** if Successful, **LT\_ERROR** on Failure.

Get the current render mode.

**See also:**

[SetRenderMode\(\)](#)

Used for: Rendering.

[RMode](#)(\* [ILTClient::GetRenderModes](#)())

**Returns:**

Pointer to [RMode](#) Linked list structure containing the valid render modes.

Get a **NULL** - terminated list of supported render modes.

**See also:**

[GetRenderMode\(\)](#)

[SetRenderMode\(\)](#)

Used for: Rendering.

[HSURFACE](#)(\* [ILTClient::GetScreenSurface](#)())

**Returns:**

Handle to the screen surface.

Get the screen surface handle.

Used for: 2D Rendering.

[LTRESULT](#)(\* [ILTClient::GetServiceList](#))([NetService](#) \*&pListHead)

**Parameters:**

*pListHead* (return) List of **NetServices**.

**Returns:**

**LT\_OK** (always)

Get a list of net services (TCP/IP, modem, etc).

You should call [FreeServiceList\(\)](#) to deallocate the returned list when you are done with it.

**See also:**

[FreeServiceList\(\)](#)

Used for: Networking.

[LTRESULT](#)(\* [ILTClient::GetSessionList](#))([NetSession](#) \*&pListHead, const char \*pInfo)

**Parameters:**

*pListHead* (return) List of **NetSessions**  
*pInfo* A string containing IP addresses to check for sessions.

**Returns:**

**LT\_NOTINITIALIZED** if the NetMgr has no main driver (failed to call InitNetworking?) or there is no query socket, or **LT\_ERROR** if it can't bind to port when starting the query process; otherwise, returns **LT\_OK**.

Fills **pListHead** with enumerated sessions.

**pInfo** is a list of particular IP addresses to check, separated by semicolons. A record containing a single asterisk (\*) means to check the local network for any broadcast sessions. A colon followed by a number designates a port to use. If the first character of an address is a numeral (between '0' and '9') it is treated as a numeric IP address; otherwise it's treated as a host name.

Internally, the TCP driver will use the StartQuery/UpdateQuery/EndQuery functions to look for sessions. It will keep looking for connections for 3.5 seconds (constant **QUERY\_TIME**), sending out queries every 1.0 seconds (constant **QUERY\_SEND\_INTERVAL**).

You should call [FreeSessionList\(\)](#) to deallocate the returned list when you are done with it.

**See also:**

[FreeSessionList\(\)](#)

Used for: Networking.

[LTRESULT](#)(\* [ILTClient::GetSkyDef](#))([SkyDef](#) \*pDef)

**Parameters:**

*pDef* (return) Sky definition structure (filled in by function).

**Returns:**

Always returns **LT\_OK**.

Get the sky definition.

Used for: Special FX.

[LTRESULT](#)(\* [ILTClient::GetSoundData](#))([HLTSOUND](#) hSound, [int16](#) \*&pSixteenBitData, [int8](#) \*&pEightBitData, [uint32](#) \*dwSamplesPerSecond, [uint32](#) \*dwChannels)

**Parameters:**

*hSound* Handle to sound.  
*pSixteenBitData* The data buffer if the buffer is 16 bits per sample, or **NULL** otherwise.  
*pEightBitData* The data buffer if the buffer is 8 bits per sample, or **NULL** otherwise.  
*dwSamplesPerSecond* Samples per second.  
*dwChannels* Number of channels. (Samples alternate between each channel.)

**Returns:**

**LT\_ERROR** when sound is in neither 8- nor 16-bit format, or **LT\_INVALIDPARAMS** if **hSound** is not available; otherwise, returns **LT\_OK**.

Get the sound data and information about the data. This will force the sound to be decompressed. fTimer is set only if **LT\_OK** is returned.

Used for: Audio.

[LTRESULT](#)(\* [ILTClient::GetSoundOffset](#))([HLTSOUND](#) hSound, [uint32](#) \*dwOffset, [uint32](#) \*dwSize)

**Parameters:**

*hSound* Handle to sound.  
*dwOffset* Current offset into sound buffer (in buffer increments: bytes if 8 bit, double bytes if 16 bit).  
*dwSize* Size of sound buffer (in bytes).

**Returns:**

**LT\_FINISHED** when the sound is not playing (may be delayed due to collisions, or done), or **LT\_INVALIDPARAMS** when **hSound** is not available; otherwise, returns **LT\_OK**.

Get current offset into sound buffer of sound. fTimer is set only if **LT\_OK** is returned.

Used for: Audio.

[LTRESULT](#)(\* [ILTClient::GetSoundTimer](#))([HLTSOUND](#) hSound, [LTFLOAT](#) \*fTimer)

**Parameters:**

*hSound* Handle to sound.  
*fTimer* Current time of sound.

**Returns:**

**LT\_FINISHED** when sound is not playing (may be delayed due to collisions), or **LT\_INVALIDPARAMS** when **hSound** is not available; otherwise, returns **LT\_OK**.

Get current time in seconds of sound. fTimer is set only if **LT\_OK** is returned.

Used for: Audio.

void(\* [ILTClient::GetStringDimensions](#))([HLTFONT](#) hFont, [HSTRING](#) hString, int \*sizeX, int \*sizeY)

**Parameters:**

*hFont* Font to use for calculation.  
*hString* The string.  
*sizeX* (return) Calculated width.

*sizeY* (return) Calculated height.

Get the dimensions that this string would take up if it were used to make a surface.

Used for: 2D Rendering.

[LTRESULT](#)(\* [ILTClient::GetSurfaceAlpha](#))([HSURFACE](#) hSurface, float &alpha)

**Parameters:**

*hSurface* Surface to query.

*alpha* (return) Alpha value (0.0=transparent, 1.0=opaque).

**Returns:**

**LT\_INVALIDPARAMS** when input surface is **NULL**; otherwise, returns **LT\_OK**.

Get surface alpha.

**Note:** Alpha is only used on optimized surfaces, and only when they are blitted to the screen.

**See also:**

[SetSurfaceAlpha\(\)](#)

Used for: 2D Rendering.

void(\* [ILTClient::GetSurfaceDims](#))([HSURFACE](#) hSurf, [uint32](#) \*pWidth, [uint32](#) \*pHeight)

**Parameters:**

*hSurf* The surface to query.

*pWidth* (return) Width of surface.

*pHeight* (return) Height of surface.

Get the dimensions of a surface. You can pass in **NULL** for **pWidth** and **pHeight**.

Used for: 2D Rendering.

void(\* [ILTClient::GetSurfaceUserData](#))([HSURFACE](#) hSurface)

**Parameters:**

*hSurface* Handle of surface to query.

**Returns:**

Void pointer to user data.

Get the user-defined surface data associated with each surface. This data can contain any information and can be formatted in any way the user sees fit.

**See also:**

[SetSurfaceUserData\(\)](#)

Used for: 2D Rendering.

[LTRESULT](#)(\* [ILTClient::GetVersionInfoExt](#))([LTVERSIONINFOEXT](#) \*pVersionInfo)

**Parameters:**

*pVersionInfo* (return) **LTVERSIONINFOEXT** struct to be filled in by the function

**Returns:**

**LT\_INVALIDPARAMS** if **pVersionInfo** is **NULL** or if it's not the expected size (i.e., different versions of the struct); otherwise, returns **LT\_OK**.

Get the extended version info.

Used for: Misc.

bool(\* [ILTClient::InitMusic](#))(const char \*szMusicDLL)

**Parameters:**

*szMusicDLL* Music dll filename.

**Returns:**

**FALSE** if the provided redbook audio manager failed to initialize **TRUE** otherwise

Initialize the music subsystem. Currently the only music dll that exists is for red book audio. IMA is no longer supported by the engine, it has been replaced with DirectMusic which can be used from the ILTDirectMusic interface. If you are not using red book audio you do not need to call this function!

Used for: Music.

[LTRESULT](#)(\* [ILTClient::InitNetworking](#))(const char \*pDriver, [uint32](#) dwFlags)

**Parameters:**

*pDriver* Name of driver. Use **NULL** for the default net driver. (currently unsupported)

*dwFlags* Flags (currently unsupported).

**Returns:**

**LT\_OK** (always).

Clear any running shells from the client manager; terminate any running network drivers; start drivers "dplay2" and "internet".

Call this function before calling any other network functions.

The parameters are ignored.

Used for: Networking.

bool(\* [LTClient::IntersectSegment](#))([IntersectQuery](#) \*pQuery, [IntersectInfo](#) \*pInfo)  
Intersections.

bool(\* [LTClient::IntersectSweptSphere](#))(const [LTVector](#) &vStart, const [LTVector](#) &vEnd, float fRadius, [LTVector](#) &vFinalPos, [LTVector](#) &vNormal)

bool(\* [LTClient::IsCommandOn](#))(int commandNum)

**Parameters:**

*commandNum* Command ID to query.

**Returns:**

A boolean indicating whether the given command is toggled on or off.

Query a command for on/off status.

Used for: Input.

bool(\* [LTClient::IsConnected](#))()

**Returns:**

**true** when connected to server; otherwise, returns false.

Determine whether this client is currently connected to a server. (The function checks whether the ClientMgr's m\_pCurShell is valid.)

Used for: Client Management.

[LTRESULT](#)(\* [LTClient::IsDeviceEnabled](#))(const char \*strDeviceName, bool \*pIsEnabled)

**Parameters:**

*strDeviceName* Device name.

*pIsEnabled* (return) **true** if enabled, or **false** if disabled.

**Returns:**

Always **LT\_OK**.

Find out if the specified device is enabled yet.

**See also:**

[EnableDevice\(\)](#)

Used for: Input.

bool(\* [LTClient::IsDone](#))([HLTSOUND](#) pSoundHandle)

**Parameters:**

*pSoundHandle* Handle to client only sound.

**Returns:**

**TRUE** if the sound is either no longer playing, or not found **FALSE** otherwise

Check whether a sound is finished playing or whether the object it was attached to has been removed.

Used for: Audio.

[LTRESULT](#)(\* [LTClient::IsLobbyLaunched](#))(const char \*sDriver)

**Parameters:**

*sDriver* String containing name of driver.

**Returns:**

**LT\_ERROR** if it tried to add a driver and that failed, or to indicate that the session was not lobby launched; otherwise, return **LT\_OK** (indicated a lobby launched session).

Determine if we were lobby-launched.

Passing **NULL** for **sDriver** will cause the "dplay2" driver to be used.

If a driver of the given name cannot be found, [IsLobbyLaunched\(\)](#) will try to add it.

Used for: Networking.

bool(\* [LTClient::LoadSong](#))(const char \*szSong)

**Parameters:**

*szSong* Name of song.

**Returns:**

**FALSE** if the redbook audio manager was not properly initialized **FALSE** is the song could not found or loaded **TRUE** otherwise

Cache a song file.

This function is now only used if you are playing red book audio. If you are using DirectMusic use the functions in the ILTDirectMusic interface.

Used for: Music.

[LTRESULT](#)(\* [ILTClient::MakeCubicEnvMap](#))([HLOCALOBJ](#) hCamera, [uint32](#) nSize, const char \*pszFilePrefix)

**Parameters:**

*hCamera* The camera to use as a position for the generation of the cubic environment map  
*nSize* The number of pixels on a side of the final output environment map  
*pszFilePrefix* The filename that will be placed at the beginning of each filename, it will append the postfix and the .bmp extension

**Returns:**

**LT\_OK**, **LT\_NOTINITIALIZED**, or **LT\_NOTIN3D**.

Given the specified position of the camera it will generate 6 images in each direction to be used for a cubic environment map

Used for: Rendering.

void(\* [ILTClient::OffsetGlobalLightScale](#))(const [LTVector](#) \*pOffset)

**Parameters:**

*pOffset* [Light](#) scale RGB offset (range: clamped to 0.0-1.0).

Change the global light scale that is applied to all rendering by an offset.

**See also:**

[GetGlobalLightScale\(\)](#)

[SetGlobalLightScale\(\)](#)

Used for: Misc.

[LTRESULT](#)(\* [ILTClient::OptimizeParticles](#))([HLOCALOBJ](#) hSystem)

**Parameters:**

*hSystem* Particle system to optimize.

**Returns:**

**LT\_INVALIDPARAMS** - *hSystem* is invalid (NULL, or not an **OT\_PARTICLESYSTEM**).

**LT\_OK** - No problems.

This is an optimization you can make to help the engine minimize its boundaries on a particle system. If you create particles in various places and they go away, you can use this every so often to recalculate where the particles are.

Used for: Special FX.

[LTRESULT](#)(\* [ILTClient::OptimizeSurface](#))([HSURFACE](#) hSurface, [HLTCOLOR](#) hTransparentColor)

**Parameters:**

*hSurface* The surface to process.  
*hTransparentColor* The color to optimize with.

**Returns:**

**LT\_INVALIDPARAMS** when the input surface is **NULL**, or **LT\_ERROR** when the render struct cannot optimize the surface; otherwise, returns **LT\_OK**.

Optimize any surfaces you use while rendering 3D stuff in the background. Then, when the 3D rendering is done, draw them in a StartOptimized2D/EndOptimized2D block. Call this function each time you change the contents.

**See also:**

[StartOptimized2D\(\)](#)

[EndOptimized2D\(\)](#)

Used for: 2D Rendering.

bool(\* [ILTClient::PauseMusic](#))()

Pause music. Can be resumed.

This function is now only used if you are playing red book audio. If you are using DirectMusic use the functions in the ILTDirectMusic interface.

**Returns:**

**FALSE** if the redbook audio manager was not properly initialized **FALSE** if the redbook audio manager failed to pause the current song **TRUE** otherwise

Used for: Music.

void(\* [ILTClient::PlayJovstickEffect](#))(const char \*pEffectName, float x, float y)

**Parameters:**

*pEffectName* Effect to play.  
*x* Data for effect.  
*y* Data for effect.

Play a force-feedback joystick effect.

Used for: Input.

bool(\* [ILTClient::PlayList](#))(const char \*szPlayList, const char \*szTransition, bool bLoop, [uint32](#) dwBoundaryFlags)

**Parameters:**

*szPlayList* Name of playlist.  
*szTransition* Name of transition song to play before playlist starts. IMA only.

*bLoop* Loop the list.

*dwBoundaryFlags* Can be **MUSIC\_IMMEDIATE**, **MUSIC\_MEASURE**, **MUSIC\_SONG**, or **MUSIC\_QUEUE**. Valid for IMA only.

**Returns:**

**FALSE** if the redbook audio manager was not properly initialized **FALSE** if the playlist could not be found or started **TRUE** otherwise

Play a playlist. All songs will be played until end of list.

This function is now only used if you are playing red book audio. If you are using DirectMusic use the functions in the ILTDirectMusic interface.

Used for: Music.

[LTRESULT](#)(\* [ILTClient::QueryGraphicDevice](#))([LTGraphicsCaps](#) \*pCaps)

**Parameters:**

*pCaps* Name of engine hook.

**Returns:**

**LT\_OK** when it succeeds, returns an error if not.

This is here so we can query the video caps on the API level.

Used for: Misc.

[LTRESULT](#)(\* [ILTClient::ReadConfigFile](#))(const char \*pFilename)

**Parameters:**

*pFilename* The configuration file.

**Returns:**

**LT\_ERROR** if the file doesn't exist or if it can't open the file for writing.

Read and parse a configuration file.

Used for: Misc.

[LTRESULT](#)(\* [ILTClient::RegisterConsoleProgram](#))(const char \*pName, [ConsoleProgramFn](#) fn)

**Parameters:**

*pName* Name of console program.

*fn* Associated function.

**Returns:**

**LT\_OK** or **LT\_ALREADYEXISTS**.

Register a console program. pName is just stored so it should either be static or allocated. When the clientshell DLL is unloaded, it gets rid of any registered programs.

Used for: Misc.

void(\* [ILTClient::RelinquishRenderModes](#))([RMode](#) \*pModes)

**Parameters:**

*pModes* An array of **pMode** variables to relinquish.

Relinquish list of supported render modes.

Used for: Rendering.

[LTRESULT](#)(\* [ILTClient::RemoveInternetDriver](#))()

void(\* [ILTClient::RemoveLine](#))([HLOCALOBJ](#) hObj, [HLTLINE](#) hLine)

**Parameters:**

*hObj* Line system to modify.

*hLine* Line to remove.

Remove a line from a line system.

Used for: Special FX.

void(\* [ILTClient::RemoveParticle](#))([HLOCALOBJ](#) hSystem, [LTParticle](#) \*pParticle)

**Parameters:**

*hSystem* Particle system to modify.

*pParticle* Particle to remove.

Remove a particle.

Used for: Special FX.

[LTRESULT](#)(\* [ILTClient::RenderCamera](#))([HLOCALOBJ](#) hCamera, float fFrameTime)

**Parameters:**

*hCamera* The camera to render.

*fFrameTime* The frame time. This is used to update things like sprites and texture scripts. Normally this can just be GetFrameTime, or can be 0 if paused

**Returns:**

**LT\_OK**, **LT\_NOTINITIALIZED**, or **LT\_NOTIN3D**.

Render from the position of the camera into its rectangle.

Used for: Rendering.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::RenderObjects\]\(#\)\)\(\[HLOCALOBJ\]\(#\) hCamera, \[HLOCALOBJ\]\(#\) \\*pObjects, int nObjects, float fFrameTime\)](#)

**Parameters:**

*hCamera* The camera to render.  
*pObjects* An array of objects to render.  
*nObjects* The size of the **pObjects** array.  
*fFrameTime* The frame time. This is used to update things like sprites and texture scripts. Normally this can just be GetFrameTime, or can be 0 if paused

**Returns:**

**LT\_OK**, **LT\_NOTINITIALIZED**, or **LT\_NOTIN3D**.

Renders the list of objects through the given camera.

Used for: Rendering.

[\*\*bool\*\*\(\\* \[ILTClient::ResumeMusic\]\(#\)\)\(\)](#)

Resume music.

This function is now only used if you are playing red book audio. If you are using DirectMusic use the functions in the ILTDirectMusic interface.

**Returns:**

**FALSE** is the redbook audio manager was not properly initialized **FALSE** if there is no current song playing, or the redbook audio manager failed to resume **TRUE** otherwise

Used for: Music.

[\*\*void\*\*\(\\* \[ILTClient::RunConsoleString\]\(#\)\)\(const char \\*pString\)](#)

**Parameters:**

*pString* The string command to execute.

Same as typing a string into the console.

Used for: Misc.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::ScaleSurfaceToSurface\]\(#\)\)\(\[HSURFACE\]\(#\) hDest, \[HSURFACE\]\(#\) hSrc, \[LRect\]\(#\) \\*pDestRect, \[LRect\]\(#\) \\*pSrcRect\)](#)

**Parameters:**

*hDest* Destination surface.  
*hSrc* Source surface.  
*pDestRect* Area of **hDest** to fill (in **hDest** coordinates). **NULL** means the whole thing.  
*pSrcRect* Area from **hSrc** to draw (in **hSrc** coordinates). **NULL** means the whole thing.

**Returns:**

**LT\_NOTINITIALIZED** when there is no render struct, or **LT\_ERROR** for general failure (problems with the surfaces, etc.); otherwise, returns **LT\_OK**.

Draw the source surface onto the destination surface, but scale **pSrcRect** to fit **pDestRect**.

Used for: 2D Rendering.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::ScaleSurfaceToSurfaceSolidColor\]\(#\)\)\(\[HSURFACE\]\(#\) hDest, \[HSURFACE\]\(#\) hSrc, \[LRect\]\(#\) \\*pDestRect, \[LRect\]\(#\) \\*pSrcRect, \[HLTCOLOR\]\(#\) hTransCol](#)

**Parameters:**

*hDest* Destination surface.  
*hSrc* Source surface.  
*pDestRect* Area of **hDest** to fill (in **hDest** coordinates). **NULL** means the whole thing.  
*pSrcRect* Area from **hSrc** to draw (in **hSrc** coordinates). **NULL** means the whole thing.  
*hTransparentColor* Transparency color.  
*hFillColor* Fill color.

**Returns:**

**LT\_NOTINITIALIZED** when there is no render struct, or **LT\_ERROR** for general failure (problems with the surfaces, etc.); otherwise, returns **LT\_OK**.

Draw the source surface onto the destination surface, but scale **pSrcRect** to fit **pDestRect** using a solid color mask.

Used for: 2D Rendering.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::ScaleSurfaceToSurfaceTransparent\]\(#\)\)\(\[HSURFACE\]\(#\) hDest, \[HSURFACE\]\(#\) hSrc, \[LRect\]\(#\) \\*pDestRect, \[LRect\]\(#\) \\*pSrcRect, \[HLTCOLOR\]\(#\) hColor\)](#)

**Parameters:**

*hDest* Destination surface.  
*hSrc* Source surface.  
*pDestRect* Area of **hDest** to fill (in **hDest** coordinates). **NULL** means the whole thing.  
*pSrcRect* Area from **hSrc** to draw (in **hSrc** coordinates). **NULL** means the whole thing.  
*hColor* Transparency color.

**Returns:**

**LT\_NOTINITIALIZED** when there is no render struct, or **LT\_ERROR** for general failure (problems with the surfaces, etc.); otherwise, returns **LT\_OK**.

Draw the source surface onto the destination surface, but scale **pSrcRect** to fit **pDestRect** with transparency.

Used for: 2D Rendering.

[\*\*LTRESULT\*\*\(\\* \[LTClient::SelectService\]\(#\)\)\(\[HNETSERVICE\]\(#\) hNetService\)](#)

**Parameters:**

*hNetService* Description of net service.

**Returns:**

**LT\_NOTINITIALIZED** if it can't find the service, or **LT\_ERROR** if the service is found but can't be selected; otherwise, returns **LT\_OK**.

Find and select the given service as the one to use.

Used for: Networking.

void(\* [LTClient::SetCameraFOV](#))([HLOCALOBJ](#) hObj, float fovX, float fovY)

**Parameters:**

*hObj* Camera to modify.

*pX* (return) X FOV (range: clamped to  $\frac{\pi}{100}$  to  $(99 \times \frac{\pi}{100})$  ).

*pY* (return) Y FOV (range: clamped to  $\frac{\pi}{100}$  to  $(99 \times \frac{\pi}{100})$  ).

Set the FOV of a camera. It defaults to  $(\frac{\pi}{2}, \frac{\pi}{2})$ .

**See also:**

[GetCameraFOV\(\)](#)

Used for: Rendering.

bool(\* [LTClient::SetCameraLightAdd](#))([HLOCALOBJ](#) hCamera, const [LTVector](#) \*pAdd)

**Parameters:**

*hCamera* Camera to modify.

*pAdd* Additive value to set (range: 0.0-1.0).

**Returns:**

**false** if **hCamera** is not a camera.

Set the camera light additive value. [Light](#) add is applied *after* scaling, so if the light is fully bright and scaling is zero, you'll only see whiteness. When the light add is nonzero, it draws a poly over the screen, so don't use it all the time.

**See also:**

[GetCameraLightAdd\(\)](#)

Used for: Rendering.

void(\* [LTClient::SetCameraRect](#))([HLOCALOBJ](#) hObj, bool bFullscreen, int left, int top, int right, int bottom)

**Parameters:**

*hObj* Camera to modify.

*bFullscreen* **true** if fullscreen camera, **false** if non-fullscreen camera.

*left* Left edge of camera rectangle (in screen coordinates).

*top* Top edge of camera rectangle (in screen coordinates).

*right* Right edge of camera rectangle (in screen coordinates).

*bottom* Bottom edge of camera rectangle (in screen coordinates).

Set the rectangle of the camera on the screen. If **bFullscreen** is **true**, then it ignores the rectangle and draws the camera full-screen. If the rectangle extends over the screen boundaries, then it is clipped.

**See also:**

[GetCameraRect\(\)](#)

Used for: Rendering.

[\*\*LTRESULT\*\*\(\\* \[LTClient::SetConsoleEnable\]\(#\)\)\(bool bEnable\)](#)

**Parameters:**

*bEnable* Enable/Disable access to the console.

**Returns:**

**LT\_OK** on success.

Enable/Disable access to the console.

Used for: Misc.

[\*\*LTRESULT\*\*\(\\* \[LTClient::SetFontExtraSpace\]\(#\)\)\(\[HLTFONT\]\(#\) hFont, int pixels\)](#)

**Parameters:**

*hFont* Handle of font.

*pixels* Number of pixels to insert.

**Returns:**

**LT\_ERROR** when the font is invalid; otherwise, returns **LT\_OK**.

Set the intercharacter spacing. Intercharacter spacing is added to each character, including break characters, when the system writes a line of text.

**See also:**

[GetFontExtraSpace\(\)](#)

Used for: 2D Rendering.

```
void(* ILTClient::SetGlobalLightScale)(const LTVector *pScale)
```

**Parameters:**

*pScale* [Light](#) scale RGB value (range: clamped to 0.0-1.0).

Set the global light scale that is applied to all rendering.

**See also:**

[GetGlobalLightScale\(\)](#)

[OffsetGlobalLightScale\(\)](#)

Used for: Misc.

```
void(* ILTClient::SetInputState)(bool bOn)
```

**Parameters:**

*bOn* **true** means to turn the input on, and **false** means to turn it off. The default is **true**.

Turn the input state on or off. This is for when the client is interacting with menus and you don't want mouse movement or keystrokes to be sent to the server.

Used for: Input.

```
void(* ILTClient::SetLightColor)(HLOCALOBJ hObj, float r, float g, float b)
```

**Parameters:**

*hObj* [Light](#) to modify.

*r* Red value (0-1).

*g* Green value (0-1).

*b* Blue value (0-1).

Set the color of a light.

**See also:**

[GetLightColor\(\)](#)

Used for: Special FX.

```
void(* ILTClient::SetLightRadius)(HLOCALOBJ hObj, float radius)
```

**Parameters:**

*hObj* [Light](#) to modify.

*radius* Radius to set.

Set the radius of a light.

**See also:**

[GetLightRadius\(\)](#)

Used for: Special FX.

```
void(* ILTClient::SetLineInfo)(HLTLINE hLine, LTLine *pLine)
```

**Parameters:**

*hLine* Line to modify.

*pLine* Line info to set.

Set the line info for an HLTLINE.

**See also:**

[GetLineInfo\(\)](#)

Used for: Special FX.

```
LTRESULT(* ILTClient::SetModelHook)(ModelHookFn fn, void *pUser)
```

**Parameters:**

*fn* Hook function.

*pUser* User data.

**Returns:**

**LT\_OK**

When this is set, the renderer will call your hook function before drawing each model. So as not to slow the renderer, either keep the function very fast or set it to **NULL**.

Used for: Rendering.

```
void(* ILTClient::SetMusicVolume)(short wVolume)
```

**Parameters:**

*wVolume* New music volume.

Set music volume (range: 0=off, 100=full).

This function is now only used if you are playing red book audio. If you are using DirectMusic use the functions in the ILTDirectMusic interface.

**See also:**[GetMusicVolume\(\)](#)

Used for: Music.

void(\* [LTClient::SetObjectColor](#))([HLOCALOBJ](#) hObject, float r, float g, float b, float a)**Parameters:**

*hObject* Object to modify.  
*r* Red value (range: 0.0-1.0).  
*g* Green value (range: 0.0-1.0).  
*b* Blue value (range: 0.0-1.0).  
*a* Alpha value (range: 0.0-1.0).

Set the color and alpha of the object.

**Note:** This function overrides a model's render style's material diffuse color.**See also:**[GetObjectColor\(\)](#)

Used for: Object.

void(\* [LTClient::SetObjectPosAndRotation](#))([HLOCALOBJ](#) hObj, const [LTVector](#) \*pPos, const [LTRotation](#) \*pRotation)void(\* [LTClient::SetObjectRotation](#))([HLOCALOBJ](#) hObj, const [LTRotation](#) \*pRotation)[LTRESULT](#)(\* [LTClient::SetObjectScale](#))([HLOCALOBJ](#) hObj, const [LTVector](#) \*pScale)void(\* [LTClient::SetObjectUserData](#))([HLOCALOBJ](#) hObj, void \*pData)**Parameters:**

*hObj* Object to modify.  
*pData* User data.

Set user data for the object. This data is completely defined by the user and has no necessary bearing on the rest of the system.

**See also:**[GetObjectUserData\(\)](#)

Used for: Object.

[LTRESULT](#)(\* [LTClient::SetOptimized2DBlend](#))([LTSurfaceBlend](#) blend)**Parameters:***blend* The blend mode.**Returns:**

**LT\_NOTINITIALIZED** when the render struct is not initialized, **LT\_NOTIN3D** when this function is called outside a [Start3D\(\)](#)/[End3D\(\)](#) code block, or **LT\_ERROR** for general failure; otherwise, returns **LT\_OK**.

Change the source/destination blend mode for rendering Optimized2D surfaces. (Defaults to LTSURFACEBLEND\_ALPHA.)

**Note:** Drawing a surface with a blend mode other than **\_ALPHA** will automatically optimize the surface.**See also:**[GetOptimized2DBlend\(\)](#)

Used for: 2D Rendering.

[LTRESULT](#)(\* [LTClient::SetOptimized2DColor](#))([HLTCOLOR](#) hColor)**Parameters:***hColor* The color to apply.**Returns:**

**LT\_NOTINITIALIZED** when the render struct is not initialized, **LT\_NOTIN3D** if this function is called outside a [Start3D\(\)](#)/[End3D\(\)](#) code block, or **LT\_ERROR** for general failure; otherwise, returns **LT\_OK**.

Change the color used on Optimized2D surfaces (i.e. apply a color filter to Optimized2D).

**See also:**[GetOptimized2DColor\(\)](#)

Used for: 2D Rendering.

[LTRESULT](#)(\* [LTClient::SetParticleSystemEffectShaderID](#))([HLOCALOBJ](#) hObj, [uint32](#) EffectShaderID)**Parameters:**

*hObj* Particle system to modify.  
*EffectID* Effect Shader ID.

**Returns:**

**LT\_INVALIDPARAMS** - *hObj* is invalid (NULL, or not an **OT\_PARTICLESYSTEM**).

**LT\_OK** - Successful.

Set the Effect Shader ID.

Used for: Special FX.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::SetPixel\]\(#\)\)\(\[HSURFACE\]\(#\) hSurface, \[uint32\]\(#\) x, \[uint32\]\(#\) y, \[HLTCOLOR\]\(#\) color\)](#)

**Parameters:**

*hSurface* Handle of surface.  
*x* Pixel x coordinate.  
*y* Pixel y coordinate.  
*color* Color to set.

**Returns:**

**LT\_INVALIDPARAMS** when the input surface is NULL or x and y coordinates are outside the surface, or **LT\_ERROR** when the surface data cannot be locked; otherwise, returns **LT\_OK**.

Set pixel color (*slow*).

**See also:**

[GetPixel\(\)](#)

Used for: 2D Rendering.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::SetPolyGridEffectShaderID\]\(#\)\)\(\[HLOCALOBJ\]\(#\) hObj, \[uint32\]\(#\) EffectShaderID\)](#)

**Parameters:**

*hObj* Polygrid to modify.  
*EffectID* Effect Shader ID.

**Returns:**

**LT\_INVALIDPARAMS** - *hObj* is invalid (NULL, or not an **OT\_POLYGRID**).

**LT\_OK** - Successful.

Set the Effect Shader ID.

Used for: Special FX.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::SetPolyGridEnvMap\]\(#\)\)\(\[HLOCALOBJ\]\(#\) hObj, const char \\*pFilename\)](#)

**Parameters:**

*hObj* Polygrid to modify.  
*pFilename* Environment map filename (.dtx file). If NULL, disable the environment map.

**Returns:**

**LT\_INVALIDPARAMS** - *hObj* is invalid (NULL, or not an **OT\_POLYGRID**).

**LT\_NOTFOUND** - *pFilename* is invalid (can't find file).

**LT\_OK** - Successful.

Set the environment map for the PolyGrid.

Used for: Special FX.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::SetPolyGridTexture\]\(#\)\)\(\[HLOCALOBJ\]\(#\) hObj, const char \\*pFilename\)](#)

**Parameters:**

*hObj* Polygrid to modify.  
*pFilename* Texture filename (.spr).

**Returns:**

**LT\_ERROR** - *hObj* is invalid (NULL, or not an **OT\_POLYGRID**).

**LT\_MISSINGSpriteFile** - *pFilename* is invalid (can't find file).

**LT\_INVALIDSpriteFile** - Invalid/corrupt texture (sprite).

**LT\_OK** - Successful.

Set the texture. The texture *must* be a sprite file. It *cannot* be a .dtx file.

Used for: Special FX.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::SetPolyGridTextureInfo\]\(#\)\)\(\[HLOCALOBJ\]\(#\) hObj, float xPan, float yPan, float xScale, float yScale, float fBaseReflection, float fVolumeIOR\)](#)

**Parameters:**

*hObj* Polygrid to modify.  
*xPan* Texture horizontal pan. (default = 0.0)  
*yPan* Texture vertical pan. (default = 0.0)  
*xScale* Texture horizontal scale. (default = 1.0)  
*yScale* Texture vertical scale. (default = 1.0)  
*fBaseReflection* Base amount of reflection on the polygrid (default = 0.5)  
*fVolumeIOR* The index of refraction for the volume being simulated (default = 1.33)

**Returns:**

**LT\_ERROR** - *hObj* is invalid (NULL, or not an **OT\_POLYGRID**).

**LT\_OK** - Successful.

Set the texture pan and scale for a PolyGrid. These are *not* auto-pan or auto-scale values, and must be adjusted whenever you want to animate them.

**See also:**

[GetPolyGridTextureInfo\(\)](#)

Used for: Special FX.

[LTRESULT\(\\* ILTClient::SetRawDeviceActuatorData\)\(uint32 deviceNum, void \\*pData\)](#)

**Parameters:**

*deviceNum* device index, from zero to numDevices, the value returned from GetNumRawDevices.  
*pData* pointer to buffer containing actuator data

**Returns:**

**LT\_OK** or **LT\_ERROR**

Sends actuator data to the input device.

**See also:**

[GetRawDeviceActuatorDataSize](#) Used for: Input.

[LTRESULT\(\\* ILTClient::SetRenderMode\)\(RMode \\*pMode\)](#)

**Parameters:**

*pMode* The requested render mode.

**Returns:**

**LT\_KEPTSAMEMODE** when the function cannot set the mode requested but was able to restore the previous mode, or  
**LT\_UNABLETORESTOREVIDEO** if it cannot set the mode and cannot restore the video mode (in which case it will give a message and shutdown afterwards); otherwise, returns **LT\_OK**.

Tells the renderer to switch to the requested render mode.

**Note:** It may not have set the *exact* mode you requested. You can check with [GetRenderMode\(\)](#) to see what the new mode is.

**See also:**

[GetRenderMode\(\)](#)

Used for: Rendering.

[LTRESULT\(\\* ILTClient::SetSurfaceAlpha\)\(HSURFACE hSurface, float alpha\)](#)

**Parameters:**

*hSurface* Surface to modify.  
*alpha* Alpha value to set (0.0=transparent, 1.0=opaque).

**Returns:**

**LT\_INVALIDPARAMS** when surface is NULL; otherwise, returns **LT\_OK**.

Set surface alpha.

**Note:** Alpha is only used on optimized surfaces, and only when they are blitted to the screen.

**See also:**

[GetSurfaceAlpha\(\)](#)

Used for: 2D Rendering.

[void\(\\* ILTClient::SetSurfaceUserData\)\(HSURFACE hSurface, void \\*pUserData\)](#)

**Parameters:**

*hSurface* Handle of surface.  
*pUserData* User data to attach.

Attach whatever user data you want to a surface. This data can contain any information and can be formatted in any way the user sees fit.

**See also:**

[GetSurfaceUserData\(\)](#)

Used for: 2D Rendering.

[HLTCOLOR\(\\* ILTClient::SetupColor1\)\(float r, float g, float b, bool bTransparent\)](#)

**Parameters:**

*r* Red component (0.0 to 1.0).  
*g* Green component (0.0 to 1.0).  
*b* Blue component (0.0 to 1.0).  
*bTransparent* Transparency on/off. Only used when the function specifies so.

**Returns:**

Handle to the created color.

This function is just for convenience. You don't have to create or delete these colors, and they're always around.

Used for: 2D Rendering.

[HLTCOLOR](#)(\* [LTClient::SetupColor2](#))(float r, float g, float b, bool bTransparent)

**Parameters:**

*r* Red component (0.0 to 1.0).  
*g* Green component (0.0 to 1.0).  
*b* Blue component (0.0 to 1.0).  
*bTransparent* Transparency on/off. Only used when the function specifies so.

**Returns:**

Handle to the created color.

This function is just for convenience. You don't have to create or delete these colors, and they're always around.

Used for: 2D Rendering.

[LTRESULT](#)(\* [LTClient::SetupParticleSystem](#))([HLOCALOBJ](#) hObj, const char \*pTextureName, float gravityAccel, [uint32](#) flags, float particleRadius)

**Parameters:**

*hObj* Particle system to modify.  
*pTextureName* Name of particle texture (.dtx or .spr).  
*gravityAccel* Gravity acceleration value.  
*flags* Flags (see PS\_ defines, above).  
*particleRadius* Particle system radius (used for visibility culling).

**Returns:**

**LT\_INVALIDPARAMS** - Either *hObj* is invalid (NULL, or not an **OT\_PARTICLESYSTEM**) or *pTextureName* is a NULL pointer,

**LT\_NOTFOUND** - *pTextureName* is invalid (file not found).

**LT\_OK** - Successful.

Set the parameters of the particle system.

Used for: Special FX.

bool(\* [LTClient::SetupPolyGrid](#))([HLOCALOBJ](#) hObj, [uint32](#) width, [uint32](#) height, [uint32](#) nPGFlags, bool \*pEnabledVerts)

**Parameters:**

*hObj* Polygrid to set up.  
*width* Width of polygrid.  
*height* Height of polygrid.  
*nPGFlags* PolyGrid specific flags, can be a bitwise combination of the PG\_ flags.  
*pEnabledVerts* A list of booleans that is width\*height large, and specifies which vertices are valid for rendering. This may be NULL, in which case all vertices are valid.

**Returns:**

**false** - *hObj* is invalid (NULL, or not an **OT\_POLYGRID**), or either width or height is invalid (<2 or >65000).

**true** - Successful.

A poly grid is basically a heightmapped grid of pixels that are drawn as polygons. Each pixel can have a value from -128 to 127. The value of the pixel defines its height and is a lookup into the color table for the vertex color. You can scale and rotate the poly grid using [SetObjectScale\(\)](#) and [SetObjectRotation\(\)](#).

Used for: Special FX.

bool(\* [LTClient::SetupVolumeEffect](#))([HLOCALOBJ](#) hObj, [VolumeEffectInfo](#) &info)

**Parameters:**

*hObj* VolumeEffect to set up.  
*info* Initialization structure for this VolumeEffect.

**Returns:**

**false** - *hObj* is invalid (NULL, or not an **OT\_VOLUMEEFFECT**), or part of the initialization structure is invalid.

**true** - Successful.

A volume effect is a generic way of defining a custom effect. An example of it's use is snow, which is implemented as a dynamic particle volume effect.

Used for: Special FX.

[LTRESULT](#)(\* [LTClient::SetVolumeEffectEffectShaderID](#))([HLOCALOBJ](#) hObj, [uint32](#) EffectShaderID)

**Parameters:**

*hObj* VolumeEffect to modify.  
*EffectID* Effect Shader ID.

**Returns:**

**LT\_INVALIDPARAMS** - *hObj* is invalid (NULL, or not an **OT\_VOLUMEEFFECT**).

**LT\_OK** - Successful.

Set the Effect Shader ID.

Used for: Special FX.

[LTRESULT](#)(\* [LTClient::ShutdownRender](#))([uint32](#) flags)

**Parameters:**

*flags* The flags to while performing the shutdown (a combination of the **RSHUTDOWN** flags in [ltcodes.h](#)).

**Returns:**

LT\_OK if Successful, LT\_ERROR on Failure.

Shut down the renderer. The renderer won't come back until you call [SetRenderMode\(\)](#).

**See also:**

[SetRenderMode\(\)](#)

Used for: Rendering.

[LTRESULT](#)(\* [LTClient::SortParticles](#))([HLOCALOBJ](#) hSystem, const [LTVector](#) &vDir, [uint32](#) nNumIters)

**Parameters:**

*hSystem* Particle system to sort.  
*vDir* Direction vector to sort on (sorts from near to far)  
*nNumIters* Number of times to pass through the list for sorting

**Returns:**

LT\_INVALIDPARAMS - *hSystem* is invalid (NULL, or not an OT\_PARTICLESYSTEM).

LT\_OK - No problems.

This function will take a particle system and a vector indicating how the particles should be sorted. It will then run through for the specified number of iterations and sort the particles from nearest to farthest.

Used for: Special FX.

[LTRESULT](#)(\* [LTClient::Start3D](#))()

**Returns:**

LT\_OK or LT\_NOTINITIALIZED or LT\_ALREADYIN3D.

Start a [Start3D\(\)](#)/End3D() block.

**Note:** You must be in a [Start3D\(\)](#)/End3D() block in order to render through any cameras.

**See also:**

[End3D\(\)](#)

Used for: Rendering.

void(\* [LTClient::StartCounter](#))([LTCounter](#) \*pCounter)

**Parameters:**

*pCounter* The counter that the engine will update.

Start a counter.

**Note:** Use this to time sections of code. Timing is done in microseconds (millionths of a second).

Start a counter.

**Note:** Use this to time sections of code. Timing is done in microseconds (millionths of a second).

**See also:**

[EndCounter\(\)](#)

Used for: Misc.

[LTRESULT](#)(\* [LTClient::StartDeviceTrack](#))([uint32](#) nDevices, [uint32](#) nBufferSize)

**Parameters:**

*nDevices* Bit-field of devices to track (**DEVICETYPE\_** defines from [ltbasedefs.h](#)).  
*nBufferSize* Number of device events that could occur between calls to [TrackDevice\(\)](#), not to exceed **MAX\_INPUT\_BUFFER\_SIZE**.

**Returns:**

LT\_ERROR if the input services have not been initialized LT\_ERROR if the buffer size is larger than MAX\_INPUT\_BUFFER\_SIZE LT\_ERROR if none of the requested devices could be acquired LT\_TRUE otherwise

Start tracking input devices. Between calls to [StartDeviceTrack\(\)](#) and [EndDeviceTrack\(\)](#), no command states will be set through the normal input.

Used for: Input.

[LTRESULT](#)(\* [LTClient::StartOptimized2D](#))()

**Returns:**

LT\_NOTINITIALIZED when the render struct is not initialized, or LT\_NOTIN3D when this function is called outside a [Start3D\(\)](#)/End3D() code block; otherwise, returns LT\_OK.

Starts a [StartOptimized2D\(\)](#)/EndOptimized2D() block.

**Note:** You must be in a [StartOptimized2D\(\)](#)/EndOptimized2D() code block to draw any optimized 2D surfaces.

**See also:**

[EndOptimized2D\(\)](#)

Used for: 2D Rendering.

void(\* [ILTClient::StopMusic](#))([uint32](#) dwBoundaryFlags)

**Parameters:**

*dwBoundaryFlags* Can be **MUSIC\_IMMEDIATE**, **MUSIC\_MEASURE**, **MUSIC\_SONG**, or **MUSIC\_QUEUE**. Valid for IMA only.

Stop music.

This function is now only used if you are playing red book audio. If you are using DirectMusic use the functions in the ILTDirectMusic interface.

Used for: Music.

[LTRESULT](#)(\* [ILTClient::TrackDevice](#))([DeviceInput](#) \*pInputArray, [uint32](#) \*pnInOut)

**Parameters:**

*pInputArray* (return) Buffer of tracking structures.

*pnInOut* (input/return) Supply the size of pInputArray. Returns with number of events filled in.

**Returns:**

**LT\_INPUTBUFFEROVERFLOW** if there were more events that occurred than the original buffer size.

Track Input Devices.

**See also:**

[StartDeviceTrack\(\)](#)

Used for: Input.

[LTRESULT](#)(\* [ILTClient::TransformSurfaceToSurface](#))([HSURFACE](#) hDest, [HSURFACE](#) hSrc, [LTFloatPt](#) \*pOrigin, int destX, int destY, float angle, float scaleX, float scaleY)

**Parameters:**

*hDest* Destination surface.

*hSrc* Source surface.

*pOrigin* Position to treat as origin (in **hDest** coordinates). A value of **NULL** means to use the centerpoint as origin.

*destX* Destination x position (in **hDest** coordinates).

*destY* Destination y position (in **hDest** coordinates).

*angle* Angle by which to rotate **hSrc**.

*scaleX* Scale to horizontally expand/contract **hSrc**.

*scaleY* Scale to vertically expand/contract **hSrc**.

**Returns:**

**LT\_NOTINITIALIZED** when the render struct is not initialized, or **LT\_INVALIDPARAMS** when surfaces are **NULL** or coordinates are bad; otherwise, returns **LT\_OK**.

Transform the source surface onto the destination surface.

Used for: 2D Rendering.

[LTRESULT](#)(\* [ILTClient::TransformSurfaceToSurfaceTransparent](#))([HSURFACE](#) hDest, [HSURFACE](#) hSrc, [LTFloatPt](#) \*pOrigin, int destX, int destY, float angle, float scaleX, float scaleY, [hColor](#) transparency)

**Parameters:**

*hDest* Destination surface.

*hSrc* Source surface.

*pOrigin* Position to treat as origin (in **hDest** coordinates). A value of **NULL** means to use the centerpoint as origin.

*destX* Destination x position (in **hDest** coordinates).

*destY* Destination y position (in **hDest** coordinates).

*angle* Angle to rotate **hSrc**.

*scaleX* Scale to horizontally expand/contract **hSrc**.

*scaleY* Scale to vertically expand/contract **hSrc**.

*transparency* hColor Transparency color.

**Returns:**

**LT\_NOTINITIALIZED** when render struct not initialized, or **LT\_INVALIDPARAMS** when surfaces are **NULL** or coordinates are bad; otherwise, returns **LT\_OK**.

Transform the source surface onto the destination surface with transparency

Used for: 2D Rendering.

[LTRESULT](#)(\* [ILTClient::UnoptimizeSurface](#))([HSURFACE](#) hSurface)

**Parameters:**

*hSurface* The surface to process.

**Returns:**

**LT\_INVALIDPARAMS** when the surface is **NULL**; otherwise, returns **LT\_OK**.

Unoptimize a surface.

Used for: 2D Rendering.

[LTRESULT](#)(\* [ILTClient::UnregisterConsoleProgram](#))(const char \*pName)

**Parameters:**

*pName* Name of console program.

**Returns:**

**LT\_OK** or **LT\_NOTFOUND**.

Unregister a console program.

Used for: Misc.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::WarpSurfaceToSurface\]\(#\)\)\(\[HSURFACE\]\(#\) hDest, \[HSURFACE\]\(#\) hSrc, \[LTWarpPt\]\(#\) \\*pCoords, int nCoords\)](#)

**Parameters:**

*hDest* Destination surface.  
*hSrc* Source surface.  
*pCoords* Array of warp points that define source-destination mapping.  
*nCoords* Number of elements in the **pCoords** array.

**Returns:**

**LT\_NOTINITIALIZED** when the render struct is not initialized, or **LT\_INVALIDPARAMS** when surfaces are **NULL** or coordinates are bad; otherwise, returns **LT\_OK**.

(Affine) warp the source polygon into the destination polygon. The source coordinates are *clamped* to be inside the polygon of the source surface, and the warp is clipped against the destination polygon (don't specify coordinates outside the source polygon, but feel free to specify them outside the destination polygon). The polygon you specify should be convex. The minimum number of coordinates is 3 and the maximum is 10.

Used for: 2D Rendering.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::WarpSurfaceToSurfaceSolidColor\]\(#\)\)\(\[HSURFACE\]\(#\) hDest, \[HSURFACE\]\(#\) hSrc, \[LTWarpPt\]\(#\) \\*pCoords, int nCoords, \[HLTCOLOR\]\(#\) hTransColor, \[HI\]\(#\)](#)

**Parameters:**

*hDest* Destination surface.  
*hSrc* Source surface.  
*pCoords* Array of warp points that define source-destination mapping.  
*nCoords* Number of elements in the **pCoords** array.  
*hTransColor* Transparency color.  
*hFillColor* Fill color.

**Returns:**

**LT\_NOTINITIALIZED** when the render struct is not initialized, or **LT\_INVALIDPARAMS** when surfaces are **NULL** or coordinates are bad; otherwise, returns **LT\_OK**.

(Affine) warp the source poly into the destination polygon using a solid color mask. The source coordinates are *clamped* to be inside the polygon of the source surface, and the warp is clipped against the destination polygon (don't specify coordinates outside the source polygon, but feel free to specify them outside the destination polygon). The polygon you specify should be convex. The minimum number of coordinates is 3 and the maximum is 10.

Used for: 2D Rendering.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::WarpSurfaceToSurfaceTransparent\]\(#\)\)\(\[HSURFACE\]\(#\) hDest, \[HSURFACE\]\(#\) hSrc, \[LTWarpPt\]\(#\) \\*pCoords, int nCoords, \[HLTCOLOR\]\(#\) hColor\)](#)

**Parameters:**

*hDest* Destination surface.  
*hSrc* Source surface.  
*pCoords* Array of warp points that define source-destination mapping.  
*nCoords* Number of elements in the **pCoords** array.  
*hColor* Transparency color.

**Returns:**

**LT\_NOTINITIALIZED** when the render struct is not initialized, or **LT\_INVALIDPARAMS** when surfaces are **NULL** or coordinates are bad; otherwise, returns **LT\_OK**.

(Affine) warp the source poly into the destination polygon with transparency. The source coordinates are *clamped* to be inside the polygon of the source surface, and the warp is clipped against the destination polygon (don't specify coordinates outside the source polygon, but feel free to specify them outside the destination polygon). The polygon you specify should be convex. The minimum number of coordinates is 3 and the maximum is 10.

Used for: 2D Rendering.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::WriteConfigFile\]\(#\)\)\(const char \\*pFilename\)](#)

**Parameters:**

*pFilename* The configuration file.

**Returns:**

**LT\_ERROR** if it can't open the file for writing.

Write configuration files.

Used for: Misc.

[\*\*LTRESULT\*\*\(\\* \[ILTClient::WriteConfigFileFields\]\(#\)\)\(const char \\*pFilename, \[uint32\]\(#\) nNumValues, const char \\*\\*pValues\)](#)

**Parameters:**

*pFilename* The configuration file.  
*nNumValues* The number of values to write out, 0 will write all of them  
*pValues* The names of each value to be saved out. There should be nNumValues of them.

**Returns:**

**LT\_ERROR** if it can't open the file for writing.

Write configuration files.

Used for: Misc.

The documentation for this class was generated from the following file:

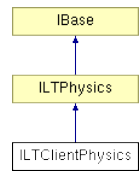
- [iltclient.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

### ILTClientPhysics Class Reference

Inheritance diagram for ILTClientPhysics:



#### Public Member Functions

virtual [LTRESULT](#) [UpdateMovement](#) ([MoveInfo](#) \*pInfo)=0  
virtual [LTRESULT](#) [MovePushObjects](#) ([HOBJECT](#) hToMove, const [LTVector](#) &p, [HOBJECT](#) \*hPushObjects, [uint32](#) nPushObjects)=0  
virtual [LTRESULT](#) [RotatePushObjects](#) ([HOBJECT](#) hToMove, const [LTRotation](#) &q, [HOBJECT](#) \*hPushObjects, [uint32](#) nPushObjects)=0

#### Member Function Documentation

virtual [LTRESULT](#) ILTClientPhysics::MovePushObjects( [HOBJECT](#) *hToMove*,  
const [LTVector](#) & *p*,  
[HOBJECT](#) \* *hPushObjects*,  
[uint32](#) *nPushObjects*  
) [pure virtual]

virtual [LTRESULT](#) ILTClientPhysics::RotatePushObjects( [HOBJECT](#) *hToMove*,  
const [LTRotation](#) & *q*,  
[HOBJECT](#) \* *hPushObjects*,  
[uint32](#) *nPushObjects*  
) [pure virtual]

virtual [LTRESULT](#) ILTClientPhysics::UpdateMovement( [MoveInfo](#) \* *pInfo* ) [pure virtual]

The documentation for this class was generated from the following file:

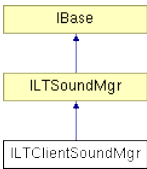
- [iltphysics.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

### ILTClientSoundMgr Class Reference

Inheritance diagram for ILTClientSoundMgr:



#### Public Types

enum { [\\_ILTClientSoundMgr\\_VERSION](#) = 0 + ILTSoundMgr::\_ILTSoundMgr\_VERSION\_ \* 100 }

#### Public Member Functions

```

virtual LTRESULT GetSound3DProviderLists (Sound3DProvider *pSound3DProviderList, bool bVerify, uint32 uiMax3DVoices)=0
virtual LTRESULT ReleaseSound3DProviderList (Sound3DProvider *pSound3DProviderList)=0
virtual LTRESULT InitSound (InitSoundInfo *pSoundInfo)=0
virtual LTRESULT SetListenerDoppler (float fVolume)=0
virtual LTRESULT GetVolume (uint16 &nVolume)=0
virtual LTRESULT SetVolume (uint16 nVolume)=0
virtual LTRESULT UpdateVolumeSettings ()=0
virtual LTRESULT SetReverbProperties (ReverbProperties *pReverbProperties)=0
virtual LTRESULT GetReverbProperties (ReverbProperties *pReverbProperties)=0
virtual LTRESULT SetSoundOcclusion (HLTSOUND hSound, LTFLOAT fLevel)=0
virtual LTRESULT GetSoundOcclusion (HLTSOUND hSound, LTFLOAT *pLevel)=0
virtual LTRESULT SetSoundObstruction (HLTSOUND hSound, LTFLOAT fLevel)=0
virtual LTRESULT GetSoundObstruction (HLTSOUND hSound, LTFLOAT *pLevel)=0
virtual LTRESULT SetSoundFilter (const char *pFilter)=0
virtual LTRESULT SetSoundFilterParam (const char *pParam, float fValue)=0
virtual LTRESULT EnableSoundFilter (bool bEnable)=0
virtual LTRESULT SetSoundPosition (HLTSOUND hSound, LTVector *pPos)=0
virtual LTRESULT GetSoundPosition (HLTSOUND hSound, LTVector *pPos)=0
virtual LTRESULT PauseSounds ()=0
virtual LTRESULT ResumeSounds ()=0
virtual LTRESULT SetSoundClassMultiplier (uint8 nSoundClass, float fMult, bool bUseGlobalVolume=true)=0
virtual LTRESULT GetSoundClassMultiplier (uint8 nSoundClass, float *pfMult)=0
virtual LTRESULT SetListener (bool bListenerInClient, LTVector *pPos, LTRotation *pRot, bool bTeleport)=0

```

---

## Detailed Description

Client-specific sound mgr interface.

---

## Member Enumeration Documentation

anonymous enum

**Enumeration values:**

[\\_ILTClientSoundMgr\\_VERSION\\_](#)

---

## Member Function Documentation

```
virtual LTRESULT ILTClientSoundMgr::EnableSoundFilter( bool bEnable ) [pure virtual]
```

**Parameters:**

*bEnable* Enable or disable current filter in engine

**Returns:**

**LT\_ERROR** when engine does not support filters, otherwise, returns **LT\_OK**.

Turn on or off the current filter

Used for: Audio.

```
virtual LTRESULT ILTClientSoundMgr::GetReverbProperties( ReverbProperties * pReverbProperties ) [pure virtual]
```

**Parameters:**

*pReverbProperties* (return) Current reverb properties.

**Returns:**

**LT\_OK**

Retrieve the current reverb properties.

Used for: Audio.

```

virtual LTRESULT ILTClientSoundMgr::GetSound3DProviderLists( Sound3DProvider * & pSound3DProviderList,
                                                             bool bVerify,
                                                             uint32 uiMax3DVoices
                                                             ) [pure virtual]

```

**Parameters:**

*pSound3DProviderList* (return) The list of available 3D sound providers.

*bVerify* Tell the engine not to report providers that aren't completely supported on the system. This takes longer and causes speaker popping.

*uiMax3DVoices* Tell the engine to check to only report back providers that can support the maximum voices required.

**Returns:**

**LT\_OK**.

Get a list of the 3D sound providers available on the system, to be used in the [InitSoundInfo](#) structure. Games should only need to use **bVerify** when a different provider is chosen. EAX support can only be checked when the provider is opened, so without the **bVerify**, the **SOUND3DPROVIDER\_CAPS\_REVERB** cannot be set. InitSound will report this info in the **m\_dwResults** flags.

**Note:** Be sure to release the list with [ReleaseSound3DProviderList\(\)](#).

**See also:**

[ReleaseSound3DProviderList\(\)](#)

Used for: Audio.

virtual [LTRESULT](#) ILTClientSoundMgr::GetSoundClassMultiplier( [uint8](#) *nSoundClass*,  
float \* *pMult*  
) [pure virtual]

**Returns:**

**LT\_ERROR** if invalid sound type, **LT\_OK** otherwise

Used for: Audio

virtual [LTRESULT](#) ILTClientSoundMgr::GetSoundObstruction( [HLTSOUND](#) *hSound*,  
[LTFLOAT](#) \* *pLevel*  
) [pure virtual]

**Parameters:**

*hSound* Handle to the client only sound.

*pLevel* Variable to return the level of obstruction into.

**Returns:**

**LT\_OK**.

Retrieve the occlusion level of a sound.

Used for: Audio.

virtual [LTRESULT](#) ILTClientSoundMgr::GetSoundOcclusion( [HLTSOUND](#) *hSound*,  
[LTFLOAT](#) \* *pLevel*  
) [pure virtual]

**Parameters:**

*hSound* Handle to the client only sound.

*pLevel* Variable to return the level of occlusion into.

**Returns:**

**LT\_OK**.

Retrieve the occlusion level of a sound.

Used for: Audio.

virtual [LTRESULT](#) ILTClientSoundMgr::GetSoundPosition( [HLTSOUND](#) *hSound*,  
[LTVector](#) \* *pPos*  
) [pure virtual]

**Parameters:**

*hSound* Handle to sound.

*pPos* (return) Position to retrieve.

**Returns:**

**LT\_INVALIDPARAMS** when there are invalid parameters, or **LT\_ERROR** when unable to find **hSound**; otherwise, returns **LT\_OK**.

Retrieve current position and orientation of a sound.

Used for: Audio.

virtual [LTRESULT](#) ILTClientSoundMgr::GetVolume( [uint16](#) & *nVolume* ) [pure virtual]

**Parameters:**

*nVolume* (return) Current volume level (range: 0 to 100).

**Returns:**

**LT\_OK**

Retrieve the global sound volume.

Used for: Audio.

virtual [LTRESULT](#) ILTClientSoundMgr::InitSound( [InitSoundInfo](#) \* *pSoundInfo* ) [pure virtual]

**Parameters:**

*pSoundInfo* The initialization parameters.

**Returns:**

**LT\_INVALIDPARAMS** when **pSoundInfo** is invalid, **LT\_ERROR** when unable to initialize the sound driver; otherwise, return **LT\_OK**.

Initialize the sound driver.

Used for: Audio.

virtual [LTRESULT](#) ILTClientSoundMgr::PauseSounds( ) [pure virtual]

**Returns:**

**LT\_ERROR** when unable to pause; otherwise, returns **LT\_OK**.

Pause sounds.

Used for: Audio.

virtual [LTRESULT](#) ILTClientSoundMgr::ReleaseSound3DProviderList( [Sound3DProvider](#) \* *pSound3DProviderList* ) [pure virtual]

**Parameters:**

*pSound3DProviderList* The 3D sound provider list to be released.

**Returns:**

LT\_OK Successful.

LT\_INVALIDPARAMS An invalid provider list was provided.

This releases a list of 3D sound providers generated by GetSound3DProviderList().

**See also:**

GetSound3DProviderList()

Used for: Audio.

virtual [LTRESULT](#) ILTClientSoundMgr::ResumeSounds( ) [pure virtual]

**Returns:**

LT\_ERROR when unable to resume; otherwise, returns LT\_OK.

Resume sounds.

Used for: Audio.

```
virtual LTRESULT ILTClientSoundMgr::SetListener( bool      bListenerInClient,
                                                LTVector * pPos,
                                                LTRotation * pRot,
                                                bool      bTeleport
                                              ) [pure virtual]
```

**Parameters:**

*bListenerInClient* If TRUE, then **pPos** and **pRot** are ignored and can be set to NULL.

*pPos* Position of listener.

*pRot* Rotation of listener.

*bTeleport* Consider the movement a teleportation (don't apply velocity).

**Returns:**

LT\_OK.

Set the listener status, position, and orientation.

Used for: Audio.

virtual [LTRESULT](#) ILTClientSoundMgr::SetListenerDoppler( float *fVolume* ) [pure virtual]

**Parameters:**

*fDoppler* Current Doppler factor (range: 0 (off) to 10)

**Returns:**

LT\_OK

Set the global Doppler effect (only used when 3D hardware sounds are played. Doppler factor is a multiple of real world Doppler effect, so 2 would twice the Doppler effect that would occur in real life.

Used for: Audio.

virtual [LTRESULT](#) ILTClientSoundMgr::SetReverbProperties( [ReverbProperties](#) \* *pReverbProperties* ) [pure virtual]

**Parameters:**

*pReverbProperties* The desired reverb properties

**Returns:**

LT\_OK

Set the reverb properties.

Used for: Audio.

```
virtual LTRESULT ILTClientSoundMgr::SetSoundClassMultiplier( uint8 nSoundClass,
                                                            float fMult,
                                                            bool bUseGlobalVolume = true
                                                          ) [pure virtual]
```

**Returns:**

LT\_ERROR if invalid sound class or multiplier LT\_OK otherwise

This function allows different sound class categories to be created. Game can create up to MAX\_SOUND\_CLASSES types. Each sound class can have it's own multiplier that is relative to the current maximum sound volume (if bUseGlobalVolume is set to true). 0 is the default sound class, and results in no multiplier.

If bUseGlobalVolume is set to false, the sound class multiplier is used as the sounds volume level allowing you to control sounds independent of the global volume setting.

Used for: Audio

virtual [LTRESULT](#) ILTClientSoundMgr::SetSoundFilter( const char \* *pFilter* ) [pure virtual]

**Parameters:**

*pFilter* Name of the filter.

returns LT\_NOTFOUND if an unknown filter is requested; otherwise, LT\_OK.

Set a current sound engine filter

Used for: Audio.

```
virtual LTRESULT ILTClientSoundMgr::SetSoundFilterParam( const char * pParam,
                                                         float      fValue
                                                         ) [pure virtual]
```

**Parameters:**

*pParam* Name of the parameter.  
*fValue* Value of the parameter.

**Returns:**

**LT\_ERROR** when engine does not support filters, **LT\_NOTFOUND** when an unknown parameter is requested; otherwise, returns **LT\_OK**.

Set a parameter of a filter.

Used for: Audio.

```
virtual LTRESULT ILTClientSoundMgr::SetSoundObstruction( HLTSOUND hSound,
                                                         LTFLOAT  fLevel
                                                         ) [pure virtual]
```

**Parameters:**

*hSound* Handle to the client only sound.  
*fLevel* Level of obstruction.

**Returns:**

**LT\_OK**.

Set the sound obstruction level.

Used for: Audio.

```
virtual LTRESULT ILTClientSoundMgr::SetSoundOcclusion( HLTSOUND hSound,
                                                       LTFLOAT  fLevel
                                                       ) [pure virtual]
```

**Parameters:**

*hSound* Handle to the client only sound.  
*fLevel* Level of occlusion.

**Returns:**

**LT\_OK**.

Set the sound occlusion level.

Used for: Audio.

```
virtual LTRESULT ILTClientSoundMgr::SetSoundPosition( HLTSOUND hSound,
                                                       LTVector * pPos
                                                       ) [pure virtual]
```

**Parameters:**

*hSound* Handle to sound.  
*pPos* New position of sound. Can be **NULL**.

**Returns:**

**LT\_INVALIDPARAMS** when there are invalid parameters, or **LT\_ERROR** when unable to find **hSound**; otherwise, returns **LT\_OK**.

Set position and orientation of a sound.

Used for: Audio.

```
virtual LTRESULT ILTClientSoundMgr::SetVolume( uint16 nVolume ) [pure virtual]
```

**Parameters:**

*nVolume* Desired volume level (range: 0 to 100).

**Returns:**

**LT\_OK**

Set the global sound volume.

Used for: Audio.

```
virtual LTRESULT ILTClientSoundMgr::UpdateVolumeSettings( ) [pure virtual]
```

**Parameters:**

*return* **LT\_OK**

Tells the sound manager to update any current sounds after volume changes are made.

Used for: Audio.

---

The documentation for this class was generated from the following file:

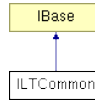
- [iltsoundmgr.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
 2006, All Rights Reserved.

## API Reference

## ILTCommon Class Reference

Inheritance diagram for ILTCommon:



### Public Types

enum { [ILTCommon\\_VERSION](#) = 4 }

### Public Member Functions

virtual [LTRESULT](#) [GetObjectType](#) ([HOBJECT](#) hObj, [uint32](#) \*type)=0  
 virtual [LTRESULT](#) [CompressVector](#) (const [LTVector](#) &vec, [CompVector](#) &CompV)=0  
 virtual [LTRESULT](#) [UncompressVector](#) (const [CompVector](#) &CompV, [LTVector](#) &vec)=0  
 virtual [LTRESULT](#) [CompressRotation](#) (const [LTRotation](#) &rot, [CompRot](#) &CompR)=0  
 virtual [LTRESULT](#) [Parse](#) ([ConParse](#) \*pParse)=0  
 virtual [LTRESULT](#) [GetModelAnimUserDims](#) ([HOBJECT](#) hObject, [LTVector](#) \*pDims, [HMODELANIM](#) hAnim)=0  
 virtual [LTRESULT](#) [GetRotationVectors](#) ([LTRotation](#) &rot, [LTVector](#) &up, [LTVector](#) &right, [LTVector](#) &forward)=0  
 virtual [LTRESULT](#) [SetupEuler](#) ([LTRotation](#) &rot, float pitch, float yaw, float roll)=0  
 virtual [LTRESULT](#) [CreateMessage](#) ([ILTMessage](#) [Write](#) \*pMsg)=0  
 virtual [LTRESULT](#) [GetPointStatus](#) (const [LTVector](#) \*pPoint)=0  
 virtual [LTRESULT](#) [GetPointShade](#) (const [LTVector](#) \*pPoint, [LTVector](#) \*pColor)=0  
 virtual [LTRESULT](#) [GetPolyTextureFlags](#) ([HPOLY](#) hPoly, [uint32](#) \*pFlags)=0  
 virtual [LTRESULT](#) [GetPolyPlane](#) ([HPOLY](#) hPoly, [LTPlane](#) \*pPlane)=0  
 virtual [LTRESULT](#) [UncompressRotation](#) (const [CompRot](#) &CompR, [LTRotation](#) &rot)=0  
 virtual [LTRESULT](#) [SetObjectResource](#) ([HOBJECT](#) hObj, [EObjectResource](#) eType, [uint32](#) nIndex, const char \*pszResource)=0  
 virtual [LTRESULT](#) [SetObjectFileNames](#) ([HOBJECT](#) hObj, [ObjectCreateStruct](#) \*pStruct)=0  
 virtual [LTRESULT](#) [GetObjectFlags](#) (const [HOBJECT](#) hObj, const [ObjFlagType](#) flagType, [uint32](#) &dwFlags)=0  
 virtual [LTRESULT](#) [SetObjectFlags](#) ([HOBJECT](#) hObj, const [ObjFlagType](#) flagType, [uint32](#) dwFlags, [uint32](#) dwMask)=0  
 virtual [LTRESULT](#) [GetAttachmentObjects](#) ([HATTACHMENT](#) hAttachment, [HOBJECT](#) &hParent, [HOBJECT](#) &hChild)=0  
 virtual [LTRESULT](#) [GetAttachments](#) ([HLOCALOBJ](#) hObj, [HLOCALOBJ](#) \*inList, [uint32](#) inListSize, [uint32](#) &outListSize, [uint32](#) &outNumAttachments)=0  
 virtual [LTRESULT](#) [GetAttachedModelNodeTransform](#) ([HATTACHMENT](#) hAttachment, [HMODELNODE](#) hNode, [LTransform](#) &transform)=0  
 virtual [LTRESULT](#) [GetAttachmentTransform](#) ([HATTACHMENT](#) hAttachment, [LTransform](#) &transform, bool bWorldSpace)=0  
 virtual [LTRESULT](#) [GetAttachedModelSocketTransform](#) ([HATTACHMENT](#) hAttachment, [HMODEL SOCKET](#) hSocket, [LTransform](#) &transform)=0  
 virtual [LTRESULT](#) [NumAttachments](#) ([HLOCALOBJ](#) hObject, [uint32](#) &dwAttachCount)=0

### Detailed Description

Define a holder to get this interface like this:

```

define_holder_to_instance(ILTCommon, your_var, Server);
define_holder_to_instance(ILTCommon, your_var, Client);
    
```

### Member Enumeration Documentation

anonymous enum

Enumeration values:

[ILTCommon\\_VERSION](#)

### Member Function Documentation

virtual [LTRESULT](#) ILTCommon::CompressRotation( const [LTRotation](#) & rot, [CompRot](#) & CompR ) [pure virtual]

Parameters:

*rot* The rotation to compress.  
*CompR* (return) The [CompRot](#) struct to pack rot into.

Returns:

[LT\\_OK](#).

Compress an [LTRotation](#) down to 6 (or sometimes 3) bytes.

See also:

[UncompressRotation\(\)](#)

Used for: Misc.

```
virtual LTRESULT ILTCommon::CompressVector( const LTVector & vec,
                                           CompVector &   CompV
                                           ) [pure virtual]
```

**Parameters:**

*vec*        The vector to compress.  
*CompV* (return) The [CompVector](#) struct to pack vec into.

**Returns:**

**LT\_OK.**

Compress an **LTVector** down to 9 bytes.

**See also:**

[UncompressVector\(\)](#)

Used for: Misc.

```
virtual LTRESULT ILTCommon::CreateMessage( ILTMessage\_Write * & pMsg ) [pure virtual]
```

Start a message. Make sure to properly deal with this object as a reference counted object.

Used for: Messaging.

```
virtual LTRESULT ILTCommon::GetAttachedModelNodeTransform( HATTACHMENT hAttachment,
                                                           HMODELNODE hNode,
                                                           LTransform &   transform
                                                           ) [pure virtual]
```

Basically ILTCommon::GetAttachmentTransform(WORLD) \* ModelLT::GetNodeTransform(LOCAL).

Used for: Object.

```
virtual LTRESULT ILTCommon::GetAttachedModelSocketTransform( HATTACHMENT hAttachment,
                                                            HMODEL SOCKET hSocket,
                                                            LTransform &   transform
                                                            ) [pure virtual]
```

Basically ILTCommon::GetAttachmentTransform(WORLD) \* ModelLT::GetSocketTransform(LOCAL).

Used for: Object.

```
virtual LTRESULT ILTCommon::GetAttachmentObjects( HATTACHMENT hAttachment,
                                                  HOBJECT &      hParent,
                                                  HOBJECT &      hChild
                                                  ) [pure virtual]
```

**Returns:**

**LT\_NOTINITIALIZED** if either of the objects doesn't exist.

Get the parent and child of an attachment.

Used for: Object.

```
virtual LTRESULT ILTCommon::GetAttachments( HLOCALOBJ hObj,
                                           HLOCALOBJ * inList,
                                           uint32      inListSize,
                                           uint32 &    outListSize,
                                           uint32 &    outNumAttachments
                                           ) [pure virtual]
```

**Parameters:**

*inList*                The list to be filled in.  
*inListSize*            How large **inList** is.  
*outListSize*           How many elements in **inList** were filled in.  
*outNumAttachments*   How many attachments **hObj** has. This number can be larger that **outListSize** if **inList** can't hold all the attachments.

Gets the objects attached to this object.

Used for: Object.

```
virtual LTRESULT ILTCommon::GetAttachmentTransform( HATTACHMENT hAttachment,
                                                    LTransform &   transform,
                                                    bool        bWorldSpace
                                                    ) [pure virtual]
```

Get the attachment transform (based off the parent object position, model socket, and attachment offset). If **bWorldSpace** is **DTRUE**, then the transform is in world space.

Used for: Object.

```
virtual LTRESULT ILTCommon::GetModelAnimUserDims( HOBJECT      hObject,
                                                  LTVector *    pDims,
                                                  HMODELANIM hAnim
                                                  ) [pure virtual]
```

Get the user dimensions of a model animation (the dimensions set in ModelEdit).

Used for: Models and Animation.

```
virtual LTRESULT ILTCommon::GetObjectFlags( const HOBJECT hObj,
```

```
const ObjFlagType flagType,
      uint32 & dwFlags
) [pure virtual]
```

Get the object's flag parameters. Specify what flag type you're accessing and the value.

Used for: Object.

```
virtual LTRESULT ILTCommon::GetObjectType( HOBJECT hObj,
      uint32 * type
) [pure virtual]
```

Tell what type of object this is.

Used for: Object.

```
virtual LTRESULT ILTCommon::GetPointShade( const LTVector * pPoint,
      LTVector * pColor
) [pure virtual]
```

**Parameters:**

*pPoint* The location to test.

*pColor* The lighting at the given point.

**Returns:**

**LT\_NOTINWORLD** when the point is outside the world.

Get the shade (RGB, 0-255) at the point you specify.

Used for: Misc.

```
virtual LTRESULT ILTCommon::GetPointStatus( const LTVector * pPoint ) [pure virtual]
```

**Parameters:**

*pPoint* The location to test.

**Returns:**

**LT\_NOTINWORLD** when no world is loaded, **LT\_INSIDE** when the point is in the world, or **LT\_OUTSIDE** when the point is not in the world.

Used for: Misc.

```
virtual LTRESULT ILTCommon::GetPolyPlane( HPOLY hPoly,
      LTPlane * pPlane
) [pure virtual]
```

**Parameters:**

*hPoly* The handle of the polygon.

*pPlane* Pointer to dest where the polygon's plane is written.

**Returns:**

**LT\_ERROR** when no world is loaded or **hPoly** is invalid; otherwise, returns **LT\_OK**.

Get the plane for a poly.

Used for: Misc.

```
virtual LTRESULT ILTCommon::GetPolyTextureFlags( HPOLY hPoly,
      uint32 * pFlags
) [pure virtual]
```

**Parameters:**

*hPoly* The handle of the polygon.

*pFlags* Pointer to dest where flags are written.

**Returns:**

**LT\_ERROR** when no world is loaded or **hPoly** is invalid; otherwise, returns **LT\_OK**.

Get the texture flags from a poly.

Used for: Misc.

```
virtual LTRESULT ILTCommon::GetRotationVectors( LTRotation & rot,
      LTVector & up,
      LTVector & right,
      LTVector & forward
) [pure virtual]
```

```
virtual LTRESULT ILTCommon::NumAttachments( HLOCALOBJ hObject,
      uint32 & dwAttachCount
) [pure virtual]
```

**Parameters:**

*hObject* Object for which attachments should be enumerated.

*typeFilter* **AttachmentType** enum describing whether to count all, server-only, or client-only attachments.

*dwAttachCount* [return] Number of attachments to **hObject**.

**Returns:**

**LT\_INVALIDPARAMS** if **hObject** is NULL.

Fetch the number of attachments that a given object has associated with it.

**Note:** The **typeFilter** parameter is ignored when the server version of this function is called. The server only knows about server-made attachments so it always counts those.

Used for: Object.

virtual [LTRESULT](#) ILTCommon::Parse( [ConParse](#) \* *pParse* ) [pure virtual]

**Returns:**

**LT\_FINISHED** when done.

Initialize the [ConParse](#) and call. Fills in **m\_Args** and **m\_nArgs** with each set of tokens.

```
Sample loop:
parse.Init(pStr);
while (pEngine->Parse2(&parse) == LT\_OK)
{
    ... process args ...
}
```

Used for: Misc.

virtual [LTRESULT](#) ILTCommon::SetObjectFileNames( [HOBJECT](#) *hObj*,  
[ObjectCreateStruct](#) \* *pStruct*  
) [pure virtual]

Changes all the appropriate files to match found in the create struct

**Parameters:**

*hObj* The object to change the filenames of  
*pStruct* The create struct which holds the appropriate names

Used for: Object.

virtual [LTRESULT](#) ILTCommon::SetObjectFlags( [HOBJECT](#) *hObj*,  
const [ObiFlagType](#) *flagType*,  
[uint32](#) *dwFlags*,  
[uint32](#) *dwMask*  
) [pure virtual]

Set the object's flag parameters. Specify what flag type you're accessing, the value, and the mask of flags to modify. Use FLAGMASK\_ALL to modify all flags.

Used for: Object.

virtual [LTRESULT](#) ILTCommon::SetObjectResource( [HOBJECT](#) *hObj*,  
[EObjectResource](#) *eType*,  
[uint32](#) *nIndex*,  
const char \* *pszResource*  
) [pure virtual]

Change the filenames of a model or a sprite.

**Parameters:**

*hObj* The object to change the resource of  
*eType* The type of resource that is being changed  
*nIndex* The index of the resource being changed  
*pszResource* The name of the resource that it should be changed to

Used for: Object.

virtual [LTRESULT](#) ILTCommon::SetupEuler( [LTRotation](#) & *rot*,  
float *pitch*,  
float *yaw*,  
float *roll*  
) [pure virtual]

virtual [LTRESULT](#) ILTCommon::UncompressRotation( const [CompRot](#) & *CompR*,  
[LTRotation](#) & *rot*  
) [pure virtual]

virtual [LTRESULT](#) ILTCommon::UncompressVector( const [CompVector](#) & *CompV*,  
[LTVector](#) & *vec*  
) [pure virtual]

**Parameters:**

*CompV* The [CompVector](#) struct to be unpacked.  
*vec* (return) The **LTVector** to hold the uncompressed data.

**Returns:**

**LT\_OK**.

Uncompress an **LTVector** which was compressed using **CompressVector**.

**See also:**

[CompressVector\(\)](#)

Used for: Misc.

The documentation for this class was generated from the following file:

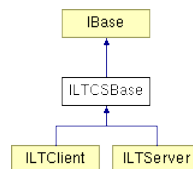
- [iltcommon.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
 2006, All Rights Reserved.

## API Reference

## ILTCSTBase Class Reference

Inheritance diagram for ILTCSTBase:



### Public Types

```
enum { \_ILTCSTBase\_VERSION = 4 }
```

### Public Member Functions

```

virtual ILTCommon * Common ()=0
virtual ILTPhysics * Physics ()=0
virtual ILTTransform * GetTransformLT ()=0
virtual ILTModel * GetModelLT ()=0
virtual ILTSoundMgr * SoundMgr ()=0
virtual void CPrint (const char *pMsg,...)=0
virtual bool GetContainerCode (HOBJECT hObj, uint16 *pCode)=0
virtual uint32 GetObjectContainers (HOBJECT hObj, HOBJECT *pContainerList, uint32 maxListSize)=0
virtual uint32 GetContainedObjects (HOBJECT hContainer, HOBJECT *pObjectList, uint32 maxListSize)=0
virtual LTRESULT OpenFile (const char *pFilename, ILTStream **pStream)=0
virtual LTRESULT CopyFile (const char *pszSourceFile, const char *pszDestFile)=0
virtual LTRESULT OpenMemoryStream (ILTStream **pStream, uint32 nCacheSize=256)=0
virtual LTRESULT GetBlindObjectData (uint32 nNum, uint32 nId, uint8 *&pData, uint32 &nSize)=0
virtual LTRESULT FreeBlindObjectData (uint32 nNum, uint32 nId)=0
virtual HSTRING FormatString (int messageCode,...)=0
virtual HSTRING CopyString (HSTRING hString)=0
virtual HSTRING CreateString (const char *pString)=0
virtual void FreeString (HSTRING hString)=0
virtual bool CompareStrings (HSTRING hString1, HSTRING hString2)=0
virtual bool CompareStringsUpper (HSTRING hString1, HSTRING hString2)=0
virtual const char * GetStringData (HSTRING hString)=0
virtual float GetVarValueFloat (HCONSOLEVAR hVar)=0
virtual const char * GetVarValueString (HCONSOLEVAR hVar)=0
virtual LTFLOAT GetTime ()=0
virtual LTFLOAT GetFrameTime ()=0
virtual LTRESULT GetSourceWorldOffset (LTVector &vVector)=0
virtual LTRESULT RemoveObject (HOBJECT hObj)=0
virtual LTRESULT GetLightGroupID (const char *pName, uint32 *pResult) const =0
virtual LTRESULT GetOccluderID (const char *pName, uint32 *pResult) const =0
virtual LTRESULT GetTextureEffectVarID (const char *pName, uint32 nStage, uint32 *pResult) const =0
virtual LTRESULT LinkObjRef (HOBJECT hObj, LTObjRef *pRef)=0
virtual LTRESULT SendTo (ILTMessage\_Read *pMsg, const char *pAddr, uint16 nPort)=0
virtual LTRESULT StartPing (const char *pAddr, uint16 nPort, uint32 *pPingID)=0
virtual LTRESULT GetPingStatus (uint32 nPingID, uint32 *pStatus, uint32 *pLatency)=0
virtual LTRESULT RemovePing (uint32 nPingID)=0
virtual LTRESULT GetObjectRotation (HLOCALOBJ hObj, LTRotation *pRotation)=0
virtual LTRESULT GetObjectPos (HLOCALOBJ hObj, LTVector *pPos)=0
virtual HMODELANIM GetAnimIndex (HOBJECT hObj, const char *pAnimName)=0
virtual void SetModelAnimation (HOBJECT hObj, HMODELANIM hAnim)=0
virtual HMODELANIM GetModelAnimation (HOBJECT hObj)=0
virtual void SetModelLooping (HOBJECT hObj, bool bLoop)=0
virtual bool GetModelLooping (HOBJECT hObj)=0
virtual LTRESULT ResetModelAnimation (HOBJECT hObj)=0
virtual uint32 GetModelPlaybackState (HOBJECT hObj)=0
virtual uint32 GetPointContainers (const LTVector *pPoint, HOBJECT *pList, uint32 maxListSize)=0
  
```

### Data Fields

```
LTRESULT(* GetTcpIpAddress) (char *sAddress, uint32 dwBufferSize, uint16 &hostPort)
```

### Member Enumeration Documentation

anonymous enum

**Enumeration values:**  
*ILTCSBase VERSION*

---

## Member Function Documentation

virtual [ILTCommon](#)\* ILTCSBase::Common( ) [pure virtual]

**Returns:**

Pointer to the [ILTCommon](#) interface.

Accesses the [ILTCommon](#) interface.

Used for: Interface.

virtual bool ILTCSBase::CompareStrings( [HSTRING](#) *hString1*,  
[HSTRING](#) *hString2*  
 ) [pure virtual]

**Parameters:**

*hString1* Handle of string one.

*hString2* Handle of string two.

Compare two **HSTRING** strings for equality.

Used for: Misc.

virtual bool ILTCSBase::CompareStringsUpper( [HSTRING](#) *hString1*,  
[HSTRING](#) *hString2*  
 ) [pure virtual]

**Parameters:**

*hString1* Handle of string one.

*hString2* Handle of string two.

Compare two **HSTRING** strings for equality.

Used for: Misc.

virtual [LTRESULT](#) ILTCSBase::CopyFile( const char \* *pszSourceFile*,  
 const char \* *pszDestFile*  
 ) [pure virtual]

**Parameters:**

*pszSourceFile* Name of the file to open.

*pszDestFile* Where to copy the file to.

Copies a file. This function is useful for copying files out of the rez file to a temporary file, so you can do special operations on it like LoadLibrary.

Used for: Misc.

virtual [HSTRING](#) ILTCSBase::CopyString( [HSTRING](#) *hString* ) [pure virtual]

**Parameters:**

*hString* Handle to the string to copy.

Copy a string. This is much more efficient than [CreateString\(\)](#).

Used for: Misc.

virtual void ILTCSBase::CPrint( const char \* *pMsg*,  
 ...  
 ) [pure virtual]

**Parameters:**

*pMsg* **printf** style format string and variable parameter list.

A better BPrint. It prints right to the console (doesn't do any network stuff) and doesn't care if you're connected yet.

Used for: Misc.

virtual [HSTRING](#) ILTCSBase::CreateString( const char \* *pString* ) [pure virtual]

**Parameters:**

*pString* Null terminated character string.

Create a new **HSTRING** from the given ASCII-terminated **char\***.

Used for: Misc.

virtual [HSTRING](#) ILTCSBase::FormatString( int *messageCode*,  
 ...  
 ) [pure virtual]

**Parameters:**

*messageCode* Index of format string to use.

In Windows, messageCode comes from your resource DLL. The messages need to be formatted with 1!s! 2!s! (the number is the argument number and the !s! says its a string). You can also use !f! and !d! for floating point and whole number.

Used for: Misc.

```
virtual LTRESULT ILTCSBase::FreeBlindObjectData( uint32 nNum,
                                                uint32 nId
                                                ) [pure virtual]
```

**Parameters:**

*nNum* Index of blind data.  
*nId* ID of blind data (will fail if doesn't match).

**Returns:**

Returns LTTRUE on success.

Free the memory used by blind data. Once all blind data has been freed, support structures will also be freed.

Used for: Misc.

```
virtual void ILTCSBase::FreeString( HSTRING hString ) [pure virtual]
```

**Parameters:**

*hString* Handle to the string to free.

Destroy an HSTRING.

Used for: Misc.

```
virtual HMODELANIM ILTCSBase::GetAnimIndex( HOBJECT hObj,
                                             const char * pAnimName
                                             ) [pure virtual]
```

**Parameters:**

*hObj* Model instance  
*pAnimName* name of animation

**Returns:**

-1 if the animation doesn't exist or if the object isn't a model.

Get an animation index from a model. The animation index is a handle to an animation associated with a model.

Used for: Models and Animation.

```
virtual LTRESULT ILTCSBase::GetBlindObjectData( uint32 nNum,
                                                uint32 nId,
                                                uint8 *& pData,
                                                uint32 & nSize
                                                ) [pure virtual]
```

**Parameters:**

*nNum* Index of blind data.  
*nId* ID of blind data (will fail if doesn't match).  
*pData* A pointer to the blind data is placed here.  
*nSize* Size of blind data is placed here.

**Returns:**

Returns LTTRUE on success.

Retrieve a pointer to a section of blind data Get a pointer to a section of blind data. Blind data should be freed with FreeBlindObjectData after it has been used.

Used for: Misc.

```
virtual uint32 ILTCSBase::GetContainedObjects( HOBJECT hContainer,
                                              HOBJECT * pObjectList,
                                              uint32 maxListSize
                                              ) [pure virtual]
```

**Parameters:**

*hContainer* The container to query.  
*pObjectList* (return) The object list array/buffer.  
*pFlagList* (return) The flag list array/buffer.  
*maxListSize* The number of elements in list arrays.

**Returns:**

The number of elements in **pContainerList** filled in.

Get the list of objects inside the container. Returns the number of elements in **pObjectList** filled in.

Used for: Object.

```
virtual bool ILTCSBase::GetContainerCode( HOBJECT hObj,
                                          uint16 * pCode
                                          ) [pure virtual]
```

**Parameters:**

*hObj* Handle to container object.  
*pCode* Address of variable that gets set.

**Returns:**

**false** when the object isn't a container.

Get the container's container code (can only be set in the [ObjectCreateStruct](#) during creation).

Used for: Misc.

virtual [LTFLOAT](#) ILTCSBase::GetFrameTime( ) [pure virtual]

**Returns:**

The time since the last shell update.

Used for: Misc.

virtual [LTRESULT](#) ILTCSBase::GetLightGroupID( const char \* *pName*,  
[uint32](#) \* *pResult*  
) const [pure virtual]

**Parameters:**

*pName* Name of the lightgroup

Function to generate the appropriate lightgroup ID given the name of the lightgroup.

**Note:** Lightgroup names are case-sensitive.

Used for: LightGroup control

virtual [HMODELANIM](#) ILTCSBase::GetModelAnimation( [HOBJECT](#) *hObj* ) [pure virtual]

**Parameters:**

*hObj* ???

**Returns:**

The animation the model is currently on, or ([HMODELANIM](#))-1 if none.

Used for: Models and Animation.

virtual bool ILTCSBase::GetModelLooping( [HOBJECT](#) *hObj* ) [pure virtual]

**Parameters:**

*hObj* ???

**Returns:**

???

Get the looping state of the model. The default state is **TRUE**.

Used for: Models and Animation.

virtual [ILTModel](#)\* ILTCSBase::GetModelLT( ) [pure virtual]

**Returns:**

Pointer to the [ILTModel](#) interface.

Accesses the [ILTModel](#) interface.

Used for: Interface.

virtual [uint32](#) ILTCSBase::GetModelPlaybackState( [HOBJECT](#) *hObj* ) [pure virtual]

**Parameters:**

*hObj* ???

**Returns:**

???

Tell what the playback state of the model is (a combination of the **MS\_** bits defined in basedefs\_de.h).

Used for: Models and Animation.

virtual [uint32](#) ILTCSBase::GetObjectContainers( [HOBJECT](#) *hObj*,  
[HOBJECT](#) \* *pContainerList*,  
[uint32](#) *maxListSize*  
) [pure virtual]

**Parameters:**

*hObj* The object to query.

*pContainerList* (return) The container list array/buffer.

*pFlagList* (return) The flag list array/buffer.

*maxListSize* The number of elements in list arrays.

**Returns:**

The number of elements in **pContainerList** filled in.

Get the list of containers the object is inside. **pFlagList** must be the same size as **pContainerList**.

Used for: Object.

virtual [LTRESULT](#) ILTCSBase::GetObjectPos( [HLOCALOBJ](#) *hObj*,  
[LTVector](#) \* *pPos*  
) [pure virtual]

Gets the position of the specified object

Used for: Object.

Implemented in [ILTClient](#), and [ILTServer](#).

```
virtual LTRESULT ILTCSBase::GetObjectRotation( HLOCALOBJ hObj,
                                              LTRotation* pRotation
                                              ) [pure virtual]
```

Gets the rotation of the specified object

Used for: Object.

Implemented in [ILTClient](#), and [ILTServer](#).

```
virtual LTRESULT ILTCSBase::GetOccluderID( const char *  pName,
                                           uint32 *    pResult
                                           ) const [pure virtual]
```

**Parameters:**

*pName* Name of the occluder

Function to generate the appropriate occluder ID given the name of the occluder.

**Note:** Occluder names are case-insensitive.

Used for: Occluder control

```
virtual LTRESULT ILTCSBase::GetPingStatus( uint32  nPingID,
                                           uint32 * pStatus,
                                           uint32 * pLatency
                                           ) [pure virtual]
```

**Parameters:**

*nPingID* Ping ID to query

*pStatus* Ping status result

*pLatency* Latency result if \*pStatus == PING\_STATUS\_SUCCESS

**Returns:**

LT\_NOTFOUND if that ping ID is unknown, LT\_OK otherwise.

```
virtual uint32 ILTCSBase::GetPointContainers( const LTVector * pPoint,
                                              HOBJECT *      pList,
                                              uint32         maxListSize
                                              ) [pure virtual]
```

**Parameters:**

*pPoint* Location to test.

*pList* Array of HOBJECT

*maxListSize* Length of the pList array.

**Returns:**

The number of containers filled in.

Find out what containers contain the given point.

Used for: Misc.

```
virtual LTRESULT ILTCSBase::GetSourceWorldOffset( LTVector & vVector ) [pure virtual]
```

The worlds are offset at processing time to be centered around the origin. This offset can be found by calling this function. This is useful for mapping position values back to the original level values for debugging and error reporting

**Parameters:**

*vVector* The vector offset from the original level. To get the original value use the equation original = position + offset

**Returns:**

Always returns LT\_OK-

Used for: Misc.

```
virtual const char* ILTCSBase::GetStringData( HSTRING hString ) [pure virtual]
```

**Parameters:**

*hString* Handle of string.

Get the string's data. You should only use this for strings that you stored off and want to pass to the engine for a filename or something. Most strings could be in some format other than ANSI.

Used for: Misc.

```
virtual LTRESULT ILTCSBase::GetTextureEffectVarID( const char *  pName,
                                                    uint32         nStage,
                                                    uint32 *      pResult
                                                    ) const [pure virtual]
```

**Parameters:**

*pName* Name of the texture effect

*nStage* The stage that is desired

Function to generate the appropriate ID for a texture effect given the name of the effect and the stage that the parameter belongs to.

**Note:** Lightgroup names are case-sensitive.

Used for: Texture Effect Variable manipulation

virtual [LTFLOAT](#) ILTCSBase::GetTime( ) [pure virtual]

**Returns:**

The current time, in seconds, since the shell started.

Used for: Misc.

virtual [ILTTransform](#)\* ILTCSBase::GetTransformLT( ) [pure virtual]

**Returns:**

Pointer to the [ILTTransform](#) interface.

Accesses the [ILTTransform](#) interface.

Used for: Interface.

virtual float ILTCSBase::GetVarValueFloat( [HCONSOLEVAR](#) hVar ) [pure virtual]

**Parameters:**

*hVar* Handle of console variable.

**Returns:**

The floating point value of a console variable, or 0 if the **hVar** is **NULL**.

Used for: Misc.

virtual const char\* ILTCSBase::GetVarValueString( [HCONSOLEVAR](#) hVar ) [pure virtual]

**Parameters:**

*hVar* Handle of console variable.

**Returns:**

The string value of a console variable, or **NULL** if the **hVar** is **NULL**.

Used for: Misc.

virtual [LTRESULT](#) ILTCSBase::LinkObjRef( [HOBJECT](#) hObj,  
[LTObjRef](#)\* pRef  
) [pure virtual]

**Parameters:**

*hObj* Object to link to

*pRef* [LTObjRef](#) object to add to the object's reference list

This function adds a reference link to the object's list, to be called when the object gets deleted.

**Note:** This function is called internally by the [LTObjRef](#) class. Do not call this function externally!

virtual [LTRESULT](#) ILTCSBase::OpenFile( const char \* pFilename,  
[ILTStream](#)\*\* pStream  
) [pure virtual]

**Parameters:**

*pFilename* Name of the file to open.

*pStream* Address of pointer to an [ILTStream](#) that gets set.

Open a file up. Pass in the relative filename. Free the file by calling DStream::Release().

Used for: Misc.

virtual [LTRESULT](#) ILTCSBase::OpenMemoryStream( [ILTStream](#)\*\* pStream,  
[uint32](#) nCacheSize = 256  
) [pure virtual]

**Parameters:**

*pStream* Address of pointer to an [ILTStream](#) that gets set.

*nCacheSize* Stream memory cache block size

Opens a memory stream. Free the file by calling DStream::Release().

Used for: Misc.

virtual [ILTPhysics](#)\* ILTCSBase::Physics( ) [pure virtual]

**Returns:**

Pointer to the [ILTPhysics](#) interface.

Accesses the [ILTPhysics](#) interface.

Used for: Interface.

virtual [LTRESULT](#) ILTCSBase::RemoveObject( [HOBJECT](#) *hObj* ) [pure virtual]

**Parameters:**

*hObj* ???

Remove the object from the world.

**Note:** The object will not be removed or deleted until the end of the frame.

Used for: Object.

Implemented in [LTServer](#).

virtual [LTRESULT](#) ILTCSBase::RemovePing( [uint32](#) *nPingID* ) [pure virtual]

**Parameters:**

*nPingID* Ping ID to remove

**Returns:**

**LT\_NOTFOUND** if that ping ID is unknown, **LT\_OK** otherwise.

virtual [LTRESULT](#) ILTCSBase::ResetModelAnimation( [HOBJECT](#) *hObj* ) [pure virtual]

**Parameters:**

*hObj* ???

**Returns:**

???

Start the current animation over.

Used for: Models and Animation.

virtual [LTRESULT](#) ILTCSBase::SendTo( [LTMessage\\_Read](#) \* *pMsg*,  
const char \* *pAddr*,  
[uint16](#) *nPort*  
) [pure virtual]

**Parameters:**

*pMsg* user data to send

*pAddr* IP address to target; can be numeric or host name.

*nPort* Port to use

**Returns:**

**LT\_NOTINITIALIZED** if there is no TCP/IP driver or the driver's socket is uninitialized, or **LT\_ERROR** if the address cannot be built; otherwise, returns **LT\_OK**.

Send a message using the TCP/IP protocol.

**sAddr** is a string containing an IP addresses to check. If the first character of the address is a numeral (between '0' and '9') it is treated as a numeric IP address; otherwise it's treated as a host name.

Used for: Networking.

virtual void ILTCSBase::SetModelAnimation( [HOBJECT](#) *hObj*,  
[HMODELANIM](#) *hAnim*  
) [pure virtual]

**Parameters:**

*hObj* The model whose animation you want to set

*hAnim* The animation you want to play

Set the current animation of the object, if it is a model.

Used for: Models and Animation.

virtual void ILTCSBase::SetModelLooping( [HOBJECT](#) *hObj*,  
bool *bLoop*  
) [pure virtual]

**Parameters:**

*hObj* ???

*bLoop* ???

Set the looping state of the model. The default state is **TRUE**.

Used for: Models and Animation.

virtual [ILTSoundMgr](#)\* ILTCSBase::SoundMgr( ) [pure virtual]

**Returns:**

Pointer to the [ILTSoundMgr](#) interface.

Accesses the [ILTSoundMgr](#) interface.

Used for: Interface.

virtual [LTRESULT](#) ILTCSBase::StartPing( const char \* *pAddr*,  
[uint16](#) *nPort*,  
[uint32](#) \* *pPingID*

) [pure virtual]

**Parameters:**

*pAddr*     IP address to target; can be numeric or host name.  
*nPort*     Port to use  
*pPingID*   Resulting Ping ID

**Returns:**

**LT\_NOTINITIALIZED** if there is no TCP/IP driver or the driver's socket is uninitialized, or **LT\_ERROR** if the address cannot be built; otherwise, returns **LT\_OK**.

Start a ping request to a TCP/IP address

Used for: Networking

Field Documentation

[LTRESULT](#)(\* [ILTCBase::GetTcpIpAddress](#))(char \*sAddress, [uint32](#) dwBufferSize, [uint16](#) &hostPort)

**Parameters:**

*sAddress*     (return) buffer to receive local IP address string in dot notation format (NNN.NNN.NNN.NNN) followed by a null byte.  
*dwBufferSize* size of sAddress buffer (this should be at least 17 bytes to ensure room for the dot notation string and a null byte) if there is not enough room the string will be truncated to (dwBufferSize - 1) bytes.  
*hostPort*     (return) filled in with host port

**Returns:**

**LT\_INVALIDPARAMS** if sAddress is null, or **LT\_NOTINITIALIZED** if there is no main driver; otherwise returns **LT\_OK**.

Get the TCP/IP address of the main driver, if it is available. If you are hosting a game, **hostPort** is filled in with the port you are hosting on. If you are not hosting, it is set to 0xFFFF.

**Note:** dwBufferSize must be greater than zero bytes and less than or equal to the size of the space allocated for sAddress. This is not checked by the current code in the engine, and it will fault if you pass in zero bytes.

Used for: Networking.

The documentation for this class was generated from the following file:

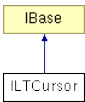
- [ilcsbase.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
 2006, All Rights Reserved.

API Reference

ILTCursor Class Reference

Inheritance diagram for ILTCursor:



Public Types

enum { [\\_ILTCursor\\_VERSION](#) = 0 }

Public Member Functions

virtual [LTRESULT SetCursorMode](#) ([CursorMode](#) cMode, bool bForce=false)=0  
 virtual [LTRESULT GetCursorMode](#) ([CursorMode](#) &cMode)=0  
 virtual [LTRESULT IsCursorModeAvailable](#) ([CursorMode](#) cMode)=0  
 virtual [LTRESULT LoadCursorBitmapResource](#) (const char \*pName, [HLTCursor](#) &hCursor)=0  
 virtual [LTRESULT FreeCursor](#) (const [HLTCursor](#) hCursor)=0  
 virtual [LTRESULT SetCursor](#) ([HLTCursor](#) hCursor)=0  
 virtual [LTRESULT IsValidCursor](#) ([HLTCursor](#) hCursor)=0  
 virtual [LTRESULT RefreshCursor](#) ()=0

Detailed Description

The ILTCursor interface provides functions to manipulate the cursor.

Define a holder to get this interface like this:

[define\\_holder](#)([ILTCursor](#), your\_var);

Member Enumeration Documentation

anonymous enum

**Enumeration values:**

*ILTCursor* *VERSION*

---

## Member Function Documentation

virtual [LTRESULT](#) ILTCursor::FreeCursor( const [HLTCursor](#) *hCursor* ) [pure virtual]

**Parameters:**

*hCursor* The handle to the cursor that was originally created using LoadCursorBitmapResource

**Returns:**

Always returns **LT\_OK**.

Free a cursor.

Used for: Input.

virtual [LTRESULT](#) ILTCursor::GetCursorMode( [CursorMode](#) & *cMode* ) [pure virtual]

**Parameters:**

*cMode* A reference to the CursorMode type of this cursor.

**Returns:**

Always returns **LT\_OK**.

Get current cursor mode. Always fills in **cMode**.

Used for: Input.

virtual [LTRESULT](#) ILTCursor::IsCursorModeAvailable( [CursorMode](#) *cMode* ) [pure virtual]

**Parameters:**

*cMode* Type of cursor mode to set (CM\_None or CM\_Hardware)

**Returns:**

Should return **LT\_YES** if a hardware cursor can be used, **LT\_NO** otherwise. Since we can't detect this, it just returns **LT\_YES** for now.

Used for: Input.

virtual [LTRESULT](#) ILTCursor::IsValidCursor( [HLTCursor](#) *hCursor* ) [pure virtual]

**Parameters:**

*hCursor* A handle to a cursor that was originally created using LoadCursorBitmapResource

**Returns:**

**LT\_YES** when the **HLTCursor** is valid; otherwise, returns **LT\_NO**.

Checks if an **HLTCursor** is a valid one.

Used for: Input.

virtual [LTRESULT](#) ILTCursor::LoadCursorBitmapResource( const char \* *pName*,  
[HLTCursor](#) & *hCursor*  
) [pure virtual]

**Parameters:**

*pName* The name of the bitmap file to open.

*hCursor* A reference to the HCURSOR handle to the cursor to be created

**Returns:**

Should return **LT\_OK** if the bitmap file could be opened, **LT\_NOTINITIALIZED** if the engine has not been initialised.

Set the current hardware cursor bitmap. The bitmap comes from cshell.dll. Used for: Input.

virtual [LTRESULT](#) ILTCursor::RefreshCursor( ) [pure virtual]

**Returns:**

Always returns **LT\_OK**.

Refreshes the cursor (displays it if type CM\_Hardware or hides it if CM\_None).

Used for: Input.

virtual [LTRESULT](#) ILTCursor::SetCursor( [HLTCursor](#) *hCursor* ) [pure virtual]

**Parameters:**

*hCursor* A handle to a cursor that was originally created using LoadCursorBitmapResource

**Returns:**

Returns **LT\_OK** if the cursor was set to the hCursor handle passed in, **LT\_INVALIDPARAMS** if the hCursor handle passed in was invalid.

Set the current cursor.

Used for: Input.

virtual [LTRESULT](#) ILTCursor::SetCursorMode( [CursorMode](#) *cMode*,

```
bool bForce = false
) [pure virtual]
```

**Parameters:**

*cMode* Type of cursor mode to set (CM\_None or CM\_Hardware)  
*bForce* true if the cursor mode should be set whether or not we think we're already in this mode

**Returns:**

An LTRESULT specifying if the cursor mode was correctly set **LT\_OK** on success or **LT\_UNSUPPORTED** on failure)

Enable/disable hardware cursor.

Used for: Input.

The documentation for this class was generated from the following file:

- [ilteursor.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

## ILTCursorInst Class Reference

### Public Member Functions

virtual [LTRESULT IsValid](#) ()=0  
virtual void [SetData](#) (const void \*pData)=0  
virtual void \* [GetData](#) ()=0

### Member Function Documentation

virtual void\* ILTCursorInst::GetData( ) [pure virtual]

**Returns:**

A pointer to the user data that was set using SetData

Get cursor data.

**See also:**

[SetData\(\)](#)

Used for: Input.

virtual [LTRESULT ILTCursorInst::IsValid](#)( ) [pure virtual]

**Returns:**

An LTRESULT specifying if the cursor is valid **LT\_YES** if valid or **LT\_NO** if invalid

Check for valid cursor.

Used for: Input.

virtual void ILTCursorInst::SetData( const void \* *pData* ) [pure virtual]

**Parameters:**

*pData* Address of user data to set the cursor to.

Set user data.

**See also:**

[GetData\(\)](#)

Used for: Input.

The documentation for this class was generated from the following file:

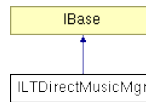
- [ilteursor.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

## ILTDirectMusicMgr Class Reference

Inheritance diagram for ILTDirectMusicMgr:



### Public Types

```
enum { ILTDirectMusicMgr\_VERSION = 0 }
```

### Public Member Functions

```
virtual LTRESULT Init ()=0
virtual LTRESULT Term ()=0
virtual LTRESULT InitLevel (const char *sWorkingDirectory, const char *sControlFileName, const char *sDefine1=NULL, const char *sDefine2=NULL, const char *sDefine3=NULL)=0
virtual LTRESULT TermLevel ()=0
virtual LTRESULT Play ()=0
virtual LTRESULT Stop (const LTDMEnactTypes nStart=LTDMEnactDefault)=0
virtual LTRESULT Pause (const LTDMEnactTypes nStart=LTDMEnactDefault)=0
virtual LTRESULT UnPause ()=0
virtual LTRESULT SetVolume (const long nVolume)=0
virtual LTRESULT ChangeIntensity (const int nNewIntensity, const LTDMEnactTypes nStart=LTDMEnactInvalid)=0
virtual LTRESULT PlaySecondary (const char *sSecondarySegment, const LTDMEnactTypes nStart=LTDMEnactDefault)=0
virtual LTRESULT StopSecondary (const char *sSecondarySegment=NULL, const LTDMEnactTypes nStart=LTDMEnactDefault)=0
virtual LTRESULT PlayMotif (const char *sStyleName, const char *sMotifName, const LTDMEnactTypes nStart=LTDMEnactDefault)=0
virtual LTRESULT StopMotif (const char *sStyleName, const char *sMotifName=NULL, const LTDMEnactTypes nStart=LTDMEnactDefault)=0
    virtual int GetCurIntensity ()=0
virtual LTDMEnactTypes StringToEnactType (const char *sName)=0
    virtual void EnactTypeToString (LTDMEnactTypes nType, char *sName)=0
    virtual int GetNumIntensities ()=0
    virtual int GetInitialIntensity ()=0
    virtual int GetInitialVolume ()=0
    virtual int GetVolumeOffset ()=0
```

### Detailed Description

The ILTDirectMusicMgr interface provides functions for using Driect Music.

Define a holder to get this interface like this:

```
define\_holder(ILTDirectMusicMgr, your\_var);
```

### Member Enumeration Documentation

anonymous enum

**Enumeration values:**

[\\_ILTDirectMusicMgr\\_VERSION\\_](#)

### Member Function Documentation

```
virtual LTRESULT ILTDirectMusicMgr::ChangeIntensity( const int nNewIntensity,
                                                    const LTDMEnactTypes nStart = LTDMEnactInvalid
                                                    ) [pure virtual]
```

**Parameters:**

*nNewIntensity* Contains the intensity number to change to. This value must be greater than zero.  
*nStart* This is an optional parameter that determines when the intensity will begin to change.

**Returns:**

LT\_OK if successful or LT\_ERROR if not successful.

Change the current intensity that is being played. If a transition is defined in the control file, the transition will be played before the new intensity starts.

Used for: Music.

```
virtual void ILTDirectMusicMgr::EnactTypeToString( LTDMEnactTypes nType,
                                                  char * sName
                                                  ) [pure virtual]
```

**Parameters:**

*nType* Enact type to convert to a string.  
*sName* Pointer to a string buffer to contain the convert string. This buffer must be at least 10 bytes to contain the largest possible enact value! If an unrecognized enact type is passed in then the string will be set to an empty string. (ie the first character will be )

Convert an enact type to a string. The following conversions are done : LTDMEnactInvalid = "Invalid"

LTDMEnactDefault = "Default"

LTDMEnactImmediately = "Immediate"

LTDMEactNextBeat = "Beat"

LTDMEactNextMeasure = "Measure"

LTDMEactNextGrid = "Grid"

LTDMEactNextSegment = "Segment"

LTDMEactNextMarker = "Marker"

Used for: Music.

virtual int ILTDirectMusicMgr::GetCurIntensity( ) [pure virtual]

**Returns:**

The current intensity that is being played.

Used for: Music.

virtual int ILTDirectMusicMgr::GetInitialIntensity( ) [pure virtual]

**Returns:**

The initial intensity value for this level.

Get the initial intensity for the current level. The return value is not defined if you are currently not in a level.

Used for: Music.

virtual int ILTDirectMusicMgr::GetInitialVolume( ) [pure virtual]

**Returns:**

The initial volume for this level.

Get the initial volume for the current level. The return value is not defined if you are currently not in a level.

Used for: Music.

virtual int ILTDirectMusicMgr::GetNumIntensities( ) [pure virtual]

**Returns:**

The number of intensities defined in the current level.

**Note:** The return value is undefined if you are not currently in a level.

Returns the number of intensities defined in the current level.

Used for: Music.

virtual int ILTDirectMusicMgr::GetVolumeOffset( ) [pure virtual]

**Returns:**

The volume offset for this level.

Get the volume offset for the current level. The return value is not defined if you are currently not in a level.

Used for: Music.

virtual [LTRESULT](#) ILTDirectMusicMgr::Init( ) [pure virtual]

**Returns:**

LT\_OK if successful or LT\_ERROR if not successful.

Initialize the LTDirectMusic interface. This should be called at the start of the program to initialize the interface. It is typically called in the user's implementation of the OnEngineInitialized function.

Used for: Music.

```
virtual LTRESULT ILTDirectMusicMgr::InitLevel( const char * sWorkingDirectory,
                                             const char * sControlFileName,
                                             const char * sDefine1 = NULL,
                                             const char * sDefine2 = NULL,
                                             const char * sDefine3 = NULL
                                             ) [pure virtual]
```

**Parameters:**

*sWorkingDirectory* The name of the default directory containing the music content including the control file.

*sControlFileName* The name of the music control file to read in for this level. If no path is specified it is read from the default directory.

*sDefine1* First optional define parameter which can be used to pass information into the control file. The control file can then use a #ifdef to check for a define. For example you might want to define "HIGH\_QUALITY" or "LOW\_WUALITY" to control the quality level of the music.

*sDefine2* Second optional define parameter.

*sDefine3* Third optional define parameter.

**Returns:**

LT\_OK if successful or LT\_ERROR if not successful.

Called at the start of each level to initialize the music for that level. The directory for all of the music files and the control file is specified in the *sWorkingDirectory* parameter. The name of the control file to use is specified in the *sControlFileName* parameter. Up to three optional defines can also be used to pass information to the control file. These defines are used when loading the control file to optionally include or exclude portions of the file. This function is typically called from the user's implementation of the DoLoadWorld function.

Used for: Music.

```
virtual LTRESULT ILTDirectMusicMgr::Pause( const LTDMEactTypes nStart = LTDMEactDefault ) [pure virtual]
```

**Parameters:**

*nStart* Optional parameter which describes when the music should stop playing.

**Returns:**

LT\_OK if successful or LT\_ERROR if not successful.

Pause music playback.

Used for: Music.

```
virtual LTRESULT ILTDirectMusicMgr::Play( ) [pure virtual]
```

**Returns:**

LT\_OK if successful or LT\_ERROR if not successful.

Called to begin playing music . This function is typically called from the users implementation of the FirstUpdate function. Optionally, a trigger in the level can be used to activate it.

Used for: Music.

```
virtual LTRESULT ILTDirectMusicMgr::PlayMotif( const char *           sStyleName,
                                              const char *           sMotifName,
                                              const LTDMEactTypes nStart = LTDMEactDefault
                                              ) [pure virtual]
```

**Parameters:**

*sStyleName* The name of the style which contains the motif we want to play.

*sMotifName* The name of the motif to play.

*nStart* This parameter defines when, relative to the current primary segment, the motif will begin playing.

**Returns:**

LT\_OK if successful or LT\_ERROR if not successful.

Begin playing a motif.

Used for: Music.

```
virtual LTRESULT ILTDirectMusicMgr::PlaySecondary( const char *           sSecondarySegment,
                                                  const LTDMEactTypes nStart = LTDMEactDefault
                                                  ) [pure virtual]
```

**Parameters:**

*sSecondarySegment* Contains the name of the secondary segment to begin playing.

*nStart* This parameter defines when, relative to the current primary segment, the secondary segment will begin playing.

**Returns:**

LT\_OK if successful or LT\_ERROR if not successful.

Begin playing the secondary segment with the name specified.

Used for: Music.

```
virtual LTRESULT ILTDirectMusicMgr::SetVolume( const long nVolume ) [pure virtual]
```

**Parameters:**

*nVolume* Contains the volume to set. Negative values make the music volume lower and positive values make it higher. The number is in fractions of a decibel so fairly large numbers are needed to cause a noticeable effect.

**Returns:**

LT\_OK if successful or LT\_ERROR if not successful.

Set the current volume adjustment. This function is usually called from a trigger in the level.

Used for: Music.

```
virtual LTRESULT ILTDirectMusicMgr::Stop( const LTDMEactTypes nStart = LTDMEactDefault ) [pure virtual]
```

**Parameters:**

*nStart* Optional parameter which describes when the music should stop playing.

**Returns:**

LT\_OK if successful or LT\_ERROR if not successful.

Called to stop the music from playing. This function is usually called from a trigger in the level.

Used for: Music.

```
virtual LTRESULT ILTDirectMusicMgr::StopMotif( const char *           sStyleName,
                                              const char *           sMotifName = NULL,
                                              const LTDMEactTypes nStart = LTDMEactDefault
                                              ) [pure virtual]
```

**Parameters:**

*sStyleName* The name of the style which contains the motif we want to stop. If this name is NULL all styles will be checked.

*sMotifName* The name of the motif to stop. If this name is NULL all motifs that are playing will be stopped.

*nStart* This parameter determines when playback will stop.

**Returns:**

LT\_OK if successful or LT\_ERROR if not successful.

Stop all motifs with the specified name in the specified style. If the Style name is NULL then all styles will be searched. If the Motif names is NULL all motifs will be stopped.

Used for: Music.

```
virtual LTRESULT ILTDirectMusicMgr::StopSecondary( const char *          sSecondarySegment = NULL,
                                                  const LTDMEnactTypes nStart = LTDMEnactDefault
                                                  ) [pure virtual]
```

**Parameters:**

*sSecondarySegment* Contains the name of the secondary segment to stop playing. If the name is NULL then all secondary segments that are currently playing will stop.

*nStart* This parameter determines when playback will stop.

**Returns:**

LT\_OK if successful or LT\_ERROR if not successful.

Stop playing all secondary segments with the name specified. If the name is NULL, then stop playing all secondary segments.

Used for: Music.

```
virtual LTDMEnactTypes ILTDirectMusicMgr::StringToEnactType( const char * sName ) [pure virtual]
```

**Parameters:**

*sName* Contains the string name to convert to an enact type.

**Returns:**

The enact type which is represented by the string that was passed in. If NULL is passed in, or an unrecognized string is passed in then LTDMEnactInvalid is returned.

Convert a string to an enact type. This is a very useful function to use when converting user strings in triggers into enact types to pass into the LTDirectMusic API. The following conversions are done :

"Invalid" = LTDMEnactInvalid

"Default" = LTDMEnactDefault

"Immediately" = LTDMEnactImmediately

"Immediate" = return LTDMEnactImmediately

"NextBeat" = return LTDMEnactNextBeat

"NextMeasure" = return LTDMEnactNextMeasure

"NextGrid" = return LTDMEnactNextGrid

"NextSegment" = return LTDMEnactNextSegment

"NextMarker" = return LTDMEnactNextMarker

"Beat" = return LTDMEnactNextBeat

"Measure" = return LTDMEnactNextMeasure

"Grid" = return LTDMEnactNextGrid

"Segment" = return LTDMEnactNextSegment

"Marker" = return LTDMEnactNextMarker

Used for: Music.

```
virtual LTRESULT ILTDirectMusicMgr::Term( ) [pure virtual]
```

**Returns:**

Always returns LT\_OK.

Terminate the LTDirectMusic interface. This should be called at the end of the program to clean up and terminate the interface. It is typically called in the user's implementation of the OnEngineTerm function.

Used for: Music.

```
virtual LTRESULT ILTDirectMusicMgr::TermLevel( ) [pure virtual]
```

**Returns:**

Always returns LT\_OK.

Called at the end of each level to terminate the music for that level. This function is typically called from the users implementation of the OnExitWorld function.

Used for: Music.

```
virtual LTRESULT ILTDirectMusicMgr::UnPause( ) [pure virtual]
```

**Returns:**

LT\_OK if successful or LT\_ERROR if not successful.

Resume music playback after a pause.

Used for: Music.

The documentation for this class was generated from the following file:

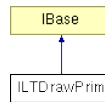
- [iltdirectmusic.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
 2006, All Rights Reserved.

## API Reference

## ILTDrawPrim Class Reference

Inheritance diagram for ILTDrawPrim:



### Public Types

```
enum { ILTDrawPrim\_VERSION = 0 }
```

### Public Member Functions

```

virtual LTRESULT BeginDrawPrim ()=0
virtual LTRESULT EndDrawPrim ()=0
virtual LTRESULT SetCamera (const HOBJECT hCamera)=0
virtual LTRESULT SetTexture (const HTEXTURE hTexture)=0
virtual LTRESULT SetTransformType (const ELTTransformType eType)=0
virtual LTRESULT SetColorOp (const ELTColorOp eColorOp)=0
virtual LTRESULT SetAlphaBlendMode (const ELTBlendMode eBlendMode)=0
virtual LTRESULT SetZBufferMode (const ELTZBufferMode eZBufferMode)=0
virtual LTRESULT SetAlphaTestMode (const ELTTestMode eTestMode)=0
virtual LTRESULT SetClipMode (const ELTClipMode eClipMode)=0
virtual LTRESULT SetFillMode (ELTDPFillMode efillmode)=0
virtual LTRESULT SetCullMode (ELTDPCullMode ecullmode)=0
virtual LTRESULT SetFogEnable (bool bFogEnable)=0
virtual LTRESULT SetReallyClose (bool bReallyClose)=0
virtual LTRESULT SetEffectShaderID (uint32 nEffectShaderID)=0
virtual void SaveViewport ()=0
virtual void RestoreViewport ()=0

virtual LTRESULT DrawPrim (LT\_POLYGT3 *pPrim, const uint32 nCount=1)=0
virtual LTRESULT DrawPrim (LT\_POLYFT3 *pPrim, const uint32 nCount=1)=0
virtual LTRESULT DrawPrim (LT\_POLYG3 *pPrim, const uint32 nCount=1)=0
virtual LTRESULT DrawPrim (LT\_POLYF3 *pPrim, const uint32 nCount=1)=0
virtual LTRESULT DrawPrim (LT\_POLYGT4 *pPrim, const uint32 nCount=1)=0
virtual LTRESULT DrawPrim (LT\_POLYGT4 *ppPrim, const uint32 nCount=1)=0
virtual LTRESULT DrawPrim (LT\_POLYFT4 *pPrim, const uint32 nCount=1)=0
virtual LTRESULT DrawPrim (LT\_POLYG4 *pPrim, const uint32 nCount=1)=0
virtual LTRESULT DrawPrim (LT\_POLYF4 *pPrim, const uint32 nCount=1)=0
virtual LTRESULT DrawPrim (LT\_LINEGT *pPrim, const uint32 nCount=1)=0
virtual LTRESULT DrawPrim (LT\_LINEFT *pPrim, const uint32 nCount=1)=0
virtual LTRESULT DrawPrim (LT\_LINEG *pPrim, const uint32 nCount=1)=0
virtual LTRESULT DrawPrim (LT\_LINEF *pPrim, const uint32 nCount=1)=0
virtual LTRESULT DrawPrimPoint (LT\_VERTGT *pVerts, const uint32 nCount=1)=0
virtual LTRESULT DrawPrimPoint (LT\_VERTG *pVerts, const uint32 nCount=1)=0
virtual LTRESULT DrawPrimFan (LT\_VERTGT *pVerts, const uint32 nCount)=0
virtual LTRESULT DrawPrimFan (LT\_VERTFT *pVerts, const uint32 nCount, LT\_VERTRGBA rgba)=0
virtual LTRESULT DrawPrimFan (LT\_VERTG *pVerts, const uint32 nCount)=0
virtual LTRESULT DrawPrimFan (LT\_VERTF *pVerts, const uint32 nCount, LT\_VERTRGBA rgba)=0
virtual LTRESULT DrawPrimStrip (LT\_VERTGT *pVerts, const uint32 nCount)=0
virtual LTRESULT DrawPrimStrip (LT\_VERTFT *pVerts, const uint32 nCount, LT\_VERTRGBA rgba)=0
virtual LTRESULT DrawPrimStrip (LT\_VERTG *pVerts, const uint32 nCount)=0
virtual LTRESULT DrawPrimStrip (LT\_VERTF *pVerts, const uint32 nCount, LT\_VERTRGBA rgba)=0

virtual void SetXY4 (LT\_POLYGT4 *pPrim, float x0, float y0, float x1, float y1, float x2, float y2, float x3, float y3)=0
virtual void SetXY4 (LT\_POLYFT4 *pPrim, float x0, float y0, float x1, float y1, float x2, float y2, float x3, float y3)=0
virtual void SetXY4 (LT\_POLYG4 *pPrim, float x0, float y0, float x1, float y1, float x2, float y2, float x3, float y3)=0
virtual void SetXY4 (LT\_POLYF4 *pPrim, float x0, float y0, float x1, float y1, float x2, float y2, float x3, float y3)=0
virtual void SetXYWH (LT\_POLYGT4 *pPrim, float x, float y, float w, float h)=0
virtual void SetXYWH (LT\_POLYFT4 *pPrim, float x, float y, float w, float h)=0
virtual void SetXYWH (LT\_POLYG4 *pPrim, float x, float y, float w, float h)=0
virtual void SetXYWH (LT\_POLYF4 *pPrim, float x, float y, float w, float h)=0
virtual void SetUV4 (LT\_POLYGT4 *pPrim, float u0, float v0, float u1, float v1, float u2, float v2, float u3, float v3)=0
virtual void SetUV4 (LT\_POLYFT4 *pPrim, float u0, float v0, float u1, float v1, float u2, float v2, float u3, float v3)=0
virtual void SetUVWH (LT\_POLYGT4 *pPrim, float u, float v, float w, float h)=0
virtual void SetUVWH (LT\_POLYFT4 *pPrim, HTEXTURE pTex, float u, float v, float w, float h)=0
virtual void SetUVWH (LT\_POLYF4 *pPrim, float u, float v, float w, float h)=0
  
```

```

virtual void SetRGB (LT\_POLYGT4 *pPrim, uint32 color)=0
virtual void SetRGB (LT\_POLYFT4 *pPrim, uint32 color)=0
virtual void SetRGB (LT\_POLYG4 *pPrim, uint32 color)=0
virtual void SetRGB (LT\_POLYF4 *pPrim, uint32 color)=0
virtual void SetRGBA (LT\_POLYGT4 *pPrim, uint32 color)=0
virtual void SetRGBA (LT\_POLYFT4 *pPrim, uint32 color)=0
virtual void SetRGBA (LT\_POLYG4 *pPrim, uint32 color)=0
virtual void SetRGBA (LT\_POLYF4 *pPrim, uint32 color)=0
virtual void SetRGB4 (LT\_POLYGT4 *pPrim, uint32 color0, uint32 color1, uint32 color2, uint32 color3)=0
virtual void SetRGB4 (LT\_POLYG4 *pPrim, uint32 color0, uint32 color1, uint32 color2, uint32 color3)=0
virtual void SetRGBA4 (LT\_POLYGT4 *pPrim, uint32 color0, uint32 color1, uint32 color2, uint32 color3)=0
virtual void SetRGBA4 (LT\_POLYG4 *pPrim, uint32 color0, uint32 color1, uint32 color2, uint32 color3)=0
virtual void SetALPHA (LT\_POLYGT4 *pPrim, uint8 alpha)=0
virtual void SetALPHA (LT\_POLYG4 *pPrim, uint8 alpha)=0
virtual void SetALPHA (LT\_POLYFT4 *pPrim, uint8 alpha)=0
virtual void SetALPHA (LT\_POLYF4 *pPrim, uint8 alpha)=0
virtual void SetALPHA4 (LT\_POLYGT4 *pPrim, uint8 alpha0, uint8 alpha1, uint8 alpha2, uint8 alpha3)=0
virtual void SetALPHA4 (LT\_POLYG4 *pPrim, uint8 alpha0, uint8 alpha1, uint8 alpha2, uint8 alpha3)=0

```

---

## Detailed Description

The ILTDrawPrim is a low level polygon based drawing/rendering interface.

Define a holder to get this interface like this:

```
define\_holder(ILTDrawPrim, your_var);
```

---

## Member Enumeration Documentation

anonymous enum

**Enumeration values:**

[\\_ILTDrawPrim\\_VERSION\\_](#)

---

## Member Function Documentation

virtual [LTRESULT](#) ILTDrawPrim::BeginDrawPrim( ) [pure virtual]

Called to begin a draw primitive block. This is for optimizing draw primitive calls. Calls to have the engine render anything not through the draw primitive cannot be made inside this block.

Used for: Rendering.

virtual [LTRESULT](#) ILTDrawPrim::DrawPrim( [LT\\_LINEF](#) \* pPrim,  
const [uint32](#) nCount = 1  
) [pure virtual]

virtual [LTRESULT](#) ILTDrawPrim::DrawPrim( [LT\\_LINEG](#) \* pPrim,  
const [uint32](#) nCount = 1  
) [pure virtual]

virtual [LTRESULT](#) ILTDrawPrim::DrawPrim( [LT\\_LINEFT](#) \* pPrim,  
const [uint32](#) nCount = 1  
) [pure virtual]

virtual [LTRESULT](#) ILTDrawPrim::DrawPrim( [LT\\_LINEGT](#) \* pPrim,  
const [uint32](#) nCount = 1  
) [pure virtual]

virtual [LTRESULT](#) ILTDrawPrim::DrawPrim( [LT\\_POLYF4](#) \* pPrim,  
const [uint32](#) nCount = 1  
) [pure virtual]

virtual [LTRESULT](#) ILTDrawPrim::DrawPrim( [LT\\_POLYG4](#) \* pPrim,  
const [uint32](#) nCount = 1  
) [pure virtual]

virtual [LTRESULT](#) ILTDrawPrim::DrawPrim( [LT\\_POLYFT4](#) \* pPrim,  
const [uint32](#) nCount = 1  
) [pure virtual]

virtual [LTRESULT](#) ILTDrawPrim::DrawPrim( [LT\\_POLYGT4](#) \*\* ppPrim,  
const [uint32](#) nCount = 1

```
) [pure virtual]
```

```
virtual LTRESULT ILTDrawPrim::DrawPrim( LT\_POLYGT4 * pPrim,  
                                         const uint32   nCount = 1  
                                         ) [pure virtual]
```

```
virtual LTRESULT ILTDrawPrim::DrawPrim( LT\_POLYF3 * pPrim,  
                                         const uint32   nCount = 1  
                                         ) [pure virtual]
```

```
virtual LTRESULT ILTDrawPrim::DrawPrim( LT\_POLYG3 * pPrim,  
                                         const uint32   nCount = 1  
                                         ) [pure virtual]
```

```
virtual LTRESULT ILTDrawPrim::DrawPrim( LT\_POLYFT3 * pPrim,  
                                         const uint32   nCount = 1  
                                         ) [pure virtual]
```

```
virtual LTRESULT ILTDrawPrim::DrawPrim( LT\_POLYGT3 * pPrim,  
                                         const uint32   nCount = 1  
                                         ) [pure virtual]
```

#### Parameters:

*pPrim* A pointer to an array of polygon structures to draw.

*nCount* The number of polygons (default value is 1) in the array identified by the **pPrim** parameter.

Draw one or more polygons of one type (line, triangle, or quadrilateral). This overloaded function supports the following structures: [LT\\_LINEF](#), [LT\\_LINEFT](#), [LT\\_LINEG](#), [LT\\_LINEGT](#), [LT\\_POLYF3](#), [LT\\_POLYFT3](#), [LT\\_POLYG3](#), [LT\\_POLYGT3](#), [LT\\_POLYF4](#), [LT\\_POLYFT4](#), [LT\\_POLYG4](#), and [LT\\_POLYGT4](#).

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::DrawPrimFan( LT\_VERTF *   pVerts,  
                                           const uint32   nCount,  
                                           LT\_VERTRGBA rgba  
                                           ) [pure virtual]
```

```
virtual LTRESULT ILTDrawPrim::DrawPrimFan( LT\_VERTG * pVerts,  
                                           const uint32   nCount  
                                           ) [pure virtual]
```

```
virtual LTRESULT ILTDrawPrim::DrawPrimFan( LT\_VERTFT * pVerts,  
                                           const uint32   nCount,  
                                           LT\_VERTRGBA rgba  
                                           ) [pure virtual]
```

```
virtual LTRESULT ILTDrawPrim::DrawPrimFan( LT\_VERTGT * pVerts,  
                                           const uint32   nCount  
                                           ) [pure virtual]
```

#### Parameters:

*pVerts* A pointer to an array structures that identify the vertices of the fan. Three or more vertices are required. The first vertex structure in this array defines the source vertex for the fan. This overloaded functions supports the following vertex structures: [LT\\_VERTF](#), [LT\\_VERTFT](#), [LT\\_VERTG](#), and [LT\\_VERTGT](#).

*nCount* The number of vertex structures in the array identified by the **pVerts** parameter.

*rgba* The color for flat shaded fans.

Draw a primitive triangle fan. Each vertex in the array creates a triangle with the previous vertex in the array and the source vertex.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::DrawPrimPoint( LT\_VERTG * pVerts,  
                                           const uint32   nCount = 1  
                                           ) [pure virtual]
```

```
virtual LTRESULT ILTDrawPrim::DrawPrimPoint( LT\_VERTGT * pVerts,  
                                           const uint32   nCount = 1  
                                           ) [pure virtual]
```

```
virtual LTRESULT ILTDrawPrim::DrawPrimStrip( LT\_VERTF *   pVerts,  
                                           const uint32   nCount,  
                                           LT\_VERTRGBA rgba  
                                           ) [pure virtual]
```

```
virtual LTRESULT ILTDrawPrim::DrawPrimStrip( LT\_VERTG * pVerts,
                                             const uint32 nCount
                                             ) [pure virtual]
```

```
virtual LTRESULT ILTDrawPrim::DrawPrimStrip( LT\_VERTFT * pVerts,
                                             const uint32 nCount,
                                             LT\_VERTRGBA rgba
                                             ) [pure virtual]
```

```
virtual LTRESULT ILTDrawPrim::DrawPrimStrip( LT\_VERTGT * pVerts,
                                             const uint32 nCount
                                             ) [pure virtual]
```

**Parameters:**

*pVerts* A pointer to an array of structures that identify the vertices of the strip. Three or more vertices are required. The first vertex structure in this array defines the source vertex for the strip. This overloaded functions supports the following vertex structures: [LT\\_VERTF](#), [LT\\_VERTFT](#), [LT\\_VERTG](#), and [LT\\_VERTGT](#).

*nCount* The number of vertex structures in the array identified by the **pVerts** parameter.

*rgba* The color for flat shaded strips.

Draw a primitive triangle strip. Each vertex in the array creates a triangle with the previous two vertices in the array.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::EndDrawPrim( ) [pure virtual]
Called to end a draw primitive block.
```

Used for: Rendering.

```
virtual void ILTDrawPrim::RestoreViewport( ) [pure virtual]
Restore the saved viewport
```

Used for: Rendering.

```
virtual void ILTDrawPrim::SaveViewport( ) [pure virtual]
Store off the current viweport
```

Used for: Rendering.

```
virtual void ILTDrawPrim::SetALPHA( LT\_POLYF4 * pPrim,
                                   uint8 alpha
                                   ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetALPHA( LT\_POLYFT4 * pPrim,
                                   uint8 alpha
                                   ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetALPHA( LT\_POLYG4 * pPrim,
                                   uint8 alpha
                                   ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetALPHA( LT\_POLYGT4 * pPrim,
                                   uint8 alpha
                                   ) [pure virtual]
```

**Parameters:**

*pPrim* (return) A pointer to the polygon structure for which the color data is set. This overloaded function supports the [LT\\_POLYF4](#), [LT\\_POLYFT4](#), [LT\\_POLYG4](#) and [LT\\_POLYGT4](#) structures.

*alpha* The alpha value for the entire quadrilateral.

Set the alpha value for a quadrilateral.

Used for: Rendering.

```
virtual void ILTDrawPrim::SetALPHA4( LT\_POLYG4 * pPrim,
                                   uint8 alpha0,
                                   uint8 alpha1,
                                   uint8 alpha2,
                                   uint8 alpha3
                                   ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetALPHA4( LT\_POLYGT4 * pPrim,
                                   uint8 alpha0,
                                   uint8 alpha1,
```

```

        uint8      alpha2,
        uint8      alpha3
    ) [pure virtual]

```

**Parameters:**

*pPrim* (return) A pointer to the polygon structure for which the alpha value is set. This overloaded function supports the [LT\\_POLYG4](#) and [LT\\_POLYGT4](#) structures.

*alpha0* The alpha value of the first (upper left) vertex of the quadrilateral.

*alpha1* The alpha value of the second vertex of the quadrilateral.

*alpha2* The alpha value of the third vertex of the quadrilateral.

*alpha3* The alpha value of the fourth vertex of the quadrilateral.

Set the alpha value for each vertex in a quadrilateral.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::SetAlphaBlendMode( const ELTBlendMode eBlendMode ) [pure virtual]
```

**Parameters:**

*eBlendMode* An **ELTBlendMode** enum value.

Set the alpha blend mode of the primitive.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::SetAlphaTestMode( const ELTTestMode eTestMode ) [pure virtual]
```

**Parameters:**

*eTestMode* An **ELTTestMode** enum value.

Set the alpha test mode for drawing primitives.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::SetCamera( const HOBJECT hCamera ) [pure virtual]
```

**Parameters:**

*hCamera* The handle of the camera.

Set the camera relative to which the primitive is drawn.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::SetClipMode( const ELTClipMode eClipMode ) [pure virtual]
```

**Parameters:**

*eClipMode* An **ELTClipMode** enum value.

Set the clip mode for drawing primitives.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::SetColorOp( const ELTColorOp eColorOp ) [pure virtual]
```

**Parameters:**

*eColorOp* An **ELTColorOp** enum value.

Set the texture-to-color behavior of the primitive.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::SetCullMode( ELTDPCullMode ecullmode ) [pure virtual]
```

**Parameters:**

*ecullmode* An **ELTDPCullMode** enum value.

Set the cull mode for drawing primitives.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::SetEffectShaderID( uint32 nEffectShaderID ) [pure virtual]
```

**Parameters:**

*nEffectShaderID* - Specifies the effect shader ID.

Specify the Effect Shader to use when rendering this prim.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::SetFillMode( ELTDPFillMode efillmode ) [pure virtual]
```

**Parameters:**

*efillmode* An **ELTDPFillMode** enum value.

Set the fill mode for drawing primitives.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::SetFogEnable( bool bFogEnable ) [pure virtual]
```

**Parameters:**

*bFogEnable* An **boolean** indicating whether or not fog should be enabled

Set the fog enabled status for drawing primitives.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::SetReallyClose( bool bReallyClose ) [pure virtual]
```

**Parameters:**

*bReallyClose* - Specifies whether or not the draw primitive will be rendered in the really close range.

Specify whether or not to be in really close space for rendering

Used for: Rendering.

```
virtual void ILTDrawPrim::SetRGB( LT\_POLYF4 * pPrim,
                                uint32      color
                                ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetRGB( LT\_POLYG4 * pPrim,
                                uint32      color
                                ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetRGB( LT\_POLYFT4 * pPrim,
                                uint32      color
                                ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetRGB( LT\_POLYGT4 * pPrim,
                                uint32      color
                                ) [pure virtual]
```

**Parameters:**

*pPrim* (return) A pointer to the polygon structure for which the color data is set. This overloaded function supports the [LT\\_POLYF4](#), [LT\\_POLYG4](#), [LT\\_POLYGT4](#) and [LT\\_POLYFT4](#) structures.

*color* The RGB color of all vertices in the quadrilateral.

Set a single RGB color for all vertices in a quadrilateral structure.

Used for: Rendering.

```
virtual void ILTDrawPrim::SetRGB4( LT\_POLYG4 * pPrim,
                                uint32      color0,
                                uint32      color1,
                                uint32      color2,
                                uint32      color3
                                ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetRGB4( LT\_POLYGT4 * pPrim,
                                uint32      color0,
                                uint32      color1,
                                uint32      color2,
                                uint32      color3
                                ) [pure virtual]
```

**Parameters:**

*pPrim* (return) A pointer to the polygon structure for which the color data is set. This overloaded function supports the [LT\\_POLYG4](#) and [LT\\_POLYGT4](#) structures.

*color0* The RGB color of the first (upper left) vertex of the quadrilateral.

*color1* The RGB color of the second vertex of the quadrilateral.

*color2* The RGB color of the third vertex of the quadrilateral.

*color3* The RGB color of the fourth vertex of the quadrilateral.

Set the RGB color for each vertex in a quadrilateral.

Used for: Rendering.

```
virtual void ILTDrawPrim::SetRGBA( LT\_POLYF4 * pPrim,
                                uint32      color
                                ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetRGBA( LT\_POLYG4 * pPrim,
                                uint32      color
                                ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetRGBA( LT\_POLYFT4 * pPrim,
                                uint32      color
                                ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetRGBA( LT\_POLYGT4 * pPrim,
                                   uint32      color
                                   ) [pure virtual]
```

**Parameters:**

*pPrim* (return) A pointer to the polygon structure for which the color data is set. This overloaded function supports the [LT\\_POLYF4](#), [LT\\_POLYG4](#), [LT\\_POLYGT4](#) and [LT\\_POLYFT4](#) structures.

*color* The RGBA color of all vertices in the quadrilateral.

Set a single RGBA color for all vertices in a quadrilateral structure.

Used for: Rendering.

```
virtual void ILTDrawPrim::SetRGBA4( LT\_POLYG4 * pPrim,
                                     uint32      color0,
                                     uint32      color1,
                                     uint32      color2,
                                     uint32      color3
                                     ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetRGBA4( LT\_POLYGT4 * pPrim,
                                     uint32      color0,
                                     uint32      color1,
                                     uint32      color2,
                                     uint32      color3
                                     ) [pure virtual]
```

**Parameters:**

*pPrim* (return) A pointer to the polygon structure for which the color data is set. This overloaded function supports the [LT\\_POLYG4](#) and [LT\\_POLYGT4](#) structures.

*color0* The RGBA color of the first (upper left) vertex of the quadrilateral.

*color1* The RGBA color of the second vertex of the quadrilateral.

*color2* The RGBA color of the third vertex of the quadrilateral.

*color3* The RGBA color of the fourth vertex of the quadrilateral.

Set the RGBA color for each vertex in a quadrilateral.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::SetTexture( const HTEXTURE hTexture ) [pure virtual]
```

**Parameters:**

*hTexture* The handle of the texture.

Set the texture used by the primitive. If a texture is not set using this function, all textured polygons will be rendered with random texture data.

Used for: Rendering.

```
virtual LTRESULT ILTDrawPrim::SetTransformType( const ELTTransformType eType ) [pure virtual]
```

**Parameters:**

*eType* An [ELTTransformType](#) enum value.

Set the transform type in which the primitive is drawn.

Used for: Rendering.

```
virtual void ILTDrawPrim::SetUV4( LT\_POLYFT4 * pPrim,
                                   float          u0,
                                   float          v0,
                                   float          u1,
                                   float          v1,
                                   float          u2,
                                   float          v2,
                                   float          u3,
                                   float          v3
                                   ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetUV4( LT\_POLYGT4 * pPrim,
                                   float          u0,
                                   float          v0,
                                   float          u1,
                                   float          v1,
                                   float          u2,
                                   float          v2,
                                   float          u3,
                                   float          v3
                                   ) [pure virtual]
```

**Parameters:**

*pPrim* (return) A pointer to the polygon structure for which the texture data is set. This overloaded function supports the [LT\\_POLYGT4](#) and [LT\\_POLYFT4](#) structures.

*u0* The u texture value for the upper left vertex of the quadrilateral.

*v0* The v texture value for the upper left vertex of the quadrilateral.

*u1*    The u texture value for the second vertex of the quadrilateral.  
*v1*    The v texture value for the second vertex of the quadrilateral.  
*u2*    The u texture value for the third vertex of the quadrilateral.  
*v2*    The v texture value for the third vertex of the quadrilateral.  
*u3*    The u texture value for the fourth vertex of the quadrilateral.  
*v3*    The v texture value for the fourth vertex of the quadrilateral.

Set the texture data for each vertex in a quadrilateral structure.

Used for: Rendering.

```
virtual void ILTDrawPrim::SetUVWH( LT\_POLYFT4 * pPrim,
                                   float      u,
                                   float      v,
                                   float      w,
                                   float      h
                                   ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetUVWH( LT\_POLYGT4 * pPrim,
                                   HTEXTURE   pTex,
                                   float      u,
                                   float      v,
                                   float      w,
                                   float      h
                                   ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetUVWH( LT\_POLYGT4 * pPrim,
                                   float      u,
                                   float      v,
                                   float      w,
                                   float      h
                                   ) [pure virtual]
```

#### Parameters:

*pPrim* (return) A pointer to the polygon structure for which the texture data is set. This overloaded function supports the [LT\\_POLYGT4](#) and [LT\\_POLYFT4](#) structures.  
*u*    The u texture value for the upper left vertex of the quadrilateral.  
*v*    The v texture value for the upper left vertex of the quadrilateral.  
*w*    The width of the texture.  
*h*    The height of the texture.

Set the texture data for each vertex in a quadrilateral structure. The upper left vertex is set explicitly by the **u** and **v** parameters. The other three vertices are calculated using the width and height (**w** and **h** parameters, respectively).

Used for: Rendering.

```
virtual void ILTDrawPrim::SetXY4( LT\_POLYF4 * pPrim,
                                  float      x0,
                                  float      y0,
                                  float      x1,
                                  float      y1,
                                  float      x2,
                                  float      y2,
                                  float      x3,
                                  float      y3
                                  ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetXY4( LT\_POLYG4 * pPrim,
                                  float      x0,
                                  float      y0,
                                  float      x1,
                                  float      y1,
                                  float      x2,
                                  float      y2,
                                  float      x3,
                                  float      y3
                                  ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetXY4( LT\_POLYFT4 * pPrim,
                                  float      x0,
                                  float      y0,
                                  float      x1,
                                  float      y1,
                                  float      x2,
                                  float      y2,
                                  float      x3,
                                  float      y3
                                  ) [pure virtual]
```

```
virtual void ILTDrawPrim::SetXY4( LT\_POLYGT4 * pPrim,
```

```

float      x0,
float      y0,
float      x1,
float      y1,
float      x2,
float      y2,
float      x3,
float      y3
) [pure virtual]

```

**Parameters:**

*pPrim* (return) A pointer to the polygon structure to which the coordinate parameters **x0**, **y0**, **x1**, **y1**, **x2**, **y2**, **x3**, and **y3** are applied. This overloaded function supports the [LT\\_POLYF4](#), [LT\\_POLYFT4](#), [LT\\_POLYG4](#), and [LT\\_POLYGT4](#) structures.

*x0* The x-coordinate of the first vertex of the quadrilateral. All eight of the coordinate parameters are applied to the quadrilateral in winding order.  
*y0* The y-coordinate of the first vertex of the quadrilateral.  
*x1* The x-coordinate of the second vertex of the quadrilateral.  
*y1* The y-coordinate of the second vertex of the quadrilateral.  
*x2* The x-coordinate of the third vertex of the quadrilateral.  
*y2* The y-coordinate of the third vertex of the quadrilateral.  
*x3* The x-coordinate of the fourth vertex of the quadrilateral.  
*y3* The y-coordinate of the fourth vertex of the quadrilateral.

Set vertex coordinates for a primitive quadrilateral.

Used for: Rendering.

```

virtual void ILTDrawPrim::SetXYWH( LT\_POLYF4 * pPrim,
float      x,
float      y,
float      w,
float      h
) [pure virtual]

```

```

virtual void ILTDrawPrim::SetXYWH( LT\_POLYG4 * pPrim,
float      x,
float      y,
float      w,
float      h
) [pure virtual]

```

```

virtual void ILTDrawPrim::SetXYWH( LT\_POLYFT4 * pPrim,
float      x,
float      y,
float      w,
float      h
) [pure virtual]

```

```

virtual void ILTDrawPrim::SetXYWH( LT\_POLYGT4 * pPrim,
float      x,
float      y,
float      w,
float      h
) [pure virtual]

```

**Parameters:**

*pPrim* (return) A pointer to the polygon structure for which the vertices is set.

*x* The x-coordinate of the upper left vertex of the quadrilateral.  
*y* The y-coordinate of the upper left vertex of the quadrilateral.  
*w* The width of the quadrilateral.  
*h* The height the quadrilateral.

Set vertex coordinates for a primitive quadrilateral give a source vertex, width, and height. The x and y coordinates for the upper left vertex are set with the **x** and **y** parameters. The other three vertices are calculated using the width and height (**w** and **h** parameters, respectively).

Used for: Rendering.

```

virtual LTRESULT ILTDrawPrim::SetZBufferMode( const ELTZBufferMode eZBufferMode ) [pure virtual]

```

**Parameters:**

*eZBufferMode* An **ELTZBufferMode** enum value.

Set the z-buffering mode for drawing primitives.

Used for: Rendering.

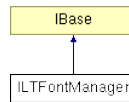
The documentation for this class was generated from the following file:

- [iltdrawprim.h](#)

## API Reference

### ILTFontManager Class Reference

Inheritance diagram for ILTFontManager:



#### Public Types

```
enum { ILTFontManager\_VERSION = 0 }
```

#### Public Member Functions

```

virtual void Init ()=0
virtual void Term ()=0
virtual bool IsInitiated ()=0
virtual CUIFont * CreateFont (HTEXTURE hTex, uint8 charw, uint8 charh, uint8 *pMap=NULL)=0
virtual CUIFont * CreateFont (HTEXTURE hTex, uint16 *pTable, uint8 charh, uint8 *pMap=NULL)=0
virtual CUIFont * CreateFont (HTEXTURE hTex, uint8 *pMap=NULL)=0
virtual CUIFont * CreateFont (char const *pszFontFile, char const *pszFontFace, uint32 pointSize, uint8 asciiStart=33, uint8 asciiEnd=126, LTFontParams *fontParams=NULL)=0
virtual CUIFont * CreateFont (char const *pszFontFile, char const *pszFontFace, uint32 pointSize, char *pCharacters, LTFontParams *fontParams=NULL)=0
virtual void DestroyFont (CUIFont *pFont)=0
virtual uint32 GetNumFontsInUse ()=0
virtual HTEXTURE CharToTexture (uint8 *pTTFFData, uint32 dataSize, uint32 character, uint32 pointSize)=0
virtual HTEXTURE StringToTexture (uint8 *pTTFFData, uint32 dataSize, char *pCharacters, uint32 pointSize)=0
virtual HTEXTURE StringToTexture (uint8 *pTTFFData, uint32 dataSize, uint32 *pCharacters, uint32 pointSize)=0
virtual CUIPolyString * CreatePolyString (CUIFont *pFont, char *pBuffer=NULL, float x=0.0, float y=0.0)=0
virtual CUIFormattedPolyString * CreateFormattedPolyString (CUIFont *pFont, char *pBuffer=NULL, float x=0.0, float y=0.0, CUI\_ALIGNMENTTYPE alignment=CUI\_HALIGN\_LEFT)=0
virtual void DestroyPolyString (CUIPolyString *pPolystr)=0

```

#### Detailed Description

The ILTFontManager interface is used to create and destroy monospace and proportional bitmap fonts, as well as vector (TrueType) fonts, CUIPolyStrings and CUIFormattedPolyStrings.

For bitmap fonts, an HTEXTURE is used to hold the font graphics, and must be 256x256 pixels or smaller.

TrueType font support is compiled into the engine by default. If you do not wish to use TrueType fonts, open up the file **engine** -2.0.1.h and *#undef* the macro FT\_ENABLE\_FREETYPE. This will save you about 80K in code/data size.

ILTFontmanager is available from the Interface Manager.

#### See also:

```

class IBase

class ILTTexInterface

class CUIFont

class CUIPolyString

class CUIFormattedPolyString

```

Used for: Text and UI.

#### Member Enumeration Documentation

anonymous enum

##### Enumeration values:

```
\_ILTFontManager\_VERSION\_
```

#### Member Function Documentation

```

virtual HTEXTURE ILTFontManager::CharToTexture( uint8 * pTTFFData,
                                              uint32  dataSize,
                                              uint32  character,
                                              uint32  pointSize
                                              ) [pure virtual]

```

##### Parameters:

```

pTTFFData  a pointer to a TrueType resource.
dataSize   the size (in bytes) of the resource pointed to by pTTFFData.
character   the character to render (multi-byte characters are supported)

```

*pointSize* the point size at which to render *character*

**Returns:**

HTEXTURE containing *character*.

Use this function to render a single character into an HTEXTURE.

Used for: Text and UI.

```
virtual CUIFont* ILTFontManager::CreateFont( char const *   pszFontFile,
                                           char const *   pszFontFace,
                                           uint32        pointSize,
                                           char *         pCharacters,
                                           LTFontParams* fontParams = NULL
                                           ) [pure virtual]
```

**Parameters:**

*pszFontFile* Filename of TTF.

*pszFontFace* Font face of TTF.

*pointSize* desired point size for the new font.

*characters* a string containing the characters which should be present in the font.

**Returns:**

pointer to a [CUIFont](#) object

This function uses a TrueType font to create an anti-aliased bitmap font. When the documentation refers to Vector Fonts, this is what is meant. All fonts are stored internally as bitmaps, but fonts can be created from TrueType resources.

This method of creating a vector font allows for easy generation of nonstandard fonts (fonts whose characters are not in ascii order)

All vector font creation functions create a font table, and this one creates a font map as well.

**See also:**

CreateFont( char const\*, char const\*, uint32, uint32, uint8, uint8)

[CUIFont::SetMap\(\)](#)

Used for: Text and UI.

```
virtual CUIFont* ILTFontManager::CreateFont( char const *   pszFontFile,
                                           char const *   pszFontFace,
                                           uint32        pointSize,
                                           uint8          asciiStart = 33,
                                           uint8          asciiEnd  = 126,
                                           LTFontParams* fontParams = NULL
                                           ) [pure virtual]
```

**Parameters:**

*pszFontFile* Filename of TTF.

*pszFontFace* Font face of TTF.

*pointSize* desired point size for the new font.

*asciiStart* (optional) specifies the beginning of the range of characters present in the font.

*asciiEnd* (optional) specifies the end of the range of characters present in the font.

**Returns:**

pointer to a [CUIFont](#) object

This function uses a TrueType font to create an anti-aliased bitmap font. When the documentation refers to Vector Fonts, this is what is meant. All fonts are stored internally as bitmaps, but fonts can be created from TrueType resources.

Additionally, when the font is created with this version of [CreateFont\(\)](#), you can optionally specify an ascii range of characters to include in your font image. The default creates a font containing characters from '!' to '~' which gives everything generally needed for North American English. To create a font containing extended ascii characters, choose different values for *asciiStart* and *asciiEnd*, or use CreateFont(uint8\*, uint32, uint32, char\*).

Used for: Text and UI.

```
virtual CUIFont* ILTFontManager::CreateFont( HTEXTURE hTex,
                                           uint8*      pMap = NULL
                                           ) [pure virtual]
```

**Parameters:**

*hTex* HTEXTURE containing a proportional bitmap font.

*pMap* (optional) specifies a character map.

**Returns:**

pointer to a [CUIFont](#) object

Creates a proportional bitmap font. Instead of relying on a user-supplied font table, the system will attempt to scan the provided texture and programmatically determine the extents of each letter. The HTEXTURE parameter must therefore be a specially prepared font graphic. Each letter is demarcated by a single transparent green pixel (ARGB = 0x0000FF00) at its lower right corner. See the images included with the [CUIFont](#) sample project.

**Note:** the HTEXTURE parameter must be a 32-bit texture for this version of [CreateFont\(\)](#).

Used for: Text and UI.

```
virtual CUIFont* ILTFontManager::CreateFont( HTEXTURE hTex,
                                           uint16*      pTable,
                                           uint8          charh,
```

```

        uint8 *      pMap = NULL
    ) [pure virtual]

```

**Parameters:**

*hTex* HTEXTURE containing a proportional bitmap font.  
*pTable* pointer to a font table (an array of uint16).  
*charh* height (in pixels) of each character in the font.  
*pMap* (optional) specifies a character map.

**Returns:**

pointer to a [CUIFont](#) object

Each character in the font is represented by 3 uint16 values in the font table. The first uint16 is the width of the character in pixels, the second and third uint16 are (u,v) texture coordinates of the character within the texture specified by *hTex*.

Used for: Text and UI.

```

virtual CUIFont* ILTFontManager::CreateFont( HTEXTURE hTex,
        uint8      charw,
        uint8      charh,
        uint8 *     pMap = NULL
    ) [pure virtual]

```

**Parameters:**

*hTex* HTEXTURE containing a monospace bitmap font.  
*charw* width (in pixels) of each character in the font.  
*charh* height (in pixels) of each character in the font.  
*pMap* (optional) specifies a character map.

**Returns:**

pointer to a [CUIFont](#) object

Creates a monospace bitmap font. No font table is necessary when creating a monospace font.

Used for: Text and UI.

```

virtual CUIFormattedPolyString* ILTFontManager::CreateFormattedPolyString( CUIFont *      pFont,
        char *      pBuffer = NULL,
        float       x = 0.0,
        float       y = 0.0,
        CUI_ALIGNMENTTYPE alignment = CUI_HALIGN_LEFT
    ) [pure virtual]

```

**Parameters:**

*pFont* pointer to a [CUIFont](#).  
*pBuffer* (optional) a null-terminated string of characters.  
*x* (optional) the x-coordinate on screen.  
*y* (optional) the y-coordinate on screen.  
*alignment* the alignment of the new text.

**Returns:**

pointer to a [CUIFormattedPolyString](#) object

This function creates a formatted (or, format-capable) [CUIPolyString](#). Unlike a regular [CUIPolyString](#), a formatted string understands newlines, word-wrap, justified text, and left-center-right alignment.

**See also:**

[CUIFontManager::CreatePolyString\(\)](#)

[CUIFontManager::DestroyPolyString\(\)](#)

Used for: Text and UI.

```

virtual CUIPolyString* ILTFontManager::CreatePolyString( CUIFont * pFont,
        char *      pBuffer = NULL,
        float       x = 0.0,
        float       y = 0.0
    ) [pure virtual]

```

**Parameters:**

*pFont* pointer to a [CUIFont](#).  
*pBuffer* (optional) a null-terminated string of characters.  
*x* (optional) the x-coordinate on screen.  
*y* (optional) the y-coordinate on screen.

**Returns:**

pointer to a [CUIPolyString](#) object

This function creates a [CUIPolyString](#)

**See also:**

[CUIFontManager::CreateFormattedPolyString\(\)](#)

[CUIFontManager::DestroyPolyString\(\)](#)

Used for: Text and UI.

```

virtual void ILTFontManager::DestroyFont( CUIFont * pFont ) [pure virtual]

```

**Parameters:**

*pFont* pointer to the font to be destroyed.

**Returns:**

none.

Destroys a font, and frees associated memory.

Used for: Text and UI.

```
virtual void ILTFontManager::DestroyPolyString( CUIPolyString * pPolystr ) [pure virtual]
```

**Parameters:**

*pPolystr* pointer to the [CUIPolyString](#) to be destroyed.

**Returns:**

none

This function destroys a [CUIPolyString](#) and frees associated memory.

**See also:**

CUIFontManager::CreatePolyString()

Used for: Text and UI.

```
virtual uint32 ILTFontManager::GetNumFontsInUse( ) [pure virtual]
```

**Returns:**

number of fonts currently in use.

Use this function to query the ILTFontManager about how many fonts are currently in use (i.e., how many valid fonts have been created but not yet destroyed). Remember that the engine uses a [CUIFont](#) for the console, so the number returned by [GetNumFontsInUse\(\)](#) may be one higher than you first expect.

Used for: Text and UI.

```
virtual void ILTFontManager::Init( ) [pure virtual]
```

**Required function call.** Called to initialize the ILTFontManager's services. You cannot call any of the ILTFontManager's functions until you have initialized it.

Currently this function is called within the engine. You can check the initialization status of the ILTFontManager with a call to [IsInitted\(\)](#). Calling this function again after the ILTFontManager has been initialized has no effect.

Used for: Text and UI.

```
virtual bool ILTFontManager::IsInitted( ) [pure virtual]
```

**Returns:**

whether ILTFontManager was initialized.

Returns **true** if the ILTFontManager is initialized, or **false** if it has not yet been initialized.

**See also:**

[Init\(\)](#)

[Term\(\)](#)

Used for: Text and UI.

```
virtual HTEXTURE ILTFontManager::StringToTexture( uint8 * pTTFFData,
                                                uint32 dataSize,
                                                uint32 * pCharacters,
                                                uint32 pointSize
                                                ) [pure virtual]
```

**Parameters:**

*pTTFFData* a pointer to a TrueType resource.

*dataSize* the size (in bytes) of the resource pointed to by *pTTFFData*.

*pCharacters* an array of multi-byte characters to render

*pointSize* the point size at which to render *pCharacters*

**Returns:**

HTEXTURE containing *character*.

Use this function to render a string of characters into an HTEXTURE.

Used for: Text and UI.

```
virtual HTEXTURE ILTFontManager::StringToTexture( uint8 * pTTFFData,
                                                uint32 dataSize,
                                                char * pCharacters,
                                                uint32 pointSize
                                                ) [pure virtual]
```

**Parameters:**

*pTTFFData* a pointer to a TrueType resource.

*dataSize* the size (in bytes) of the resource pointed to by *pTTFFData*.

*pCharacters* an array of 8-bit characters to render

*pointSize* the point size at which to render *pCharacters*

**Returns:**

HTEXTURE containing *character*.

Use this function to render a string of characters into an HTEXTURE.

Used for: Text and UI.

virtual void ILTFontManager::Term( ) [pure virtual]

**Required function call.** Called to close down the ILTFontManager's services. If you do not call [Term\(\)](#) when the client shell is terminating, you risk memory leaks or worse. Currently this function is called within the engine. You can check the initialization status of the ILTFontManager with a call to [IsInitted\(\)](#). Calling this function again after the ILTFontManager has been terminated has no effect.

Used for: Text and UI.

The documentation for this class was generated from the following file:

- [iltfontmanager.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

## ILTMath Class Reference

Inheritance diagram for ILTMath:



### Public Types

enum { [\\_ILTMath\\_VERSION\\_](#) = 0 }

### Public Member Functions

virtual [LTRESULT](#) [AlignRotation](#) ([LTRotation](#) &q, [LTVector](#) &vf, [LTVector](#) &vu)=0  
virtual [LTRESULT](#) [EulerRotateX](#) ([LTRotation](#) &q, float angle)=0  
virtual [LTRESULT](#) [EulerRotateY](#) ([LTRotation](#) &q, float angle)=0  
virtual [LTRESULT](#) [EulerRotateZ](#) ([LTRotation](#) &q, float angle)=0  
virtual [LTRESULT](#) [GetRotationVectors](#) ([LTRotation](#) &q, [LTVector](#) &R0, [LTVector](#) &R1, [LTVector](#) &R2)=0  
virtual [LTRESULT](#) [InterpolateRotation](#) ([LTRotation](#) &q0, [LTRotation](#) &q1, float u)=0  
virtual [LTRESULT](#) [SetupEuler](#) ([LTRotation](#) &q, float tx, float ty, float tz)=0  
virtual [LTRESULT](#) [SetupRotationMatrix](#) ([LTMatrix](#) &M, [LTRotation](#) &q)=0  
virtual [LTRESULT](#) [SetupTransformationMatrix](#) ([LTMatrix](#) &M, [LTVector](#) &O, [LTRotation](#) &q)=0  
virtual [LTRESULT](#) [SetupTranslationFromMatrix](#) ([LTVector](#) &t, [LTMatrix](#) &M)=0  
virtual [LTRESULT](#) [SetupTranslationMatrix](#) ([LTMatrix](#) &M, [LTVector](#) &t)=0  
virtual [LTRESULT](#) [SetupRotationFromMatrix](#) ([LTRotation](#) &q, [LTMatrix](#) &M)=0  
virtual [LTRESULT](#) [SetupRotationAroundPoint](#) ([LTMatrix](#) &M, [LTRotation](#) &q, [LTVector](#) &pl)=0  
virtual [LTRESULT](#) [RotateAroundAxis](#) ([LTRotation](#) &q, [LTVector](#) &u, float angle)=0  
virtual [LTRESULT](#) [GetRotationVectorsFromMatrix](#) ([LTMatrix](#) &M, [LTVector](#) &R0, [LTVector](#) &R1, [LTVector](#) &R2)=0

### Detailed Description

The ILTMath interface exposes various math routines for dealing with Euler angles, quaternions and matrices.

Define a holder to get this interface like this:

[define\\_holder](#)([ILTMath](#), your\_var);

Used for: Obsolete.

### Member Enumeration Documentation

anonymous enum

**Enumeration values:**

[\\_ILTMath\\_VERSION\\_](#)

### Member Function Documentation

virtual [LTRESULT](#) ILTMath::AlignRotation( [LTRotation](#) & q,  
                                          [LTVector](#) & vf,  
                                          [LTVector](#) & vu  
                                          ) [pure virtual]

```
virtual LTRESULT ILTMath::EulerRotateX( LTRotation & q,
                                         float      angle
                                     ) [pure virtual]
```

```
virtual LTRESULT ILTMath::EulerRotateY( LTRotation & q,
                                         float      angle
                                     ) [pure virtual]
```

```
virtual LTRESULT ILTMath::EulerRotateZ( LTRotation & q,
                                         float      angle
                                     ) [pure virtual]
```

```
virtual LTRESULT ILTMath::GetRotationVectors( LTRotation & q,
                                              LTVector & R0,
                                              LTVector & R1,
                                              LTVector & R2
                                           ) [pure virtual]
```

```
virtual LTRESULT ILTMath::GetRotationVectorsFromMatrix( LTMatrix & M,
                                                       LTVector & R0,
                                                       LTVector & R1,
                                                       LTVector & R2
                                                    ) [pure virtual]
```

```
virtual LTRESULT ILTMath::InterpolateRotation( LTRotation & qu,
                                              LTRotation & q0,
                                              LTRotation & q1,
                                              float      u
                                           ) [pure virtual]
```

```
virtual LTRESULT ILTMath::RotateAroundAxis( LTRotation & q,
                                             LTVector & u,
                                             float      angle
                                           ) [pure virtual]
```

```
virtual LTRESULT ILTMath::SetupEuler( LTRotation & q,
                                      float      tx,
                                      float      ty,
                                      float      tz
                                    ) [pure virtual]
```

```
virtual LTRESULT ILTMath::SetupRotationAroundPoint( LTMatrix & M,
                                                    LTRotation & q,
                                                    LTVector & pl
                                                  ) [pure virtual]
```

```
virtual LTRESULT ILTMath::SetupRotationFromMatrix( LTRotation & q,
                                                  LTMatrix & M
                                                ) [pure virtual]
```

```
virtual LTRESULT ILTMath::SetupRotationMatrix( LTMatrix & M,
                                              LTRotation & q
                                            ) [pure virtual]
```

```
virtual LTRESULT ILTMath::SetupTransformationMatrix( LTMatrix & M,
                                                     LTVector & O,
                                                     LTRotation & q
                                                  ) [pure virtual]
```

```
virtual LTRESULT ILTMath::SetupTranslationFromMatrix( LTVector & t,
                                                      LTMatrix & M
                                                    ) [pure virtual]
```

```
virtual LTRESULT ILTMath::SetupTranslationMatrix( LTMatrix & M,
                                                  LTVector & t
                                                ) [pure virtual]
```

The documentation for this class was generated from the following file:

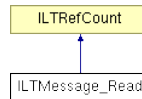
- [iltmath.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
 2006, All Rights Reserved.

## API Reference

### ILTMessage\_Read Class Reference

Inheritance diagram for ILTMessage\_Read:



#### Public Member Functions

```

virtual ILTMessage\_Read * Clone () const =0
virtual ILTMessage\_Read * SubMsg (uint32 nPos) const =0
virtual ILTMessage\_Read * SubMsg (uint32 nPos, uint32 nLength) const =0
    virtual uint32 Size () const =0
    virtual void Seek (int32 nOffset)=0
    virtual void SeekTo (uint32 nPos)=0
    virtual uint32 Tell () const =0
    virtual uint32 TellEnd () const =0
    virtual bool EOM () const =0
    virtual uint32 ReadBits (uint32 nBits)=0
    virtual uint64 ReadBits64 (uint32 nBits)=0
    virtual void ReadData (void *pData, uint32 nBits)=0
virtual ILTMessage\_Read * ReadMessage ()=0
    virtual uint32 ReadString (char *pDest, uint32 nMaxLen)=0
    virtual HSTRING ReadHString ()=0
    virtual LTVector ReadCompLTVector ()=0
    virtual LTVector ReadCompPos ()=0
    virtual LTRotation ReadCompLTRotation ()=0
    virtual uint32 ReadHStringAsString (char *pDest, uint32 nMaxLen)=0
    virtual HOBJECT ReadObject ()=0
    virtual LTRotation ReadYRotation ()=0
        bool Readbool ()
        uint8 Readuint8 ()
        uint16 Readuint16 ()
        uint32 Readuint32 ()
        uint64 Readuint64 ()
        int8 Readint8 ()
        int16 Readint16 ()
        int32 Readint32 ()
        int64 Readint64 ()
        float Readfloat ()
        double Readdouble ()
        LTVector ReadLTVector ()
        LTRotation ReadLTRotation ()
template<class T>
    void ReadType (T *pValue)
    virtual uint32 PeekBits (uint32 nBits) const =0
    virtual uint64 PeekBits64 (uint32 nBits) const =0
    virtual void PeekData (void *pData, uint32 nBits) const =0
virtual ILTMessage\_Read * PeekMessage () const =0
    virtual uint32 PeekString (char *pDest, uint32 nMaxLen) const =0
    virtual HSTRING PeekHString () const =0
    virtual LTVector PeekCompLTVector () const =0
    virtual LTVector PeekCompPos () const =0
    virtual LTRotation PeekCompLTRotation () const =0
    virtual uint32 PeekHStringAsString (char *pDest, uint32 nMaxLen) const =0
    virtual HOBJECT PeekObject () const =0
    virtual LTRotation PeekYRotation () const =0
        bool Peekbool () const
        uint8 Peekuint8 () const
        uint16 Peekuint16 () const
        uint32 Peekuint32 () const
        uint64 Peekuint64 () const
        int8 Peekint8 () const
        int16 Peekint16 () const
        int32 Peekint32 () const
        int64 Peekint64 () const
        float Peekfloat () const
        double Peekdouble () const
        LTVector PeekLTVector () const
        LTRotation PeekLTRotation () const
template<class T>
    void PeekType (T *pValue) const
  
```

## Member Function Documentation

```
virtual ILTMessage\_Read* ILTMessage_Read::Clone( ) const [pure virtual]

virtual bool ILTMessage_Read::EOM( ) const [pure virtual]

virtual uint32 ILTMessage_Read::PeekBits( uint32 nBits ) const [pure virtual]

virtual uint64 ILTMessage_Read::PeekBits64( uint32 nBits ) const [pure virtual]

bool ILTMessage_Read::Peekbool( ) const [inline]

virtual LTRotation ILTMessage_Read::PeekCompLTRotation( ) const [pure virtual]

virtual LTVector ILTMessage_Read::PeekCompLTVector( ) const [pure virtual]

virtual LTVector ILTMessage_Read::PeekCompPos( ) const [pure virtual]

virtual void ILTMessage_Read::PeekData( void * pData,
                                       uint32 nBits
                                     ) const [pure virtual]

double ILTMessage_Read::Peekdouble( ) const [inline]

float ILTMessage_Read::Peekfloat( ) const [inline]

virtual HSTRING ILTMessage_Read::PeekHString( ) const [pure virtual]

virtual uint32 ILTMessage_Read::PeekHStringAsString( char * pDest,
                                                  uint32 nMaxLen
                                                ) const [pure virtual]

int16 ILTMessage_Read::Peekint16( ) const [inline]

int32 ILTMessage_Read::Peekint32( ) const [inline]

int64 ILTMessage_Read::Peekint64( ) const [inline]

int8 ILTMessage_Read::Peekint8( ) const [inline]

LTRotation ILTMessage_Read::PeekLTRotation( ) const [inline]

LTVector ILTMessage_Read::PeekLTVector( ) const [inline]

virtual ILTMessage\_Read* ILTMessage_Read::PeekMessage( ) const [pure virtual]

virtual HOBJECT ILTMessage_Read::PeekObject( ) const [pure virtual]

virtual uint32 ILTMessage_Read::PeekString( char * pDest,
                                       uint32 nMaxLen
                                     ) const [pure virtual]
```

```

template<class T>
void ILTMessage_Read::PeekType( T * pValue ) const [inline]

uint16 ILTMessage_Read::Peekuint16( ) const [inline]

uint32 ILTMessage_Read::Peekuint32( ) const [inline]

uint64 ILTMessage_Read::Peekuint64( ) const [inline]

uint8 ILTMessage_Read::Peekuint8( ) const [inline]

virtual LTRotation ILTMessage_Read::PeekYRotation( ) const [pure virtual]

virtual uint32 ILTMessage_Read::ReadBits( uint32 nBits ) [pure virtual]

virtual uint64 ILTMessage_Read::ReadBits64( uint32 nBits ) [pure virtual]

bool ILTMessage_Read::Readbool( ) [inline]

virtual LTRotation ILTMessage_Read::ReadCompLTRotation( ) [pure virtual]

virtual LTVector ILTMessage_Read::ReadCompLTVector( ) [pure virtual]

virtual LTVector ILTMessage_Read::ReadCompPos( ) [pure virtual]

virtual void ILTMessage_Read::ReadData( void * pData,
                                         uint32 nBits
                                     ) [pure virtual]

double ILTMessage_Read::Readdouble( ) [inline]

float ILTMessage_Read::Readfloat( ) [inline]

virtual HSTRING ILTMessage_Read::ReadHString( ) [pure virtual]

virtual uint32 ILTMessage_Read::ReadHStringAsString( char * pDest,
                                                    uint32 nMaxLen
                                                ) [pure virtual]

int16 ILTMessage_Read::Readint16( ) [inline]

int32 ILTMessage_Read::Readint32( ) [inline]

int64 ILTMessage_Read::Readint64( ) [inline]

int8 ILTMessage_Read::Readint8( ) [inline]

LTRotation ILTMessage_Read::ReadLTRotation( ) [inline]

```

```
LTVector ILTMessage_Read::ReadLTVector( ) [inline]

virtual ILTMessage\_Read* ILTMessage_Read::ReadMessage( ) [pure virtual]

virtual HOBJECT ILTMessage_Read::ReadObject( ) [pure virtual]

virtual uint32 ILTMessage_Read::ReadString( char * pDest,
                                         uint32 nMaxLen
                                         ) [pure virtual]

template<class T>
void ILTMessage_Read::ReadType( T * pValue ) [inline]

uint16 ILTMessage_Read::Readuint16( ) [inline]

uint32 ILTMessage_Read::Readuint32( ) [inline]

uint64 ILTMessage_Read::Readuint64( ) [inline]

uint8 ILTMessage_Read::Readuint8( ) [inline]

virtual LTRotation ILTMessage_Read::ReadYRotation( ) [pure virtual]

virtual void ILTMessage_Read::Seek( int32 nOffset ) [pure virtual]

virtual void ILTMessage_Read::SeekTo( uint32 nPos ) [pure virtual]

virtual uint32 ILTMessage_Read::Size( ) const [pure virtual]

virtual ILTMessage\_Read* ILTMessage_Read::SubMsg( uint32 nPos,
                                                uint32 nLength
                                                ) const [pure virtual]

virtual ILTMessage\_Read* ILTMessage_Read::SubMsg( uint32 nPos ) const [pure virtual]

virtual uint32 ILTMessage_Read::Tell( ) const [pure virtual]

virtual uint32 ILTMessage_Read::TellEnd( ) const [pure virtual]
```

---

The documentation for this class was generated from the following file:

- [iltmessage.h](#)

---

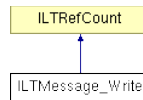
Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

---

### ILTMessage\_Write Class Reference

Inheritance diagram for ILTMessage\_Write:



## Public Member Functions

```

virtual void Reset ()=0
virtual ILTMessage\_Read * Read ()=0
virtual uint32 Size () const =0
virtual void WriteBits (uint32 nValue, uint32 nSize)=0
virtual void WriteBits64 (uint64 nValue, uint32 nSize)=0
virtual void WriteData (const void *pData, uint32 nSize)=0
virtual void WriteMessage (const ILTMessage\_Read *pMsg)=0
virtual void WriteMessageRaw (const ILTMessage\_Read *pMsg)=0
virtual void WriteString (const char *pString)=0
virtual void WriteHString (HSTRING hString)=0
virtual void WriteCompLTVector (const LTVector &vVec)=0
virtual void WriteCompPos (const LTVector &vPos)=0
virtual void WriteCompLTRotation (const LTRotation &cRotation)=0
virtual void WriteHStringFormatted (int nStringCode,...)=0
virtual void WriteHStringArgList (int nStringCode, va_list *pList)=0
virtual void WriteStringAsHString (const char *pString)=0
virtual void WriteObject (HOBJECT hObj)=0
virtual void WriteYRotation (const LTRotation &cRotation)=0
void Writebool (bool bValue)
void Writeuint8 (uint8 nValue)
void Writeuint16 (uint16 nValue)
void Writeuint32 (uint32 nValue)
void Writeuint64 (uint64 nValue)
void Writeint8 (int8 nValue)
void Writeint16 (int16 nValue)
void Writeint32 (int32 nValue)
void Writeint64 (int32 nValue)
void Writefloat (float fValue)
void Writedouble (double fValue)
void WriteLTVector (const LTVector &vValue)
void WriteLTRotation (const LTRotation &cValue)

template<class T>
void WriteType (const T &tValue)
  
```

## Member Function Documentation

```
virtual ILTMessage\_Read * ILTMessage\_Write::Read( ) [pure virtual]
```

```
virtual void ILTMessage\_Write::Reset( ) [pure virtual]
```

```
virtual uint32 ILTMessage\_Write::Size( ) const [pure virtual]
```

```
virtual void ILTMessage\_Write::WriteBits(uint32 nValue,
uint32 nSize
) [pure virtual]
```

```
virtual void ILTMessage\_Write::WriteBits64(uint64 nValue,
uint32 nSize
) [pure virtual]
```

```
void ILTMessage\_Write::Writebool( bool bValue ) [inline]
```

```
virtual void ILTMessage\_Write::WriteCompLTRotation( const LTRotation & cRotation ) [pure virtual]
```

```
virtual void ILTMessage\_Write::WriteCompLTVector( const LTVector & vVec ) [pure virtual]
```

```
virtual void ILTMessage\_Write::WriteCompPos( const LTVector & vPos ) [pure virtual]
```

```
virtual void ILTMessage\_Write::WriteData( const void * pData,
uint32 nSize
) [pure virtual]
```

```
void ILTMessage\_Write::Writedouble( double fValue ) [inline]
```

```

void ILTMessage_Write::Writefloat( float fValue ) [inline]

virtual void ILTMessage_Write::WriteHString( HSTRING hString ) [pure virtual]

virtual void ILTMessage_Write::WriteHStringArgList( int      nStringCode,
                                                    va_list *pList
                                                    ) [pure virtual]

virtual void ILTMessage_Write::WriteHStringFormatted( int nStringCode,
                                                    ...
                                                    ) [pure virtual]

void ILTMessage_Write::Writeint16( int16 nValue ) [inline]

void ILTMessage_Write::Writeint32( int32 nValue ) [inline]

void ILTMessage_Write::Writeint64( int32 nValue ) [inline]

void ILTMessage_Write::Writeint8( int8 nValue ) [inline]

void ILTMessage_Write::WriteLTRotation( const LTRotation &cValue ) [inline]

void ILTMessage_Write::WriteLTVector( const LTVector &vValue ) [inline]

virtual void ILTMessage_Write::WriteMessage( const ILTMessage\_Read *pMsg ) [pure virtual]

virtual void ILTMessage_Write::WriteMessageRaw( const ILTMessage\_Read *pMsg ) [pure virtual]

virtual void ILTMessage_Write::WriteObject( HOBJECT hObj ) [pure virtual]

virtual void ILTMessage_Write::WriteString( const char *pString ) [pure virtual]

virtual void ILTMessage_Write::WriteStringAsHString( const char *pString ) [pure virtual]

template<class T>
void ILTMessage_Write::WriteType( const T &tValue ) [inline]

void ILTMessage_Write::Writeuint16( uint16 nValue ) [inline]

void ILTMessage_Write::Writeuint32( uint32 nValue ) [inline]

void ILTMessage_Write::Writeuint64( uint64 nValue ) [inline]

void ILTMessage_Write::Writeuint8( uint8 nValue ) [inline]

virtual void ILTMessage_Write::WriteYRotation( const LTRotation &cRotation ) [pure virtual]

```

---

The documentation for this class was generated from the following file:

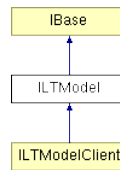
- [iltmessage.h](#)
-

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
 2006, All Rights Reserved.

## API Reference

### ILTModel Class Reference

Inheritance diagram for ILTModel:



### Public Types

enum { [ILTModel\\_VERSION](#) = 0 }

### Public Member Functions

virtual [LTRESULT](#) [CacheModelDB](#) (const char \*Filename, [HMODELDB](#) &hModelDB)=0  
 virtual [LTRESULT](#) [UncacheModelDB](#) ([HMODELDB](#) &hModelDB)=0  
 virtual [LTRESULT](#) [IsModelDBLoaded](#) ([HMODELDB](#) hModelDB)=0  
 virtual [LTRESULT](#) [AddChildModelDB](#) ([HOBJECT](#) hObj, [HMODELDB](#) hModelDB)  
 virtual [LTRESULT](#) [GetSocket](#) ([HOBJECT](#) hObj, const char \*pSocketName, [HMODEL\\_SOCKET](#) &hSocket)  
 virtual [LTRESULT](#) [GetSocketTransform](#) ([HOBJECT](#) hObj, [HMODEL\\_SOCKET](#) hSocket, [LTransform](#) &transform, bool bWorldSpace)  
 virtual [LTRESULT](#) [GetNumPieces](#) ([HOBJECT](#) hObj, [uint32](#) &NumPieces)  
 virtual [LTRESULT](#) [GetPiece](#) ([HOBJECT](#) hObj, const char \*pPieceName, [HMODELPIECE](#) &hPiece)  
 virtual [LTRESULT](#) [GetPieceHideStatus](#) ([HOBJECT](#) hObj, [HMODELPIECE](#) hPiece, bool &bHidden)  
 virtual [LTRESULT](#) [SetPieceHideStatus](#) ([HOBJECT](#) hObj, [HMODELPIECE](#) hPiece, bool bHidden)  
 virtual [LTRESULT](#) [GetNode](#) ([HOBJECT](#) hObj, const char \*pNodeName, [HMODELNODE](#) &hNode)  
 virtual [LTRESULT](#) [GetNodeName](#) ([HOBJECT](#) hObj, [HMODELNODE](#) hNode, char \*name, [uint32](#) maxlen)  
 virtual [LTRESULT](#) [GetNodeTransform](#) ([HOBJECT](#) hObj, [HMODELNODE](#) hNode, [LTransform](#) &transform, bool bWorldSpace)  
 virtual [LTRESULT](#) [GetNextNode](#) ([HOBJECT](#) hObject, [HMODELNODE](#) hNode, [HMODELNODE](#) &pNext)  
 virtual [LTRESULT](#) [GetRootNode](#) ([HOBJECT](#) hObj, [HMODELNODE](#) &hNode)  
 virtual [LTRESULT](#) [GetNumChildren](#) ([HOBJECT](#) hObj, [HMODELNODE](#) hNode, [uint32](#) &NumChildren)  
 virtual [LTRESULT](#) [GetChild](#) ([HOBJECT](#) hObj, [HMODELNODE](#) parent, [uint32](#) index, [HMODELNODE](#) &child)  
 virtual [LTRESULT](#) [GetParent](#) ([HOBJECT](#) hObj, [HMODELNODE](#) node, [HMODELNODE](#) &parent)  
 virtual [LTRESULT](#) [GetNumNodes](#) ([HOBJECT](#) hObj, [uint32](#) &num\_nodes)  
 virtual [LTRESULT](#) [GetBindPoseNodeTransform](#) ([HOBJECT](#) hObj, [HMODELNODE](#) node, [LMatrix](#) &mat)  
 virtual [LTRESULT](#) [AddNodeControlFn](#) ([HOBJECT](#) hObj, [HMODELNODE](#) hNode, [NodeControlFn](#) pFn, void \*pUserData)  
 virtual [LTRESULT](#) [AddNodeControlFn](#) ([HOBJECT](#) hObj, [NodeControlFn](#) pFn, void \*pUserData)  
 virtual [LTRESULT](#) [RemoveNodeControlFn](#) ([HOBJECT](#) hObj, [HMODELNODE](#) hNode, [NodeControlFn](#) pFn, void \*pUserData)  
 virtual [LTRESULT](#) [RemoveNodeControlFn](#) ([HOBJECT](#) hObj, [NodeControlFn](#) pFn, void \*pUserData)  
 virtual [LTRESULT](#) [UpdateMainTracker](#) ([HOBJECT](#) hObj, float fUpdateDelta)  
 virtual [LTRESULT](#) [GetAnimLength](#) ([HOBJECT](#) hModel, [HMODELANIM](#) hAnim, [uint32](#) &length)  
 virtual [LTRESULT](#) [FindWeightSet](#) ([HOBJECT](#) hObj, const char \*pSetName, [HMODELWEIGHTSET](#) &hSet)  
 virtual [LTRESULT](#) [GetMainTracker](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) &TrackerID)  
 virtual [LTRESULT](#) [GetPlaybackState](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID, [uint32](#) &flags)  
 virtual [LTRESULT](#) [AddTracker](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID)  
 virtual [LTRESULT](#) [RemoveTracker](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID)  
 virtual [LTRESULT](#) [GetAnimIndex](#) ([HOBJECT](#) hModel, const char \*pAnimName, [uint32](#) &anim\_index)  
 virtual [LTRESULT](#) [GetCurAnim](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID, [HMODELANIM](#) &hAnim)  
 virtual [LTRESULT](#) [SetCurAnim](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID, [HMODELANIM](#) hAnim)  
 virtual [LTRESULT](#) [ResetAnim](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID)  
 virtual [LTRESULT](#) [GetLooping](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID)  
 virtual [LTRESULT](#) [SetLooping](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID, bool bLooping)  
 virtual [LTRESULT](#) [GetPlaying](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID)  
 virtual [LTRESULT](#) [SetPlaying](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID, bool bPlaying)  
 virtual [LTRESULT](#) [GetCurAnimLength](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID, [uint32](#) &length)  
 virtual [LTRESULT](#) [GetCurAnimTime](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID, [uint32](#) &curTime)  
 virtual [LTRESULT](#) [SetCurAnimTime](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID, [uint32](#) curTime)  
 virtual [LTRESULT](#) [SetHintNode](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID, [HMODELNODE](#) hNode)  
 virtual [LTRESULT](#) [SetAnimRate](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID, float fRate)  
 virtual [LTRESULT](#) [GetAnimRate](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID, float &fRate)  
 virtual [LTRESULT](#) [SetWeightSet](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID, [HMODELWEIGHTSET](#) &hSet)  
 virtual [LTRESULT](#) [SetWeightSet](#) ([HOBJECT](#) hModel, [ANIMTRACKERID](#) TrackerID, [HMODELWEIGHTSET](#) hSet)  
 virtual [LTRESULT](#) [GetNumLODs](#) ([HOBJECT](#) hModel, [HMODELPIECE](#) hPiece, [uint32](#) &num\_lods)  
 virtual [LTRESULT](#) [GetLODValFromDist](#) ([HOBJECT](#) hModel, [HMODELPIECE](#) hPiece, float dist, [uint32](#) &lod\_index)  
 virtual [LTRESULT](#) [ApplyAnimations](#) ([HOBJECT](#) hModel)  
 virtual [LTRESULT](#) [GetFileNames](#) ([HOBJECT](#) hObj, char \*pFilename, [uint32](#) fileBufLen, char \*pSkinName, [uint32](#) skinBufLen)=0  
 virtual [LTRESULT](#) [GetModelDBFilename](#) ([HOBJECT](#) hObj, char \*pRetFilename, [uint32](#) fileBufLen)=0  
 virtual [LTRESULT](#) [GetSkinFilename](#) ([HOBJECT](#) hObj, [uint32](#) skin\_index, char \*pRetFileNames, [uint32](#) fileBufLen)=0  
 virtual [LTRESULT](#) [GetNumModelOBBS](#) ([HOBJECT](#) hObj, [uint32](#) &num\_obbs)  
 virtual [LTRESULT](#) [GetModelOBBCoov](#) ([HOBJECT](#) hObj, [ModelOBB](#) \*)  
 virtual [LTRESULT](#) [UpdateModelOBB](#) ([HOBJECT](#) hObj, [ModelOBB](#) \*)

### Detailed Description

The **ILTModel** interface contains functions for manipulating models and animations.

There are two implemenations of this interface; one for server, one for client.

Define a holder to get this interface like this:

```
define_holder_to_instance(ILTModel, your_var, Client);
define_holder_to_instance(ILTModel, your_var, Server);
```

## Member Enumeration Documentation

anonymous enum

**Enumeration values:**

`_ILTModel_VERSION_`

## Member Function Documentation

```
virtual LTRESULT ILTModel::AddChildModelDB( HOBJECT hObj,
                                             HMODELDB hModelDB
                                             ) [inline, virtual]
```

**Parameters:**

*hObj, handle* to engine object representing a model instance  
*hModelDB* handle to a model database.

results : LT\_ERROR could be returned for several reasons, check the console for an error message. LT\_OK success LT\_NOTFOUND when the call is inappropriate for the context.

Notes: 1. This call only works on the server side. 2. This call operates on a modelinstance's underlying model database. It is possible to add a child model to any number of modelinstances, as long as the underlying db is the same, no harm is done.

```
virtual LTRESULT ILTModel::AddNodeControlFn( HOBJECT hObj,
                                              NodeControlFn pFn,
                                              void * pUserData
                                              ) [virtual]
```

**Parameters:**

*hObj* Model to process.  
*fn* Function to call.  
*pUserData* User data.

**Returns:**

LT\_INVALIDPARAMS if hObj is either NULL, or not of type OT\_MODEL LT\_OK otherwise

Set a callback node processing for every node in a model. See above AddNodeControlFn function for details.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::AddNodeControlFn( HOBJECT hObj,
                                              HMODELNODE hNode,
                                              NodeControlFn pFn,
                                              void * pUserData
                                              ) [virtual]
```

**Parameters:**

*hObj* Model to process.  
*hNode* Node to process.  
*fn* Function to call.  
*pUserData* User data.

**Returns:**

LT\_INVALIDPARAMS if hObj is either NULL, or not of type OT\_MODEL LT\_OK otherwise

Set callback that gets the world position of the animation node just after it has been evaluated.

**Note:** Your function should return as quickly as possible -- it will be called often. The NodeControlFn will be called for each node on the model, allowing you to translate and rotate it.

**To disable the NodeControlFn, use the RemoveNodeControlFn functions**

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::AddTracker( HOBJECT hModel,
                                       ANIMTRACKERID TrackerID
                                       ) [virtual]
```

**Parameters:**

*hModel* Model instance handle.  
*hModel* handle to a model.  
*TrackerID* Animation tracker identifier to be associated with new tracker.

**Returns:**

LT\_INVALIDPARAMS on invalid hModel, LT\_OVERFLOW if there are already MAX\_TRACKERS\_PER\_MODEL attached,  
 LT\_INVALIDVERSION if *not* using SMP networking, LT\_ALREADYEXISTS if the TrackerID is already in use for the model instance, otherwise  
 LT\_OK.

Add trackers on the model.

Used for: Models and Animation.

virtual [LTRESULT](#) ILTModel::ApplyAnimations( [HOBJECT](#) *hModel* ) [virtual]

**Parameters:**

*hModel,model* instance handle

**Returns:**

,

Just update the current transformation in regards to the current animation state.

virtual [LTRESULT](#) ILTModel::CacheModelDB( const char \* *Filename*,  
[HMODELDB](#) & *hModelDB*  
) [pure virtual]

**Parameters:**

*filename,file* to load

*out* param : hModelDB, model database handle.

Loads a model with out associating it with a game object. returns a handle to the model database.

Used for: Models and Animation.

virtual [LTRESULT](#) ILTModel::FindWeightSet( [HOBJECT](#) *hObj*,  
const char \* *pSetName*,  
[HMODELWEIGHTSET](#) & *hSet*  
) [virtual]

**Parameters:**

*hObj* Model instance.

*pSetName* name of animation weight set.

*hSet* return reference parameter to weight set.

**Returns:**

**LT\_NOTINITIALIZED** if hObj is invalid, **LT\_OK**, success, **LT\_NOTFOUND**, weight set not found.

Find the animation blending weight set with the given name.

Used for: Models and Animation.

virtual [LTRESULT](#) ILTModel::GetAnimIndex( [HOBJECT](#) *hModel*,  
const char \* *pAnimName*,  
[uint32](#) & *anim\_index*  
) [virtual]

**Parameters:**

*hModel* Model instance handle.

*pAnimName* Name of the animation

*anim\_index* return value: index of animation in model database.

**Returns:**

**LT\_INVALIDPARAMS** on invalid hModel, or if there is no model database associated with hmodel. **LT\_OK**

Used for: Models and Animation.

virtual [LTRESULT](#) ILTModel::GetAnimLength( [HOBJECT](#) *hModel*,  
[HMODELANIM](#) *hAnim*,  
[uint32](#) & *length*  
) [virtual]

**Parameters:**

*hAnim* the animation of choice.

*length* the length in miliseconds of the animatoin.

**Returns:**

**LT\_OK** operation succeeded. **LT\_INVALIDPARAMS** if pModel is null. **LT\_NOTINITIALIZED** if the pTracker is not currently associated with animation data. **LT\_INVALIDVERSION** if using SMP networking.

Get the length of the specified animation (in milliseconds).

Used for: Models and Animation.

virtual [LTRESULT](#) ILTModel::GetAnimRate( [HOBJECT](#) *hModel*,  
[ANIMTRACKERID](#) *TrackerID*,  
float & *fRate*  
) [virtual]

**Parameters:**

*hModel* Model instance handle.

*TrackerID* Animation tracker identifier

*rate* a float return value indicating the rate modifier on the animation.

**Returns:**

**LT\_INVALID\_PARAMS** if TrackerID is invalid, **LT\_INVALIDVERSION** if *not* using SMP networking; otherwise, returns **LT\_OK**.

Gets the current rate of change between frames.

Used for: Models and Animation.

virtual [LTRESULT](#) ILTModel::GetBindPoseNodeTransform( [HOBJECT](#) *hObj*,

```

HMODELNODE node,
LTMatrix & mat
) [virtual]

```

**Parameters:**

*hObj* modelinstance handle  
*iNode* which node.  
*return* value: matrix representing bind pose

Gets the bind pose's tranform for the node.

Used for: Models and Animation

```

virtual LTRESULT ILTModel::GetChild( HOBJECT hObj,
HMODELNODE parent,
uint32 index,
HMODELNODE & child
) [virtual]

```

**Parameters:**

*hObj* ModelInstance handle  
*hNode* a handle to the parent node  
*index*  
*NumChildren* return value of query.

**Returns:**

if object is either invalid, or not an OT\_MODEL

GetChild returns a handle to the nth child of parent indicated.

Used for: Models and Animation.

```

virtual LTRESULT ILTModel::GetCurAnim( HOBJECT hModel,
ANIMTRACKERID TrackerID,
HMODELANIM & hAnim
) [virtual]

```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier.  
*hAnim* animation handle.

**Returns:**

**LT\_INVALIDPARAMS** on invalid TrackerID, **LT\_INVALIDVERSION** if *not* using SMP networking, otherwise **LT\_OK**.

Get the current animation for the tracker.

Used for: Models and Animation.

```

virtual LTRESULT ILTModel::GetCurAnimLength( HOBJECT hModel,
ANIMTRACKERID TrackerID,
uint32 & length
) [virtual]

```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier of choice.  
*length* the length in miliseconds of the animation.

**Returns:**

**LT\_OK** operation succeeded. **LT\_INVALIDPARAMS** if TrackerID is null. **LT\_NOTINITIALIZED** if the TrackerID is not currently associated with animation data. **LT\_INVALIDVERSION** if *not* using SMP networking.

Get the length of the current animation (in milliseconds).

Used for: Models and Animation.

```

virtual LTRESULT ILTModel::GetCurAnimTime( HOBJECT hModel,
ANIMTRACKERID TrackerID,
uint32 & curTime
) [virtual]

```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier of choice.  
*length* the length in miliseconds of the animatoin.

**Returns:**

**LT\_OK** operation succeeded. **LT\_INVALIDPARAMS** if TrackerID is null. **LT\_NOTINITIALIZED** if the TrackerID is not currently associated with animation data. **LT\_INVALIDVERSION** if *not* using SMP networking.

Get the length of the current animation (in milliseconds).

Used for: Models and Animation.

```

virtual LTRESULT ILTModel::GetFileNames( HOBJECT hObj,
char * pFilename,
uint32 fileBufLen,
char * pSkinName,

```

```

        uint32    skinBufLen
    ) [pure virtual]

```

**Parameters:**

*hObj* Model to query.  
*pFilename* (return) Buffer for model filename (can use **LTNULL**).  
*fileBufLen* The **pFilename** buffer length.  
*pSkinName* (return) Buffer for skin filename (can use **LTNULL**).  
*skinBufLen* The **pSkinName** buffer length.

**Returns:**

**LT\_INVALIDPARAMS** if the object is not a model. **LT\_UNSUPPORTED** if called from the client. **LT\_OK** otherwise.

Get the model filenames. Initializes **pFilename** or **pSkinName** to zero length if the model doesn't have a filename or skin.

**Note:** This function is only valid on the server.

Used for: Models and Animation.

```

virtual LTRESULT ILTModel::GetLODValFromDist( HOBJECT    hModel,
                                              HMODELPIECE hPiece,
                                              float       dist,
                                              uint32 &    lod_index
                                              ) [virtual]

```

**Parameters:**

*hModel,model* instance handle.  
*hPiece,piece* handle  
*distance.*  
*lod\_number,return* value.

**Returns:**

**LT\_OK** operation succeeded. **LT\_ERROR** if hpiece is invalid. **LT\_INVALIDPARAMS** hModel or hPiece is not valid. **LT\_FAIL** if operation failed for some other reason. Returns the level of detail index at the distance requested for the piece.

Used for: Models and Animation

```

virtual LTRESULT ILTModel::GetLooping( HOBJECT    hModel,
                                       ANIMTRACKERID TrackerID
                                       ) [virtual]

```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier.

**Returns:**

**LT\_NO** if not looping, or an error, **LT\_INVALIDVERSION** if *not* using SMP networking; otherwise, returns **LT\_YES**.

Used for: Models and Animation.

```

virtual LTRESULT ILTModel::GetMainTracker( HOBJECT    hModel,
                                           ANIMTRACKERID & TrackerID
                                           ) [virtual]

```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* return parameter, identifier for the main tracker.

**Returns:**

**LT\_INVALIDPARAMS** on invalid hModel, **LT\_INVALIDVERSION** if *not* using SMP networking; otherwise **LT\_OK**.

Get ref for the main animation tracker of a model.

**Note:** The main tracker always uses identifier **MAIN\_TRACKER**, so this function is not really needed and will be deprecated.

Used for: Models and Animation.

```

virtual LTRESULT ILTModel::GetModelDBFilename( HOBJECT hObj,
                                              char *    pRetFilename,
                                              uint32    fileBufLen
                                              ) [pure virtual]

```

**Parameters:**

*hObj* Model to query  
*pRetFilename,memory* buffer size of filebuflen.

Fills pRetFilename with the filename associated with the model database.

```

virtual LTRESULT ILTModel::GetModelOBBCopy( HOBJECT    hObj,
                                           ModelOBB *
                                           ) [virtual]

```

```

virtual LTRESULT ILTModel::GetNextNode( HOBJECT    hObject,
                                       HMODELNODE  hNode,
                                       HMODELNODE  & pNext
                                       ) [virtual]

```

**Parameters:**

*hObject* Model Instance.  
*hNode* Current node. Use **INVALID\_MODEL\_NODE** to get first node.  
*pNext* (return) Next node in list. **LT\_FINISHED** means the end of the list.

**Returns:**

**LT\_FINISHED** when there are no more; otherwise, returns **LT\_OK**.

Iterate through the nodes of the model. The order of the nodes is as if the hierarchy was traversed using pre-order traversal.

**Example:**

```
HMODELNODE hCurNode = INVALID_MODEL_NODE;
while (interface->GetNextModelNode(hModel, hCurNode, &hCurNode) == LT_OK)
{ ... }
```

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetNode( HOBJECT      hObject,
                                   const char *  pNodeName,
                                   HMODELNODE & hNode
                                   ) [virtual]
```

**Parameters:**

*hObj* Model instance name.  
*pNodeName* name of node in the model's model transformation hierarchy.  
*hNode* return param holding a handle to node.

**Returns:**

**LT\_OK** on success **LT\_INVALIDPARAMS** if *hObj* is not an OT\_MODEL **LT\_NOTINITIALIZED** if *hObj* is not currently associated with a model.  
**LT\_NOTFOUND** if the requested node is not found.

Get a handle on named node.

**See also:**

[GetNodeTransform](#) Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetNodeName( HOBJECT      hObject,
                                         HMODELNODE hNode,
                                         char *       name,
                                         uint32      maxlen
                                         ) [virtual]
```

**Parameters:**

*hObject* Model to query.  
*hNode* Node to query.  
*pName* (return) Buffer to hold name.  
*maxLen* Length of buffer.

**Returns:**

**LT\_INVALIDPARAMS** if a parameter is null, else **LT\_OK**

Get the name of a model transformation node.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetNodeTransform( HOBJECT      hObject,
                                              HMODELNODE hNode,
                                              LTransform & transform,
                                              bool          bWorldSpace
                                              ) [virtual]
```

**Parameters:**

*hObj* Model instance handle.  
*hNode* Model node handle.  
*transform* transform  
*bWorldSpace*

**Returns:**

**LT\_OK** on success **LT\_INVALIDPARAMS** if *hObj* is not an OT\_MODEL **LT\_NOTINITIALIZED** if *hObj* is not currently associated with a model.  
**LT\_ERROR** if a) The transform hierarchy could not be evaluated for the model b) The resultant transformation is invalid.

Get the transformation for a node on a model. If **bWorldSpace** is **TRUE**, then the transform is returned in world space. Otherwise, the transform is local to the model.

**See also:**

[GetNode](#)

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetNumChildren( HOBJECT      hObject,
                                           HMODELNODE hNode,
                                           uint32 &    NumChildren
                                           ) [virtual]
```

**Parameters:**

*hObj* ModelInstance handle  
*hNode* a handle to the node  
*NumChildren* return value of query.

**Returns:**

if object is either invalid, or not an OT\_MODEL

Returns the number of children this node has. If it is a leaf in the transform hierarchy the value will be zero.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetNumLODs( HOBJECT hModel,
                                       HMODELPIECE hPiece,
                                       uint32 & num_lods
                                       ) [virtual]
```

**Parameters:**

*hModel,model* instance handle.  
*hPiece,piece* handle  
*NumLODs,return* value, number of lods in the piece indicated.

**Returns:**

**LT\_OK** operation succeeded. **LT\_ERROR** if hpiece is invalid. **LT\_INVALIDPARAMS** hModel or hPiece is not valid. **LT\_FAIL** if operation failed for some other reason.

Returns the number of level of detail geometries the model piece has.

Used for: Models and Animation

```
virtual LTRESULT ILTModel::GetNumModelOBBS( HOBJECT hObj,
                                              uint32 & num_obbs
                                              ) [virtual]
```

```
virtual LTRESULT ILTModel::GetNumNodes( HOBJECT hObj,
                                         uint32 & num_nodes
                                         ) [virtual]
```

**Parameters:**

*hObj* ModelInstance handle  
*NumNodes* return value.

**Returns:**

if object is either invalid, or not an OT\_MODEL

Returns the number of transformation nodes the model has.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetNumPieces( HOBJECT hObj,
                                          uint32 & NumPieces
                                          ) [virtual]
```

**Parameters:**

*hObj* Model instance reference.  
*NumPieces* return value, representing the number of pieces the model has.

**Returns:**

**LT\_INVALIDPARAM** invalid model instance **LT\_OK** operation is done.

Get the number of pieces a model has.

```
virtual LTRESULT ILTModel::GetParent( HOBJECT hObj,
                                      HMODELNODE node,
                                      HMODELNODE & parent
                                      ) [virtual]
```

**Parameters:**

*hObj* ModelInstance handle  
*hNode* a handle to the node  
*parent* a handle to the parent of the node.

**Returns:**

if object is either invalid, or not an OT\_MODEL

Given a node, return a handle to its parent. The value INVALID\_MODEL\_NODE will be returned if the node is the root node.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetPiece( HOBJECT hObj,
                                     const char * pPieceName,
                                     HMODELPIECE & hPiece
                                     ) [virtual]
```

**Parameters:**

*hObj* Model instance reference.  
*pPieceName* name of model piece.  
*hPiece* return param handle.

**Returns:**

**LT\_OK** is success . **LT\_NOTINITIALIZED** if there is no model associated with hObj. **LT\_NOTFOUND** if the requested piece does not exist.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetPieceHideStatus( HOBJECT      hObj,
                                              HMODELPIECE hPiece,
                                              bool &      bHidden
                                              ) [virtual]
```

**Parameters:**

*hObj* Model instance handle.  
*hPiece* Piece instance handle.  
*bHidden* return parameter on hidden status.

**Returns:**

**LT\_OK** is success . **LT\_INVALIDPARAMS** if hObj is not associated with an OT\_MODEL

Discover whether a piece on the model is hidden. Only supports the first 32 pieces in the model. [SetPieceHideStatus\(\)](#) returns **LT\_NOCHANGE** if the piece is already hidden (not an error).

**See also:**

[SetPieceHideStatus\(\)](#)

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetPlaybackState( HOBJECT      hModel,
                                           ANIMTRACKERID TrackerID,
                                           uint32 &      flags
                                           ) [virtual]
```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier.  
*flags* state flags.

**Returns:**

**LT\_INVALIDPARAMS** on invalid hModel, **LT\_INVALIDVERSION** if *not* using SMP networking, otherwise **LT\_OK**.

Fills in flags with a combination of the **MS\_** flags in basedefs\_de.h.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetPlaying( HOBJECT      hModel,
                                      ANIMTRACKERID TrackerID
                                      ) [virtual]
```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier.

**Returns:**

**LT\_NO** if not Playing, or an error, **LT\_INVALIDVERSION** if *not* using SMP networking; otherwise, returns **LT\_YES**.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetRootNode( HOBJECT      hObj,
                                       HMODELNODE & hNode
                                       ) [virtual]
```

**Parameters:**

*hObj* ModelInstance handle  
*hNode* a handle to the root node.

**Returns:**

if object is either invalid, or not an OT\_MODEL

GetRootNode returns a handle to the root of the transformation hierarchy for the model.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetSkinFilename( HOBJECT hObj,
                                           uint32    skin_index,
                                           char *    pRetFileNames,
                                           uint32    fileBufLen
                                           ) [pure virtual]
```

```
virtual LTRESULT ILTModel::GetSocket( HOBJECT      hObj,
                                     const char *  pSocketName,
                                     HMODEL SOCKET & hSocket
                                     ) [virtual]
```

**Parameters:**

*hObj* Model Reference.  
*pSocketName* name of the socket to be fetched.  
*hSocket* return parameter of socket handle.

**Returns:**

**LT\_NOTINITIALIZED** if the Model Reference is invalid. **LT\_NOTFOUND** if socket it not found in the model eference.

Method used to get sockets handles from models by name.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetSocketTransform( HOBJECT hObj,
                                              HMODELCKET hSocket,
                                              LTransform & transform,
                                              bool bWorldSpace
                                              ) [virtual]
```

**Parameters:**

*hObj* Model instance handle.  
*hSocket* Socket instance handle.  
*transform* Transform  
*bWorldSpace* What space the transform is in in relation to parent, true is in world space, false the transform is local to model space.

**Returns:**

**LT\_OK** if operation succeeded. **LT\_ERROR** one of these caused the error : 1. The transform hierarchy has become invalid, by having a transformation that is not invertable. 2. the socket is invalid. **LT\_NOTINITIALIZED** if there is no model associated with hObj.

Fill **pos** and **rot** with the current transform for the socket. If **bWorldSpace** is **TRUE**, then the transform is returned in world space. Otherwise, the transform is local to the model (based on its position and rotation).

**Note:** This is potentially an expensive process: the model's animation has to be (partially) reevaluated for every call to get GetSocketTransform. If this function is going to be called a lot, consider caching the animation data by enabling the transformation cache.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::GetWeightSet( HOBJECT hModel,
                                         ANIMTRACKERID TrackerID,
                                         HMODELWEIGHTSET & hSet
                                         ) [virtual]
```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier  
*hSet* Model animation blending weight set handle.

**Returns:**

**LT\_OK** operation succeeded. **LT\_INVALIDPARAMS** TrackerID is invalid **LT\_INVALIDVERSION** if *not* using SMP networking

Get the weight set for this tracker.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::IsModelDBLoaded( HMODELDB hModelDB ) [pure virtual]
```

**Parameters:**

*hModelDB,handle* to model database

Test if hModelDB points to a model database.

```
virtual LTRESULT ILTModel::RemoveNodeControlFn( HOBJECT hObj,
                                                  NodeControlFn pFn,
                                                  void * pUserData
                                                  ) [virtual]
```

**Parameters:**

*hObj* Model to have function removed from.  
*fn* Function to remove.  
*pUserData* User data of function to remove (or NULL for any).

**Returns:**

**LT\_INVALIDPARAMS** if hObj is either NULL, or not of type **OT\_MODEL** **LT\_OK** otherwise

Given a model it will look through its registered callbacks, and upon finding any that matches the specified criteria, it will remove it from the list of callback functions.

**Note:** If the pUserData value is NULL, the user data of the callbacks will not be checked for matching criteria, but if it is non-null it will be checked.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::RemoveNodeControlFn( HOBJECT hObj,
                                                  HMODELNODE hNode,
                                                  NodeControlFn pFn,
                                                  void * pUserData
                                                  ) [virtual]
```

**Parameters:**

*hObj* Model to have function removed from.  
*hNode* Node to have function removed from.  
*fn* Function to remove.  
*pUserData* User data of function to remove (or NULL for any).

**Returns:**

**LT\_INVALIDPARAMS** if hObj is either NULL, or not of type **OT\_MODEL** **LT\_OK** otherwise

Given a node it will look through its registered callbacks, and upon finding any that matches the specified criteria, it will remove it from the list of callback functions.

**Note:** If the pUserData value is NULL, the user data of the callbacks will not be checked for matching criteria, but if it is non-null it will be checked.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::RemoveTracker( HOBJECT hModel,
                                           ANIMTRACKERID TrackerID
                                           ) [virtual]
```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier associated with tracker to remove.

**Returns:**

**LT\_INVALIDPARAMS** on invalid hModel or if you try to remove the main tracker, **LT\_INVALIDVERSION** if *not* using SMP networking, **LT\_NOTFOUND** if TrackerID not in use for the model instance; otherwise **LT\_OK**.

Remove trackers on the model.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::ResetAnim( HOBJECT hModel,
                                       ANIMTRACKERID TrackerID
                                       ) [virtual]
```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier.

**Returns:**

**LT\_INVALIDVERSION** if *not* using SMP networking, otherwise **LT\_OK**

Resets all animation to first frame.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::SetAnimRate( HOBJECT hModel,
                                         ANIMTRACKERID TrackerID,
                                         float fRate
                                         ) [virtual]
```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier  
*rate* a float value indicating the rate modifier on the animation.

**Returns:**

**LT\_OK** **LT\_INVALID\_PARAMS**. if TrackerID is invalid **LT\_INVALIDVERSION** if *not* using SMP networking

Sets the current rate of change between frames. Zero rate would stop the animation. Minus one, runs the animation backwards. This is effects only the rate during updates.

Functional area: Model Animations.

```
virtual LTRESULT ILTModel::SetCurAnim( HOBJECT hModel,
                                         ANIMTRACKERID TrackerID,
                                         HMODELANIM hAnim
                                         ) [virtual]
```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier.  
*hAnim* Animation Handle

**Returns:**

**LT\_INVALIDPARAMS** on invalid TrackerID, **LT\_INVALIDVERSION** if *not* using SMP networking, otherwise **LT\_OK**.

Set the current animation for the tracker.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::SetCurAnimTime( HOBJECT hModel,
                                             ANIMTRACKERID TrackerID,
                                             uint32 curTime
                                             ) [virtual]
```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier of choice.  
*curTime* time in miliseconds

**Returns:**

**LT\_OK** or if TrackerID is invalid **LT\_INVALID\_PARAMS** if TrackerID is null **LT\_NOTINITIALIZED** if the TrackerID is not currently associated with animation data **LT\_INVALIDVERSION** if *not* using SMP networking.

Moves the animation's current time to curTime.

Used for: Models and Animation.

```
virtual LTRESULT ILTModel::SetHintNode( HOBJECT hModel,
```

```

    ANIMTRACKERID TrackerID,
    HMODELNODE hNode
) [virtual]

```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier of choice.  
*hNode* Node to use for movement encoding transform hints

**Returns:**

**LT\_OK** if TrackerID is invalid **LT\_INVALID\_PARAMS** if TrackerID is null **LT\_NOTINITIALIZED** if the TrackerID is not currently associated with animation data **LT\_INVALIDVERSION** if *not* using SMP networking.

Sets node to use for movement encoding hints.

Used for: Models and Animation.

```

virtual LTRESULT ILTModel::SetLooping( HOBJECT hModel,
    ANIMTRACKERID TrackerID,
    bool bLooping
) [virtual]

```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier.  
*bLooping* looping flag, **LT\_YES** loops, **LT\_NO**, stops looping.

**Returns:**

**LT\_INVALIDVERSION** if *not* using SMP networking, otherwise **LT\_OK**

Set whether the current animation on this tracker is looping.

Used for: Models and Animation.

```

virtual LTRESULT ILTModel::SetPieceHideStatus( HOBJECT hObj,
    HMODELPIECE hPiece,
    bool bHidden
) [virtual]

```

**Parameters:**

*hObj* Model instance handle.  
*hPiece* Model piece instance handle.  
*bHidden* true/false param to switch the hidden status of piece.

**Returns:**

**LT\_OK** is success . **LT\_INVALIDPARAMS** if hObj is not associated with an OT\_MODEL **LT\_NOCHANGE** piece already hidden.

Hide or unhide a piece on the model (they are all unhidden by default). Only supports the first 32 pieces in the model. [SetPieceHideStatus\(\)](#) returns **LT\_NOCHANGE** if the piece is already hidden (not an error).

**See also:**

[GetPieceHideStatus\(\)](#)

Used for: Models and Animation.

```

virtual LTRESULT ILTModel::SetPlaying( HOBJECT hModel,
    ANIMTRACKERID TrackerID,
    bool bPlaying
) [virtual]

```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier of choice.  
*bPlaying* boolean flag, true sets animtracker to play, false to not play.

**Returns:**

**LT\_INVALIDPARAMS** if TrackerID is invalid, **LT\_INVALIDVERSION** if *not* using SMP networking, otherwise, returns **LT\_OK**.

Set whether this tracker is playing its animation.

Used for: Models and Animation.

```

virtual LTRESULT ILTModel::SetWeightSet( HOBJECT hModel,
    ANIMTRACKERID TrackerID,
    HMODELWEIGHTSET hSet
) [virtual]

```

**Parameters:**

*hModel* Model instance handle.  
*TrackerID* Animation tracker identifier  
*hSet* Model animation blending weight set handle.

**Returns:**

**LT\_OK** operation succeeded. **LT\_INVALIDPARAMS** TrackerID is invalid **LT\_INVALIDVERSION** if *not* using SMP networking

Set the animation blending weight set for this tracker.

Used for: Models and Animation.

virtual [LTRESULT](#) ILTModel::UncacheModelDB( [HMODELDB](#) & *hModelDB* ) [pure virtual]

**Parameters:**

*hModelDB*, *handle* to model database.

Turns this model into a normal model : If there are no refs associated with it, it gets released. Resets hModelDB to zero. return LT\_OK on success.

virtual [LTRESULT](#) ILTModel::UpdateMainTracker( [HOBJECT](#) *hObj*,  
float *fUpdateDelta*  
) [virtual]

**Parameters:**

*pObj* Model Instance handle.

*fUpdateDelta* time in miliseconds.

**Returns:**

LT\_INVALIDPARAMS on invalid hObj, otherwise LT\_OK.

Update the main anim tracker by **fUpdateDelta** seconds (This is useful for updating the animation on a model outside of a client update.)

Used for: Models and Animation.

virtual [LTRESULT](#) ILTModel::UpdateModelOBB( [HOBJECT](#) *hObj*,  
[ModelOBB](#) \*  
) [virtual]

The documentation for this class was generated from the following file:

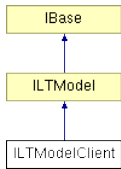
- [iltmodel.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

## ILTModelClient Class Reference

Inheritance diagram for ILTModelClient:



### Public Types

enum { [ILTModelClient\\_VERSION](#) = 0 + ILTModel :: \_ILTModel\_VERSION\_ \* 100 }

### Public Member Functions

virtual [LTRESULT](#) [GetRenderStyle](#) ( [HOBJECT](#) hModel, [uint32](#) hPiece, [CRenderStyle](#) \*\*ppRenderStyle)=0

virtual [LTRESULT](#) [SetRenderStyle](#) ( [HOBJECT](#) pModel, [uint32](#) hPiece, [CRenderStyle](#) \*pRenderStyle)=0

virtual [LTRESULT](#) [UncacheModelDB](#) (const char \*filename)=0

### Detailed Description

The **ILTModelClient** interface contains client only functions for manipulating models and animations.

Define a holder to get this interface like this:

[define\\_holder](#)([ILTModelClient](#), your\_var);

### Member Enumeration Documentation

anonymous enum

**Enumeration values:**

[\\_ILTModelClient\\_VERSION\\_](#)

### Member Function Documentation

virtual [LTRESULT](#) ILTModelClient::GetRenderStyle( [HOBJECT](#) *hModel*,  
[uint32](#) *hPiece*,  
[CRenderStyle](#) \*\* *ppRenderStyle*

) [pure virtual]

**Parameters:**  
*hModel* ModelInstance handle.  
*index* Index into model instance's list of renderstyles.  
*pRenderStyle* Pointer to the render style.

**Returns:**  
**LT\_OK**

Get's the PieceLOD's render style.

**See also:**  
[SetPieceHideStatus\(\)](#)

Used for: Models Rendering.

virtual [LTRESULT](#) ILTModelClient::SetRenderStyle( [HOBJECT](#) *pModel*,  
[uint32](#) *hPiece*,  
[CRenderStyle](#) \* *pRenderStyle*  
) [pure virtual]

**Parameters:**  
*hModel* Model instance handle.  
*index* Index into model instance's list of renderstyles.  
*pRenderStyle* Pointer to the render style.

**Returns:**  
**LT\_OK**

Set's the PieceLOD's render style.

**See also:**  
[GetPieceHideStatus\(\)](#)

Used for: Models Rendering.

virtual [LTRESULT](#) ILTModelClient::UncacheModelDB( const char \* *filename* ) [pure virtual]

**Parameters:**  
*hModel* Model instance handle.  
*filename* model filename

**Returns:**  
**LT\_OK**

**See also:**  
[UncacheModelDB\(\)](#)

Client side only, uncache a file by filename.

Used for: Models and Animation.

The documentation for this class was generated from the following file:

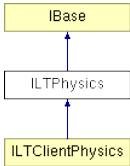
- [iltmodel.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

## ILTPhysics Class Reference

Inheritance diagram for ILTPhysics:



**Public Types**  
enum { [\\_ILTPhysics\\_VERSION](#) = 0 }

**Public Member Functions**  
virtual [LTRESULT](#) IsWorldObject( [HOBJECT](#) hObj)  
virtual [LTRESULT](#) GetStairHeight( float &fHeight)=0  
virtual [LTRESULT](#) SetStairHeight( float fHeight)=0

virtual [LTRESULT](#) [GetMass](#) ([HOBJECT](#) hObj, float \*m)  
 virtual [LTRESULT](#) [SetMass](#) ([HOBJECT](#) hObj, float m)  
 virtual [LTRESULT](#) [GetFrictionCoefficient](#) ([HOBJECT](#) hObj, float \*u)  
 virtual [LTRESULT](#) [SetFrictionCoefficient](#) ([HOBJECT](#) hObj, float u)  
 virtual [LTRESULT](#) [GetObjectDims](#) ([HOBJECT](#) hObj, [LTVector](#) \*d)  
 virtual [LTRESULT](#) [SetObjectDims](#) ([HOBJECT](#) hObj, [LTVector](#) \*d, [uint32](#) flag)=0  
 virtual [LTRESULT](#) [GetVelocity](#) ([HOBJECT](#) hObj, [LTVector](#) \*v)  
 virtual [LTRESULT](#) [SetVelocity](#) ([HOBJECT](#) hObj, const [LTVector](#) \*v)=0  
 virtual [LTRESULT](#) [GetForceIgnoreLimit](#) ([HOBJECT](#) hObj, float &f)  
 virtual [LTRESULT](#) [SetForceIgnoreLimit](#) ([HOBJECT](#) hObj, float f)  
 virtual [LTRESULT](#) [GetAcceleration](#) ([HOBJECT](#) hObj, [LTVector](#) \*a)  
 virtual [LTRESULT](#) [SetAcceleration](#) ([HOBJECT](#) hObj, const [LTVector](#) \*a)=0  
 virtual [LTRESULT](#) [MoveObject](#) ([HOBJECT](#) hObj, const [LTVector](#) \*p, [uint32](#) flag)=0  
 virtual [LTRESULT](#) [GetStandingOn](#) ([HOBJECT](#) hObj, [CollisionInfo](#) \*info)  
 virtual [LTRESULT](#) [GetGlobalForce](#) ([LTVector](#) &a)=0  
 virtual [LTRESULT](#) [SetGlobalForce](#) (const [LTVector](#) &a)=0

## Protected Attributes

[ClientServerType](#) m\_ClientServerType

---

## Member Enumeration Documentation

anonymous enum

**Enumeration values:**

[\\_ILTPysics\\_VERSION\\_](#)

---

## Member Function Documentation

virtual [LTRESULT](#) ILTPysics::GetAcceleration( [HOBJECT](#) hObj,  
[LTVector](#) \* a  
 ) [virtual]

virtual [LTRESULT](#) ILTPysics::GetForceIgnoreLimit( [HOBJECT](#) hObj,  
 float & f  
 ) [virtual]

virtual [LTRESULT](#) ILTPysics::GetFrictionCoefficient( [HOBJECT](#) hObj,  
 float \* u  
 ) [virtual]

virtual [LTRESULT](#) ILTPysics::GetGlobalForce( [LTVector](#) & a ) [pure virtual]

virtual [LTRESULT](#) ILTPysics::GetMass( [HOBJECT](#) hObj,  
 float \* m  
 ) [virtual]

virtual [LTRESULT](#) ILTPysics::GetObjectDims( [HOBJECT](#) hObj,  
[LTVector](#) \* d  
 ) [virtual]

virtual [LTRESULT](#) ILTPysics::GetStairHeight( float & fHeight ) [pure virtual]

virtual [LTRESULT](#) ILTPysics::GetStandingOn( [HOBJECT](#) hObj,  
[CollisionInfo](#) \* info  
 ) [virtual]

virtual [LTRESULT](#) ILTPysics::GetVelocity( [HOBJECT](#) hObj,  
[LTVector](#) \* v  
 ) [virtual]

virtual [LTRESULT](#) ILTPysics::IsWorldObject( [HOBJECT](#) hObj ) [virtual]

virtual [LTRESULT](#) ILTPysics::MoveObject( [HOBJECT](#) hObj,  
 const [LTVector](#) \* p,  
[uint32](#) flag  
 ) [pure virtual]

```
virtual LTRESULT ILTPysics::SetAcceleration( HOBJECT hObj,
                                             const LTVector * a
                                             ) [pure virtual]

virtual LTRESULT ILTPysics::SetForceIgnoreLimit( HOBJECT hObj,
                                                  float f
                                                  ) [virtual]

virtual LTRESULT ILTPysics::SetFrictionCoefficient( HOBJECT hObj,
                                                    float u
                                                    ) [virtual]

virtual LTRESULT ILTPysics::SetGlobalForce( const LTVector & a ) [pure virtual]

virtual LTRESULT ILTPysics::SetMass( HOBJECT hObj,
                                     float m
                                     ) [virtual]

virtual LTRESULT ILTPysics::SetObjectDims( HOBJECT hObj,
                                           LTVector * d,
                                           uint32 flag
                                           ) [pure virtual]

virtual LTRESULT ILTPysics::SetStairHeight( float fHeight ) [pure virtual]

virtual LTRESULT ILTPysics::SetVelocity( HOBJECT hObj,
                                         const LTVector * v
                                         ) [pure virtual]
```

## Field Documentation

[ClientServerType](#) [ILTPysics::m\\_ClientServerType](#) [protected]

The documentation for this class was generated from the following file:

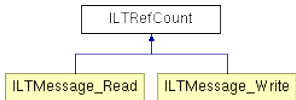
- [iltphysics.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

### ILTRefCount Class Reference

Inheritance diagram for ILTRefCount:



#### Public Member Functions

```
ILTRefCount ()
virtual ~ILTRefCount ()
virtual void IncRef ()
virtual void DecRef ()
virtual void StaticDecRef ()
virtual uint32 GetRefCount () const
```

#### Private Member Functions

```
virtual void Free ()=0
```

#### Private Attributes

[uint32](#) m\_nRefCount

## Detailed Description

This file defines the standard reference counting interface, as well as a convenient class for properly using objects which implement this interface.

## Constructor & Destructor Documentation

ILRefCount::ILRefCount( ) [inline]

virtual ILRefCount::~ILRefCount( ) [inline, virtual]

## Member Function Documentation

virtual void ILRefCount::DecRef( ) [inline, virtual]

virtual void ILRefCount::Free( ) [private, pure virtual]

virtual [uint32](#) ILRefCount::GetRefCount( ) const [inline, virtual]

virtual void ILRefCount::IncRef( ) [inline, virtual]

virtual void ILRefCount::StaticDecRef( ) [inline, virtual]

## Field Documentation

[uint32](#) [ILRefCount::m\\_nRefCount](#) [private]

The documentation for this class was generated from the following file:

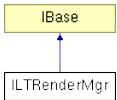
- [iltrefcount.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

## ILTRenderMgr Class Reference

Inheritance diagram for ILTRenderMgr:



### Public Types

enum { [\\_ILTRenderMgr\\_VERSION](#) = 0 }

### Public Member Functions

virtual void [Init](#) ()=0  
virtual void [Term](#) ()=0  
virtual [LTRESULT](#) [AddEffectShader](#) (const char \*pFileName, int EffectShaderID, const [uint32](#) \*pVertexElements, [uint32](#) VertexElementsSize, [HEFFECTPOOL](#) EffectPoolID)=0  
virtual [LTEffectShader](#) \* [GetEffectShader](#) (int EffectShaderID)=0  
virtual [LTRESULT](#) [CreateEffectPool](#) ([HEFFECTPOOL](#) EffectPoolID)=0  
virtual [LTRESULT](#) [CreateRenderTarget](#) ([uint32](#) nWidth, [uint32](#) nHeight, [ERenderTargetFormat](#) eRenderTargetFormat, [EStencilBufferFormat](#) eStencilBufferFormat, [HRENDERTARGET](#) hRenderTarget)=0  
virtual [LTRESULT](#) [InstallRenderTarget](#) ([HRENDERTARGET](#) hRenderTarget)=0  
virtual [LTRESULT](#) [RemoveRenderTarget](#) ([HRENDERTARGET](#) hRenderTarget)=0  
virtual [LTRESULT](#) [StretchRectRenderTargetToBackBuffer](#) ([HRENDERTARGET](#) hRenderTarget)=0  
virtual [LTRESULT](#) [GetRenderTargetDims](#) ([HRENDERTARGET](#) hRenderTarget, [uint32](#) &nWidth, [uint32](#) nHeight)=0

```
virtual LTRESULT StoreDefaultRenderTarget ()=0
virtual LTRESULT RestoreDefaultRenderTarget ()=0
virtual LTRESULT SnapshotCurrentFrame ()=0
virtual LTRESULT SaveCurrentFrameToPrevious ()=0
virtual LTRESULT UploadCurrentFrameToEffect (LTEffectShader *pEffect, const char *szParam)=0
virtual LTRESULT UploadPreviousFrameToEffect (LTEffectShader *pEffect, const char *szParam)=0
```

---

## Detailed Description

The ILTRenderMgr interface.

Define a holder to get this interface like this:

```
define\_holder(ILTRenderMgr, your_var);
```

---

## Member Enumeration Documentation

anonymous enum

**Enumeration values:**

[\\_ILTRenderMgr\\_VERSION](#)

---

## Member Function Documentation

```
virtual LTRESULT ILTRenderMgr::AddEffectShader( const char *    pFileName,
                                              int              EffectShaderID,
                                              const uint32 *    pVertexElements,
                                              uint32           VertexElementsSize,
                                              HEFFECTPOOL       EffectPoolID
                                              ) [pure virtual]
```

**Parameters:**

*pFileName*            Filename of the Effect file.  
*EffectShaderID*      Effect Shader ID to create.  
*pVertexElements*    **D3DVERTEXELEMENT9** array pointer.  
*VertexElementsSize* Size in bytes, of the **D3DVERTEXELEMENT9** array.  
*EffectPoolID*        ID handle of the **HEFFECTPOOL** for this effect. If none is desired, pass in **NO\_EFFECT\_POOL**.

**Returns:**

**LT\_INVALIDPARAMS** if the filename pointer is invalid, returns **LT\_NOTFOUND** if the file is not found, returns **LT\_ERROR** for any other problem otherwise, returns **LT\_OK**.

Create a new EffectShader

```
virtual LTRESULT ILTRenderMgr::CreateEffectPool( HEFFECTPOOL EffectPoolID ) [pure virtual]
```

**Parameters:**

*EffectPoolID*      EffectPool ID to create.

**Returns:**

**LT\_OK** if successful, **LT\_ERROR** on failure.

Create an EffectPool.

```
virtual LTRESULT ILTRenderMgr::CreateRenderTarget( uint32      nWidth,
                                                  uint32      nHeight,
                                                  ERenderTargetFormat eRenderTargetFormat,
                                                  EStencilBufferFormat eStencilBufferFormat,
                                                  HRENDERTARGET    hRenderTarget
                                                  ) [pure virtual]
```

**Parameters:**

*nWidth*              Width of the render target. (in pixels)  
*nHeight*             Height of the render target. (in pixels)  
*eRenderTargetFormat* Format of the render target.  
*eStencilBufferFormat* Format of the depth/stencil buffer. Pass in **STFMT\_UNKNOWN** to use the default depth stencil buffer format.  
*hRenderTarget*       ID of the render target.

**Returns:**

**LT\_OK** if successful, **LT\_ERROR** on failure.

Create a new RenderTarget.

```
virtual LTEffectShader* ILTRenderMgr::GetEffectShader( int EffectShaderID ) [pure virtual]
```

**Parameters:**

*EffectShaderID*      Effect Shader ID to find.

**Returns:**

Valid [LTEffectShader](#) pointer if the EffectShader is found, otherwise returns **NULL**.

Find an EffectShader in the database.

```
virtual LTRESULT ILTRenderMgr::GetRenderTargetDims( HRENDERTARGET hRenderTarget,
```

```

        uint32 & nWidth,
        uint32 nHeight
    ) [pure virtual]

```

**Parameters:**

*hRenderTarget* Render target handle.  
*nWidth* Holds the result for the width of the render target.  
*nHeight* Holds the result for the height of the render target.

**Returns:**

**LT\_OK** if successful, **LT\_ERROR** on failure.

Get the dimensions (in pixels) of the render target surface.

```
virtual void ILTRenderMgr::Init( ) [pure virtual]
```

```
virtual LTRESULT ILTRenderMgr::InstallRenderTarget( HRENDERTARGET hRenderTarget ) [pure virtual]
```

**Parameters:**

*hRenderTarget* Render target handle.

**Returns:**

**LT\_OK** if successful, **LT\_ERROR** on failure.

Set the current render target.

```
virtual LTRESULT ILTRenderMgr::RemoveRenderTarget( HRENDERTARGET hRenderTarget ) [pure virtual]
```

**Parameters:**

*hRenderTarget* Render target handle.

**Returns:**

**LT\_OK** if successful, **LT\_ERROR** on failure.

Delete this render target.

```
virtual LTRESULT ILTRenderMgr::RestoreDefaultRenderTarget( ) [pure virtual]
```

**Returns:**

**LT\_OK** if successful, **LT\_ERROR** on failure.

Restore the default render target. This should be done before `ILTCClient::End3d()` is called. This will restore the back buffer.

```
virtual LTRESULT ILTRenderMgr::SaveCurrentFrameToPrevious( ) [pure virtual]
```

**Returns:**

**LT\_OK** if successful, **LT\_ERROR** on failure.

Take the current snapshot and save it for the next frame to use. This **MUST** be done after [SnapshotCurrentFrame\(\)](#) is called.

```
virtual LTRESULT ILTRenderMgr::SnapshotCurrentFrame( ) [pure virtual]
```

**Returns:**

**LT\_OK** if successful, **LT\_ERROR** on failure.

Take the current contents of the back buffer and save them to a texture. This can be used for effects such a refraction.

```
virtual LTRESULT ILTRenderMgr::StoreDefaultRenderTarget( ) [pure virtual]
```

**Returns:**

**LT\_OK** if successful, **LT\_ERROR** on failure.

Store the current render target. This should be used at the beginning of a render frame before other render targets are applied.

```
virtual LTRESULT ILTRenderMgr::StretchRectRenderTargetToBackBuffer( HRENDERTARGET hRenderTarget ) [pure virtual]
```

**Parameters:**

*hRenderTarget* Render target handle.

**Returns:**

**LT\_OK** if successful, **LT\_ERROR** on failure.

Use `StretchRect()` to copy the render target surface to the back buffer.

```
virtual void ILTRenderMgr::Term( ) [pure virtual]
```

```
virtual LTRESULT ILTRenderMgr::UploadCurrentFrameToEffect( LTEffectShader * pEffect,
    const char * szParam
) [pure virtual]
```

**Parameters:**

*pEffect* [LTEffectShader](#) pointer.  
*szParam* Parameter name to use to pass into the EffectShader.

**Returns:**

**LT\_OK** if successful, **LT\_ERROR** on failure.

This will call SetTexture on the ID3DXEffect using szParam as the constant name. This is usually done right after a call to [SnapshotCurrentFrame\(\)](#).

```
virtual LTRESULT ILTRenderMgr::UploadPreviousFrameToEffect( LTEffectShader * pEffect,
                                                         const char * szParam
                                                         ) [pure virtual]
```

**Parameters:**  
*pEffect* [LTEffectShader](#) pointer.  
*szParam* Parameter name to use to pass into the EffectShader.

**Returns:**  
**LT\_OK** if successful, LT\_ERROR on failure.

This will call SetTexture on the ID3DXEffect using szParam as the constant name. The previous frame **MUST** be saved on a previous frame or the function will fail.

The documentation for this class was generated from the following file:

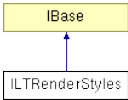
- [iltrendermgr.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

# API Reference

## ILTRenderStyles Class Reference

Inheritance diagram for ILTRenderStyles:



### Public Types

enum { [ILTRenderStyles\\_VERSION](#) = 0 }

### Public Member Functions

virtual [CRenderStyle](#) \* [DuplicateRenderStyle](#) ([CRenderStyle](#) \*pRenderStyle)=0  
virtual [CRenderStyle](#) \* [CreateRenderStyle](#) (bool bSetToDefault=true)=0  
virtual [CRenderStyle](#) \* [LoadRenderStyle](#) (const char \*szFilename)=0  
virtual void [FreeRenderStyle](#) ([CRenderStyle](#) \*pRenderStyle)=0

### Detailed Description

ILTRenderStyles Interface. The RenderStyles interface allows users to create, query, modify, and destroy individual render styles.

### Member Enumeration Documentation

anonymous enum

**Enumeration values:**  
*ILTRenderStyles\_VERSION*

### Member Function Documentation

virtual [CRenderStyle](#) \* ILTRenderStyles::CreateRenderStyle( bool *bSetToDefault* = true ) [pure virtual]  
Create a render style. You can then set all it's internals yourself.

**Parameters:**  
*bSetToDefault* A Boolean value indicating whether or not to set default values for the returned [CRenderStyle](#).

**Returns:**  
A [CRenderStyle](#) pointer to the newly created render style.

Used For: Rendering.

virtual [CRenderStyle](#) \* ILTRenderStyles::DuplicateRenderStyle( [CRenderStyle](#) \* *pRenderStyle* ) [pure virtual]  
Duplicate a render style (maybe you want to change it just for a certain object - you better dup it first, then change it, and set it).

**Parameters:**  
*pRenderStyle* A [CRenderStyle](#) pointer to the render style to be duplicated.

**Returns:**  
A [CRenderStyle](#) pointer to the duplicate render style.

Used For: Rendering.

virtual void ILTRenderStyles::FreeRenderStyle( [CRenderStyle](#) \* *pRenderStyle* ) [pure virtual]

Free a render style - you no longer need it (may or may not be internally freed - depending on the ref count).

**Parameters:**

*pRenderStyle* A pointer to the render style to be freed.

Used For: Rendering.

virtual [CRenderStyle](#)\* ILTRenderStyles::LoadRenderStyle( const char \* *szFilename* ) [pure virtual]

Load a render style (ltb file) and create a render object for it.

**Parameters:**

*szFilename* A pointer to a string containing the path to the render style file.

**Returns:**

A [CRenderStyle](#) pointer to the new render style.

Used For: Rendering.

The documentation for this class was generated from the following file:

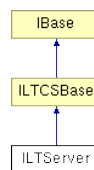
- [iltrenderstyles.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
 2006, All Rights Reserved.

## API Reference

## ILTServer Class Reference

Inheritance diagram for ILTServer:



### Public Types

enum { [ILTServer\\_VERSION](#) = 4 + ILTCSBase::\_ILTCSBase\_VERSION\_ \* 100 }

### Public Member Functions

virtual [LTRESULT](#) GetWorldBox( [LTVector](#) &min, [LTVector](#) &max)=0  
 virtual [LTRESULT](#) FindWorldModelObjectIntersections( [HOBJECT](#) hWorldModel, const [LTVector](#) &vNewPos, const [LTRotation](#) &rNewRot, [BaseObjArray](#)<  
   [HOBJECT](#)> &objArray)=0  
 virtual [LTRESULT](#) GetObjectPos( [HLOCALOBJ](#) hObj, [LTVector](#) \*pPos)=0  
 virtual [LTRESULT](#) GetObjectRotation( [HLOCALOBJ](#) hObj, [LTRotation](#) \*pRotation)=0  
 virtual [LTRESULT](#) MoveObject( [HOBJECT](#) hObj, const [LTVector](#) \*pNewPos)=0  
 virtual [LTRESULT](#) GetStandingOn( [HOBJECT](#) hObj, [CollisionInfo](#) \*pInfo)=0  
 virtual [LTRESULT](#) FindNamedObjects( const char \*pName, [BaseObjArray](#)< [HOBJECT](#)> &objArray, [uint32](#) \*nTotalFound=NULL)=0  
 virtual [LTRESULT](#) SendToObject( [ILTMMessage\\_Read](#) \*pMsg, [HOBJECT](#) hSender, [HOBJECT](#) hSendTo, [uint32](#) flags)=0  
 virtual [LTRESULT](#) SendToServer( [ILTMMessage\\_Read](#) \*pMsg, [HOBJECT](#) hSender, [uint32](#) flags)=0  
 virtual [LTRESULT](#) SetObjectSFXMessage( [HOBJECT](#) hObject, [ILTMMessage\\_Read](#) \*pMsg)=0  
 virtual [LTRESULT](#) SendToClient( [ILTMMessage\\_Read](#) \*pMsg, [HCLIENT](#) hSendTo, [uint32](#) flags)=0  
 virtual [LTRESULT](#) SendSFXMessage( [ILTMMessage\\_Read](#) \*pMsg, const [LTVector](#) &pos, [uint32](#) flags)=0  
 virtual [LTRESULT](#) GetClientPing( [HCLIENT](#) hClient, float &ping)=0  
 virtual bool GetClientData( [HCLIENT](#) hClient, [uint8](#) \*pData, int &maxLen)=0  
 virtual bool SetClientData( [HCLIENT](#) hClient, [uint8](#) const \*pData, int len)=0  
 virtual [LTRESULT](#) GetClientAddr( [HCLIENT](#) hClient, [uint8](#) pAddr[4], [uint16](#) \*pPort)=0  
 virtual void BreakInterObjectLink( [HOBJECT](#) hOwner, [HOBJECT](#) hLinked)=0  
 virtual void BreakInterObjectLink( [HOBJECT](#) hObj, [HOBJECT](#) hLinked, bool bNotify)=0  
 virtual [LTRESULT](#) GetObjectName( [HOBJECT](#) hObject, char \*pName, [uint32](#) nameBufSize)=0  
 virtual [LTRESULT](#) GetNetFlags( [HOBJECT](#) hObj, [uint32](#) &flags)=0  
 virtual [LTRESULT](#) SetNetFlags( [HOBJECT](#) hObj, [uint32](#) flags)=0  
 virtual [LTRESULT](#) GetSaveFileVersion( [uint32](#) &nSaveFileVersion)=0  
 virtual [LTRESULT](#) GetNumClassProps( const [HCLASS](#) hClass, [uint32](#) &count)=0  
 virtual [LTRESULT](#) GetClassProp( const [HCLASS](#) hClass, const [uint32](#) iProp, [ClassPropInfo](#) &info)=0  
 virtual [LTRESULT](#) GetClassName( const [HCLASS](#) hClass, char \*pName, [uint32](#) maxNameBytes)=0  
 virtual [LTRESULT](#) RemoveObject( [HOBJECT](#) hObj)=0  
 virtual [LTRESULT](#) RemoveObject( const [HCLASS](#) hClass, [ILTBaseClass](#) \*pObject)=0  
 virtual [LTRESULT](#) ThreadLoadFile( const char \*pFilename, [uint32](#) type)=0  
 virtual [LTRESULT](#) UnloadFile( const char \*pFilename, [uint32](#) type)=0  
 virtual [LTRESULT](#) GetHPolyObject( const [HPOLY](#) hPoly, [HOBJECT](#) &hObject)=0

### Data Fields

```

bool(* IntersectSegment )(IntersectQuery *pQuery, IntersectInfo *pInfo)
bool(* CastRay )(IntersectQuery *pQuery, IntersectInfo *pInfo)
ObjectList *(* GetPointAreas )(const LTVector *pPoint)
ObjectList *(* GetBoxIntersectors )(const LTVector *pMin, const LTVector *pMax)
ObjectList *(* FindObjectsTouchingSphere )(const LTVector *pPosition, float radius)
LTRESULT(* SetObjectRotation )(HOBJECT hObj, const LTRotation *pRotation)
void(* SetObjectPos )(HOBJECT hObj, const LTVector *pPos)
LTRESULT(* RotateObject )(HOBJECT hObj, const LTRotation *pRotation)
void(* TiltToPlane )(HOBJECT hObj, const LTVector *pNormal)
LTRESULT(* TeleportObject )(HOBJECT hObj, const LTVector *pNewPos)
LTRESULT(* GetLastCollision )(CollisionInfo *pInfo)
bool(* GetPointShade )(const LTVector *pPoint, LTVector *pColor)
void(* RelinquishList )(ObjectList *pList)
ObjectList *(* CreateObjectList )()
ObjectLink *(* AddObjectToList )(ObjectList *pList, HOBJECT hObj)
void(* RemoveObjectFromList )(ObjectList *pList, HOBJECT hObj)
LTRESULT(* AttachClient )(HCLIENT hParent, HCLIENT hChild)
LTRESULT(* DetachClient )(HCLIENT hClient)
HCLIENT(* GetNextClient )(HCLIENT hPrev)
HCLIENTREF(* GetNextClientRef )(HCLIENTREF hRef)
uint32(* GetClientRefInfoFlags )(HCLIENTREF hClient)
bool(* GetClientRefName )(HCLIENTREF hClient, char *pName, int maxLen)
HOBJECT(* GetClientRefObject )(HCLIENTREF hClient)
HCLIENT(* GetClientHandle )(uint32 clientID)
uint32(* GetClientID )(HCLIENT hClient)
bool(* GetClientName )(HCLIENT hClient, char *pName, int maxLen)
bool(* SetClientName )(HCLIENT hClient, const char *pName, int MaxLen)
void(* SetClientInfoFlags )(HCLIENT hClient, uint32 dwClientFlags)
uint32(* GetClientInfoFlags )(HCLIENT hClient)
void(* SetClientUserData )(HCLIENT hClient, void *pData)
void(* GetClientUserData )(HCLIENT hClient)
LTRESULT(* KickClient )(HCLIENT hClient)
LTRESULT(* SetClientViewPos )(HCLIENT hClient, const LTVector *pPos)
void(* RunGameConString )(const char *pString)
void(* SetGameConVar )(const char *pName, const char *pVal)
HCONVAR(* GetGameConVar )(const char *pName)
LTRESULT(* GetPropString )(const char *pPropName, char *pRet, int maxLen)
LTRESULT(* GetPropVector )(const char *pPropName, LTVector *pRet)
LTRESULT(* GetPropColor )(const char *pPropName, LTVector *pRet)
LTRESULT(* GetPropReal )(const char *pPropName, float *pRet)
LTRESULT(* GetPropFlags )(const char *pPropName, uint32 *pRet)
LTRESULT(* GetPropBool )(const char *pPropName, bool *pRet)
LTRESULT(* GetPropLongInt )(const char *pPropName, int32 *pRet)
LTRESULT(* GetPropRotation )(const char *pPropName, LTRotation *pRet)
LTRESULT(* GetPropRotationEuler )(const char *pPropName, LTVector *pAngles)
LTRESULT(* GetPropGeneric )(const char *pPropName, GenericProp *pProp)
LTRESULT(* DoesPropExist )(const char *pPropName, int32 *pPropType)
HOBJECT(* ObjectToHandle )(ILTBaseClass *pObject)
ILTBaseClass(* HandleToObject )(HOBJECT hObject)
bool(* UpperStrcmp )(const char *pStr1, const char *pStr2)
int(* Parse )(const char *pCommand, const char **pNewCommandPos, char *argBuffer, char **argPointers, int *nArgs)
LTRESULT(* CreateInterObjectLink )(HOBJECT hOwner, HOBJECT hLinked)
LTRESULT(* CreateAttachment )(HOBJECT hParent, HOBJECT hChild, const char *pSocketName, LTVector *pOffset, LTRotation *pRotationOffset, HATTACHMENT *pAttachment)
LTRESULT(* RemoveAttachment )(HATTACHMENT hAttachment)
LTRESULT(* FindAttachment )(HOBJECT hParent, HOBJECT hChild, HATTACHMENT *hAttachment)
bool(* GetObjectColor )(HOBJECT hObject, float *r, float *g, float *b, float *a)
bool(* SetObjectColor )(HOBJECT hObject, float r, float g, float b, float a)
HOBJECT(* GetNextObject )(HOBJECT hObj)
HOBJECT(* GetNextInactiveObject )(HOBJECT hObj)
void(* SetNextUpdate )(HOBJECT hObj, LTFLOAT nextUpdate)
LTRESULT(* GetObjectScale )(HOBJECT hObj, LTVector *pScale)
void(* ScaleObject )(HOBJECT hObj, const LTVector *pNewScale)
void(* SetObjectState )(HOBJECT hObj, int state)
int(* GetObjectState )(HOBJECT hObj)
void(* SetBlockingPriority )(HOBJECT hObj, uint8 pri)
uint8(* GetBlockingPriority )(HOBJECT hObj)
LTRESULT(* ClipSprite )(HOBJECT hObj, HPOLY hPoly)
void(* GetLightColor )(HOBJECT hObj, float *r, float *g, float *b)
void(* SetLightColor )(HOBJECT hObj, float r, float g, float b)
float(* GetLightRadius )(HOBJECT hObj)
void(* SetLightRadius )(HOBJECT hObj, float radius)
LTRESULT(* UpdateSessionName )(const char *sName)
LTRESULT(* GetSessionName )(char *sName, uint32 dwBufferSize)
LTRESULT(* SetModelNodeHideStatus )(HOBJECT hObj, char *pNodeName, bool bHidden)
LTRESULT(* GetModelNodeHideStatus )(HOBJECT hObj, char *pNodeName, bool *bHidden)
const char *(* GetAnimName )(HOBJECT hObject, HMODELANIM hAnim)
void(* SetModelPlaying )(HOBJECT hObj, bool bPlaying)
bool(* GetModelPlaying )(HOBJECT hObj)
bool(* GetModelFilenames )(HOBJECT hObj, char *pFilename, int fileBufLen, char *pSkinName, int skinBufLen)
LTRESULT(* SaveObjects )(const char *pszSaveFileName, ObjectList *pList, uint32 dwParam, uint32 flags)
LTRESULT(* RestoreObjects )(const char *pszRestoreFileName, uint32 dwParam, uint32 flags)
LTRESULT(* LoadWorld )(const char *pszWorldFileName, uint32 flags)
LTRESULT(* RunWorld )()
LTRESULT(* SendToServerApp )(ILTMessage Read &msg)
LTRESULT(* GetMaxConnections )(uint32 &nMaxConnections)
FileEntry *(* GetFileList )(const char *pDirName)
void(* FreeFileList )(FileEntry *pHead)
LTRESULT(* GetGameInfo )(void **ppData, uint32 *pLen)
HCLASS(* GetClass )(const char *pName)
HCLASS(* GetObjectClass )(HOBJECT hObject)
LTRESULT(* GetStaticObject )(HCLASS hClass, HOBJECT *obj)

```

```

        bool(* IsKindOf)(HCLASS hClass, HCLASS hTest)
ILTBaseClass *(* CreateObject)(HCLASS hClass, ObjectCreateStruct *pStruct)
ILTBaseClass *(* CreateObjectProps)(HCLASS hClass, ObjectCreateStruct *pStruct, const char *pszProps)
uint32(* GetServerFlags)()
uint32(* SetServerFlags)(uint32 flags)
LTRESULT(* CacheFile)(uint32 fileType, const char *pFilename)
void(* StartCounter)(LTCounter *pCounter)
uint32(* EndCounter)(LTCounter *pCounter)
LTFLOAT(* Random)(LTFLOAT min, LTFLOAT max)
int(* IntRandom)(int min, int max)
LTFLOAT(* RandomScale)(LTFLOAT scale)
void(* BPrint)(const char *pMsg,...)
void(* DebugOut)(const char *pMsg,...)
LTRESULT(* GetSkyDef)(SkyDef *pDef)
LTRESULT(* SetSkyDef)(SkyDef *pDef)
LTRESULT(* AddObjectToSky)(HOBJECT hObj, uint32 index)
LTRESULT(* RemoveObjectFromSky)(HOBJECT hObj)
LTRESULT(* GetGlobalLightObject)(HOBJECT *hObj)
LTRESULT(* SetGlobalLightObject)(HOBJECT hObj)

```

---

## Detailed Description

ILTServer interface. This is the interface to most server-side LithTech functionality.

---

## Member Enumeration Documentation

anonymous enum

**Enumeration values:**

`_ILTServer_VERSION_`

---

## Member Function Documentation

```

virtual void ILTServer::BreakInterObjectLink(
    HOBJECT hObj,
    HOBJECT hLinked,
    bool bNotify
) [pure virtual]

```

**Parameters:**

*hObj* Object that links will be broken from.

*hLinked* Linked object.

*bNotify* Should the linked object be notified?

Break all inter-object links between the object and the linked object, if one exists. If *hLinked* is **LTNUL** all links to *hObj* are broken.

**Note:**

**MID\_LINKBROKEN** will *not* be called if *bNotify* is **false**.

Used for: Object.

```

virtual void ILTServer::BreakInterObjectLink(
    HOBJECT hOwner,
    HOBJECT hLinked
) [pure virtual]

```

**Parameters:**

*hOwner* Owner object.

*hLinked* Linked object.

Break an inter-object link between the owner and the linked object, if one exists. You can only break a link from the owner, since the linked object doesn't know it is linked to the owner.

**Note:**

**MID\_LINKBROKEN** will *not* be called.

Used for: Object.

```

virtual LTRESULT ILTServer::FindNamedObjects( const char *
    BaseObjArray< HOBJECT > & objArray,
    uint32 *
    nTotalFound = NULL
) [pure virtual]

```

**Parameters:**

*pName* Name to search for.

*objArray* (return) An array of objects.

*nTotalFound* (return) Number of objects in array.

Get all objects with the given name.

Used for: Object.

```

virtual LTRESULT ILTServer::FindWorldModelObjectIntersections(
    HOBJECT
    const LTVector &
    const LTRotation &
    BaseObjArray< HOBJECT > & objArray
    [pure virtual]
    hWorldModel,
    vNewPos,
    rNewRot,
)

```

)

```
virtual LTRESULT ILTServer::GetClassName( const HCLASS hClass,
                                         char *      pName,
                                         uint32      maxNameBytes
                                       ) [pure virtual]
```

**Parameters:**

*hClass*            Class to query.  
*pName*            (return) Buffer for name.  
*maxNameBytes* (return) Size of buffer.

**Returns:**

LT\_OK

Get the name of a class.

Used for: Server Management.

```
virtual LTRESULT ILTServer::GetClassProp( const HCLASS hClass,
                                         const uint32 iProp,
                                         ClassPropInfo & info
                                       ) [pure virtual]
```

**Parameters:**

*hClass*    Class to query.  
*iProp*    Property to query.  
*info*    (return) Property info.

**Returns:**

LT\_OK

Get the [ClassDef](#) info from a class. This is useful if you want to access the property names of the class.

Used for: Server Management.

```
virtual LTRESULT ILTServer::GetClientAddr( HCLIENT hClient,
                                           uint8      pAddr[4],
                                           uint16 *   pPort
                                         ) [pure virtual]
```

**Parameters:**

*hClient*    Client to query  
*pAddr*    Array for holding the client's address  
*pPort*    [uint16](#) for holding the client's port

**Returns:**

LT\_ERROR if an invalid parameter is provided; otherwise, returns LT\_OK

Retrieve the TCP/IP communications address and port for the given client

Note: The local client will return 0.0.0.0:0

Used for: Client Management

```
virtual bool ILTServer::GetClientData( HCLIENT hClient,
                                       uint8 *   pData,
                                       int &      maxLen
                                       ) [pure virtual]
```

**Parameters:**

*hClient*    Client to query.  
*pData*    (out) Buffer to hold client data. Pass NULL for length returned in maxLen.  
*maxLen*    (in,out) Buffer length. Sets bytes copied.

**Returns:**

true if successful; otherwise, returns false.

Get the name of a client.

Used for: Server Management.

```
virtual LTRESULT ILTServer::GetClientPing( HCLIENT hClient,
                                           float &   ping
                                         ) [pure virtual]
```

**Parameters:**

*hClient*    Client to query.  
*ping*    (return) Ping time.

**Returns:**

LT\_NOTINITIALIZED if client is not connected. Otherwise, LT\_OK.

Get the ping time of a client.

Used for: Server Management.

```
virtual LTRESULT ILTServer::GetHPolyObject( const HPOLY hPoly,
                                           HOBJECT & hObject
                                           ) [pure virtual]
```

**Parameters:**

*hPoly* Polygon to query.

*hObject* (return) Object containing hPoly.

Get the world object that a particular [HPOLY](#) comes from.

Used for: Misc.

```
virtual LTRESULT ILTServer::GetNetFlags( HOBJECT hObj,
                                          uint32 & flags
                                          ) [pure virtual]
```

**Parameters:**

*hObj* The object to query.

*flags* (return) Flags (a combination of [NETFLAG\\_](#) defines).

**Returns:**

[LT\\_INVALIDPARAMS](#) if hObj is not found, else [LT\\_OK](#)

Get the net flags of an object.

Used for: Object.

```
virtual LTRESULT ILTServer::GetNumClassProps( const HCLASS hClass,
                                              uint32 & count
                                              ) [pure virtual]
```

**Parameters:**

*hClass* Class to query.

*count* (return) Number of properties.

**Returns:**

[LT\\_OK](#)

Get the number of properties defined by the given class.

Used for: Server Management.

```
virtual LTRESULT ILTServer::GetObjectName( HOBJECT hObject,
                                           char * pName,
                                           uint32 nameBufSize
                                           ) [pure virtual]
```

**Parameters:**

*hObject* Object to query.

*pName* (return) Buffer to hold name.

*nameBufSize* Buffer size.

**Returns:**

[LT\\_INVALIDPARAMS](#) if the buffer is not large enough, else [LT\\_OK](#)

Get the name of an object.

Used for: Object.

```
virtual LTRESULT ILTServer::GetObjectPos( HLOCALOBJ hObj,
                                          LTVector * pPos
                                          ) [pure virtual]
```

Gets the position of the specified object

Used for: Object.

Implements [ILTCSBase](#).

```
virtual LTRESULT ILTServer::GetObjectRotation( HLOCALOBJ hObj,
                                              LTRotation * pRotation
                                              ) [pure virtual]
```

Gets the rotation of the specified object

Used for: Object.

Implements [ILTCSBase](#).

```
virtual LTRESULT ILTServer::GetSaveFileVersion( uint32 & nSaveFileVersion ) [pure virtual]
```

**Parameters:**

*nSaveFileVersion* - The save file version number.

**Returns:**

[LT\\_OK](#) on success.

Access the engine's save file version.

Used for: Misc.

```
virtual LTRESULT ILTServer::GetStandingOn( HOBJECT hObj,
                                           CollisionInfo * pInfo
                                           ) [pure virtual]
```

```
virtual LTRESULT ILTServer::GetWorldBox( LTVector & min,
                                           LTVector & max
                                           ) [pure virtual]
```

```
virtual LTRESULT ILTServer::MoveObject( HOBJECT hObj,
                                         const LTVector * pNewPos
                                         ) [pure virtual]
```

```
virtual LTRESULT ILTServer::RemoveObject( const HCLASS hClass,
                                           LTBaseClass * pObject
                                           ) [pure virtual]
```

**Parameters:**

*hClass* Class of the object

*pObject* Class-only object to remove

**Returns:**

LT\_OK for valid objects, LT\_ERROR otherwise

Delete a class-only object

Used for: Class-only Objects

```
virtual LTRESULT ILTServer::RemoveObject( HOBJECT hObj ) [pure virtual]
```

Re-declaration of [ILTCSBase::RemoveObject\(HOBJECT\)](#)

Implements [ILTCSBase](#).

```
virtual LTRESULT ILTServer::SendSFXMessage( ILTMessage_Read * pMsg,
                                             const LTVector & pos,
                                             uint32 flags
                                             ) [pure virtual]
```

**Parameters:**

*pMsg* SFX Message to send

*pos* Position of message reference (in world coordinates).

*flags* Message flags.

**Returns:**

LT\_INVALIDPARAMS if the message is invalid; otherwise, returns LT\_OK.

**See also:**

[SpecialEffectNotify\(\)](#)

[ILTCommon::CreateMessage\(\)](#)

[ILTMessage\\_Write::Read\(\)](#)

Send an "instant" special effect message to all the clients who can see **pos**. The clients will receive the message through the [IClientShell::SpecialEffectNotify\(\)](#) function.

Used for: Messaging.

```
virtual LTRESULT ILTServer::SendToClient( ILTMessage_Read * pMsg,
                                           HCLIENT hSendTo,
                                           uint32 flags
                                           ) [pure virtual]
```

**Parameters:**

*pMsg* Message to send to the client.

*hSendTo* Destination client. Use NULL to send to all clients.

*flags* Message flags.

**Returns:**

LT\_INVALIDPARAMS if the message is invalid; otherwise, returns LT\_OK.

**See also:**

[OnMessage\(\)](#)

[ILTCommon::CreateMessage\(\)](#)

[ILTMessage\\_Write::Read\(\)](#)

Send the message to the client. The client shell will receive the message through its OnMessage() function.

Used for: Messaging.

```
virtual LTRESULT ILTServer::SendToObject( ILTMessage_Read * pMsg,
                                           HOBJECT hSender,
```

```

        HOBJECT      hSendTo,
        uint32      flags
    ) [pure virtual]

```

**Parameters:**

*pMsg*     **Message** to send to the object.  
*hSender* Source object (can be **NULL**).  
*hSendTo* Destination object.  
*flags*     Message flags.

**Returns:**

**LT\_INVALIDPARAMS** if the message is invalid; otherwise, returns **LT\_OK**.

**See also:**

ObjectMessageFn()

[ILTCommon::CreateMessage\(\)](#)

[ILTMessage\\_Write::Read\(\)](#)

Send a message to an object. The object will receive the message through its ObjectMessageFn function.

Used for: Messaging.

```

virtual LTRESULT ILTServer::SendToServer( ILTMessage\_Read * pMsg,
        HOBJECT      hSender,
        uint32      flags
    ) [pure virtual]

```

**Parameters:**

*pMsg*     **Message** to send to the server.  
*hSender* Source object (cannot be **NULL**).  
*flags*     Message flags.

**Returns:**

**LT\_INVALIDPARAMS** if the message is invalid, or the server shell is invalid; otherwise, returns **LT\_OK**.

**See also:**

OnObjectMessage()

[ILTCommon::CreateMessage\(\)](#)

[ILTMessage\\_Write::Read\(\)](#)

Send the message to the server shell. The server shell will receive the message through its OnObjectMessage() function.

Used for: Messaging.

```

virtual bool ILTServer::SetClientData( HCLIENT  hClient,
        uint8 const * pData,
        int      len
    ) [pure virtual]

```

**Parameters:**

*hClient* Client to query.  
*pData*   Buffer holding desired data  
*len*     Buffer length.

**Returns:**

**true** if successful; otherwise, returns **false**.

Set the data of a client.

Used for: Server Management.

```

virtual LTRESULT ILTServer::SetNetFlags( HOBJECT hObj,
        uint32 flags
    ) [pure virtual]

```

**Parameters:**

*hObj*   The object to modify.  
*flags*   Flags (a combination of **NETFLAG\_** defines).

**Returns:**

**LT\_INVALIDPARAMS** if hObj is not found, else **LT\_OK**

Set the net flags of an object.

Used for: Object.

```

virtual LTRESULT ILTServer::SetObjectSFXMessage( HOBJECT      hObject,
        ILTMessage\_Read * pMsg
    ) [pure virtual]

```

**Parameters:**

*hObject* Object to modify.  
*pMsg*     **Message** to associate with this object.

**Returns:**

**LT\_INVALIDPARAMS** if the message is invalid; otherwise, returns **LT\_OK**.

Sets up a special effect message. If your object has a special effect message, the `SpecialEffectNotify()` of the client shell will be called.

**Note:** An object can have only **one** associated special effect message. If the object already has a special effect message, then calling this function will replace it.

**See also:**

`SpecialEffectNotify()`

[ILTCommon::CreateMessage\(\)](#)

[ILTMessage\\_Write::Read\(\)](#)

Used for: Messaging.

```
virtual LTRESULT ILTServer::ThreadLoadFile( const char * pFilename,
                                           uint32      type
                                           ) [pure virtual]
```

**Parameters:**

*pFilename* File to load.

*type* File type (one of the **FT\_** defines in [itcodes.h](#)). Types currently supported are **FT\_MODEL** and **FT\_TEXTURE**. (Textures are not loaded on the server. A message is sent to the clients to load the texture in their thread.)

**Returns:**

**LT\_UNSUPPORTED** if you pass an invalid type, **LT\_MISSINGFILE** if the file is missing, **LT\_ALREADYEXISTS** if the file is already in memory, or **LT\_INPROGRESS** if it is already loading in the thread. (**Note:** Many of these returns don't signify errors.)

Load the specified file in the loader thread. Also send a message to clients saying to do the same. When the operation is completed, this function calls [IServerShell::FileLoadNotify](#) with a result (which can be an error if the file is missing). Objects can be created using the file but they will not be visible until all files they need are loaded.

Used for: Misc.

```
virtual LTRESULT ILTServer::UnloadFile( const char * pFilename,
                                         uint32      type
                                         ) [pure virtual]
```

**Parameters:**

*pFilename* File to unload.

*type* File type (one of the **FT\_** defines in [itcodes.h](#)).

**Returns:**

**LT\_ERROR** if the file is being used and can't be unloaded, or **LT\_NOTFOUND** if the file isn't currently loaded.

Unload the data.

Used for: Misc.

## Field Documentation

[ObjectLink](#)\*([ILTServer::AddObjectToList](#))([ObjectList](#) \*pList, [HOBJECT](#) hObj)

**Parameters:**

*pList* Object list to modify.

*hObj* Object to add.

**Returns:**

a pointer to the new [ObjectLink](#)

Add objects to a list.

Used for: Object.

[LTRESULT](#)\*([ILTServer::AddObjectToSky](#))([HOBJECT](#) hObj, [uint32](#) index)

**Parameters:**

*hObj* Object to add.

*index* Index of sky object. Each object should have a unique index.

**Returns:**

**LT\_ERROR** - Either the server object is NULL, or the index is beyond the maximum range allowed (i.e. *index* >= **MAX\_SKYOBJECTS**).

**LT\_OK** - Successful.

Add an object to the sky list.

Used for: Special FX.

[LTRESULT](#)\*([ILTServer::AttachClient](#))([HCLIENT](#) hParent, [HCLIENT](#) hChild)

**Parameters:**

*hParent* Parent client.

*hChild* Child client.

**Returns:**

**LT\_INVALIDPARAMS** if *hParent* == *hChild*, otherwise **LT\_OK**

Attach one client to another. Messages sent to **hParent** will also go to **hChild**, and the client object of **hChild** will be reported as the object of **hParent**. This is useful for demo playback when you want to watch another player's view.

Used for: Server Management.

void(\* [LTServer::BPrint](#))(const char \*pMsg,...)

**Parameters:**

*pMsg* The printf-styled format string to display. This parameter can be followed by the usual format parameters.

Only use this for debugging. This sends an **STC\_BPRINT** message with the string in it.

Used for: Misc.

[LTRESULT](#)(\* [LTServer::CacheFile](#))([uint32](#) fileType, const char \*pFilename)

**Parameters:**

*fileType* Type of file (one of the **FT** defines in [ltcodes.h](#)).

*pFilename* File to cache.

**Returns:**

**LT\_NOTFOUND** if it can't find the file.

Cache the given file. Only call this from ServerShell::CacheFiles.

Used for: Misc.

bool(\* [LTServer::CastRay](#))([IntersectQuery](#) \*pQuery, [IntersectInfo](#) \*pInfo)

[LTRESULT](#)(\* [LTServer::ClipSprite](#))([HOBJECT](#) hObj, [HPOLY](#) hPoly)

**Parameters:**

*hObj* Sprite to modify.

*hPoly* Polygon to clip to. Use **INVALID\_POLY** to un-clip the sprite.

**Returns:**

**LT\_ERROR** - *hObj* is invalid (NULL, or not an **OT\_SPRITE**).

**LT\_OK** - Successful.

Clips the sprite onto a polygon.

Used for: Special FX.

[LTRESULT](#)(\* [LTServer::CreateAttachment](#))([HOBJECT](#) hParent, [HOBJECT](#) hChild, const char \*pSocketName, [LTVector](#) \*pOffset, [LTRotation](#) \*pRotationOffset, [HAttachment](#) \*pAttachment)

**Parameters:**

*hParent* Parent object.

*hChild* Child object.

*pSocketName* Socket name (on parent).

*pOffset* Position offset (in local parent coordinates).

*pRotationOffset* Rotation offset (in local parent coordinates).

*pAttachment* (return) Attachment handle.

**Returns:**

**LT\_OK**, **LT\_NODENOTFOUND**, or **LT\_INVALIDPARAMS**.

Attaches **hChild** to **hParent**. If **pSocketName** is set, this function will attach **hChild** to a specific node on **hParent**. LithTech will automatically detach if you remove **hParent**, but it will *not* automatically detach if you remove **hChild**. If you remove the child without removing the attachment, the results are undefined.

Used for: Object.

[LTRESULT](#)(\* [LTServer::CreateInterObjectLink](#))([HOBJECT](#) hOwner, [HOBJECT](#) hLinked)

**Parameters:**

*hOwner* Owner object.

*hLinked* Linked object.

**Returns:**

**LT\_ERROR** if trying to make a link to itself.

If you want to hold on to an **HOBJECT** and receive notification when the object is removed from the world, you can create an inter-object link between the two. The owner will get **MID\_LINKBROKEN** notification when the link is being broken (by either the owner or linked object being removed). If a link between the two objects already exists, the function will not create another link and return **LT\_OK**.

Used for: Object.

[LTBaseClass](#)(\* [LTServer::CreateObject](#))([HCLASS](#) hClass, [ObjectCreateStruct](#) \*pStruct)

**Parameters:**

*hClass* Class to use for object creation.

*pStruct* Description of the object.

**Returns:**

Handle to object.

Create an object using the default type of the object. If you pass in **LTNUL** for pStruct, OnPreCreate will not be called, and the object will not be added to the

world. (This is only valid for creating CF\_CLASSONLY objects.)

Used for: Object.

[ObjectList](#)\*([\\* ILTServer::CreateObjectList](#)())()

**Returns:**

A pointer to the allocated list.

Allocate a list.

**Note:** Free the list with [RelinquishList\(\)](#).

**See also:**

[RelinquishList\(\)](#)

Used for: Object.

[ILTBaseClass](#)\*([\\* ILTServer::CreateObjectProps](#))([HCLASS](#) hClass, [ObjectCreateStruct](#) \*pStruct, const char \*pszProps)

**Parameters:**

*hClass* Class to use for object creation.

*pStruct* Description of object.

*pszProps* Properties string.

**Returns:**

Handle to object.

Create an object, using a string to specify the properties of the object.

Used for: Object.

void([\\* ILTServer::DebugOut](#))(const char \*pMsg,...)

**Parameters:**

*pMsg* The printf-styled format string to display. This parameter can be followed by the usual format parameters.

Output a TRACE message to the Debug Output window. Newlines must be explicitly used.

Used for: Misc.

[LTRESULT](#)\*([\\* ILTServer::DetachClient](#))([HCLIENT](#) hClient)

**Parameters:**

*hClient* Child client to detach.

**Returns:**

LT\_OK

Detach a child client from its parent attachment.

Used for: Server Management.

[LTRESULT](#)\*([\\* ILTServer::DoesPropExist](#))(const char \*pPropName, [int32](#) \*pPropType)

**Parameters:**

*pPropName* Property name (case-sensitive).

*pPropType* (return) Property type to retrieve (one of the **PT** defines).

**Returns:**

Returns **LT\_NOTFOUND** if it doesn't exist.

See if a property exists.

Used for: Object.

[uint32](#)\*([\\* ILTServer::EndCounter](#))([LTCounter](#) \*pCounter)

**Parameters:**

*pCounter* The counter that the engine will stop.

**Returns:**

Microseconds since StartCounter was called.

Stop a counter. Use this to time sections of code. Timing is done in microseconds (millionths of a second).

Used for: Misc.

[LTRESULT](#)\*([\\* ILTServer::FindAttachment](#))([HOBJECT](#) hParent, [HOBJECT](#) hChild, [HATTACHMENT](#) \*hAttachment)

**Parameters:**

*hParent* Parent object.

*hChild* Child object.

*hAttachment* (return) Attachment handle, or **NULL** if there is an error.

**Returns:**

**LT\_ERROR**, **LT\_NOTFOUND**, or **LT\_OK** if it found it.

Look for an attachment on the parent.

Used for: Object.

[ObjectList](#)\*([ILTServer::FindObjectsTouchingSphere](#))(const [LTVector](#) \*pPosition, float radius)

void([ILTServer::FreeFileList](#))([FileEntry](#) \*pHead)

**Parameters:**

*pHead* Head of file list.

Use when you're done with each list you get from [GetFileList\(\)](#).

**See also:**

[GetFileList\(\)](#)

Used for: Misc.

const char\*([ILTServer::GetAnimName](#))([HOBJECT](#) hObject, [HMODELANIM](#) hAnim)

**Parameters:**

*hObject* The model to query.

*hAnim* The animation to query.

**Returns:**

Pointer to the name of the animation, or [LTNULL](#) if the handle is invalid.

Get the animation name from an animation.

Used for: Models and Animation.

[uint8](#)\*([ILTServer::GetBlockingPriority](#))([HOBJECT](#) hObj)

**Parameters:**

*hObj* The object to query.

**Returns:**

The blocking priority.

Get the blocking priority (defaults to 0). See the FAQ for a description of how this works.

Used for: Object.

[ObjectList](#)\*([ILTServer::GetBoxIntersecters](#))(const [LTVector](#) \*pMin, const [LTVector](#) \*pMax)

[HCLASS](#)\*([ILTServer::GetClass](#))(const char \*pName)

**Parameters:**

*pName* Name of class to retrieve.

**Returns:**

[LTNULL](#) if the class doesn't exist; otherwise, returns the handle of the requested class.

Used for: Server Management.

[HCLIENT](#)\*([ILTServer::GetClientHandle](#))([uint32](#) clientID)

**Parameters:**

*clientID* Client to query.

**Returns:**

Client handle.

Get the handle of the client identified by clientID. [GetClientHandle\(\)](#) will return [NULL](#) if clientID is not valid.

Used for: Server Management.

[uint32](#)\*([ILTServer::GetClientID](#))([HCLIENT](#) hClient)

**Parameters:**

*hClient* Client to query.

**Returns:**

Client ID.

Get the unique ID of a client.

Used for: Server Management.

[uint32](#)\*([ILTServer::GetClientInfoFlags](#))([HCLIENT](#) hClient)

**Parameters:**

*hClient* Client to query.

**Returns:**

Client info flags (a combination of the [CIF\\_](#) flags above).

Get client info flags.

Used for: Server Management.

bool(\* [ILTServer::GetClientName](#))([HCLIENT](#) hClient, char \*pName, int maxLen)

**Parameters:**

*hClient* Client to query.  
*pName* (return) Buffer to hold client name.  
*maxLen* Buffer length.

**Returns:**

**true** if successful; otherwise, returns **false**.

Get the name of a client.

Used for: Server Management.

uint32(\* [ILTServer::GetClientRefInfoFlags](#))([HCLIENTREF](#) hClient)

**Parameters:**

*hClient* Client to query.

**Returns:**

Client reference info flags.

**HCLIENTREF** objects are used for comparison to **HCLIENT** objects.

Used for: Server Management.

bool(\* [ILTServer::GetClientRefName](#))([HCLIENTREF](#) hClient, char \*pName, int maxLen)

**Parameters:**

*hClient* Client to query.  
*pName* (return) Buffer to hold client reference name.  
*maxLen* Buffer length.

**Returns:**

false if the client ref is invalid. Otherwise, true.

An **HCLIENTREF** is used for comparison to an **HCLIENT**.

Used for: Server Management.

[HOBJECT](#)(\* [ILTServer::GetClientRefObject](#))([HCLIENTREF](#) hClient)

**Parameters:**

*hClient* Client to query.

**Returns:**

Handle to client reference object.

An **HCLIENTREF** is used for comparison to an **HCLIENT**.

Used for: Server Management.

void(\* [ILTServer::GetClientUserData](#))([HCLIENT](#) hClient)

**Parameters:**

*hClient* Client to query.

**Returns:**

Pointer to user data.

Get user data for **HCLIENT** objects.

Used for: Server Management.

[FileEntry](#)(\* [ILTServer::GetFileList](#))(const char \*pDirName)

**Parameters:**

*pDirName* The directory path.

**Returns:**

A pointer to a non-circular linked list of filenames terminated by **DNULL**.

Get a list of files and directories in a directory (pass in "" to start with). Use [FreeFileList\(\)](#) to avoid memory leaks.

**See also:**

[FreeFileList\(\)](#)

Used for: Misc.

[HCONVAR](#)(\* [ILTServer::GetGameConVar](#))(const char \*pName)

**Parameters:**

*pName* Console variable name.

**Returns:**

Handle to console variable object.

Get a handle to a console variable.

Used for: Misc.

[LTRESULT](#)(\* [ILTServer::GetGameInfo](#))(void \*\*ppData, [uint32](#) \*pLen)

Grants access to the **m\_pGameInfo** from the [StartGameRequest](#) structure.

**Parameters:**

*ppData* Filled with a pointer to the GameInfo data.  
*pLen* Filled with the length of the GameInfo data.

**Returns:**

LT\_OK

**Note:** This is *not* copy of the data. The pointer filled in can be NULL if there is no game info data.

Used for: Server Management.

[LTRESULT](#)(\* [ILTServer::GetGlobalLightObject](#))([HOBJECT](#) \*hObj)

**Parameters:**

*hObj* (return) Global light object to query.

**Returns:**

LT\_ERROR - *hObj* is NULL.

LT\_OK - Successful.

Get the global light object.

Used for: Special FX.

[LTRESULT](#)(\* [ILTServer::GetLastCollision](#))([CollisionInfo](#) \*pInfo)

void(\* [ILTServer::GetLightColor](#))([HOBJECT](#) hObj, float \*r, float \*g, float \*b)

**Parameters:**

*hObj* [Light](#) to query.  
*r* (return) Red value (0-1).  
*g* (return) Green value (0-1).  
*b* (return) Blue value (0-1).

Get the color of a light. When you create a light, its color defaults to (0,0,0).

**Note:** This merely calls [GetObjectColor\(\)](#).

**See also:**

[SetLightColor\(\)](#)

[GetObjectColor\(\)](#)

Used for: Special FX.

float(\* [ILTServer::GetLightRadius](#))([HOBJECT](#) hObj)

**Parameters:**

*hObj* [Light](#) to query.

**Returns:**

If *hObj* is an **OT\_LIGHT**, returns the radius of the light. Otherwise returns 1.0.

Get the radius of a light. When you create a light, its radius defaults to 100.

**See also:**

[SetLightRadius\(\)](#)

Used for: Special FX.

[LTRESULT](#)(\* [ILTServer::GetMaxConnections](#))([uint32](#) &nMaxConnections)

**Parameters:**

*nMaxConnections* (return) Filled in with maximum number of remote connections allowed.

**Returns:**

LT\_NOTINITIALIZED if the main network driver is uninitialized; otherwise, returns LT\_OK.

Get the maximum number of remote clients allowed for the session.

Used for: Networking.

bool(\* [ILTServer::GetModelFileNames](#))([HOBJECT](#) hObj, char \*pFilename, int fileBufLen, char \*pSkinName, int skinBufLen)

**Parameters:**

*hObj* Model to query.

*pFilename* (return) Buffer for model filename (can use **LTNUL**).  
*fileBufLen* The **pFilename** buffer length.  
*pSkinName* (return) Buffer for skin filename (can use **LTNUL**).  
*skinBufLen* The **pSkinName** buffer length.

**Returns:**

**false** if the object is not a model.

Get the model filenames. Initializes **pFilename** or **pSkinName** to zero length if the model doesn't have a filename or skin.

Used for: Models and Animation.

[\*\*LRESULT\*\*\(\\* \*\*ILTSer\*\*::\*\*GetModelNodeHideStatus\*\*\)\(\*\*HOBJECT\*\* hObj, char \\*pNodeName, bool \\*bHidden\)](#)

**Parameters:**

*hObj* The model to query.  
*pNodeName* The node to query.  
*bHidden* (return) **true** for hidden, **false** for unhidden.

**Returns:**

**LT\_OK**, **LT\_ERROR**, **LT\_NOCHANGE**, or **LT\_NODENOTFOUND**.

Get the hide/unhide status of a node on the model (they are all unhidden by default).

Used for: Models and Animation.

bool(\* [\*\*ILTSer\*\*::\*\*GetModelPlaying\*\*](#))(**HOBJECT** hObj)

**Parameters:**

*hObj* Model to query.

**Returns:**

**true** if the model is playing; otherwise, returns **false**.

Get the playing state of the model. The default state is **false**.

Used for: Models and Animation.

[\*\*HCLIENT\*\*\(\\* \*\*ILTSer\*\*::\*\*GetNextClient\*\*\)\(\*\*HCLIENT\*\* hPrev\)](#)

**Parameters:**

*hPrev* (input/return) The client to start from. Use **LTNUL** to get the first client.

**Returns:**

Handle to the next client after **hPrev**. **LTNUL** when there are no more.

Get next client.

Used for: Server Management.

[\*\*HCLIENTREF\*\*\(\\* \*\*ILTSer\*\*::\*\*GetNextClientRef\*\*\)\(\*\*HCLIENTREF\*\* hRef\)](#)

**Parameters:**

*hRef* (input/return) The client reference to start from. Use **LTNUL** to get the first client reference.

**Returns:**

Handle to the next client reference after **hRef**, or **LTNUL** when there are no more.

Get next saved client reference. (These are created when a world is saved.)

Used for: Server Management.

[\*\*HOBJECT\*\*\(\\* \*\*ILTSer\*\*::\*\*GetNextInactiveObject\*\*\)\(\*\*HOBJECT\*\* hObj\)](#)

**Parameters:**

*hObj* Object to start from. Use **NUL** to get the first inactive object.

**Returns:**

**NUL** if there are no more inactive objects; otherwise, returns the next inactive object.

Same as [\*\*GetNextObject\(\)\*\*](#), but this iterates over all the inactive objects.

**See also:**

[\*\*GetNextObject\(\)\*\*](#)

Used for: Object.

[\*\*HOBJECT\*\*\(\\* \*\*ILTSer\*\*::\*\*GetNextObject\*\*\)\(\*\*HOBJECT\*\* hObj\)](#)

**Parameters:**

*hObj* Object to start from. Use **NUL** to get first object.

**Returns:**

**NUL** if there are no more objects; otherwise, returns the next object.

Iterate through all the objects in the world. This is generally a bad idea for speed reasons, but sometimes it is necessary.

**See also:**

[\*\*GetNextInactiveObject\(\)\*\*](#)

Used for: Object.

[HCLASS](#)(\* [ILTServer::GetObjectClass](#))([HOBJECT](#) hObject)

**Parameters:**

*hObject* Object to query.

**Returns:**

The class of the object.

Get the class of an object.

Used for: Object.

bool(\* [ILTServer::GetObjectColor](#))([HOBJECT](#) hObject, float \*r, float \*g, float \*b, float \*a)

**Parameters:**

*hObject* Object to query.

*r* (return) Red value (range: 0.0-1.0).

*g* (return) Green value (range: 0.0-1.0).

*b* (return) Blue value (range: 0.0-1.0).

*a* (return) Alpha value (range: 0.0-1.0).

**Returns:**

**false** if hObject is NULL, else **true**

Get object color. All objects default to (0,0,0,1).

Used for: Object.

[LTRESULT](#)(\* [ILTServer::GetObjectScale](#))([HOBJECT](#) hObj, [LTVector](#) \*pScale)

**Parameters:**

*hObj* Object to query.

*pScale* (return) Scale.

**Returns:**

**LT\_ERROR** if jObj or pScale is invalid, else **LT\_OK**

Get the scale of the object (only works on models and sprites).

Used for: Object.

int(\* [ILTServer::GetObjectState](#))([HOBJECT](#) hObj)

**Parameters:**

*hObj* The object to query.

**Returns:**

The object state (a combination of **OBJSTATE\_** defines).

Get the state of an object.

Used for: Object.

[ObjectList](#)(\* [ILTServer::GetPointAreas](#))(const [LTVector](#) \*pPoint)

bool(\* [ILTServer::GetPointShade](#))(const [LTVector](#) \*pPoint, [LTVector](#) \*pColor)

**Parameters:**

*pPoint* The point in world coordinates.

*pColor* (return) The color value at pPoint.

**Returns:**

**DFALSE** if the point is outside the world.

Calculate the lighting shade (RGB, range 0-255) at the point you specify.

Used for: Misc.

[LTRESULT](#)(\* [ILTServer::GetPropBool](#))(const char \*pPropName, bool \*pRet)

**Parameters:**

*pPropName* Property name (case-sensitive).

*pRet* (return) Bool property to retrieve.

**Returns:**

**LT\_NOTFOUND** if property doesn't exist.

Get a boolean property value from the world file.

Used for: Object.

[LTRESULT](#)(\* [ILTServer::GetPropColor](#))(const char \*pPropName, [LTVector](#) \*pRet)

**Parameters:**

*pPropName* Property name (case-sensitive).  
*pRet* (return) Color property to retrieve.

**Returns:**

**LT\_NOTFOUND** if property doesn't exist.

Get a color property value from the world file.

Used for: Object.

[LTRESULT](#)(\* [LTServer::GetPropFlags](#))(const char \*pPropName, [uint32](#) \*pRet)

**Parameters:**

*pPropName* Property name (case-sensitive).  
*pRet* (return) Flags property to retrieve.

**Returns:**

**LT\_NOTFOUND** if property doesn't exist.

Get a flags property value from the world file.

Used for: Object.

[LTRESULT](#)(\* [LTServer::GetPropGeneric](#))(const char \*pPropName, [GenericProp](#) \*pProp)

**Parameters:**

*pPropName* Property name (case-sensitive).  
*pProp* (return) Generic property to retrieve.

**Returns:**

**LT\_NOTFOUND** if the named property is not found, else **LT\_OK**

Fills in the [GenericProp](#) for the different data types. For a list of which property types map to which [GenericProp](#) variable, see the [GenericProp](#) structure.

**Note:** If the property exists, it always initializes the members in the prop first, so if the [GenericProp](#) variable doesn't support that property type, it will be zero

Used for: Object.

[LTRESULT](#)(\* [LTServer::GetPropLongInt](#))(const char \*pPropName, [int32](#) \*pRet)

**Parameters:**

*pPropName* Property name (case-sensitive).  
*pRet* (return) Long integer property to retrieve.

**Returns:**

**LT\_NOTFOUND** if property doesn't exist.

Get a long integer property value from the world file.

Used for: Object.

[LTRESULT](#)(\* [LTServer::GetPropReal](#))(const char \*pPropName, float \*pRet)

**Parameters:**

*pPropName* Property name (case-sensitive).  
*pRet* (return) Real or float property to retrieve.

**Returns:**

**LT\_NOTFOUND** if property doesn't exist.

Get a real or float property value from the world file.

Used for: Object.

[LTRESULT](#)(\* [LTServer::GetPropRotation](#))(const char \*pPropName, [LTRotation](#) \*pRet)

**Parameters:**

*pPropName* Property name (case-sensitive).  
*pRet* (return) Quaternion rotation property to retrieve.

**Returns:**

**LT\_NOTFOUND** if property doesn't exist.

Get a quaternion rotation property value from the world file.

Used for: Object.

[LTRESULT](#)(\* [LTServer::GetPropRotationEuler](#))(const char \*pPropName, [LTVector](#) \*pAngles)

**Parameters:**

*pPropName* Property name (case-sensitive).  
*pAngles* (return) Euler rotation property to retrieve.

**Returns:**

**LT\_NOTFOUND** if property doesn't exist.

Get an Euler rotation property value from the world file.

Used for: Object.

[LTRESULT](#)(\* [ILTServer::GetPropString](#))(const char \*pPropName, char \*pRet, int maxLen)

**Parameters:**

*pPropName* Property name (case-sensitive).  
*pRet* (return) Buffer to hold string property.  
*maxLen* Buffer length.

**Returns:**

**LT\_NOTFOUND** if property doesn't exist.

Get a string property value from the world file.

Used for: Object.

[LTRESULT](#)(\* [ILTServer::GetPropVector](#))(const char \*pPropName, [LTVector](#) \*pRet)

**Parameters:**

*pPropName* Property name (case-sensitive).  
*pRet* (return) Vector property to retrieve.

**Returns:**

**LT\_NOTFOUND** if property doesn't exist.

Get a vector property value from the world file.

Used for: Object.

[uint32](#)(\* [ILTServer::GetServerFlags](#))()

**Returns:**

Current state flags (a combination of the **SS\_** flags above).

Get server state flags.

Used for: Server Management.

[LTRESULT](#)(\* [ILTServer::GetSessionName](#))(char \*sName, [uint32](#) dwBufferSize)

**Parameters:**

*sName* (return) String to fill with session name.  
*dwBufferSize* Size of sName parameter.

**Returns:**

**LT\_NOTINITIALIZED** if the main network driver is uninitialized, or **LT\_ERROR** if session is invalid or the buffer is too small to contain the session name; otherwise, returns **LT\_OK**.

Get the name of the session.

Used for: Networking.

[LTRESULT](#)(\* [ILTServer::GetSkyDef](#))([SkyDef](#) \*pDef)

**Parameters:**

*pDef* (return) Sky definition structure (filled in by function).

**Returns:**

Always returns **LT\_OK**.

Get the sky definition.

Used for: Special FX.

[LTRESULT](#)(\* [ILTServer::GetStaticObject](#))([HCLASS](#) hClass, [HOBJECT](#) \*obj)

**Parameters:**

*hClass* Class to query.  
*obj* (return) Static object.

**Returns:**

**LT\_NOTFOUND** if the server doesn't have a static object of this class.

Get the static object of a class.

Used for: Server Management.

[ILTBaseClass](#)\*([ILTServer::HandleToObject](#))([HOBJECT](#) hObject)

**Parameters:**

*pObject* Handle of object to query.

**Returns:**

Object.

Get handles from objects.

**Note:**

[HandleToObject\(\)](#) will return **NULL** if the given object does not reside on this server (i.e. if it is controlled remotely).

Used for: Server Management.

bool(\* [ILTServer::IntersectSegment](#))([IntersectQuery](#) \*pQuery, [IntersectInfo](#) \*pInfo)  
World control.

int(\* [ILTServer::IntRandom](#))(int min, int max)

**Parameters:**

*min* Minimum extent of range.  
*max* Maximum extent of range.

**Returns:**

A random integer number in the given range.

Used for: Misc.

bool(\* [ILTServer::IsKindOf](#))([HCLASS](#) hClass, [HCLASS](#) hTest)

**Parameters:**

*hClass* Class to query.  
*hTest* Test base class.

**Returns:**

**TRUE** if **hClass** is an **hTest**, or is derived from **hTest**. Returns **FALSE** if **hClass** is not an **hTest**, if **hClass** is not derived from **hTest**, or if **hTest** is **LTNULL**.

Tells if **hClass** is the class of **hTest** (or is derived from **hTest**). If you pass in **LTNULL** for **hTest** (for example, if [GetClass\(\)](#) returns **LTNULL** in [IsKindOf](#)(hClass, GetClass("TestClass"))), then [IsKindOf](#)() will return **FALSE**.

Used for: Object.

[LTRESULT](#)(\* [ILTServer::KickClient](#))([HCLIENT](#) hClient)

**Parameters:**

*hClient* Client to kick.

Kick a client off the server. [OnRemoveClient\(\)](#) will be called.

**See also:**

[OnRemoveClient\(\)](#)

Used for: Server Management.

[LTRESULT](#)(\* [ILTServer::LoadWorld](#))(const char \*pszWorldFileName, [uint32](#) flags)

**Parameters:**

*pszWorldFileName* Name and path in the rez of the world file to load.  
*flags* A combination of the **LOADWORLD\_** flags.

**Returns:**

**LT\_MISSINGWORLDFILE** is the world file cannot be found. Otherwise, **LT\_OK**.

Used for: Server Management.

[HOBJECT](#)(\* [ILTServer::ObjectToHandle](#))([ILTBaseClass](#) \*pObject)

**Parameters:**

*pObject* Object to query.

**Returns:**

Handle to **pObject**.

Get objects from handles.

Used for: Object.

int(\* [ILTServer::Parse](#))(const char \*pCommand, const char \*\*pNewCommandPos, char \*argBuffer, char \*\*argPointers, int \*nArgs)

**Parameters:**

*pCommand* String to parse.  
*pNewCommandPos* (return) New ending position (after semicolon or end of string).  
*argBuffer* Memory for scratchpad.  
*argPointers* (return) Array of pointers to arguments found.  
*nArgs* (return) Number of arguments found (size of array).

**Returns:**

**true** if a semicolon is found and there are more arguments in the string, or **false** if there are no more arguments.

Parse a string for arguments. This function works like a command line parser. Arguments within quotation marks are considered one argument.

Used for: Misc.

[LTFLOAT](#)(\* [ILTServer::Random](#))([LTFLOAT](#) min, [LTFLOAT](#) max)

**Parameters:**

*min* Minimum extent of range.  
*max* Maximum extent of range.

**Returns:**

A random floating-point number in the given range.

Used for: Misc.

[LTFLOAT](#)(\* [ILTServer::RandomScale](#))([LTFLOAT](#) scale)

**Parameters:**

*scale* Maximum extent of range.

**Returns:**

A floating-point number from 0.0 to **scale**.

Used for: Misc.

void(\* [ILTServer::RelinquishList](#))([ObjectList](#) \*pList)

**Parameters:**

*pList* Object list to relinquish.

Any time the engine gives you a list, give the list back with this function, or else you will have significant memory leaks.

Used for: Misc.

[LTRESULT](#)(\* [ILTServer::RemoveAttachment](#))([HATTACHMENT](#) hAttachment)

**Parameters:**

*hAttachment* Attachment to remove.

**Returns:**

**LT\_INVALIDPARAMS** if hAttachment is NULL, **LT\_ERROR** if hAttachment has no parent, **LT\_NOTFOUND** if the parent is not found, else **LT\_OK**

Removes an attachment.

**Note:** An attachment becomes invalid when you remove the parent so this function will crash if you call it with an attachment with a parent you've removed.

Used for: Object.

void(\* [ILTServer::RemoveObjectFromList](#))([ObjectList](#) \*pList, [HOBJECT](#) hObj)

**Parameters:**

*pList* Object list to modify.

*hObj* Object to remove.

Remove an object from the list given the handle.

**Note:**

**Caution:** LithTech will do a search over all objects in the list.

Used for: Object.

[LTRESULT](#)(\* [ILTServer::RemoveObjectFromSky](#))([HOBJECT](#) hObj)

**Parameters:**

*hObj* Object to remove.

**Returns:**

**LT\_ERROR** - The server object is NULL.

**LT\_OK** - Successful.

Remove an object from the sky list.

Used for: Special FX.

[LTRESULT](#)(\* [ILTServer::RestoreObjects](#))(const char \*pszRestoreFileName, [uint32](#) dwParam, [uint32](#) flags)

**Parameters:**

*pszRestoreFileName* The restore filename.

*dwParam* User parameters (**dwParam** gets passed into **MID\_LOADOBJECT** as **fData**).

*flags* The save flags (**RESTOREOBJECTS\_** defines).

**Returns:**

**LT\_OK** on success.

Restore objects from a file.

Used for: Misc.

[LTRESULT](#)(\* [ILTServer::RotateObject](#))([HOBJECT](#) hObj, const [LTRotation](#) \*pRotation)

void(\* [ILTServer::RunGameConString](#))(const char \*pString)

**Parameters:**

*pString* Console command to execute.

Run a string through the game console. The syntax is the same as any other console string (semicolons separate commands, and new variable names create new

**HCONVAR** handles).

Used for: Misc.

[\*\*LTRESULT\*\*\(\\* \[ILTServer::RunWorld\]\(#\)\)\(\)](#)

**Returns:**

LT\_NOTINWORLD if world is not loaded. Otherwise, LT\_OK.

Run the loaded world.

Used for: Server Management.

[\*\*LTRESULT\*\*\(\\* \[ILTServer::SaveObjects\]\(#\)\)\(const char \\*pszSaveFileName, \[ObjectList\]\(#\) \\*pList, \[uint32\]\(#\) dwParam, \[uint32\]\(#\) flags\)](#)

**Parameters:**

*pszSaveFileName* The save filename.

*pList* The list of objects to save.

*dwParam* User parameters (**dwParam** gets passed into **MID\_SAVEOBJECT** as **fData**).

*flags* The save flags (**SAVEOBJECTS\_** defines).

**Returns:**

LT\_ERROR if file could not be opened. LT\_OK on success.

Save objects to a file.

Used for: Misc.

[void](#)(\* [ILTServer::ScaleObject](#))([HOBJECT](#) hObj, const [LTVector](#) \*pNewScale)

**Parameters:**

*hObj* Object to modify.

*pNewScale* New scale to set.

Scale the object (only works on models and sprites).

Used for: Object.

[\*\*LTRESULT\*\*\(\\* \[ILTServer::SendToServerApp\]\(#\)\)\(\[ILTMessage\\\_Read\]\(#\) &msg\)](#)

**Parameters:**

*msg* Message to send.

**Returns:**

LT\_NOTFOUND if there is no server app or if it isn't set up to receive the message; otherwise, returns LT\_OK.

Send a message to the standalone server app. It will receive it through the [ServerAppHandler::ShellMessageFn\(\)](#) function.

Used for: Networking.

[void](#)(\* [ILTServer::SetBlockingPriority](#))([HOBJECT](#) hObj, [uint8](#) pri)

**Parameters:**

*hObj* The object to modify.

*pri* The blocking priority.

Set the blocking priority (defaults to 0). See the FAQ for a description of how this works.

Used for: Object.

[void](#)(\* [ILTServer::SetClientInfoFlags](#))([HCLIENT](#) hClient, [uint32](#) dwClientFlags)

**Parameters:**

*hClient* Client to modify.

*dwClientFlags* Client info flags (a combination of the **CIF\_** flags above).

Set client info flags.

Used for: Server Management.

[bool](#)(\* [ILTServer::SetClientName](#))([HCLIENT](#) hClient, const char \*pName, int MaxLen)

**Parameters:**

*hClient* Client to query.

*pName* Buffer holding desired name

*maxLen* Buffer length.

**Returns:**

**true** if successful; otherwise, returns **false**.

Set the name of a client.

Used for: Server Management.

[void](#)(\* [ILTServer::SetClientUserData](#))([HCLIENT](#) hClient, void \*pData)

**Parameters:**

*hClient* Client to modify.

*pData* User data.

Set user data for **HCLIENT** objects.

Used for: Server Management.

[\*\*LTRESULT\*\*\(\\* \[ILTServer::SetClientViewPos\]\(#\)\)\(\[HCLIENT\]\(#\) hClient, const \[LTVector\]\(#\) \\*pPos\)](#)

**Parameters:**

*hClient* Client to modify.  
*pPos* Position (in world coordinates).

**Returns:**

LT\_INVALIDPARAMS if hClient is not valid. Otherwise, LT\_OK.

Set where the client is seeing and hearing from. This function controls what the server sends the client. You should set this every frame.

Used for: Server Management.

void(\* [ILTServer::SetGameConVar](#))(const char \*pName, const char \*pVal)

**Parameters:**

*pName* Console variable name.  
*pVal* Console variable value.

Set a console variable. Creates a new one if it doesn't exist.

Used for: Misc.

[\*\*LTRESULT\*\*\(\\* \[ILTServer::SetGlobalLightObject\]\(#\)\)\(\[HOBJECT\]\(#\) hObj\)](#)

**Parameters:**

*hObj* [Light](#) object to set as the global light.

**Returns:**

Always returns **LT\_OK**.

Set the global light object.

Used for: Special FX.

void(\* [ILTServer::SetLightColor](#))([HOBJECT](#) hObj, float r, float g, float b)

**Parameters:**

*hObj* [Light](#) to modify.  
*r* Red value (0-1).  
*g* Green value (0-1).  
*b* Blue value (0-1).

Set the color of a light.

**Note:** This merely calls [SetObjectColor\(\)](#).

**See also:**

[GetLightColor\(\)](#)  
[SetObjectColor\(\)](#)

Used for: Special FX.

void(\* [ILTServer::SetLightRadius](#))([HOBJECT](#) hObj, float radius)

**Parameters:**

*hObj* [Light](#) to modify.  
*radius* Radius to set.

Set the radius of a light.

**See also:**

[GetLightRadius\(\)](#)

Used for: Special FX.

[\*\*LTRESULT\*\*\(\\* \[ILTServer::SetModelNodeHideStatus\]\(#\)\)\(\[HOBJECT\]\(#\) hObj, char \\*pNodeName, bool bHidden\)](#)

**Parameters:**

*hObj* Model to modify.  
*pNodeName* Node to modify.  
*bHidden* **TRUE** means to hide; **FALSE** means to unhide.

**Returns:**

LT\_OK, LT\_ERROR, LT\_NOCHANGE, or LT\_NODENOTFOUND.

Hide or unhide a node on the model (they are all unhidden by default).

Used for: Models and Animation.

void(\* [ILTServer::SetModelPlaying](#))([HOBJECT](#) hObj, bool bPlaying)

**Parameters:**

*hObj* Model to modify.  
*bPlaying* **true** for playing, or **false** for not playing.

Set the Playing state of the model. The default state is **false**.

Used for: Models and Animation.

void(\* [LTServer::SetNextUpdate](#))([HOBJECT](#) hObj, [LTFLOAT](#) nextUpdate)

**Parameters:**

*hObj* Object to modify.  
*nextUpdate* Next update time.

This is a counter that controls when each object has its Update() function called. It is set to 0.0001 when an object is created, so Update() is called right away by default.

Used for: Object.

bool(\* [LTServer::SetObjectColor](#))([HOBJECT](#) hObject, float r, float g, float b, float a)

**Parameters:**

*hObject* Object to modify.  
*r* Red value (range: 0.0-1.0).  
*g* Green value (range: 0.0-1.0).  
*b* Blue value (range: 0.0-1.0).  
*a* Alpha value (range: 0.0-1.0).

**Returns:**

**false** if hObject is NULL, else **true**

Set object color. All objects default to (0,0,0,1) For lights, this changes the color of the light. For models, this brightens the shading of a model.

**Note:** This function overrides a model's render style's material diffuse color.

Used for: Object.

void(\* [LTServer::SetObjectPos](#))([HOBJECT](#) hObj, const [LTVector](#) \*pos)

[LTRESULT](#)(\* [LTServer::SetObjectRotation](#))([HOBJECT](#) hObj, const [LTRotation](#) \*pRotation)

void(\* [LTServer::SetObjectState](#))([HOBJECT](#) hObj, int state)

**Parameters:**

*hObj* The object to modify.  
*state* The object state (a combination of **OBJSTATE\_** defines).

Set the state of an object.

Used for: Object.

[uint32](#)(\* [LTServer::SetServerFlags](#))([uint32](#) flags)

**Parameters:**

*flags* Flags to set (a combination of the **SS\_** flags above).

**Returns:**

New flags (some may not be accepted).

Set server state flags.

Used for: Server Management.

[LTRESULT](#)(\* [LTServer::SetSkyDef](#))([SkyDef](#) \*pDef)

**Parameters:**

*pDef* Sky definition structure.

**Returns:**

Always returns **LT\_OK**.

Set the sky definition.

Used for: Special FX.

void(\* [LTServer::StartCounter](#))([LTCounter](#) \*pCounter)

**Parameters:**

*pCounter* The counter that the engine will update.

Start a counter. Use this to time sections of code. Timing is done in microseconds (millionths of a second).

Used for: Misc.

[LTRESULT](#)(\* [LTServer::TeleportObject](#))([HOBJECT](#) hObj, const [LTVector](#) \*pNewPos)

void(\* [ILTServer::TiltToPlane](#))([HOBJECT](#) hObj, const [LTVector](#) \*pNormal)

[LTRESULT](#)(\* [ILTServer::UpdateSessionName](#))(const char \*sName)

**Parameters:**

*sName* New session name to use.

**Returns:**

**LT\_NOTINITIALIZED** if the main network driver is uninitialized, or **LT\_ERROR** if the session name cannot be set; otherwise, returns **LT\_OK**.

Update the session name.

Used for: Networking.

bool(\* [ILTServer::UpperStrcmp](#))(const char \*pStr1, const char \*pStr2)

**Parameters:**

*pStr1* String 1 for comparison.

*pStr2* String 2 for comparison.

**Returns:**

**true** if the strings are the same, **false** otherwise.

Does a fast upper-case string comparison of the two strings.

Used for: Misc.

The documentation for this class was generated from the following file:

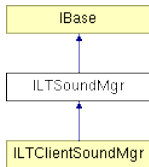
- [iltserver.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

## ILTSoundMgr Class Reference

Inheritance diagram for ILTSoundMgr:



### Public Types

enum { [\\_ILTSoundMgr\\_VERSION\\_](#) = 0 }

### Public Member Functions

virtual [LTRESULT](#) [PlaySound](#) ([PlaySoundInfo](#) \*pPlaySoundInfo, [HLTSOUND](#) &hResult)=0

virtual [LTRESULT](#) [GetSoundDuration](#) ([HLTSOUND](#) hSound, [LTFLOAT](#) &fDuration)=0

virtual [LTRESULT](#) [IsSoundDone](#) ([HLTSOUND](#) hSound, bool &bDone)=0

virtual [LTRESULT](#) [KillSound](#) ([HLTSOUND](#) hSound)=0

virtual [LTRESULT](#) [KillSoundLoop](#) ([HLTSOUND](#) hSound)=0

### Detailed Description

Base sound mgr interface. (shared between client and server)

### Member Enumeration Documentation

anonymous enum

**Enumeration values:**

[\\_ILTSoundMgr\\_VERSION\\_](#)

### Member Function Documentation

virtual [LTRESULT](#) [ILTSoundMgr::GetSoundDuration](#)([HLTSOUND](#) *hSound*,

[LTFLOAT](#) & *fDuration*  
) [pure virtual]

**Parameters:**

*hSound* Handle to sound.  
*fDuration* (return) Duration of sound.

**Returns:**

**LT\_INVALIDPARAMS** when **hSound** is not available; otherwise, returns **LT\_OK**.

Get total length (in seconds) of sound.

Used for: Audio.

virtual [LTRESULT](#) ILTSoundMgr::IsSoundDone( [HLTSOUND](#) *hSound*,  
bool & *bDone*  
) [pure virtual]

**Parameters:**

*hSound* Handle to client only sound.  
*bDone* (return) Finished state of sound. A return value of **TRUE** means the sound is done; a return value of **FALSE** means it is still playing.

**Returns:**

**LT\_INVALIDPARAMS** when **hSound** is invalid; otherwise, returns **LT\_OK**.

Check whether the sound is finished playing or if the object it was attached to was removed.

Used for: Audio.

virtual [LTRESULT](#) ILTSoundMgr::KillSound( [HLTSOUND](#) *hSound* ) [pure virtual]

**Parameters:**

*hSound* Handle to sound.

**Returns:**

**LT\_INVALIDPARAMS** when **hSound** is invalid or not playing; otherwise, returns **LT\_OK**.

Kill a sound.

Used for: Audio.

virtual [LTRESULT](#) ILTSoundMgr::KillSoundLoop( [HLTSOUND](#) *hSound* ) [pure virtual]

**Parameters:**

*hSound* Handle to sound.

**Returns:**

**LT\_INVALIDPARAMS** when **hSound** is invalid or not playing; otherwise, returns **LT\_OK**.

Kill a looping sound. The sound will continue to play until it reaches the end, then remove itself.

Used for: Audio.

virtual [LTRESULT](#) ILTSoundMgr::PlaySound( [PlaySoundInfo](#) \* *pPlaySoundInfo*,  
[HLTSOUND](#) & *hResult*  
) [pure virtual]

**Parameters:**

*pPlaySoundInfo* [Sound](#) control structure.  
*hResult* (return) Newly created sound handle.

**Returns:**

**LT\_INVALIDPARAMS** when **pPlaySoundInfo** is invalid, **LT\_MISSINGFILE** when the sound file was not found, or **LT\_ERROR** when unable to play the sound; otherwise, returns **LT\_OK**.

Play a sound with full control.

Specifying the **PLAYSOUND\_GETHANDLE** flag in **pPlaySoundInfo** will fill *pPlaySoundInfo->m\_hSound* with the handle to the client sound, and prevent the sound from being automatically deleted when the sound is done playing. If it is specified, you should subsequently call [KillSound\(\)](#).

**See also:**

[KillSound\(\)](#)

Used for: Audio.

---

The documentation for this class was generated from the following file:

- [iltoundmgr.h](#)

---

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

---

## Public Member Functions

```
virtual LTRESULT GetNumAnims (uint32 &nAnims)=0
virtual LTRESULT GetNumFrames (uint32 iAnim, uint32 &nFrames)=0
virtual LTRESULT GetCurPos (uint32 &iAnim, uint32 &iFrame)=0
virtual LTRESULT SetCurPos (uint32 iAnim, uint32 iFrame)=0
virtual LTRESULT GetFlags (uint32 &flags)=0
virtual LTRESULT SetFlags (uint32 flags)=0
virtual LTRESULT GetAnimLength (uint32 &msLen, const uint32 iAnim)=0
virtual LTRESULT GetFrameTextureHandle (HTEXTURE &hTex, const uint32 iAnim, const uint32 iFrame)=0
```

[illegible]

*msLen* (return) Length (in milliseconds).  
*iAnim* Animation to query.

**LT\_ERROR** - *iAnim* is invalid (i.e. too high).

**LT\_OK** - Successful.

Retrieves the length of a sprite animation in milliseconds.

Used for: Special FX.

[illegible]

*iAnim* (return) Current animation.  
*iFrame* (return) Current animation.

Always returns **LT\_OK**.

Get the currently playing animation and frame.

Used for: Special FX.

virtual [LTRESULT](#) ILTSpriteControl::GetFlags( [uint32](#) & *flags* ) [pure virtual]

*flags* (return) Flags (see SC\_ defines, above).

Always returns **LT\_OK**.

Get the flags for this sprite.

Used for: Special FX.

[illegible]

*hTex* (return) Texture handle.  
*iAnim* Animation to query.  
*iFrame* Frame to query.

**LT\_ERROR** - *iAnim* is invalid (i.e. too high), or some other internal error occurred.

**LT OK** - Successful.

Retrieve an HTEXTURE for a specific frame in this sprite.

Used for: Special FX.

```
virtual LTRESULT ILTSpriteControl::GetNumAnims( uint32 & nAnims ) [pure virtual]
```

*nAnims* (return) Number of animations.

Always returns **LT OK**.

Get the number of animations in this sprite.

Used for: Special FX.

```
virtual LTRESULT ILTSpriteControl::GetNumFrames( uint32 iAnim,
                                                uint32 & nFrames
                                              ) [pure virtual]
```

**Parameters:**

*iAnim* Animation to query.  
*nFrames* (return) Number of frames in animation.

**Returns:**

**LT\_INVALIDPARAMS** - *iAnim* is invalid (i.e. too high).

**LT\_OK** - Successful.

Get the number of frames in specified animation.

Used for: Special FX.

```
virtual LTRESULT ILTSpriteControl::SetCurPos( uint32 iAnim,
                                                uint32 iFrame
                                              ) [pure virtual]
```

**Parameters:**

*iAnim* Index of animation to set.  
*iFrame* Animation frame to set.

**Returns:**

**LT\_INVALIDPARAMS** - *iAnim* is invalid (i.e. too high) and/or *iFrame* is invalid (i.e. too high).

**LT\_OK** - Successful.

Set the currently playing animation and frame.

Used for: Special FX.

```
virtual LTRESULT ILTSpriteControl::SetFlags( uint32 flags ) [pure virtual]
```

**Parameters:**

*flags* Flags (see SC defines, above).

**Returns:**

Always returns **LT\_OK**.

Set the flags for this sprite.

Used for: Special FX.

---

The documentation for this class was generated from the following file:

- [iltspritecontrol.h](#)

---

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
 2006, All Rights Reserved.

## API Reference

---

## ILTStream Class Reference

### Public Member Functions

```
virtual void Release ()=0
virtual LTRESULT Read (void *pData, uint32 size)=0
virtual LTRESULT ReadString (char *pStr, uint32 maxBytes)=0
virtual LTRESULT ErrorStatus ()=0
virtual LTRESULT SeekTo (uint32 offset)=0
virtual LTRESULT GetPos (uint32 *offset)=0
virtual LTRESULT GetLen (uint32 *len)=0
virtual LTRESULT WriteStream (ILTStream &dsSource, uint32 dwMin=0xFFFFFFFF, uint32 dwMax=0xFFFFFFFF)=0
    uint32 GetPos ()
    uint32 GetLen ()
template<class T>
    ILTStream & operator>> (T &toRead)
virtual LTRESULT Write (const void *pData, uint32 size)=0
virtual LTRESULT WriteString (const char *pStr)=0
template<class T>
    ILTStream & WriteVal (const T toWrite)
template<class T>
    ILTStream & operator<< (T &toWrite)
```

### Protected Member Functions

virtual [~ILStream](#) ()

---

## Constructor & Destructor Documentation

virtual ILStream::~ILStream( ) [inline, protected, virtual]

---

## Member Function Documentation

virtual [LTRESULT](#) ILStream::ErrorStatus( ) [pure virtual]

### Returns:

**LT\_OK** if there are no errors; otherwise, an error occurred.

Used for: Misc.

[uint32](#) ILStream::GetLen( ) [inline]

### Returns:

the length of the file.

Get the length of the file.

Used for: Misc.

virtual [LTRESULT](#) ILStream::GetLen( [uint32](#) \* *len* ) [pure virtual]

### Parameters:

*len* Address of variable to be filled in.

### Returns:

**LT\_OK** on success.

Get the length of the file.

Used for: Misc.

[uint32](#) ILStream::GetPos( ) [inline]

### Returns:

the current offset into the file.

Get the current offset into the file.

Used for: Misc.

virtual [LTRESULT](#) ILStream::GetPos( [uint32](#) \* *offset* ) [pure virtual]

### Parameters:

*offset* Address of variable to be filled in.

### Returns:

**LT\_OK** on success.

Get the current offset into the file.

Used for: Misc.

template<class T>

[ILStream](#)& ILStream::operator<<( T & *toWrite* ) [inline]

### Parameters:

*toWrite* Primitive to write.

### Returns:

**LT\_OK** on success.

Write data to the stream. The stream operation will only work on non-pointer data types. It writes out the data using sizeof() to determine how much to write. Only enabled in writeable streams.

Used for: Misc.

template<class T>

[ILStream](#)& ILStream::operator>>( T & *toRead* ) [inline]

### Returns:

A reference to the **ILStream** instance.

### Parameters:

*toRead* Reference to the item that is the destination for the read.

A stream operator for reading in data.

Used for: Misc.

virtual [LTRESULT](#) ILStream::Read( void \* *pData*,  
[uint32](#) *size*)

```
) [pure virtual]
```

**Parameters:**

*pData* Address of buffer to read into.  
*size* Length of the pData buffer.

**Returns:**

**LT\_ERROR** when you read past the end.

Read in data. *Always* fills in **pData** with 0s if it returns an error so it can safely read everything in without checking the return value every time and checking [ErrorStatus\(\)](#) at the end.

Used for: Misc.

```
virtual LTRESULT ILTStream::ReadString( char * pStr,
                                       uint32 maxBytes
                                       ) [pure virtual]
```

**Parameters:**

*pStr* Address of buffer string to read into.  
*maxBytes* Length of string buffer.

**Returns:**

**LT\_ERROR** if you read past the end.

Read in data. *Always* fills in **pData** with 0s if it returns an error so it can safely read everything in without checking the return value every time and checking [ErrorStatus\(\)](#) at the end.

Used for: Misc.

```
virtual void ILTStream::Release( ) [pure virtual]
```

Call this when you're done with it (it's the same as deleting it).

Used for: Misc.

```
virtual LTRESULT ILTStream::SeekTo( uint32 offset ) [pure virtual]
```

**Parameters:**

*offset* Number of bytes.

**Returns:**

**LT\_OK** on success.

Seek the file pointer to the given offset.

Used for: Misc.

```
virtual LTRESULT ILTStream::Write( const void * pData,
                                  uint32 size
                                  ) [pure virtual]
```

**Parameters:**

*pData* Address of data buffer to write.  
*size* Number of bytes to write.

**Returns:**

**LT\_OK** on success.

Write data of the given length to the stream. Only enabled in writeable streams.

Used for: Misc.

```
virtual LTRESULT ILTStream::WriteStream( ILTStream & dsSource,
                                         uint32 dwMin = 0xFFFFFFFF,
                                         uint32 dwMax = 0xFFFFFFFF
                                         ) [pure virtual]
```

**Parameters:**

*dsSource* Stream to read from.  
*dwMin* Start position in **dsSource**.  
*dwMax* End position in **dsSource**.

**Returns:**

**LT\_ERROR** if either stream has an [ErrorStatus\(\)](#) != **LT\_OK**, or returns an error from Read or Write.

Read the data in **dsSource** and write it to this stream. You can specify the range of **dsSource** to read, or leave it at **WRITESTREAM\_DEFAULT** and it will write *all* of **dsSource**.

Used for: Misc.

```
virtual LTRESULT ILTStream::WriteString( const char * pStr ) [pure virtual]
```

**Parameters:**

*pStr* Address of null terminated string to write.

**Returns:**

**LT\_OK** on success.

Write a null-terminated string to the stream. Only enabled in writeable streams.

Used for: Misc.

```
template<class T>
ILTStream& ILTStream::WriteVal( const T toWrite ) [inline]
```

**Parameters:**  
*toWrite* Primitive to write.

**Returns:**  
**LT\_OK** on success.

Write data to the stream. This function will only work on non-pointer data types. It writes out the data using sizeof() to determine how much to write. Only enabled in writeable streams.

Used for: Misc.

The documentation for this class was generated from the following file:

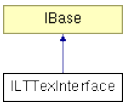
- [iltstream.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

## ILTTexInterface Class Reference

Inheritance diagram for ILTTexInterface:



### Public Types

```
enum { \_ILTTexInterface\_VERSION = 0 }
```

### Public Member Functions

```
virtual LTRESULT FindTextureFromName ( HTEXTURE &hTexture, const char *pFilename)=0
virtual LTRESULT CreateTextureFromName ( HTEXTURE &hTexture, const char *pFilename)=0
virtual LTRESULT CreateTextureFromData ( HTEXTURE &hTexture, ETextureType eTextureType, uint32 TextureFlags, uint8 *pData, uint32 nWidth, uint32 nHeight, uint32 nAutoGenMipMaps=1)=0
virtual LTRESULT GetTextureData (const HTEXTURE hTexture, const uint8 *pData, uint32 &nWidth, uint32 &nHeight, uint32 &nPitch, ETextureType &eType)=0
virtual LTRESULT FlushTextureData (const HTEXTURE hTexture, ETextureMod eChanged=TEXTURE_DATAANDPALETTECHANGED, uint32 nMipMap=0)=0
virtual LTRESULT GetTextureDims (const HTEXTURE hTexture, uint32 &nWidth, uint32 &nHeight)=0
virtual LTRESULT GetTextureType (const HTEXTURE hTexture, ETextureType &eTextureType)=0
virtual bool ReleaseTextureHandle (const HTEXTURE hTexture)=0
virtual uint32 AddRefTextureHandle (const HTEXTURE hTexture)=0
```

### Detailed Description

The ILTTexInterface interface exposes texture manipulation functionality to the game code. It enables direct access to texture data. This direct access means that the texture won't be converted to a new format, and will therefore result in faster texture rendering.

Define a holder to get this interface like this:

```
define\_holder(ILTTexInterface, your_var);
```

### Member Enumeration Documentation

anonymous enum

**Enumeration values:**  
[\\_ILTTexInterface\\_VERSION](#)

### Member Function Documentation

```
virtual uint32 ILTTexInterface::AddRefTextureHandle( const HTEXTURE hTexture ) [pure virtual]
```

**Parameters:**  
*hTexture* Texture handle

**Returns:**  
**Reference** count of texture

Call `AddRefTextureHandle` to artificially increment the reference count of a texture.

Used for: Special FX.

```
virtual LTRESULT ILTTextureInterface::CreateTextureFromData( HTEXTURE & hTexture,
                                                         ETextureType eTextureType,
                                                         uint32 TextureFlags,
                                                         uint8 * pData,
                                                         uint32 nWidth,
                                                         uint32 nHeight,
                                                         uint32 nAutoGenMipMaps = 1
                                                         ) [pure virtual]
```

**Parameters:**

*hTexture* (return) Texture handle.  
*eTextureType* Type (see `TEXTURETYPE_` enum, above).  
*TextureFlags* Flags (see `TEXTUREFLAG_` values, above).  
*pData* Pointer to the texture data.  
*nWidth* Width.  
*nHeight* Height.  
*nAutoGenMipMaps* Request auto creation of `nAutoGenMipMaps` number of mip maps. They will be down sampled. Set this param to 0 to autogen all the way down to 1x1.

**Returns:**

**LT\_ERROR** - Texture could not be created.

**LT\_OK** - Successful.

Creates a texture from the passed in data.

Used for: Special FX.

```
virtual LTRESULT ILTTextureInterface::CreateTextureFromName( HTEXTURE & hTexture,
                                                           const char * pFilename
                                                           ) [pure virtual]
```

**Parameters:**

*hTexture* The address of the converted texture.  
*pFilename* Points to a char array containing the relative path to the resource directory in which the .DTX file is located.

**Returns:**

**LT\_MISSINGFILE** - *pFilename* is invalid (can't find file).

**LT\_ERROR** - Internal error.

**LT\_OK** - Successful.

Creates a texture from a .DTX file. A .DTX file is created using the DEdit tool (see the Game Content Creation Guide). If the .DTX file was created with the `DTX_32BITSYSYSCOPY` bit set, the texture will not be converted (the system memory copy will remain `RGBA_8888`). To determine the format type of the new texture, use [ILTTextureInterface::GetTextureType](#).

Used for: Special FX.

```
virtual LTRESULT ILTTextureInterface::FindTextureFromName( HTEXTURE & hTexture,
                                                          const char * pFilename
                                                          ) [pure virtual]
```

**Parameters:**

*hTexture* (return) Texture handle.  
*pFilename* Texture filename.

**Returns:**

**LT\_ERROR** - Texture not found.

**LT\_OK** - Texture found.

Retrieves a handle to an existing texture given a filename.

Used for: Special FX.

```
virtual LTRESULT ILTTextureInterface::FlushTextureData( const HTEXTURE hTexture,
                                                         ETextureMod eChanged = TEXTURE_DATAANDPALETTECHANGED,
                                                         uint32 nMipMap = 0
                                                         ) [pure virtual]
```

**Parameters:**

*hTexture* Texture handle.  
*eChanged* Indicate is the texture pixel data and/or palette has changed.  
*nMipMap* Mipmap to unlock.

**Returns:**

**LT\_OK** - Successful.

Let the engine know that we are finished modifying the texture

Used for: Special FX.

```
virtual LTRESULT ILTTextureInterface::GetTextureData( const HTEXTURE hTexture,
```

```

        const uint8 *&    pData,
        uint32 &         nWidth,
        uint32 &         nHeight,
        uint32 &         nPitch,
        ETextureType &   eType
    ) [pure virtual]

```

**Parameters:**

*hTexture* Texture handle.  
*pData* (return) Texture pixel data pointer  
*nWidth* (return) The width of the surface that is locked in pixels.  
*nHeight* (return) The height of the surface that is locked in pixels.  
*nPitch* (return) Size (in bytes) of a row of pixel data.  
*eType* (return) Format of the texture data

**Returns:**

**LT\_OK** - Successful.

Fetch a pointer to the texture RGB raw data. This data cannot be modified and the pointer should only be used immediately after obtaining it without any other texture calls since the engine is free to unload the texture if it rebinds it

Used for: Special FX.

```

virtual LTRESULT ILTTexInterface::GetTextureDims( const HTEXTURE hTexture,
        uint32 &         nWidth,
        uint32 &         nHeight
    ) [pure virtual]

```

**Parameters:**

*hTexture* Texture handle.  
*nWidth* (return) Texture width.  
*nHeight* (return) Texture height.

**Returns:**

**LT\_OK** - Successful.

Get information about the texture.

Used for: Special FX.

```

virtual LTRESULT ILTTexInterface::GetTextureType( const HTEXTURE hTexture,
        ETextureType &   eTextureType
    ) [pure virtual]

```

**Parameters:**

*hTexture* Texture handle.  
*eTextureType* (return) The texture type.

**Returns:**

**LT\_OK** - Successful.

Get information about the texture.

Used for: Special FX.

```

virtual bool ILTTexInterface::ReleaseTextureHandle( const HTEXTURE hTexture ) [pure virtual]

```

**Parameters:**

*hTexture* Texture handle.

**Returns:**

**LT\_OK** - Successful.

You should call ReleaseTextureHandle after you are done with your texture so that it can be freed. If it is not also being used by the engine, it will be freed.

Used for: Special FX.

The documentation for this class was generated from the following file:

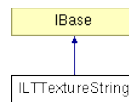
- [ilttexinterface.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
 2006, All Rights Reserved.

## API Reference

## ILTTextureString Class Reference

Inheritance diagram for ILTTextureString:



## Public Types

```
enum { \_ILTextureString\_VERSION = 0 }
```

## Public Member Functions

```

ILTextureString ()
virtual ~ILTextureString ()
virtual HCUSTOMFONTFILE RegisterCustomFontFile (const char *pszRelResource)=0
virtual LTRESULT UnregisterCustomFontFile (HCUSTOMFONTFILE hCustomFontFile)=0
virtual HTEXTURESTRING CreateTextureString (const wchar_t *pszString, const CFontInfo &Font)=0
virtual HTEXTURESTRING CreateTextureSubString (const wchar_t *pszString, HTEXTURESTRING hSrcString)=0
virtual LTRESULT WordWrapString (HTEXTURESTRING hString, uint32 nWidth)=0
virtual LTRESULT RecreateTextureString (HTEXTURESTRING hString, const wchar_t *pszString, const CFontInfo &Font)=0
virtual LTRESULT RecreateTextureSubString (HTEXTURESTRING hString, const wchar_t *pszString, HTEXTURESTRING hSrcTexture)=0
virtual LTRESULT GetStringExtents (HTEXTURESTRING hString, LRect2n &rExtents)=0
virtual LTRESULT GetStringLength (HTEXTURESTRING hString, uint32 *pnLength)=0
virtual LTRESULT GetString (HTEXTURESTRING hString, wchar_t *pszBuffer, uint32 nBufferLen)=0
virtual LTRESULT GetFont (HTEXTURESTRING hString, CFontInfo &Font)=0
virtual LTRESULT GetTextureImage (HTEXTURESTRING hString, HTEXTURE &hTexture)=0
virtual LTRESULT GetCharRect (HTEXTURESTRING hString, uint32 nCharIndex, LRect2n &rPlacementRect, LRect2n &rBlackBox, LVector2f &vUVPos, LVector2f &vUVDims)=0
virtual void ReleaseTextureString (HTEXTURESTRING hString)=0
virtual void AddRefTextureString (HTEXTURESTRING hString)=0
virtual LTRESULT SetupTextRendering (ILTDrawPrim *pDrawPrim)=0
virtual LTRESULT RenderString (HTEXTURESTRING hString, ILTDrawPrim *pDrawPrim, const LVector2f &vAnchor, uint32 nColor=0xFFFFFFFF, const LVector2f &vAnchorScale=LVector2f(0.0f, 0.0f), const LVector2f &vGround=LVector2f(1.0f, 0.0f), const LVector2f &vDown=LVector2f(0.0f, 1.0f), const LVector2f &vStretch=LVector2f(1.0f, 1.0f), bool bSnapAnchor=true)=0
virtual LTRESULT RenderStringClipped (HTEXTURESTRING hString, ILTDrawPrim *pDrawPrim, const LRect2n &rClipRect, const LVector2f &vAnchor, uint32 nColor=0xFFFFFFFF, const LVector2f &vAnchorScale=LVector2f(0.0f, 0.0f), bool bSnapAnchor=true)=0
virtual LTRESULT RenderSubString (HTEXTURESTRING hString, const wchar_t *pszString, ILTDrawPrim *pDrawPrim, const LVector2f &vAnchor, uint32 nColor=0xFFFFFFFF, const LVector2f &vGround=LVector2f(1.0f, 0.0f), const LVector2f &vDown=LVector2f(0.0f, 1.0f), const LVector2f &vStretch=LVector2f(1.0f, 1.0f))=0
virtual LTRESULT RenderSubStringClipped (HTEXTURESTRING hString, const wchar_t *pszString, ILTDrawPrim *pDrawPrim, const LRect2n &rClipRect, const LVector2f &vAnchor, uint32 nColor=0xFFFFFFFF)=0
  
```

## Member Enumeration Documentation

anonymous enum

Enumeration values:

[\\_ILTextureString\\_VERSION](#)

## Constructor & Destructor Documentation

```
ILTextureString::ILTextureString( ) [inline]
```

```
virtual ILTextureString::~ILTextureString( ) [inline, virtual]
```

## Member Function Documentation

```
virtual void ILTextureString::AddRefTextureString( HTEXTURESTRING hString ) [pure virtual]
```

```
virtual HTEXTURESTRING ILTextureString::CreateTextureString( const wchar_t *    pszString,
                                                            const CFontInfo & Font
                                                            ) [pure virtual]
```

```
virtual HTEXTURESTRING ILTextureString::CreateTextureSubString( const wchar_t *    pszString,
                                                                HTEXTURESTRING hSrcString
                                                                ) [pure virtual]
```

```
virtual LTRESULT ILTextureString::GetCharRect( HTEXTURESTRING hString,
                                             uint32          nCharIndex,
                                             LRect2n &        rPlacementRect,
                                             LRect2n &        rBlackBox,
                                             LVector2f &      vUVPos,
                                             LVector2f &      vUVDims
                                             ) [pure virtual]
```

```
virtual LTRESULT ILTextureString::GetFont( HTEXTURESTRING hString,
                                          CFontInfo &      Font
                                          )
```

```
) [pure virtual]
```

```
virtual LTRESULT ILTTextureString::GetString( HTEXTURESTRING hString,
                                             wchar_t *      pszBuffer,
                                             uint32         nBufferLen
                                             ) [pure virtual]
```

```
virtual LTRESULT ILTTextureString::GetStringExtents( HTEXTURESTRING hString,
                                                    LRect2n &      rExtents
                                                    ) [pure virtual]
```

```
virtual LTRESULT ILTTextureString::GetStringLength( HTEXTURESTRING hString,
                                                    uint32 *      pnLength
                                                    ) [pure virtual]
```

```
virtual LTRESULT ILTTextureString::GetTextureImage( HTEXTURESTRING hString,
                                                    HTEXTURE &      hTexture
                                                    ) [pure virtual]
```

```
virtual LTRESULT ILTTextureString::RecreateTextureString( HTEXTURESTRING hString,
                                                         const wchar_t *      pszString,
                                                         const CFontInfo &      Font
                                                         ) [pure virtual]
```

```
virtual LTRESULT ILTTextureString::RecreateTextureSubstring( HTEXTURESTRING hString,
                                                            const wchar_t *      pszString,
                                                            HTEXTURESTRING hSrcTexture
                                                            ) [pure virtual]
```

```
virtual HCUSTOMFONTFILE ILTTextureString::RegisterCustomFontFile( const char * pszRelResource ) [pure virtual]
```

```
virtual void ILTTextureString::ReleaseTextureString( HTEXTURESTRING hString ) [pure virtual]
```

```
virtual LTRESULT ILTTextureString::RenderString( HTEXTURESTRING hString,
                                                ILDrawPrim *      pDrawPrim,
                                                const LVector2f & vAnchor,
                                                uint32         nColor = 0xFFFFFFFF,
                                                const LVector2f & vAnchorScale = LVector2f(0.0f, 0.0f),
                                                const LVector2f & vGround = LVector2f(1.0f, 0.0f),
                                                const LVector2f & vDown = LVector2f(0.0f, 1.0f),
                                                const LVector2f & vStretch = LVector2f(1.0f, 1.0f),
                                                bool             bSnapAnchor = true
                                                ) [pure virtual]
```

```
virtual LTRESULT ILTTextureString::RenderStringClipped( HTEXTURESTRING hString,
                                                       ILDrawPrim *      pDrawPrim,
                                                       const LRect2n &      rClipRect,
                                                       const LVector2f & vAnchor,
                                                       uint32         nColor = 0xFFFFFFFF,
                                                       const LVector2f & vAnchorScale = LVector2f(0.0f, 0.0f),
                                                       bool             bSnapAnchor = true
                                                       ) [pure virtual]
```

```
virtual LTRESULT ILTTextureString::RenderSubString( HTEXTURESTRING hString,
                                                    const wchar_t *      pszString,
                                                    ILDrawPrim *      pDrawPrim,
                                                    const LVector2f &      vAnchor,
                                                    uint32         nColor = 0xFFFFFFFF,
                                                    const LVector2f &      vGround = LVector2f(1.0f, 0.0f),
                                                    const LVector2f &      vDown = LVector2f(0.0f, 1.0f),
                                                    const LVector2f &      vStretch = LVector2f(1.0f, 1.0f)
                                                    ) [pure virtual]
```

```
virtual LTRESULT ILTTextureString::RenderSubStringClipped( HTEXTURESTRING hString,
                                                           const wchar_t *      pszString,
                                                           ILDrawPrim *      pDrawPrim,
                                                           const LRect2n &      rClipRect,
                                                           const LVector2f &      vAnchor,
                                                           uint32         nColor = 0xFFFFFFFF
```

) [pure virtual]

virtual [LTRESULT](#) ILTTextureString::SetupTextRendering( [ILTDrawPrim](#) \* *pDrawPrim* ) [pure virtual]

virtual [LTRESULT](#) ILTTextureString::UnregisterCustomFontFile( [HCUSTOMFONTFILE](#) *hCustomFontFile* ) [pure virtual]

virtual [LTRESULT](#) ILTTextureString::WordWrapString( [HTEXTURESTRING](#) *hString*,  
[uint32](#) *nWidth*  
) [pure virtual]

The documentation for this class was generated from the following file:

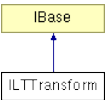
- [ilttexturestring.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

## ILTTransform Class Reference

Inheritance diagram for ILTTransform:



### Public Types

enum { [ILTTransform\\_VERSION](#) = 0 }

### Public Member Functions

virtual [LTRESULT](#) [Get](#) ([LTransform](#) &T, [LTVector](#) &p, [LTRotation](#) &q)=0  
virtual [LTRESULT](#) [Set](#) ([LTransform](#) &T, [LTVector](#) &p, [LTRotation](#) &q)=0  
virtual [LTRESULT](#) [GetPos](#) ([LTransform](#) &T, [LTVector](#) &p)=0  
virtual [LTRESULT](#) [GetRot](#) ([LTransform](#) &T, [LTRotation](#) &q)=0  
virtual [LTRESULT](#) [ToMatrix](#) ([LTransform](#) &T, [LTMatrix](#) &M)=0  
virtual [LTRESULT](#) [FromMatrix](#) ([LTransform](#) &T, [LTMatrix](#) &M)=0  
virtual [LTRESULT](#) [Inverse](#) ([LTransform](#) &T\_inv, [LTransform](#) &T)=0  
virtual [LTRESULT](#) [Multiply](#) ([LTransform](#) &Tab, [LTransform](#) &Ta, [LTransform](#) &Tb)=0  
virtual [LTRESULT](#) [Difference](#) ([LTransform](#) &diff, [LTransform](#) &t1, [LTransform](#) &t2)=0

### Detailed Description

The ILTTransform interface exposes various routines for dealing with LTransforms, which can used to represent local coordinate frames.

Define a holder to get this interface like this:

[define\\_holder](#)([ILTTransform](#), your\_var);

### Member Enumeration Documentation

anonymous enum

Enumeration values:

[\\_ILTTransform\\_VERSION\\_](#)

### Member Function Documentation

virtual [LTRESULT](#) ILTTransform::Difference( [LTransform](#) & *diff*,  
[LTransform](#) & *t1*,  
[LTransform](#) & *t2*  
) [pure virtual]

Parameters:

*diff*  $\mathbf{T}_b^{-1} * \mathbf{T}_a$   
*Ta*  $\mathbf{T}_a$   
*Tb*  $\mathbf{T}_b$

Given two [LTransform](#) objects  $T_a$  and  $T_b$ , compute the composite transformation

$$T_{diff} = T_b^{-1} * T_a.$$

This is equivalent to rotating and translating by  $T_a$ , then rotating and translating in the direction opposite to  $T_b$ .

Used for: Math.

```
virtual LTRESULT ILTTransform::FromMatrix( LTransform & T,
                                           LTMatrix & M
                                           ) [pure virtual]
```

**Parameters:**

$T$  the [LTransform](#) object  
 $M$  the 4x4 transformation matrix

**Returns:**

LT\_OK

Given a 4x4 matrix of the form

$$M = \begin{bmatrix} 1 - 2y^2 - 2z^2 & 2xy - 2rz & 2xz + 2ry & O_z \\ 2xy + 2rz & 1 - 2x^2 - 2z^2 & 2yz - 2rx & O_y \\ 2xz - 2ry & 2yz + 2rx & 1 - 2x^2 - 2y^2 & O_x \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

create the corresponding [LTransform](#)  $T$ .

Used for: Math.

```
virtual LTRESULT ILTTransform::Get( LTransform & T,
                                    LTVector & p,
                                    LTRotation & q
                                    ) [pure virtual]
```

**Parameters:**

$T$  the [LTransform](#) object from which to copy  $P$  and  $q$   
 $p$  the position of  $T$   
 $q$  the orientation of  $T$

**Returns:**

LT\_OK

Copy the information contained in  $T$  to  $P$  and  $q$ .

Used for: Math.

```
virtual LTRESULT ILTTransform::GetPos( LTransform & T,
                                       LTVector & p
                                       ) [pure virtual]
```

**Parameters:**

$T$  the [LTransform](#) object from which to copy  $P$   
 $p$  the position of  $T$

**Returns:**

LT\_OK

Copy the position of  $T$  to  $P$ .

Used for: Math.

```
virtual LTRESULT ILTTransform::GetRot( LTransform & T,
                                       LTRotation & q
                                       ) [pure virtual]
```

**Parameters:**

$T$  the [LTransform](#) object from which to copy  $q$   
 $q$  the orientation of  $T$

**Returns:**

LT\_OK

Copy the rotation of  $T$  to  $q$ .

Used for: Math.

```
virtual LTRESULT ILTTransform::Inverse( LTransform & T_inv,
                                       LTransform & T
                                       ) [pure virtual]
```

**Parameters:**

$T_{inv}$   $T^{-1}$   
 $T$   $T$

**Returns:**

LT\_OK

Given a [LTransform](#) object  $T$ , compute its inverse  $T^{-1}$  such that the product  $T * T^{-1}$  results in a transformation equal to  $P = (0, 0, 0)$ ,  $q = (0, 0, 0, 1)$ .

Used for: Math.

```
virtual LTRESULT ILTTransform::Multiply( LTransform & Tab,
                                         LTransform & Ta,
                                         LTransform & Tb
                                         ) [pure virtual]
```

**Parameters:**  
*Tab*   **T<sub>a</sub>** \* **T<sub>b</sub>**  
*Ta*   **T<sub>a</sub>**  
*Tb*   **T<sub>b</sub>**

**Returns:**  
**LT\_OK**

Given two [LTransform](#) objects **T<sub>a</sub>** and **T<sub>b</sub>** , compute the composite transformation

**T<sub>ab</sub>** = **T<sub>a</sub>** \* **T<sub>b</sub>**.

Used for: Math.

```
virtual LTRESULT ILTTransform::Set( LTransform & T,
                                    LTVector & p,
                                    LTRotation & q
                                    ) [pure virtual]
```

**Parameters:**  
*T* the [LTransform](#) object to which to copy **P** and **q**  
*p* a position  
*q* an orientation

**Returns:**  
**LT\_OK**

Copy the position **P** and the rotation **q** to **T** .

Used for: Math.

```
virtual LTRESULT ILTTransform::ToMatrix( LTransform & T,
                                         LTMatrix & M
                                         ) [pure virtual]
```

**Parameters:**  
*T* the [LTransform](#) object  
*M* the 4x4 transformation matrix

**Returns:**  
**LT\_OK**

Given a transform **T** , create a 4x4 matrix of the form

$$M = \begin{bmatrix} 1 - 2y^2 - 2z^2 & 2xy - 2rz & 2xz + 2ry & O_x \\ 2xy + 2rz & 1 - 2x^2 - 2z^2 & 2yz - 2rx & O_y \\ 2xz - 2ry & 2yz + 2rx & 1 - 2x^2 - 2y^2 & O_z \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

This matrix transforms a homogeneous point **P** = (*x*, *y*, *z*, 1) in a local coordinate frame to a homogeneous point in its parent coordinate frame according to the formula

**p<sub>parent</sub>** = **M** \* **p<sub>local</sub>**.

Used for: Math.

The documentation for this class was generated from the following file:

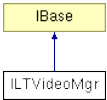
- [ilttransform.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

## ILTVideoMgr Class Reference

Inheritance diagram for ILTVideoMgr:



## Public Types

```
enum { \_ILTVideoMgr\_VERSION = 0 }
```

## Public Member Functions

```
virtual LTRESULT StartOnScreenVideo( const char *pFilename, uint32 flags, HVIDEO &hVideo)=0
virtual LTRESULT UpdateVideo( HVIDEO hVideo)=0
virtual LTRESULT GetVideoStatus( HVIDEO hVideo)=0
virtual LTRESULT StopVideo( HVIDEO hVideo)=0
```

---

## Detailed Description

The ILTVideoMgr interface.

Define a holder to get this interface like this:

```
define\_holder(ILTVideoMgr, your_var);
```

Note: all routines in here will return **LT\_NOTINITIALIZED** if the engine can't initialize the system's video modules or the runtime DLLs are missing.

---

## Member Enumeration Documentation

anonymous enum

**Enumeration values:**

[\\_ILTVideoMgr\\_VERSION](#)

---

## Member Function Documentation

```
virtual LTRESULT ILTVideoMgr::GetVideoStatus( HVIDEO hVideo ) [pure virtual]
```

**Parameters:**

*hVideo* Handle to video.

**Returns:**

**LT\_NOTINITIALIZED** - Video module not initialized. Can generally be fixed by making sure that `binkw32.dll` is loaded by placing it in the working directory.

**LT\_INVALIDPARAMS** - *hVideo* is invalid.

**LT\_FINISHED** - Finished playing.

**LT\_OK** - Still playing.

When it's done playing, it will sit at the last frame. You *must* call [StopVideo\(\)](#) to free a video's resources.

**See also:**

[StopVideo\(\)](#)

Used for: Special FX.

```
virtual LTRESULT ILTVideoMgr::StartOnScreenVideo( const char * pFilename,
                                                uint32 flags,
                                                HVIDEO & hVideo
                                                ) [pure virtual]
```

**Parameters:**

*pFilename* Video filename.

*flags* Video playback flags (see **PLAYBACK\_** defines in [ltcodes.h](#)).

*hVideo* (return) Handle to video.

**Returns:**

**LT\_NOTINITIALIZED** - Video and/or renderer module not initialized. Can generally be fixed by making sure that `binkw32.dll` is loaded by placing it in the working directory.

**LT\_INVALIDPARAMS** - *pFilename* is NULL.

**LT\_OUTOFMEMORY** - Could not load video file.

**LT\_MISSINGFILE** - *pFilename* is invalid (can't find file).

**LT\_ERROR** - Internal DirectDraw error while initializing screen.

**LT\_OK** - Successful.

Used for: Special FX.

```
virtual LTRESULT ILTVideoMgr::StopVideo( HVIDEO hVideo ) [pure virtual]
```

**Parameters:**

*hVideo* Handle to video.

**Returns:**

**LT\_NOTINITIALIZED** - Video module not initialized. Can generally be fixed by making sure that `binkw32.dll` is loaded by placing it in the working directory.

**LT\_INVALIDPARAMS** - *hVideo* is invalid.

**LT\_OK** - Successful.

Stop playing a video. For a texture video, this restores the original textures on any polygons to which it is bound.

Used for: Special FX.

virtual [LTRESULT](#) ILTVideoMgr::UpdateVideo( [HVIDEO](#) *hVideo* ) [pure virtual]

**Parameters:**

*hVideo* Handle to video.

**Returns:**

**LT\_NOTINITIALIZED** - Video module not initialized or some other internal rendering internal error. Can generally be fixed by making sure that binkw32.dll is loaded by placing it in the working directory.

**LT\_INVALIDPARAMS** - *hVideo* is invalid.

**LT\_OK** - Successful.

For on-screen videos, this draws the current video to the screen.

Used for: Special FX.

The documentation for this class was generated from the following file:

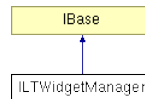
- [iltvideomgr.h](#)

Touchdown Entertainment, Inc.  
[Send feedback regarding this page.](#)  
2006, All Rights Reserved.

## API Reference

## ILTWidgetManager Class Reference

Inheritance diagram for ILTWidgetManager:



### Public Types

enum { [ILTWidgetManager\\_VERSION](#) = 0 }

### Public Member Functions

virtual [~ILTWidgetManager](#) ()  
virtual [CUI\\_RESULTTYPE](#) [SetCurrentSkin](#) ([HTEXTURE](#) hTex)=0  
virtual [HTEXTURE](#) [GetCurrentSkin](#) ()=0  
virtual void [Init](#) ([uint16](#) scrWidth, [uint16](#) scrHeight)=0  
virtual void [Term](#) ()=0  
virtual bool [IsInitied](#) ()=0  
virtual [CUIBase](#) \* [CreateWidget](#) ([CUIBase](#) \*pParent, [CUI\\_WIDGETTYPE](#) type, [CUIRECT](#) \*pRect, [CUIBase](#) \*pRelativeTo, [uint32](#) flags=[CUIIS\\_VISIBLE|CUIIS\\_ENABLED|CUIIS\\_TREE\\_VISIBLE](#))=0  
virtual [CUI\\_RESULTTYPE](#) [DestroyWidget](#) ([CUIBase](#) \*pWidget)=0  
virtual [CUIBase](#) \* [GetRootWidget](#) ()=0  
virtual [CUIBase](#) \* [GetWidgetAtCoords](#) ([int16](#) x, [int16](#) y)=0  
virtual [CUI\\_RESULTTYPE](#) [SetFocusWidget](#) ([CUIBase](#) \*pWidget)=0  
virtual [CUIBase](#) \* [GetFocusWidget](#) ()=0  
virtual bool [IsWidgetValid](#) ([CUIBase](#) \*pWidget)=0  
virtual void [Update](#) ()=0  
virtual [CUI\\_RESULTTYPE](#) [EnqueueMessage](#) ([CUIMessage](#) \*pMessage)=0  
virtual [CUI\\_RESULTTYPE](#) [DequeueMessage](#) ([CUIMessage](#) \*pMessage)=0  
virtual [CUI\\_RESULTTYPE](#) [PeekMessage](#) ([CUIMessage](#) \*pMessage)=0  
virtual [CUI\\_RESULTTYPE](#) [SetUserCallback](#) ([CUI\\_CALLBACKTYPE](#) pUserCallback, void \*pUserCallbackObject, void \*pUserCallbackData)=0  
virtual [CUI\\_CALLBACKTYPE](#) [GetUserCallback](#) (void \*\*ppUserCallbackObject, void \*\*ppUserCallbackData)=0  
virtual void \* [GetUserCallbackObject](#) ()=0  
virtual void \* [GetUserCallbackData](#) ()=0  
virtual [CUI\\_RESULTTYPE](#) [FocusNext](#) ()=0  
virtual [CUI\\_RESULTTYPE](#) [FocusUp](#) ()=0  
virtual [CUI\\_RESULTTYPE](#) [FocusDown](#) ()=0

### Detailed Description

ILTWidgetManager interface. This interface is used to create and destroy CUI Widgets, as well as pass CUIMessages around.

The widget manager maintains a hierarchy ui ui widgets which is used in rendering and message handling. When you create a widget via the [ILTWidgetManager::CreateWidget\(\)](#) function, you pass in an (optionally NULL) parent widget to secure the new widget's place in the hierarchy. (parent-child

relationships can be changed at run-time, too). The hierarchy has two basic rules:

1. Parent widgets render before (behind) their children.
2. Messages are passed to children before the parent attempts to handle them.

If you want to use the `ILTWidgetManager`, you must call the [ILTWidgetManager::Update\(\)](#) fuction once per frame. This will trigger the following actions:

1. Any messages waiting in the queue will be handled
2. The UI system will render itself.
3. Widgets scheduled for destruction ( [ILTWidgetManager::DestroyWidget\(\)](#) ) will be released.

`ILTWidgetManager` is available from the Interface Database.

**See also:**

class [IBase](#)

class [CUIBase](#)

class [CUIMessage](#)

Used for: Text and UI.

## Member Enumeration Documentation

anonymous enum

**Enumeration values:**

[\\_ILTWidgetManager\\_VERSION\\_](#)

## Constructor & Destructor Documentation

virtual `ILTWidgetManager::~ILTWidgetManager( )` [`inline`, `virtual`]

## Member Function Documentation

```
virtual CUIBase* ILTWidgetManager::CreateWidget( CUIBase *      pParent,
                                              CUI\_WIDGETTYPE type,
                                              CUIRECT *      pRect,
                                              CUIBase *      pRelativeTo,
                                              uint32         flags = CUIS_VISIBLE|CUIS_ENABLED|CUIS_TREE_VISIBLE
                                              ) [pure virtual]
```

**Parameters:**

*pParent*      parent of the new widget (can be NULL).  
*type*          the type of widget to create.  
*pRect*        initial size and position of the widget.  
*pRelativeTo* specifies which widget (if any) the initial rectangle coordinates are relative to.  
*flags*        (optional) specifies initial status flags.

**Returns:**

pointer to a [CUIBase](#) object

Creates a new widget. If the *parent* is NULL, the widget will not be part of the rendering or messaging hierarchy, and it will be left up to the programmer to ensure that the widget is drawn. If *pRelativeTo* is NULL, then *pRect* will be relative to the upper left corner of the display (0,0).

To create a widget that is visible and will receive messages by default, the macro `CUI_DEFAULT_WIDGET_FLAGS` is passed in for the *flags* parameter. See [cuitypes.h](#) for the macro definition and other flags.

**See also:**

[CUI\\_WIDGETTYPE](#)

[CUI\\_STATETYPE](#)

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE ILTWidgetManager::DequeueMessage( CUIMessage * pMessage ) [pure virtual]
```

**Parameters:**

*pMessage* address of an empty [CUIMessage](#) object

**Returns:**

`CUIR_OK` if successful.

This function dequeues (removes) a message from the internal CUI message handler. You must provide a [CUIMessage](#) object to receive the data.

**See also:**

[CUIMessage](#).

[CUI\\_MESSAGETYPE](#).

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE ILTWidgetManager::DestroyWidget( CUIBase * pWidget ) [pure virtual]
```

**Parameters:**

*pWidget* the address of the widget to destroy.

**Returns:**

CUIR\_OK if successful.

Schedules a widget for destruction. The widget is not immediately destroyed, so that any last rendering calls or messages can be handled. At the end of the [ILTWidgetManager::Update\(\)](#) function, scheduled widgets are deleted. Destroying a widget recursively destroys all child widgets, so if you do not want to release all children, you must remove them from the widget's child list.

Used for: Text and UI.

virtual [CUI\\_RESULTTYPE](#) ILTWidgetManager::EnqueueMessage( [CUIMessage](#) \* *pMessage* ) [pure virtual]

**Parameters:**

*pMessage* address of a [CUIMessage](#) object

**Returns:**

CUIR\_OK if successful.

This function enqueues a message in the internal CUI message handler. An internal copy of the message is created, so you do not have to keep this message around.

**See also:**

[CUIMessage](#).

[CUI\\_MESSAGE\\_TYPE](#).

Used for: Text and UI.

virtual [CUI\\_RESULTTYPE](#) ILTWidgetManager::FocusDown( ) [pure virtual]

**Returns:**

CUIR\_OK if successful.

This function sets the focus to the first child of the currently focused widget, if any.

Used for: Text and UI.

virtual [CUI\\_RESULTTYPE](#) ILTWidgetManager::FocusNext( ) [pure virtual]

**Returns:**

CUIR\_OK if successful.

This function sets the focus to the next sibling of the currently focused widget, if any.

Used for: Text and UI.

virtual [CUI\\_RESULTTYPE](#) ILTWidgetManager::FocusUp( ) [pure virtual]

**Returns:**

CUIR\_OK if successful.

This function sets the focus to the parent of the currently focused widget, if any.

Used for: Text and UI.

virtual [HTEXTURE](#) ILTWidgetManager::GetCurrentSkin( ) [pure virtual]

virtual [CUIBase](#)\* ILTWidgetManager::GetFocusWidget( ) [pure virtual]

**Parameters:**

*pWidget* the widget to focus.

**Returns:**

CUIR\_OK if successful.

Setting the focus widget has no visible effect, but it is a way to keep track of a widget for whatever purpose you may have in mind.

**See also:**

[GetFocusWidget\(\)](#).

Used for: Text and UI.

virtual [CUIBase](#)\* ILTWidgetManager::GetRootWidget( ) [pure virtual]

**Returns:**

the address of the root widget.

The root widget is the parent of all top-level widgets.

Used for: Text and UI.

virtual [CUI\\_CALLBACKTYPE](#) ILTWidgetManager::GetUserCallback( void \*\* *ppUserCallbackObject*,  
void \*\* *ppUserCallbackData*  
) [pure virtual]

(out) address of user-defined callback object (out) address of user-defined callback data

**Returns:**

address of user-defined callback function

This function retrieves the user-defined callback, object, and data specified when the user callback was set.

**See also:**

[SetUserCallback\(\)](#).

virtual void\* ILTWidgetManager::GetUserCallbackData( ) [pure virtual]

**Returns:**

user-defined callback data

This function retrieves the user-defined callback data specified when the user callback was set.

**See also:**

[SetUserCallback\(\)](#).

virtual void\* ILTWidgetManager::GetUserCallbackObject( ) [pure virtual]

**Returns:**

user-defined callback object

This function retrieves the user-defined callback object specified when the user callback was set.

**See also:**

[SetUserCallback\(\)](#).

virtual [CUIBase](#)\* ILTWidgetManager::GetWidgetAtCoords( [int16](#) x,  
[int16](#) y  
) [pure virtual]

**Returns:**

the address of the topmost widget at screen position (x,y).

Use this function to find the topmost widgets at screen position (x,y).

Used for: Text and UI.

virtual void ILTWidgetManager::Init( [uint16](#) scrWidth,  
[uint16](#) scrHeight  
) [pure virtual]

**Parameters:**

*scrWidth* the LithTech window's width, in pixels

*scrHeight* the LithTech window's height, in pixels

**Required Function.** Called to initialize the ILTWidgetManager's services. You cannot call any of the ILTWidgetManager's functions until you have initialized it. The client is responsible for initializing the ILTWidgetManager. ILTClientShell::OnEngineInitialized() is a good place.

virtual bool ILTWidgetManager::IsInitted( ) [pure virtual]

**Returns:**

whether ILTWidgetManager was initialized.

Returns **true** if the ILTWidgetManager is initialized, or **false** if it has not yet been initialized.

**See also:**

[Init\(\)](#)

[Term\(\)](#)

Used for: Text and UI.

virtual bool ILTWidgetManager::IsValidWidget( [CUIBase](#) \* pWidget ) [pure virtual]

**Parameters:**

*pWidget* address of the widget to check

**Returns:**

true or false

Function returns true if the widget's CUIGUID is found in the internal GUID tree. If the function returns false, then the widget was previously destroyed and the pointer is no longer valid.

Used for: Text and UI.

virtual [CUI\\_RESULTTYPE](#) ILTWidgetManager::PeekMessage( [CUIMessage](#) \* pMessage ) [pure virtual]

**Parameters:**

*pMessage* address of a [CUIMessage](#) object

**Returns:**

CUIR\_OK if successful.

This function looks at the next message in the queue, but does not delete it. You must provide a [CUIMessage](#) object to receive the data.

**See also:**

[CUIMessage](#).

[CUI\\_MESSAGE\\_TYPE](#).

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE ILTWidgetManager::SetCurrentSkin( HTEXTURE hTex ) [pure virtual]
```

```
virtual CUI\_RESULTTYPE ILTWidgetManager::SetFocusWidget( CUIBase * pWidget ) [pure virtual]
```

**Parameters:**

*pWidget* the widget to focus.

**Returns:**

CUIR\_OK if successful.

Setting the focus widget has no visible effect, but it is a way to keep track of a widget for whatever purpose you may have in mind.

**See also:**

[GetFocusWidget\(\)](#).

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE ILTWidgetManager::SetUserCallback( CUI\_CALLBACKTYPE pUserCallback,
                                                         void * pUserCallbackObject,
                                                         void * pUserCallbackData
                                                         ) [pure virtual]
```

**Parameters:**

*pUserCallback* address of a user-defined callback,

*pUserCallbackObject* a user-defined object.

*pUserCallbackData* user-specified data that gets added to every [CUIMessage](#) going to this callback

**Returns:**

CUIR\_OK if successful.

This function installs a system-wide message callback if the user wishes to intercept all messages before they are sent to any widgets. the *pUserCallbackObject* parameter is used to get around the limitation of not being able to use dynamic member functions as callbacks. For example:

Within foo::init\_ui()

Call the function [ILTWidgetManager::SetUserCallback](#)(foo::static\_callback, **this**, pUserData).

Within static foo::static\_callback(CUIMessage\* pMessage)

```
foo* theFoo = (foo*)pMessage->m_pUserObject;
```

And you can call foo's member functions.

**See also:**

[CUI\\_CALLBACKTYPE](#).

Used for: Text and UI.

```
virtual void ILTWidgetManager::Term( ) [pure virtual]
```

**Required Function.** Called to close down the ILTWidgetManager's services. If you do not call [Term\(\)](#) when the client shell is terminating, you risk memory leaks or worse. The client is responsible for terminating the ILTWidgetManager. ILTClientShell::OnEngineTerm() is a good place.

```
virtual void ILTWidgetManager::Update( ) [pure virtual]
```

**Returns:**

none.

Call this function once per frame to render the UI, process waiting messages, and handle the deletion of destroyed widgets.

Used for: Text and UI.

---

The documentation for this class was generated from the following file:

- [iltwidgetmanager.h](#)